Control for Robots

A 12-Week M.Sc. Lecture Course on Advanced Control for Manipulators, Mobile Robots, and Quadrotors

Dr. Abdul Manan Khan

Academic Year 2025–26

Advanced Robotics Programme

Preface

This book is the lecture-notes companion to the M.Sc. module Control for Robots. Over twelve weeks it develops a complete toolkit for controlling manipulators, mobile robots, and quadrotors — from classical computed-torque control and model-predictive control, through adaptive, robust, backstepping, and sliding-mode control, to a unifying framework of nonlinear control optimization that ties every controller to a single optimization problem with free parameters. The final four weeks bring the theory into ROS 2: each controller is implemented as a ROS 2 node, wrapped into navigation pipelines, deployed on quadrotor dynamics, and compared head-to-head in the capstone.

Every lecture is supported by a working MATLAB (or in one case, Python) simulation. All figures in this book are reproducible from the simulation scripts available in the companion repository at `https://github.com/abdul-mannan-khan/control-for-robots`. A Docker runtime is provided so students can reproduce every result without fighting library installation.

Dr. Abdul Manan Khan
Advanced Robotics Programme, 2026

Contents

Contents iii

Control Theory Block 1

1 Manipulator Dynamics and Computed-Torque Control (Week 1) 2
1.1 Overview . . . 2
1.1.1 Learning Outcomes . . . 2
1.1.2 Session Roadmap (3 Hours) . . . 2
1.2 Manipulator Dynamics . . . 3
1.2.1 Why Dynamics Matter . . . 3
1.2.2 The Euler–Lagrange Equation of Motion . . . 4
1.2.3 A Concrete Example: 2-DOF Planar Manipulator . . . 4
1.2.4 Key Properties of the Dynamic Model . . . 5
1.3 Control Design for the Manipulator . . . 5
1.3.1 Approach 1: Simple PD Control . . . 5
1.3.2 Approach 2: PD + Gravity Compensation . . . 6
1.3.3 Approach 3: Computed Torque Control (Inverse Dynamics) . . . 6
1.3.4 Gain Selection: Placing Closed-Loop Poles . . . 6
1.3.5 Lyapunov Stability Proof . . . 7
1.4 MATLAB Implementation . . . 8
1.4.1 System Parameters . . . 8
1.4.2 Desired Trajectory Functions . . . 8
1.4.3 Dynamics with Computed Torque Controller . . . 8
1.4.4 Run Simulation & Plot . . . 9
1.5 Comparing Control Strategies . . . 9
1.5.1 Controller A: PD Only . . . 10
1.5.2 Controller B: PD + Gravity Compensation . . . 10
1.5.3 Controller C: Computed Torque (Full) . . . 10
1.6 Robustness Analysis . . . 10
1.6.1 Sources of Model Uncertainty . . . 10
1.6.2 Mathematical Framework . . . 11
1.6.3 Experiment: Payload Mismatch . . . 11
1.6.4 Experiment: Adding Joint Friction . . . 12
1.6.5 Experiment: External Disturbance Pulse . . . 12
1.6.6 Summary of Robustness Results . . . 12
1.7 Summary and Next Steps . . . 13
1.7.1 Hands-On Activities . . . 13
1.7.2 Activity 3: Robustness Investigation (20 min) . . . 13
1.7.3 Activity 4: Controller Comparison Dashboard (25 min) . . . 13
1.7.4 Activity 5: Design Challenge (15 min) . . . 13
1.7.5 End-of-Lecture Quiz . . . 14
1.7.6 Week 1 Summary . . . 14
1.7.7 Key Equations to Remember . . . 15
1.7.8 Next Week: Advanced Control Techniques . . . 15
1.7.9 Preparation for Next Lecture . . . 15

2 Model Predictive Control — Introduction (Week 2) 17
2.1 Overview . . . 17
2.1.1 Week 1 Recall: Computed Torque Control — Exam Practice . . . 17
2.1.2 Learning Outcomes . . . 18
2.1.3 Session Roadmap (3 Hours) . . . 18
2.2 MPC Theory & Derivation . . . 20
2.2.1 What Is Model Predictive Control? . . . 20
2.2.2 The Optimisation Problem . . . 20
2.2.3 State Prediction over the Horizon . . . 21
2.2.4 Quadratic Cost in Matrix Form . . . 22
2.2.5 The Optimal Control Law . . . 22
2.2.6 Receding Horizon: Apply Only the First Input . . . 23
2.2.7 Effect of Tuning Parameters . . . 23
2.3 MPC for 2-DOF Manipulator . . . 24
2.3.1 The Nonlinear Dynamics . . . 24
2.3.2 Linearisation around an Operating Point . . . 24
2.3.3 Discretisation . . . 24
2.3.4 MPC Design for the Manipulator . . . 25
2.3.5 MATLAB Implementation — Manipulator MPC . . . 25
2.4 MPC for Differential-Drive Mobile Robot . . . 25
2.4.1 Kinematic Model . . . 25
2.4.2 Linearisation around a Reference Trajectory . . . 26
2.4.3 Constraints . . . 26
2.4.4 MPC Design . . . 26
2.4.5 MATLAB Implementation — Mobile Robot MPC . . . 26
2.5 MPC for Quadrotor Drone . . . 27
2.5.1 The 12-State Quadrotor Model . . . 27
2.5.2 Linearised Hover Dynamics . . . 27
2.5.3 MPC Design for the Quadrotor . . . 28
2.5.4 Constraints . . . 28
2.5.5 MATLAB Implementation — Quadrotor MPC . . . 28
2.6 MATLAB Implementation & Activities . . . 28
2.6.1 Activity 1: Prediction Horizon Tuning (10 min) . . . 28
2.6.2 Activity 2: Three-System Comparison (15 min) . . . 29
2.6.3 Activity 3: MPC vs Computed Torque Discussion (5 min) . . . 29
2.7 Summary and Next Steps . . . 29
2.7.1 Summary & Key Takeaways . . . 29
2.7.2 Core MPC Equations . . . 29
2.7.3 MPC vs Computed Torque Control . . . 29
2.7.4 Next Week: Adaptive Control . . . 30
2.7.5 Preparation . . . 30

3 Model Predictive Control — Design and Simulation (Week 3) 31
3.1 Overview . . . 31
3.1.1 Week 1 Recall: Computed Torque Control — Exam Practice . . . 31
3.1.2 Learning Outcomes . . . 32
3.1.3 Session Roadmap (3 Hours) . . . 32
3.2 MPC Theory & Derivation . . . 34
3.2.1 What Is Model Predictive Control? . . . 34
3.2.2 The Optimisation Problem . . . 34
3.2.3 State Prediction over the Horizon . . . 35
3.2.4 Quadratic Cost in Matrix Form . . . 36
3.2.5 The Optimal Control Law . . . 36
3.2.6 Receding Horizon: Apply Only the First Input . . . 37

3.2.7 Effect of Tuning Parameters . . . 37
3.3 MPC for 2-DOF Manipulator . . . 38
3.3.1 The Nonlinear Dynamics . . . 38
3.3.2 Linearisation around an Operating Point . . . 38
3.3.3 Discretisation . . . 39
3.3.4 MPC Design for the Manipulator . . . 39
3.3.5 MATLAB Implementation — Manipulator MPC . . . 39
3.4 MPC for Differential-Drive Mobile Robot . . . 40
3.4.1 Kinematic Model . . . 40
3.4.2 Linearisation around a Reference Trajectory . . . 40
3.4.3 Constraints . . . 41
3.4.4 MPC Design . . . 41
3.4.5 MATLAB Implementation — Mobile Robot MPC . . . 41
3.5 MPC for Quadrotor Drone . . . 42
3.5.1 The 12-State Quadrotor Model . . . 42
3.5.2 Nonlinear Equations of Motion . . . 42
3.5.3 Linearisation around Hover . . . 43
3.5.4 MPC Design for the Quadrotor . . . 43
3.5.5 Constraints . . . 44
3.5.6 MATLAB Implementation — Quadrotor MPC . . . 44
3.6 MATLAB Implementation & Activities . . . 45
3.6.1 Activity 1: Prediction Horizon Tuning (10 min) . . . 45
3.6.2 Activity 2: Three-System Comparison (15 min) . . . 45
3.6.3 Activity 3: MPC vs Computed Torque Discussion (5 min) . . . 46
3.7 Summary and Next Steps . . . 46
3.7.1 Summary & Key Takeaways . . . 46
3.7.2 Core MPC Equations . . . 46
3.7.3 MPC vs Computed Torque Control . . . 46
3.7.4 Next Week: Adaptive Control . . . 47
3.7.5 Preparation . . . 47

4 Adaptive Control (Week 4) 48
4.1 Overview . . . 48
4.1.1 Week 3 Recall — Model Predictive Control — Exam Practice . . . 48
4.1.2 Learning Outcomes . . . 50
4.1.3 Session Roadmap (3 Hours) . . . 50
4.2 Adaptive Control Theory . . . 51
4.2.1 Why Adaptive Control? . . . 51
4.2.2 The Linearity-in-Parameters Property . . . 52
4.2.3 Concrete Example: 2-DOF Planar Manipulator . . . 53
4.2.4 The Adaptive Control Law . . . 53
4.2.5 Parameter Update Law . . . 54
4.2.6 Lyapunov Stability Proof . . . 54
4.3 Adaptive Control for 2-DOF Manipulator . . . 57
4.3.1 System Setup . . . 57
4.3.2 Building the Regressor Matrix . . . 57
4.3.3 Complete Control Algorithm . . . 58
4.3.4 MATLAB Implementation . . . 58
4.4 Adaptive Control for Mobile Robot . . . 59
4.4.1 Parametric Uncertainty in Mobile Robots . . . 59
4.4.2 Linearity in Parameters for the Kinematic Model . . . 59
4.4.3 Adaptive Tracking Controller . . . 59
4.4.4 MATLAB Implementation . . . 60

4.5 Adaptive Control for Quadrotor . . . 61
4.5.1 Quadrotor Dynamics with Unknown Parameters . . . 61
4.5.2 Linearity in Parameters . . . 61
4.5.3 Attitude Adaptive Control . . . 62
4.5.4 Update Laws . . . 62
4.5.5 MATLAB Implementation . . . 62
4.6 MATLAB Implementation & Activities . . . 62
4.6.1 Activity 1: Parameter Convergence (10 minutes) . . . 62
4.6.2 Activity 2: Adaptive vs Computed Torque Under Uncertainty (15 minutes) . . . 63
4.6.3 Quiz . . . 63
4.7 Summary and Next Steps . . . 64
4.7.1 Summary — Control Methods Comparison . . . 64

5 Robust Control (Week 5) 65
5.1 Overview . . . 65
5.1.1 Week 4 Recall: Adaptive Control — Exam Practice (20 min) . . . 65
5.1.2 Learning Outcomes . . . 68
5.1.3 Session Roadmap (3 Hours) . . . 68
5.2 Robust Control Theory . . . 69
5.2.1 Why Robust Control? . . . 69
5.2.2 Problem Setup . . . 70
5.2.3 Robust Computed Torque + Robust Term . . . 70
5.2.4 Robust Term Design . . . 71
5.2.5 Lyapunov Stability Proof . . . 71
5.3 Robust Control for 2-DOF Manipulator . . . 73
5.3.1 System Description . . . 73
5.3.2 Uncertainty Model . . . 74
5.3.3 Robust Controller . . . 74
5.3.4 Guaranteed Tracking Error Bound . . . 74
5.3.5 MATLAB Implementation . . . 75
5.4 Robust Control for Mobile Robot . . . 76
5.4.1 Sources of Disturbance . . . 76
5.4.2 Disturbed Unicycle Model . . . 76
5.4.3 Robust Tracking Controller . . . 76
5.4.4 Stability Analysis . . . 77
5.4.5 MATLAB Implementation . . . 77
5.5 Robust Control for Quadrotor Drone . . . 78
5.5.1 Wind Disturbance Model . . . 78
5.5.2 Simplified Model for Control Design . . . 78
5.5.3 Robust Position Controller . . . 78
5.5.4 Robust Attitude Controller . . . 79
5.5.5 MATLAB Implementation . . . 79
5.6 Summary and Next Steps . . . 80
5.6.1 MATLAB Activities . . . 80
5.6.2 Activity 1: Robust vs Non-Robust Manipulator (10 min) . . . 80
5.6.3 Activity 2: Robust vs Adaptive — All Three Systems (15 min) . . . 81
5.6.4 Quick Quiz . . . 81
5.6.5 Summary: Comparison of All Four Controllers . . . 81

6 Backstepping Control (Week 6) 83
6.1 Overview . . . 83
6.1.1 Week 5 Recall: Robust Control — Exam Practice (20 min) . . . 83
6.1.2 Question 1 — Robust Sliding-Mode Analysis . . . 83
6.1.3 Question 2 — Robust vs. Adaptive: Mathematical Comparison . . . 85

6.1.4 Learning Outcomes . . . 85
6.1.5 Session Roadmap (3 Hours) . . . 86
6.2 Backstepping Theory . . . 87
6.2.1 Motivation — Why Backstepping? . . . 87
6.2.2 Strict-Feedback Form . . . 87
6.2.3 Two-Step Backstepping — Complete Derivation . . . 87
6.2.4 Generalisation to \(n\)-Step Backstepping . . . 89
6.2.5 Why Backstepping Works — Intuition . . . 89
6.3 Backstepping for Robot Manipulator . . . 90
6.3.1 Strict-Feedback Form of Manipulator Dynamics . . . 90
6.3.2 MATLAB Implementation . . . 92
6.4 Backstepping for Mobile Robot . . . 92
6.4.1 Unicycle Model and Error Coordinates . . . 92
6.4.2 Backstepping Design . . . 93
6.4.3 MATLAB Implementation . . . 94
6.5 Backstepping for Quadrotor Drone . . . 94
6.5.1 Quadrotor Cascade Structure . . . 94
6.5.2 Backstepping Design — Outer Loop (Position Control) . . . 95
6.5.3 Backstepping Design — Inner Loop (Attitude Control) . . . 95
6.5.4 MATLAB Implementation . . . 96
6.6 Summary and Next Steps . . . 96
6.6.1 Hands-on Activities & Quiz . . . 96
6.6.2 Activity 1: Gain Tuning for Manipulator Backstepping (10 min) . . . 96
6.6.3 Activity 2: Comparative Study (15 min) . . . 97
6.6.4 Summary: Comparison of All Five Control Methods . . . 98

7 Sliding Mode Control (Week 7) 99
7.1 Overview . . . 99
7.1.1 Week 6 Recall: Backstepping Control — Exam Practice . . . 99
7.1.2 Question 1 — Scalar Strict-Feedback System . . . 99
7.1.3 Question 2 — 2-DOF Manipulator Backstepping . . . 100
7.1.4 Learning Outcomes . . . 101
7.1.5 Session Roadmap . . . 101
7.2 Sliding Mode Control Theory . . . 102
7.2.1 1.1 Motivation and Core Idea . . . 102
7.2.2 1.2 Two Phases of SMC . . . 103
7.2.3 1.3 Sliding Surface Design . . . 103
7.2.4 1.4 Reaching Law . . . 103
7.2.5 1.5 Control Law Decomposition . . . 104
7.2.6 1.6 Existence Condition (Reaching Condition) . . . 105
7.2.7 1.7 The Chattering Problem . . . 105
7.2.8 1.8 Multi-Variable Extension . . . 107
7.3 SMC for Robot Manipulator . . . 107
7.3.1 2.1 System Model . . . 107
7.3.2 2.2 Sliding Surface . . . 108
7.3.3 2.3 Control Law Derivation . . . 108
7.3.4 2.4 Lyapunov Stability Proof . . . 109
7.3.5 2.5 MATLAB Implementation — 2-DOF Manipulator SMC . . . 109
7.4 SMC for Mobile Robot . . . 110
7.4.1 3.1 Unicycle Kinematic Model . . . 110
7.4.2 3.2 Position Tracking Formulation . . . 110
7.4.3 3.3 Sliding Surface Design . . . 110
7.4.4 3.4 SMC Control Law . . . 111
7.4.5 3.5 MATLAB Implementation . . . 111

7.5 SMC for Quadrotor Drone . . . 112
7.5.1 4.1 Quadrotor Dynamic Model . . . 112
7.5.2 4.2 Hierarchical SMC Design . . . 112
7.5.3 4.3 MATLAB Implementation . . . 114
7.6 Summary and Next Steps . . . 114
7.6.1 Lab Activities . . . 114
7.6.2 Activity 1 — Chattering Observation (10 min) . . . 114
7.6.3 Activity 2 — Disturbance Rejection Comparison (15 min) . . . 114
7.6.4 Activity 3 — Super-Twisting Algorithm (5 min) . . . 115
7.6.5 Summary — All Six Control Methods Compared . . . 115

8 Nonlinear Control Optimization for Robotic Systems (Week 8) 117
8.1 Overview . . . 117
8.1.1 Learning Outcomes . . . 117
8.1.2 Session Roadmap . . . 117
8.1.3 MATLAB Lab Activities . . . 118
8.1.4 Setup . . . 118
8.1.5 Activity 1 — SMC Pareto sweep (15 min) . . . 119
8.1.6 Activity 3 — Robust vs. average-case (15 min) . . . 119
8.1.7 Activity 4 — NMPC horizon sweep (15 min) . . . 119
8.1.8 Appendix: Week 7 Recall — Sliding Mode Control (Exam Practice) . . . 119
8.1.9 Course Recap & What's Next . . . 121
8.1.10 The nonlinear control theory journey . . . 121
8.1.11 Unifying themes . . . 121
8.1.12 What's next — Automation block . . . 122
8.2 Why Robots Demand Nonlinear Control Optimization . . . 122
8.2.1 Robot dynamics are globally nonlinear . . . 122
8.2.2 Why Jacobian linearization is not enough . . . 123
8.2.3 Two meanings of “control optimization” . . . 123
8.3 Foundations of Nonlinear Control Optimization . . . 123
8.3.1 Notation used throughout Section 4 . . . 123
8.3.2 The nonlinear optimal control problem . . . 124
8.3.3 Pontryagin's Minimum Principle (PMP) . . . 124
8.3.4 Hamilton–Jacobi–Bellman (nonlinear) . . . 125
8.3.5 Direct methods: shooting and collocation . . . 126
8.3.6 iLQR and DDP — the robot trajectory optimizer of choice . . . 126
8.3.7 Nonlinear MPC (NMPC) . . . 129
8.4 Optimization of Sliding Mode Control . . . 129
8.4.1 Free parameters of SMC . . . 129
8.4.2 Optimization formulation . . . 130
8.4.3 Worked numerical example — single-link manipulator . . . 132
8.4.4 Simulation results (manipulator / mobile / quadrotor) . . . 134
8.5 Optimization of Adaptive Control . . . 134
8.5.1 Free parameters . . . 134
8.5.2 Full Slotine–Li derivation — where the adaptation law comes from . . . 135
8.5.3 Worked numerical example — single-link manipulator with unknown payload . . . 137
8.5.4 Optimization formulation (of \(\Gamma, \sigma\)) . . . 139
8.5.5 Simulation results . . . 139
8.6 Optimization of Robust Control for Robotic Systems . . . 140
8.6.1 Where uncertainty comes from in a real robot . . . 140
8.6.2 Robust controller structures used in this lab . . . 140
8.6.3 Full derivation: why the robust term \(-\rho\tanh(s/\varepsilon)\) works . . . 142
8.6.4 Worked numerical example — single-link arm with payload uncertainty . . . 143
8.6.5 Min-max optimization problem . . . 144

8.6.6 Link to LMI and \(H_\infty\) design . . . 144
8.6.7 MATLAB implementation . . . 144
8.6.8 Simulation results . . . 146
8.7 Optimization of Backstepping Control . . . 146
8.7.1 Free parameters: the virtual-control gains . . . 146
8.7.2 Full 2-step backstepping derivation for the manipulator . . . 147
8.7.3 Worked numerical example — backstepping on the single-link arm . . . 148
8.7.4 CLF-based optimization cost . . . 149
8.7.5 Command-filtered backstepping . . . 150
8.7.6 Simulation results . . . 150
8.8 Optimization of Nonlinear Model Predictive Control . . . 150
8.8.1 The design knobs . . . 150
8.8.2 Terminal ingredients and recursive feasibility . . . 151
8.8.3 Horizon vs. compute: the real-time trade . . . 151
8.8.4 Full derivation of the iLQR algorithm . . . 151
8.8.5 Worked numerical example — iLQR on the single-link arm . . . 154
8.8.6 Simulation results (iLQR-based NMPC) . . . 156
8.9 Unified View — Every Nonlinear Controller as an Optimization Problem . . . 157
8.9.1 The common template . . . 157
8.9.2 Master summary table . . . 157
8.9.3 Cross-cutting optimization tools . . . 157

ROS 2 Integration Block 159

9 ROS 2 Control Implementation (Week 9) 160
9.1 Overview . . . 160
9.1.1 Company Brief: SkyDynamics Robotics . . . 160
9.1.2 Provided Bag Data . . . 160
9.1.3 Deliverables . . . 160
9.1.4 Evaluation Criteria . . . 161
9.1.5 Skills You Will Demonstrate . . . 161
9.1.6 Recommended Approach (3-hour session) . . . 161
9.2 Why ROS2 for Control Engineers? . . . 163
9.2.1 ROS2 vs ROS1 for Real-Time Control . . . 163
9.2.2 The ros2_control Framework . . . 164
9.2.3 Industry Adoption . . . 164
9.2.4 The Control Loop in ROS2 . . . 164
9.2.5 ROS2 Control Architecture Diagram . . . 165
9.3 Implementing PID Controller as a ROS2 Node . . . 165
9.3.1 Theory Recap . . . 165
9.3.2 ROS2 Implementation . . . 165
9.3.3 Real-Time Considerations . . . 166
9.4 Implementing LQR Controller as a ROS2 Node . . . 167
9.4.1 Theory Recap . . . 167
9.4.2 ROS2 Implementation . . . 167
9.4.3 PID vs LQR Comparison . . . 169
9.5 Implementing MPC Controller as a ROS2 Node . . . 169
9.5.1 Theory Recap . . . 169
9.5.2 ROS2 Implementation . . . 169
9.5.3 MPC Node Internal Loop . . . 171
9.5.4 Discussion: Computation Time vs Horizon Length . . . 171
9.6 NMPC via iLQR as a ROS2 Node — Week 8 Integration . . . 171
9.6.1 4b.1 NMPC Node Structure . . . 172

9.6.2 4b.2 Running the NMPC Node . . . 172
9.6.3 4b.3 Mapping to Week 8 ğ9 Derivation . . . 172
9.6.4 4b.4 Simulation results . . . 173
9.7 Extending to All 7 Controllers as ROS2 Nodes . . . 174
9.7.1 Implementation Guides . . . 174
9.7.2 Controller Complexity Comparison . . . 175
9.7.3 Unified Launch File . . . 175
9.8 Summary and Next Steps . . . 176
9.8.1 ROS2 Lab Exercises . . . 176
9.8.2 Lab Analysis Tasks (Python/MATLAB-style) . . . 176
9.8.3 ROS2 Applied Exercises . . . 178
9.8.4 Hands-On Activities . . . 178
9.8.5 Activity 1: PID on TurtleBot3 Simulation . . . 178
9.8.6 Activity 2: LQR with Different Q/R Matrices . . . 179
9.8.7 Activity 3: MPC Horizon Study . . . 179
9.8.8 Summary . . . 180
9.8.9 Key Takeaways . . . 180
9.8.10 MATLAB vs ROS2: When to Use Which . . . 181
9.8.11 Running the ROS2 Demo . . . 181
9.8.12 Build and Source . . . 181
9.8.13 Launch the Demo . . . 182
9.8.14 ROS2 Topics . . . 182
9.8.15 Expected RViz2 Output . . . 182
9.8.16 Troubleshooting . . . 182
9.8.17 Appendix: Review Material — LQR/LQG Optimal Control . . . 182

10 ROS 2 Quadrotor Control (Week 10) 186
10.1 Overview . . . 186
10.1.1 Company Brief: AeroVision UAV . . . 186
10.1.2 Provided Bag Data . . . 186
10.1.3 Deliverables . . . 186
10.1.4 Evaluation Criteria . . . 187
10.1.5 Skills You Will Demonstrate . . . 187
10.1.6 Recommended Approach (3-Hour Session) . . . 187
10.2 Quadrotor Dynamics Review . . . 188
10.2.1 State Vector . . . 189
10.2.2 Translational Dynamics . . . 189
10.2.3 Rotational Dynamics . . . 189
10.2.4 Motor Mixing Matrix . . . 190
10.2.5 Quadrotor Plus-Configuration Diagram . . . 190
10.3 Cascaded Control Architecture in ROS2 . . . 190
10.3.1 Three-Loop Cascade . . . 190
10.3.2 Cascaded Architecture Block Diagram . . . 191
10.3.3 ROS2 Topic Structure . . . 191
10.3.4 Launch File for Cascaded System . . . 192
10.4 Implementing PID Attitude Controller in ROS2 . . . 192
10.4.1 Full PID Attitude Controller Node . . . 192
10.4.2 Complementary Filter for Attitude Estimation . . . 193
10.5 Applying All 7 Controllers to Quadrotor . . . 194
10.5.1 Computed Torque Control (CTC) . . . 194
10.5.2 Model Predictive Control (MPC) . . . 194
10.5.3 Adaptive Control . . . 195
10.5.4 Robust Control (H-infinity / Sliding Boundary Layer) . . . 195
10.5.5 Backstepping Control . . . 195

10.5.6 Sliding Mode Control (SMC) . . . 196
10.5.7 Nonlinear Control Optimization — iLQR / NMPC (Week 8) . . . 196
10.5.8 Comprehensive Controller Comparison . . . 197
10.6 Quadrotor Simulation in ROS2 . . . 197
10.6.1 Simulation Environment . . . 197
10.6.2 ROS2 Integration . . . 198
10.6.3 Visualization in RViz2 . . . 198
10.6.4 Recording Flight Data with rosbag2 . . . 198
10.6.5 Lab Exercises from ROS2 Robotics Course (Week 6) . . . 198
10.7 Summary and Next Steps . . . 199
10.7.1 Hands-On Activities . . . 199
10.7.2 Activity 1: PID Hover Test . . . 199
10.7.3 Activity 2: Trajectory Tracking — PID vs LQR vs MPC . . . 199
10.7.4 Activity 3: Wind Disturbance Rejection — SMC vs Robust vs PID . . . 200
10.7.5 Summary . . . 202
10.7.6 Key Takeaways . . . 202
10.7.7 Preparation for Week 11 . . . 203
10.7.8 Running the ROS2 Demo . . . 203
10.7.9 Build and Source . . . 203
10.7.10 Launch the Demo . . . 203
10.7.11 ROS2 Topics . . . 204
10.7.12 Expected RViz2 Output . . . 204
10.7.13 Troubleshooting . . . 204
10.7.14 Appendix: Review Material — ROS2 Control Implementation . . . 204

11 ROS 2 Navigation Control (Week 11) 207
11.1 Overview . . . 207
11.1.1 Company Brief: AutoNav Solutions . . . 207
11.1.2 Provided Bag Data . . . 207
11.1.3 Deliverables . . . 207
11.1.4 Evaluation Criteria . . . 208
11.1.5 Skills You Will Demonstrate . . . 208
11.1.6 Recommended Approach (3-Hour Session) . . . 208
11.2 The ROS2 Navigation Stack (Nav2) . . . 210
11.2.1 Nav2 Core Components . . . 210
11.2.2 Nav2 Architecture Diagram . . . 211
11.2.3 Key Nav2 ROS2 Topics . . . 211
11.3 DWA: The Local Controller as Optimization . . . 211
11.3.1 DWA Velocity Space . . . 211
11.3.2 Dynamic Window Constraint . . . 211
11.3.3 DWA Objective Function . . . 212
11.3.4 Connection to MPC (Week 3) . . . 212
11.3.5 ROS2 Implementation — Simplified DWA Local Planner Node . . . 213
11.4 AMCL: State Estimation for Control . . . 214
11.4.1 Connection to Nonlinear Optimization (Week 8) . . . 214
11.4.2 AMCL Algorithm . . . 215
11.4.3 AMCL Cycle Diagram . . . 216
11.4.4 ROS2 AMCL Configuration . . . 216
11.5 Costmap: Constraints for Control . . . 216
11.5.1 Costmaps as Control Constraints . . . 216
11.5.2 Costmap Layers . . . 217
11.5.3 Inflation Cost Function . . . 217
11.5.4 Nav2 Costmap Configuration . . . 217

11.6 Behavior Trees for Mission Control . . . 218
11.6.1 Why Behavior Trees? . . . 218
11.6.2 BT Node Types . . . 218
11.6.3 Navigation Behavior Tree Diagram . . . 218
11.6.4 Nav2 BT XML Configuration . . . 218
11.7 Applying Control Theory to Navigation . . . 219
11.7.1 Detailed Mapping . . . 219
11.8 Summary and Next Steps . . . 220
11.8.1 ROS2 Lab Exercises . . . 220
11.8.2 Python Lab Exercises (Algorithmic Foundations) . . . 221
11.8.3 ROS2 Applied Exercises (Node-Based) . . . 221
11.8.4 Hands-On Activities . . . 221
11.8.5 Activity 1: Configure Nav2 for TurtleBot3 Navigation . . . 221
11.8.6 Activity 2: Compare DWA vs MPC for Navigation . . . 222
11.8.7 Activity 3: Waypoint Navigation with Recovery Behaviors . . . 223
11.8.8 Summary — Key Takeaways . . . 225
11.8.9 Running the ROS2 Demo . . . 226
11.8.10 Build and Source . . . 226
11.8.11 Launch the Demo . . . 226
11.8.12 ROS2 Topics . . . 226
11.8.13 Expected RViz2 Output . . . 227
11.8.14 Troubleshooting . . . 227
11.8.15 Appendix: Review Material — Quadrotor Control . . . 227

12 Controller Comparison and Industry Applications (Week 12) 230
12.1 Overview . . . 230
12.1.1 Company Brief: Autonomous Systems Corp . . . 230
12.1.2 Provided Bag Data . . . 230
12.1.3 Deliverables . . . 230
12.1.4 Evaluation Criteria . . . 231
12.1.5 Skills You Will Demonstrate . . . 231
12.1.6 Recommended Approach (3-Hour Capstone) . . . 231
12.2 Comprehensive Controller Comparison . . . 233
12.2.1 The Seven Controllers at a Glance . . . 233
12.2.2 Performance Metrics for Controller Comparison . . . 235
12.2.3 Head-to-Head Comparison Table . . . 236
12.2.4 Controller Selection Decision Flowchart . . . 237
12.2.5 Comparison by Robot System . . . 237
12.2.6 When to Use Which Controller — Practitioner's Decision Guide . . . 239
12.2.7 Practical Decision Guide . . . 239
12.3 Industry Applications & Real-World Case Studies . . . 240
12.3.1 Automotive & Self-Driving Vehicles . . . 240
12.3.2 Aerospace & UAVs . . . 241
12.3.3 Industrial Robotics & Manufacturing . . . 243
12.3.4 Warehouse & Logistics Robotics . . . 244
12.3.5 Legged Robotics . . . 245
12.3.6 Medical & Surgical Robotics . . . 246
12.3.7 Marine & Underwater Robotics . . . 247
12.3.8 Space & Satellite Systems . . . 248
12.3.9 Energy & Process Control . . . 249
12.3.10 Consumer & Emerging Applications . . . 250
12.3.11 Industry-Controller Matrix . . . 250
12.3.12 Controller Popularity by Domain . . . 251

12.4 ROS 2 System Integration . . . 251
12.4.1 Full Autonomous System Architecture . . . 251
12.4.2 ROS 2 Course Control Course Alignment Map . . . 251
12.4.3 Tier 1 — Direct Alignment (Must-Do) . . . 251
12.4.4 Tier 2 — Strong Complementary . . . 252
12.4.5 Tier 3 — Supporting Context . . . 252
12.4.6 Building Your Interview Demo Portfolio . . . 252
12.4.7 Reference to ROS 2 Capstone (src/ros2_robotics_course/week_12) . . . 253
12.5 Summary and Next Steps . . . 254
12.5.1 Activity 1: Controller Selection Challenge . . . 254
12.5.2 Scenario 1: FANUC Paint Line with Variable Paint Guns (0.5–2.5 kg) . . . 254
12.5.3 Scenario 2: Delivery Drone in 30+ mph Crosswinds . . . 254
12.5.4 Scenario 3: Self-Driving Car on Highway with Comfort Constraints . . . 254
12.5.5 Scenario 4: Surgical Robot Tracking Breathing Tumour (0.1 mm precision) . . . 254
12.5.6 Scenario 5: Geostationary Satellite Station-Keeping (Fuel-Limited) . . . 255
12.5.7 Activity 2: Industry Research Project . . . 255
12.5.8 Activity 3: Grand Demo — Portfolio Video . . . 255
12.5.9 Video Structure . . . 256
12.5.10 Course Wrap-Up: 12 Weeks of Control for Robots . . . 257
12.5.11 Summary of All 12 Weeks . . . 257
12.5.12 What to Study Next . . . 257
12.5.13 Career Advice . . . 258
12.5.14 Running the ROS2 Demo . . . 258
12.5.15 Build and Source . . . 258
12.5.16 Launch the Demo . . . 259
12.5.17 ROS2 Topics . . . 259
12.5.18 Expected RViz2 Output . . . 259
12.5.19 Troubleshooting . . . 259
12.5.20 Appendix: Review Material — Navigation Control . . . 259

List of Figures

List of Tables

Control Theory Block

1 Manipulator Dynamics and Computed-Tor (Week 1)

1.1 Overview 2
1.1.1 Learning Outcomes . . . 2
1.1.2 Session Roadmap (3 Hours) 2
1.2 Manipulator Dynamics . 3
1.2.1 Why Dynamics Matter . 3
1.2.2 The Euler–Lagrange Equation of Motion . . . 4
1.2.3 A Concrete Example: 2-DOF Planar Manipulator 4
1.2.4 Key Properties of the Dynamic Model 5
1.3 Control Design for the Manipulator 5
1.3.1 Approach 1: Simple PD Control 5
1.3.2 Approach 2: PD + Gravity Compensation 6
1.3.3 Approach 3: Computed Torque Control (Inverse Dynamics) 6
1.3.4 Gain Selection: Placing Closed-Loop Poles 6
1.3.5 Lyapunov Stability Proof 7
1.4 MATLAB Implementation 8
1.4.1 System Parameters . . . 8
1.4.2 Desired Trajectory Functions 8
1.4.3 Dynamics with Computed Torque Controller 8
1.4.4 Run Simulation & Plot . 9
1.5 Comparing Control Strategies 9
1.5.1 Controller A: PD Only . 10
1.5.2 Controller B: PD + Gravity Compensation . 10
1.5.3 Controller C: Computed Torque (Full) 10
1.6 Robustness Analysis . . . 10
1.6.1 Sources of Model Uncertainty 10
1.6.2 Mathematical Framework 11
1.6.3 Experiment: Payload Mismatch . . . 11

From Equations of Motion to Computed Torque Control

Control for Robots | University of West London
AY 2025–26 | 3-Hour Lecture + Activity Session

1.1 Overview

1.1.1 Learning Outcomes

By the end of this session, you will be able to:

- Derive the Euler–Lagrange equations of motion for a 2-DOF robot manipulator
- Identify the mass matrix \(M(q)\), Coriolis matrix \(C(q,\dot{q})\), and gravity vector \(G(q)\) and explain their physical roles
- Design a computed torque (inverse dynamics) controller and prove its stability using Lyapunov analysis
- Implement computed torque control in MATLAB/Simulink and interpret tracking error plots
- Analyse robustness of the controller against model uncertainties, payload variations, and disturbances
- Compare PD, PD + gravity compensation, and full computed torque control performance

1.1.2 Session Roadmap (3 Hours)

0:00 – 0:40

1.1.2.1 Manipulator Dynamics

- Review of feedback vs feedforward control
- Euler–Lagrange formulation
- Properties of the dynamics: \(M(q)\), \(C(q,\dot{q})\), \(G(q)\)
- Skew-symmetry property & its role in control

0:40 – 1:20

1.1.2.2 Control Design

- PD control and its limitations
- PD + gravity compensation
- Computed torque (inverse dynamics) control
- Lyapunov stability proof

1:20 – 1:30

1.1.2.3 Break

- 10-minute break

1:30 – 2:15

1.1.2.4 MATLAB Implementation

- Guided MATLAB coding of computed torque controller
- Running ODE45 simulations
- Interpreting tracking and error plots

2:15 – 3:00

1.1.2.5 Robustness Analysis & Activities

- Effect of model uncertainty on computed torque
- Payload variation experiments
- Disturbance rejection analysis
- Group activity & wrap-up quiz

1.2 Manipulator Dynamics

1.2.1 Why Dynamics Matter

Kinematics tells us where the robot moves; dynamics tells us what forces and torques are required to produce that motion. Without a dynamic model we cannot design controllers that truly cancel the nonlinear effects of inertia, coupling, Coriolis forces, and gravity.

1.2.1.1 Feedback Control

Uses sensor measurements (encoders, IMUs) to adjust torques in real time. Robust against model errors but reacts after errors occur.

1.2.1.2 Feedforward Control

Applies precomputed torques from the dynamic model before the error appears. Requires an accurate model but gives faster response.

1.2.1.3 Combined (Computed Torque)

Feedforward cancels known dynamics; feedback corrects remaining error. This is the gold standard for manipulator control.

1.2.2 The Euler–Lagrange Equation of Motion

For an n-DOF rigid-body manipulator, the Lagrangian $\mathcal{L} = T - V$ (kinetic minus potential energy) yields the standard form:

$$ M(q)\,\ddot{q} \;+\; C(q,\dot{q})\,\dot{q} \;+\; G(q) \;=\; \tau $$

Symbol	Name	Dimension	Physical Meaning
q	Joint positions	$n \times 1$	Generalised coordin
$M(q)$	Mass (inertia) matrix	$n \times n$	Relates joint accele
$C(q,\dot{q})$	Coriolis & centrifugal matrix	$n \times n$	Velocity-dependent
$G(q)$	Gravity vector	$n \times 1$	Torques required to
τ	Joint torques	$n \times 1$	Control inputs (wha

1.2.3 A Concrete Example: 2-DOF Planar Manipulator

Consider a two-link planar arm with link masses m_1, m_2, link lengths l_1, l_2, and distances to centres of mass l_{c1}, l_{c2}. For simplicity we take $l_{ci} = l_i$ (point masses at link tips).

1.2.3.1 Mass Matrix $M(q)$

$$ M(q) = \begin{bmatrix} m_1 l_1^2 + m_2(l_1^2 + l_2^2 + 2l_1 l_2 \cos q_2) & m_2(l_2^2 + l_1 l_2 \cos q_2) \\ m_2(l_2^2 + l_1 l_2 \cos q_2) & m_2 l_2^2 \end{bmatrix} $$

Notice: $M(q)$ depends on q_2 only (the relative angle), and is always symmetric positive-definite so it is invertible.

Coriolis & Centrifugal Matrix $C(q,\dot{q})$ Using Christoffel symbols of the first kind:

$$ C(q,\dot{q}) = \begin{bmatrix} -m_2 l_1 l_2 \sin(q_2)\,\dot{q}_2 & -m_2 l_1 l_2 \sin(q_2)(\dot{q}_1 + \dot{q}_2) \\ m_2 l_1 l_2 \sin(q_2)\,\dot{q}_1 & 0 \end{bmatrix} $$

1.2.3.2 Gravity Vector $G(q)$

$$ G(q) = \begin{bmatrix} (m_1 + m_2)\,g\,l_1 \cos q_1 + m_2\,g\,l_2 \cos(q_1 + q_2) \\ m_2\,g\,l_2 \cos(q_1 + q_2) \end{bmatrix} $$

1.2.4 Key Properties of the Dynamic Model

1.2.4.1 Property 1: Symmetry & Positive-Definiteness

\(M(q) = M(q)^T > 0\) for all \(q\). This guarantees the system always has finite inertia and the dynamics are well-posed.

1.2.4.2 Property 2: Skew-Symmetry

The matrix \(\dot{M}(q) - 2C(q,\dot{q})\) is skew-symmetric, i.e., \(x^T[\dot{M} - 2C]x = 0\) for all \(x\). This property is essential for Lyapunov-based stability proofs.

1.2.4.3 Property 3: Linearity in Parameters

The dynamics can be written as \(M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = Y(q,\dot{q},\ddot{q})\,\theta\) where \(\theta\) contains the physical parameters. This enables adaptive control.

1.2.4.4 Think–Pair–Share (5 min)

Question: If the second link mass \(m_2\) doubles (e.g., the robot picks up a heavy payload), which terms in \(M(q)\), \(C(q,\dot{q})\), and \(G(q)\) are affected? Which are not affected?

Discuss with your neighbour for 2 minutes, then we will share answers as a class.

1.3 Control Design for the Manipulator

1.3.1 Approach 1: Simple PD Control

The simplest approach is a joint-space PD controller:

$$\tau = K_P\,e + K_D\,\dot{e}$$

where \(e = q_d - q\) is the tracking error and \(K_P, K_D \succ 0\) are diagonal gain matrices.

This treats the manipulator as if it were a collection of independent linear joints. It ignores nonlinear coupling, Coriolis forces, and gravity entirely.

Limitation: PD control alone cannot guarantee zero steady-state error in the presence of gravity. The arm will droop under its own weight because no term compensates for \(G(q)\).

1.3.2 Approach 2: PD + Gravity Compensation

$$\tau = K_P\,e + K_D\,\dot{e} + G(q)$$

Adding the gravity term $G(q)$ eliminates the steady-state droop at the desired position. However, during fast motions the Coriolis and inertia coupling terms still cause significant tracking errors.

1.3.3 Approach 3: Computed Torque Control (Inverse Dynamics)

The computed torque controller uses the full dynamic model to cancel all nonlinear dynamics and then imposes a desired linear closed-loop behaviour.

Computed Torque Control Law

$$\tau = M(q)\,\bigl[\ddot{q}_d + K_D\,\dot{e} + K_P\,e\bigr] + C(q,\dot{q})\,\dot{q} + G(q)$$

1.3.3.1 Derivation — Step by Step

Step 1: Start from the robot dynamics:

$$M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} + G(q) = \tau$$

Step 2: Substitute the control law into the dynamics:

$$M(q)\,\ddot{q} + \cancel{C(q,\dot{q})\,\dot{q}} + \cancel{G(q)} = M(q)\bigl[\ddot{q}_d + K_D\dot{e} + K_P\ e\bigr] + \cancel{C(q,\dot{q})\,\dot{q}} + \cancel{G(q)}$$

Step 3: Since $M(q)$ is invertible, left-multiply both sides by $M^{-1}(q)$:

$$\ddot{q} = \ddot{q}_d + K_D\,\dot{e} + K_P\,e$$

Step 4: Since $\ddot{e} = \ddot{q}_d - \ddot{q}$, rearrange to get the closed-loop error dynamics:

$$\ddot{e} + K_D\,\dot{e} + K_P\,e = 0$$

This is a linear, decoupled, second-order ODE for each joint — identical to a mass–spring–damper system. With proper gain selection the error converges to zero exponentially!

1.3.4 Gain Selection: Placing Closed-Loop Poles

For each joint, the characteristic equation is $s^2 + k_d\,s + k_p = 0$. Comparing with the standard second-order form $s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$:

$$k_p = \omega_n^2, \qquad k_d = 2\zeta\omega_n$$

Design Choice	ζ	ω_n	K_P	K_D	Behaviour
Underdamped	0.5	10	100	10	Fast but oscillat

Design Choice	\(\zeta\)	\(\omega_n\)	\(K_P\)	\(K_D\)	Behaviour
Critically damped	1.0	10	100	20	Fastest without overshoot
Overdamped	2.0	10	100	40	Sluggish but smooth

1.3.5 Lyapunov Stability Proof

Even though the computed torque controller linearises the system, we can also prove stability directly using Lyapunov theory — which is more rigorous and generalises to uncertain models.

1.3.5.1 Lyapunov Proof for Computed Torque Control

Step 1 — Choose a Lyapunov candidate:

\[V(e, \dot{e}) = \frac{1}{2}\dot{e}^T M(q)\,\dot{e} + \frac{1}{2}e^T K_P\,e \]

Since \(M(q) \succ 0\) and \(K_P \succ 0\), we have \(V > 0\) for all \((e, \dot{e}) \ne (0,0)\).

Step 2 — Differentiate:

\[\dot{V} = \dot{e}^T M(q)\,\ddot{e} + \frac{1}{2}\dot{e}^T \dot{M}(q)\,\dot{e} + e^T K_P\,\dot{e} \]

Step 3 — Substitute the closed-loop dynamics \(M(q)\ddot{e} = -C(q,\dot{q})\dot{e} - K_D\dot{e} - K_P e\):

\[\dot{V} = \dot{e}^T\bigl[-C\dot{e} - K_D\dot{e} - K_P e\bigr] + \frac{1}{2}\dot{e}^T\dot{M}\,\dot{e} + e^T K_P\dot{e} \]

Step 4 — Apply the skew-symmetry property \(\dot{e}^T[\dot{M} - 2C]\dot{e} = 0\):

\[\dot{V} = -\dot{e}^T K_D\,\dot{e} \;\leq\; 0 \]

Step 5 — Invoke LaSalle's invariance principle: \(\dot{V} = 0 \implies \dot{e} = 0 \implies \ddot{e} = 0 \implies K_P e = 0 \implies e = 0\). Therefore the equilibrium \((e, \dot{e}) = (0, 0)\) is asymptotically stable.

Key Insight: The skew-symmetry of \(\dot{M} - 2C\) is what makes the cross terms cancel in the Lyapunov derivative. This is the most important structural property in robot control!

1.3.5.2 Quick Check (2 min)

1. What happens if \(K_P = 0\) in the Lyapunov function? Can we still guarantee that \(e \to 0\)?
2. In the computed torque law, which term handles gravity? Which handles Coriolis coupling?
3. If \(M(q)\) were identity (unit mass everywhere), how would the computed torque law simplify?

1.4 MATLAB Implementation

We will now implement the computed torque controller for a 2-DOF manipulator step by step. The robot must track a sinusoidal trajectory: \(q_d(t) = \begin{bmatrix} 0.5\sin(t) \\ 0.5\cos(t) \end{bmatrix}\).

1.4.1 System Parameters

% ===== System Parameters ===== clear all; close all; clc; params.m1 = 1; % Mass of link 1 (kg) params.m2 = 1; % Mass of link 2 (kg) params.l1 = 1; % Length of link 1 (m) params.l2 = 1; % Length of link 2 (m) params.g = 9.81; % Gravitational acceleration (m/s^2) % Control gains (critically damped, wn = 7.07) params.Kp = diag([50 50]); % Kp = wn^2 params.Kd = diag([20 20]); % Kd = 2*zeta*wn (zeta 1.4) % Simulation setup t_span = 0:0.01:20; % 20 seconds, 10 ms step q0 = [0; 0]; % Initial joint angles dq0 = [0; 0]; % Initial joint velocities x0 = [q0; dq0]; % State vector [q1; q2; dq1; dq2]

1.4.2 Desired Trajectory Functions

% ===== Desired Trajectory ===== qd = @(t) [0.5*sin(t); 0.5*cos(t)]; dqd = @(t) [0.5*cos(t); -0.5*sin(t)]; ddqd = @(t) [-0.5*sin(t); -0.5*cos(t)];

1.4.3 Dynamics with Computed Torque Controller

1.4.3.1 Pseudocode

PSEUDOCODE — Computed Torque Controller

FUNCTION manipulator_dynamics(t, state, params): EXTRACT q = state[1:2], dq = state[3:4] COMPUTE desired trajectory: qd(t), dqd(t), ddqd(t) COMPUTE tracking error: e = qd - q COMPUTE velocity error: de = dqd - dq BUILD M(q): 2Œ2 mass matrix using m1, m2, l1, l2, cos(q2) BUILD C(q,dq): 2Œ2 Coriolis matrix using m2, l1, l2, sin(q2), dq BUILD G(q): 2Œ1 gravity vector using g, m1, m2, l1, l2 COMPUTE feedforward: a_cmd = ddqd + Kd*de + Kp*e COMPUTE torque: tau = M * a_cmd + C * dq + G COMPUTE acceleration: ddq = M \ (tau - C*dq - G) RETURN [dq; ddq]

1.4.3.2 MATLAB Code

function dx = manipulator_dynamics(t, x, params, qd, dqd, ddqd) q = x(1:2); % Joint angles dq = x(3:4); % Joint velocities % Desired trajectory at current time q_des = qd(t); dq_des = dqd(t); ddq_des = ddqd(t); % Tracking errors e = q_des - q; de = dq_des - dq; % --- Build the Dynamic Model --- m1 = params.m1; m2 = params.m2; l1 = params.l1; l2 = params.l2; g = params.g; % Mass matrix M(q) M11 = m1*l1^2 + m2*(l1^2 + l2^2 + 2*l1*l2*cos(q(2))); M12 = m2*(l2^2 + l1*l2*cos(q(2))); M22 = m2*l2^2; M = [M11, M12; M12, M22]; % Coriolis matrix C(q, dq) h = m2*l1*l2*sin(q(2)); C

= [-h*dq(2), -h*(dq(1)+dq(2)); h*dq(1), 0]; % Gravity vector G(q) G = [(m1+m2)*g*l1*cos(q(1)) + m2*g*l2*cos(q(1)+q(2)); m2*g*l2*cos(q(1)+q(2))
]; % --- Computed Torque Control Law --- a_cmd = ddq_des + params.Kd*de + params.Kp*e; tau = M * a_cmd + C * dq + G; % --- Forward Dynamics --- ddq = M \ (tau - C*dq - G); dx = [dq; ddq]; end

Why **M\b** instead of **inv(M)*b**? The backslash operator is numerically more stable and faster. Never invert the mass matrix explicitly in production code.

1.4.4 Run Simulation & Plot

% ===== Run ODE45 ===== [t, x] = ode45(@(t,x) manipulator_dynamics(t, x, params, qd, dqd, ddqd), ... t_span, x0); % ===== Compute desired trajectory for plotting ===== qd_vals = zeros(length(t), 2); for i = 1:length(t) qd_vals(i,:) = qd(t(i))'; end % ===== Plot: Joint Angles ===== figure('Position', [100 100 1200 500]); subplot(1,2,1); plot(t, x(:,1), 'b', 'LineWidth', 1.5); hold on; plot(t, x(:,2), 'r', 'LineWidth', 1.5); plot(t, qd_vals(:,1), 'b--', 'LineWidth', 1.2); plot(t, qd_vals(:,2), 'r--', 'LineWidth', 1.2); xlabel('Time (s)'); ylabel('Angle (rad)'); title('Joint Angles: Actual vs Desired'); legend('q_1', 'q_2', 'q_{1,d}', 'q_{2,d}'); grid on; % ===== Plot: Tracking Error ===== subplot(1,2,2); e_vals = qd_vals - x(:,1:2); plot(t, e_vals(:,1), 'b', 'LineWidth', 1.5); hold on; plot(t, e_vals(:,2), 'r', 'LineWidth', 1.5); xlabel('Time (s)'); ylabel('Error (rad)'); title('Tracking Error'); legend('e_1', 'e_2'); grid on; sgtitle('Computed Torque Control — 2-DOF Manipulator');

Expected Result: After a brief transient (1–2 s), both joint errors should converge to near zero and the actual trajectory should overlay the desired trajectory almost perfectly.

1.4.4.1 Hands-On Activity 1 (10 min)

Task: Run the code above in MATLAB. Then modify the gains as follows and observe the effect on the tracking error plot:

1. Set \(K_P = \text{diag}([10, 10])\) and \(K_D = \text{diag}([5, 5])\) — what happens?
2. Set \(K_P = \text{diag}([200, 200])\) and \(K_D = \text{diag}([5, 5])\) — is there overshoot?
3. Set \(K_P = \text{diag}([50, 50])\) and \(K_D = \text{diag}([50, 50])\) — is the response sluggish?

Record the peak error for each case. We will compare results as a class.

1.5 Comparing Control Strategies

To appreciate why computed torque is superior, let us compare three controllers on the same trajectory.

1.5.1 Controller A: PD Only

% Inside dynamics function, replace control law with: tau = params.Kp * e + params.Kd * de; % No model terms — ignores M, C, G completely

1.5.2 Controller B: PD + Gravity Compensation

% Inside dynamics function, replace control law with: tau = params.Kp * e + params.Kd * de + G; % Compensates gravity but ignores Coriolis and inertia coupling

1.5.3 Controller C: Computed Torque (Full)

% Full computed torque (already implemented above): a_cmd = ddq_-des + params.Kd * de + params.Kp * e; tau = M * a_cmd + C * dq + G;

Controller	Model Knowledge	Steady-State Error	Transient Error	C
PD Only	None	Non-zero (gravity droop)	Large	P
PD + Gravity	\(G(q)\)	Zero at setpoints	Moderate	M
Computed Torque	\(M, C, G\)	Zero	Small (tunable)	E

1.5.3.1 Hands-On Activity 2 (15 min)

Task: Implement all three controllers. Plot their tracking errors on the same figure using `subplot` or overlaid with different colours. Answer:

1. Which controller has the largest peak error during the first 2 seconds?
2. Which controller has a non-zero steady-state error? Why?
3. At what joint velocity does the PD controller start to diverge noticeably from computed torque?

1.6 Robustness Analysis

Computed torque control delivers perfect tracking when the model is exact. In practice, we never know the model perfectly. Robustness analysis studies what happens when the model used in the controller differs from the true plant.

1.6.1 Sources of Model Uncertainty

1.6.1.1 Payload Variation

The mass \(m_2\) changes when the robot picks up or drops objects. The controller may still use the nominal value.

1.6.1.2 Friction

Real joints exhibit Coulomb and viscous friction not captured in the rigid-body model.

1.6.1.3 Parameter Errors

Link lengths and centres of mass are measured imprecisely. Actuator dynamics are neglected.

1.6.1.4 External Disturbances

Contact forces, wind loads, or interaction forces when collaborating with humans.

1.6.2 Mathematical Framework

Let the true dynamics be governed by \((M, C, G)\) and the estimated (controller) model be \((\hat{M}, \hat{C}, \hat{G})\). The control law uses the estimated model:

\[\tau = \hat{M}(q)\bigl[\ddot{q}_d + K_D\dot{e} + K_P e\bigr] + \hat{C}(q,\dot{q})\dot{q} + \hat{G}(q) \]

Substituting into the true dynamics:

\[M\,\ddot{q} + C\,\dot{q} + G = \hat{M}\bigl[\ddot{q}_d + K_D\dot{e} + K_P e\bigr] + \hat{C}\,\dot{q} + \hat{G} \]

Defining model errors \(\Delta M = M - \hat{M}\), \(\Delta C = C - \hat{C}\), \(\Delta G = G - \hat{G}\), the closed-loop error dynamics become:

\[M\,\ddot{e} + (C + K_D^*)\,\dot{e} + K_P^*\,e = \underbrace{\Delta M\,\ddot{q}_d + \Delta C\,\dot{q} + \Delta G}_{\text{perturbation } d(t)} \]

The error no longer converges to zero — it is driven by the perturbation \(d(t)\).

Key Result: With model mismatch, computed torque control guarantees bounded errors (the errors stay small if the gains are high and the model errors are small), but not asymptotic tracking to zero.

1.6.3 Experiment: Payload Mismatch

We simulate the scenario where the true plant has \(m_2 = 2\,\text{kg}\) but the controller still assumes \(m_2 = 1\,\text{kg}\).

% ===== Robustness Test: Payload Mismatch ===== % True plant parameters (what reality is) params_true = params; params_true.m2 = 2; % True mass of link 2 is DOUBLE % Controller still uses params (m2 = 1) for its model % We need a modified dynamics function: function dx = manipulator_robust_test(t, x, params_ctrl, params_true, qd, dqd, ddqd) q = x(1:2); dq = x(3:4); q_des = qd(t);

dq_des = dqd(t); ddq_des = ddqd(t); e = q_des - q; de = dq_des - dq; % --- Controller uses ESTIMATED model (params_ctrl) --- [M_hat, C_hat, G_hat] = build_model(q, dq, params_ctrl); a_cmd = ddq_des + params_ctrl.Kd*de + params_ctrl.Kp*e; tau = M_hat * a_cmd + C_hat * dq + G_hat; % --- Plant uses TRUE model (params_true) --- [M_true, C_true, G_true] = build_model(q, dq, params_true); ddq = M_true \ (tau - C_true*dq - G_true); dx = [dq; ddq]; end function [M, C, G] = build_model(q, dq, p) M11 = p.m1*p.l1^2 + p.m2*(p.l1^2 + p.l2^2 + 2*p.l1*p.l2*cos(q(2))); M12 = p.m2*(p.l2^2 + p.l1*p.l2*cos(q(2))); M22 = p.m2*p.l2^2; M = [M11, M12; M12, M22]; h = p.m2*p.l1*p.l2*sin(q(2)); C = [-h*dq(2), -h*(dq(1)+dq(2)); h*dq(1), 0]; G = [(p.m1+p.m2)*p.g*p.l1*cos(q(1)) + p.m2*p.g*p.l2*cos(q(1)+q(2)); p.m2*p.g*p.l2*cos(q(1)+q(2))]; end

1.6.4 Experiment: Adding Joint Friction

% ===== Add Coulomb + Viscous Friction ===== % After computing ddq from the model, add friction terms: b_viscous = 0.5; % Viscous friction coefficient f_coulomb = 0.3; % Coulomb friction magnitude % Friction torque (not modelled by controller) tau_friction = b_viscous * dq + f_coulomb * sign(dq); % Modified acceleration (true plant sees friction) ddq = M_true \ (tau - C_true*dq - G_true - tau_friction);

1.6.5 Experiment: External Disturbance Pulse

% ===== External Disturbance ===== % Simulate a sudden impact at t = 10 s lasting 0.5 s if t >= 10 && t <= 10.5 tau_dist = [5; 3]; % Disturbance torque (Nům) else tau_dist = [0; 0]; end ddq = M_true \ (tau - C_true*dq - G_true + tau_dist);

1.6.6 Summary of Robustness Results

Scenario	Peak Error (rad)	Steady-State Error	Stability
Perfect model	0.02 (transient only)	0	Asymptotically st
Payload Œ2	0.08 – 0.15	Small oscillatory	Bounded (stable)
Joint friction	0.03 – 0.06	Small bias	Bounded (stable)
Disturbance pulse	0.10 (at impact)	Returns to 0	ISS (input-to-stat
Payload Œ5 + friction	0.3 – 0.5	Significant	May become unst

Rule of Thumb: Higher gains (\(K_P, K_D\)) improve robustness by rejecting perturbations faster, but they also amplify sensor noise and demand higher torques. Controller design is always a trade-off between performance, robustness, and actuator limits.

1.7 Summary and Next Steps

1.7.1 Hands-On Activities

1.7.2 Activity 3: Robustness Investigation (20 min)

1. Start with the nominal computed torque controller (\(m_2 = 1\,\text{kg}\) in both plant and controller).
2. Change the true \(m_2\) to 1.5, 2.0, 3.0, and 5.0 kg while keeping the controller model at \(m_2 = 1\).
3. For each case, record the maximum tracking error \(\|e\|_\infty\) from the error plot.
4. Plot \(\|e\|_\infty\) vs payload ratio \(m_{2,\text{true}} / m_{2,\text{ctrl}}\).
5. Bonus: At what payload ratio does the system become unstable? Increase \(K_P\) and \(K_D\) — does this extend the stability boundary?

Expected Observation: Tracking error grows approximately linearly with payload mismatch for small mismatches, but growth accelerates for large mismatches.

1.7.3 Activity 4: Controller Comparison Dashboard (25 min)

Create a single MATLAB figure with a 3Œ2 subplot layout:

	Column 1: Joint Angles	Column 2: Tracking Error
Row 1	PD Only — actual vs desired	PD Only — error
Row 2	PD + Gravity — actual vs desired	PD + Gravity — error
Row 3	Computed Torque — actual vs desired	Computed Torque — error

Use the same gains and trajectory for all three. Include a title, legends, and axis labels on every subplot.

1.7.4 Activity 5: Design Challenge (15 min)

1.7.4.1 Challenge: Robust Trajectory Tracking

Design a controller that can track the reference trajectory with \(\|e\|_\infty < 0.05\,\text{rad}\) under the following simultaneous conditions:

- True \(m_2 = 2\,\text{kg}\), controller assumes \(m_2 = 1\,\text{kg}\)
- Viscous friction \(b = 0.5\,\text{Nůmůs/rad}\) (not modelled)
- An external disturbance of \(\tau_d = [3; 2]\,\text{Nům}\) applied from \(t = 8\) to \(t = 8.5\,\text{s}\)

Hint: You may increase gains, add integral action, or add a robust term \(\tau_{\text{robust}} = K_r\,\text{sign}(\dot{e})\).

1.7.5 End-of-Lecture Quiz

1.7.5.1 Test Your Understanding

1. True or False: The mass matrix $M(q)$ is always diagonal for a 2-DOF manipulator.
   Think about what the off-diagonal terms represent physically.
2. Fill in the blank: The property that $\dot{M} - 2C$ is __-_____ is essential for the Lyapunov stability proof of computed torque control.
3. Short answer: Name two practical reasons why computed torque control may not achieve perfect tracking on a real robot.
4. Calculation: For a desired natural frequency $\omega_n = 15\,\text{rad/s}$ and critical damping ($\zeta = 1$), compute K_P and K_D per joint.
5. Design question: If you double all link masses simultaneously, but the controller model remains unchanged, how does the perturbation term $d(t)$ scale? Would increasing K_P alone fix the problem? Why or why not?

Click to reveal answers

1. False. The off-diagonal terms $M_{12} = M_{21}$ capture inertia coupling between joints. They are zero only in special configurations.
2. Skew-symmetric.
3. Any two of: model parameter uncertainty, unmodelled friction, actuator saturation/dynamics, sensor noise, discretisation effects, joint flexibility.
4. $K_P = \omega_n^2 = 225$, $K_D = 2 \times 1 \times 15 = 30$.
5. The perturbation scales proportionally with the mass error ΔM, which doubles. Increasing K_P alone gives faster error correction but does not cancel the feedforward mismatch — you also need higher K_D to damp the resulting oscillations, or better yet, update the model.

1.7.6 Week 1 Summary

1.7.6.1 Manipulator Dynamics

- Euler–Lagrange formulation
- Mass matrix $M(q)$ — SPD
- Coriolis matrix $C(q,\dot{q})$
- Gravity vector $G(q)$
- Skew-symmetry property

1.7.6.2 Control Design

- PD control & limitations
- PD + gravity compensation
- Computed torque derivation
- Gain selection via pole placement
- Lyapunov stability proof

1.7.6.3 Implementation

- MATLAB ODE45 simulation
- Building M, C, G in code
- Tracking & error plots
- Controller comparison
- Gain tuning experiments

1.7.6.4 Robustness

- Payload mismatch analysis
- Unmodelled friction effects
- Disturbance rejection
- Bounded error guarantees
- Gain–robustness trade-off

1.7.7 Key Equations to Remember

Equation	Description
$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau$	Robot dynamics (Euler–Lagrange)
$\tau = M(q)[\ddot{q}_d + K_D\dot{e} + K_P e] + C\dot{q} + G$	Computed torque control law
$\ddot{e} + K_D\dot{e} + K_P e = 0$	Closed-loop error dynamics (perfect model)
$V = \tfrac{1}{2}\dot{e}^T M \dot{e} + \tfrac{1}{2}e^T K_P e$	Lyapunov function
$\dot{V} = -\dot{e}^T K_D \dot{e} \leq 0$	Lyapunov derivative (guarantees stability)

1.7.8 Next Week: Advanced Control Techniques

Building on this week's foundation, we will cover:

- Adaptive control — automatically estimating M, C, G online
- Sliding mode control — guaranteed robustness despite bounded uncertainties
- Task-space (operational space) control
- Impedance and force control for contact tasks

1.7.9 Preparation for Next Lecture

- Complete all MATLAB activities from today and save your scripts
- Review Lyapunov stability theory (especially LaSalle's invariance principle)
- Read about sliding mode control fundamentals
- Think about: "If I don't know m_2, how could the controller learn it during operation?"

Skills Achieved Today: You can now derive manipulator dynamics, design and implement a computed torque controller, prove its stability using Lyapunov theory, and analyse its robustness to model uncertainties — the foundational skill set for all advanced robot control!

Control for Robots

University of West London | Academic Year 2025–26

Week 1: Manipulator Dynamics & Control Design

This lecture material is for educational use only.

2 Model Predictive Control — Introduction (Week 2)

From Receding-Horizon Optimisation to Real-Time Robotic Control

Control for Robots | University of West London
AY 2025–26 | 3-Hour Lecture + Activity Session

2.1 Overview 17
2.1.1 Week 1 Recall: Computed Torque Control — Exam Practice 17
2.1.2 Learning Outcomes . . . 18
2.1.3 Session Roadmap (3 Hours) 18
2.2 MPC Theory & Derivation 20
2.2.1 What Is Model Predictive Control? 20
2.2.2 The Optimisation Problem 20
2.2.3 State Prediction over the Horizon 21
2.2.4 Quadratic Cost in Matrix Form 22
2.2.5 The Optimal Control Law 22
2.2.6 Receding Horizon: Apply Only the First Input . . 23
2.2.7 Effect of Tuning Parameters 23
2.3 MPC for 2-DOF Manipulator 24
2.3.1 The Nonlinear Dynamics 24
2.3.2 Linearisation around an Operating Point 24
2.3.3 Discretisation 24
2.3.4 MPC Design for the Manipulator 25
2.3.5 MATLAB Implementation — Manipulator MPC 25
2.4 MPC for Differential-Drive Mobile Robot . . . 25
2.4.1 Kinematic Model 25
2.4.2 Linearisation around a Reference Trajectory . . 26
2.4.3 Constraints 26
2.4.4 MPC Design 26
2.4.5 MATLAB Implementation — Mobile Robot MPC 26
2.5 MPC for Quadrotor Drone 27

2.1 Overview

2.1.1 Week 1 Recall: Computed Torque Control — Exam Practice

Before we begin new material, you must demonstrate command of the Week 1 content. Take out pen and paper. You have 20 minutes to answer both questions below by hand, showing every step. These are representative of the questions you will face in the final exam.

2.1.1.1 Question 1 — Computed Torque Derivation (10 marks)

Given a 2-DOF planar manipulator with Euler–Lagrange dynamics

\[M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} + G(q) = \tau \]

derive the computed torque control law

\[\tau = M(q)\bigl[\ddot{q}_d + K_D\,\dot{e} + K_P\,e\bigr] + C(q,\dot{q})\,\dot{q} + G(q) \]

where \(e = q_d - q\) is the tracking error.

Step-by-step requirements:

1. Substitute the control law \(\tau\) into the equation of motion.
2. Cancel the Coriolis and gravity terms on both sides.
3. Left-multiply both sides by \(M^{-1}(q)\) and justify why \(M(q)\) must be invertible (state the positive-definiteness property).
4. Rearrange using \(\ddot{e} = \ddot{q}_d - \ddot{q}\) to obtain the closed-loop error dynamics:

\[\ddot{e} + K_D\,\dot{e} + K_P\,e = 0 \]

Explain the physical significance of this result: what kind of system does it represent, and what does it guarantee about the tracking error?

2.1.1.2 Question 2 — Lyapunov Stability Proof (10 marks)

For the computed torque controller derived in Question 1, construct the Lyapunov candidate function:

\[V(e, \dot{e}) = \frac{1}{2}\,\dot{e}^T\,M(q)\,\dot{e} \;+\; \frac{1}{2}\,e^T\,K_P\,e \]

1. State why \(V > 0\) for all \((e, \dot{e}) \neq (0, 0)\).
2. Differentiate \(V\) with respect to time. Show every term in \(\dot{V}\).
3. Substitute the closed-loop dynamics \(M(q)\,\ddot{e} = -C(q,\dot{q})\,\dot{e} - K_D\,\dot{e} - K_P\,e\).
4. Apply the skew-symmetry property: \(\dot{e}^T\bigl[\dot{M}(q) - 2C(q,\dot{q})\bigr]\dot{e} = 0\).
5. Show that \(\dot{V} = -\dot{e}^T\,K_D\,\dot{e} \leq 0\).
6. Invoke LaSalle's invariance principle to conclude asymptotic stability of the equilibrium \((e, \dot{e}) = (0, 0)\). State the invariance argument explicitly.

You have 20 minutes. If you can reproduce both derivations cleanly, you will be able to do it in the exam. If you struggle, revise your Week 1 notes tonight. These two derivations form the foundation for everything that follows in this module.

2.1.2 Learning Outcomes

By the end of this session, you will be able to:

- Formulate MPC optimisation problems for robotic systems with state and control constraints
- Derive the MPC control law from quadratic cost minimisation over a finite prediction horizon
- Implement MPC for a 2-DOF manipulator, a differential-drive mobile robot, and a quadrotor drone
- Tune the prediction horizon \(N\), state weight \(Q\), and control weight \(R\), and understand their effects on tracking performance, control effort, and computational cost
- Simulate MPC controllers in MATLAB and interpret the resulting trajectory and error plots
- Compare MPC with computed torque control in terms of model requirements, constraint handling, and computational demand

2.1.3 Session Roadmap (3 Hours)

0:00 – 0:20

2.1.3.1 Previous Week Exam Practice

- Computed torque derivation — pen and paper
- Lyapunov stability proof — pen and paper

0:20 – 0:50

2.1.3.2 MPC Theory & Derivation

- What is MPC? The predict–optimise–apply paradigm
- Optimisation problem formulation
- State prediction over the horizon
- Quadratic cost and the optimal control law
- Receding horizon strategy

0:50 – 1:20

2.1.3.3 MPC for 2-DOF Manipulator

- Linearisation and discretisation of manipulator dynamics
- MPC formulation with 4 states and 2 controls
- Full derivation and MATLAB code

1:20 – 1:30

2.1.3.4 Break

- 10-minute break

1:30 – 2:00

2.1.3.5 MPC for Mobile Robot

- Differential-drive kinematic model
- Linearisation around reference trajectory
- MPC with velocity and turning-rate constraints
- Full derivation and MATLAB code

2:00 – 2:30

2.1.3.6 MPC for Quadrotor Drone

- 12-state quadrotor model linearised around hover
- MPC with thrust and torque constraints
- Full derivation and MATLAB code

2:30 – 3:00

2.1.3.7 MATLAB Implementation & Activities

- Activity 1: Horizon length tuning experiment
- Activity 2: Side-by-side comparison of all three systems
- Think–Pair–Share and summary

2.2 MPC Theory & Derivation

2.2.1 What Is Model Predictive Control?

Model Predictive Control (MPC) is an optimisation-based control strategy that operates on a simple but powerful principle: at every time step, predict the future behaviour of the system over a finite horizon, optimise a control sequence to minimise a cost function, and apply only the first control action before repeating the process at the next time step.

2.2.1.1 1. Predict

Use the system model $x_{k+1} = A\,x_k + B\,u_k$ to propagate the state forward N steps into the future, generating a predicted state trajectory $\{x_{k+1}, x_{k+2}, \ldots, x_{k+N}\}$.

2.2.1.2 2. Optimise

Find the control sequence $U = [u_k, u_{k+1}, \ldots, u_{k+N-1}]^T$ that minimises a cost function penalising state deviations from the reference and control effort, subject to constraints.

2.2.1.3 3. Apply

Apply only the first element u_k^* of the optimal sequence to the plant. At the next time step $k+1$, measure the new state and repeat the entire process. This is the receding horizon strategy.

Figure 1: The MPC receding horizon concept. At each time step, the controller predicts future states over a finite horizon, optimises a control sequence, applies only the first input, then shifts the window forward.

2.2.2 The Optimisation Problem

Consider a discrete-time linear time-invariant system:

$$x_{k+1} = A\,x_k + B\,u_k$$

where $x_k \in \mathbb{R}^n$ is the state, $u_k \in \mathbb{R}^m$ is the control input, $A \in \mathbb{R}^{n \times n}$ and $B \in \mathbb{R}^{n \times m}$.

The MPC optimisation problem at time step k is:

MPC Optimisation Problem

$$\min_{U} \; J = \sum_{i=1}^{N} \bigl(x_{k+i} - x_{\text{ref}}\bigr)^T Q \bigl(x_{k+i} - x_{\text{ref}}\bigr) \;+\; \sum_{i=0}^{N-1} u_{k+i}^T R\,u_{k+i}$$

subject to: $x_{k+i+1} = A\,x_{k+i} + B\,u_{k+i}$, $u_{\min} \leq u_{k+i} \leq u_{\max}$, $x_{\min} \leq x_{k+i} \leq x_{\max}$

Here \(Q \succeq 0\) is the state cost matrix (penalises deviation from the reference), \(R \succ 0\) is the control cost matrix (penalises large control inputs), and \(N\) is the prediction horizon length.

Figure 2: MPC closed-loop control architecture. The optimizer uses a system model and constraints to compute the optimal control sequence; only the first input is applied to the plant. State feedback closes the loop.

2.2.3 State Prediction over the Horizon

By recursively applying the dynamics, we can express all future states in terms of the current state \(x_k\) and the control sequence \(U\):

2.2.3.1 Recursive State Prediction

Step 1: One step ahead:

\[x_{k+1} = A\,x_k + B\,u_k \]

Step 2: Two steps ahead:

\[x_{k+2} = A\,x_{k+1} + B\,u_{k+1} = A^2\,x_k + A\,B\,u_k + B\,u_{k+1} \]

Step 3: General \(i\) steps ahead:

\[x_{k+i} = A^i\,x_k + \sum_{j=0}^{i-1} A^{i-1-j}\,B\,u_{k+j} \]

Step 4: Stack all predictions into a single matrix equation. Define:

\[\mathbf{X} = \begin{bmatrix} x_{k+1} \\ x_{k+2} \\ \vdots \\ x_{k+N} \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} u_k \\ u_{k+1} \\ \vdots \\ u_{k+N-1} \end{bmatrix} \]

Then:

\[\mathbf{X} = \mathcal{A}\,x_k + \mathcal{B}\,\mathbf{U} \]

where:

\[\mathcal{A} = \begin{bmatrix} A \\ A^2 \\ A^3 \\ \vdots \\ A^N \end{bmatrix}, \quad \mathcal{B} = \begin{bmatrix} B & 0 & 0 & \cdots & 0 \\ AB & B & 0 & \cdots & 0 \\ A^2B & AB & B & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & A^{N-3}B & \cdots & B \end{bmatrix} \]

Figure 3: State prediction visualization. The optimizer evaluates multiple control sequences (grey dashed) and selects the one minimizing the cost (green). Only the first control action u_k* is applied; the rest are discarded and re-computed at the next step.

2.2.4 Quadratic Cost in Matrix Form

Define the stacked reference \(\mathbf{X}_{\text{ref}} = [x_{\text{ref}}^T, x_{\text{ref}}^T, \ldots, x_{\text{ref}}^T]^T \in \mathbb{R}^{Nn}\) and the block-diagonal weight matrices:

\[\mathbf{Q}_N = \text{blkdiag}(Q, Q, \ldots, Q) \in \mathbb{R}^{Nn \times Nn}, \quad \mathbf{R}_N = \text{blkdiag}(R, R, \ldots, R) \in \mathbb{R}^{Nm \times Nm} \]

Substituting \(\mathbf{X} = \mathcal{A}\,x_k + \mathcal{B}\,\mathbf{U}\) into the cost function:

2.2.4.1 Cost Function Expansion

\[J = (\mathbf{X} - \mathbf{X}_{\text{ref}})^T \mathbf{Q}_N (\mathbf{X} - \mathbf{X}_{\text{ref}}) + \mathbf{U}^T \mathbf{R}_N \mathbf{U} \]

Let \(\mathbf{E} = \mathbf{X}_{\text{ref}} - \mathcal{A}\,x_k\) (the "free-response error"). Then \(\mathbf{X} - \mathbf{X}_{\text{ref}} = \mathcal{B}\,\mathbf{U} - \mathbf{E}\), so:

\[J = (\mathcal{B}\,\mathbf{U} - \mathbf{E})^T \mathbf{Q}_N (\mathcal{B}\,\mathbf{U} - \mathbf{E}) + \mathbf{U}^T \mathbf{R}_N \mathbf{U} \]

Expanding:

\[J = \mathbf{U}^T \underbrace{(\mathcal{B}^T \mathbf{Q}_N \mathcal{B} + \mathbf{R}_N)}_{H} \mathbf{U} - 2\,\mathbf{E}^T \mathbf{Q}_N \mathcal{B}\,\mathbf{U} + \mathbf{E}^T \mathbf{Q}_N \mathbf{E} \]

This is a strictly convex quadratic in \(\mathbf{U}\) (since \(H \succ 0\) when \(R \succ 0\)), so the unique minimum is found by setting \(\frac{\partial J}{\partial \mathbf{U}} = 0\):

\[2\,H\,\mathbf{U}^* - 2\,\mathcal{B}^T \mathbf{Q}_N \mathbf{E} = 0 \]

2.2.5 The Optimal Control Law

MPC Optimal Control Sequence

\[\mathbf{U}^* = \bigl(\mathcal{B}^T \mathbf{Q}_N \mathcal{B} + \mathbf{R}_N\bigr)^{-1} \mathcal{B}^T \mathbf{Q}_N \bigl(\mathbf{X}_{\text{ref}} - \mathcal{A}\,x_k\bigr) \]

The matrix \(H = \mathcal{B}^T \mathbf{Q}_N \mathcal{B} + \mathbf{R}_N\) is of dimension \(Nm \times Nm\). It must be inverted (or factorised) at every time step. This is the primary computational cost of MPC.

2.2.6 Receding Horizon: Apply Only the First Input

The full optimal sequence is $\mathbf{U}^* = [u_k^{*T}, u_{k+1}^{*T}, \ldots, u_{k+N-1}^{*T}]^T$. In the receding-horizon strategy, we apply only the first element:

$$u_k = u_k^* = \text{first } m \text{ entries of } \mathbf{U}^*$$

At the next time step, the state x_{k+1} is measured (or estimated), and the entire optimisation is re-solved. This feedback mechanism gives MPC its robustness to disturbances and model errors.

Figure 4: MPC flowchart showing the predict-optimise-apply loop. The key insight is that only the first control input is applied (highlighted in green), and the entire optimisation is re-solved at every time step.

2.2.6.1 Think–Pair–Share (3 min)

Question: Why does MPC re-solve the optimisation at every time step instead of applying the entire precomputed sequence $\mathbf{U}^*$?

Hint: Think about what happens if the model is imperfect, or if a disturbance pushes the state away from the predicted trajectory. Compare with open-loop vs closed-loop control.

PSEUDOCODE — MPC Receding Horizon Algorithm

INITIALISE: state x_0, horizon N, weights Q and R BUILD: prediction matrices A_cal and B_cal from (A, B, N) BUILD: block-diagonal weights Q_N = blkdiag(Q,...,Q), R_N = blkdiag(R,...,R) COMPUTE: H = B_cal' * Q_N * B_cal + R_N COMPUTE: H_inv = inv(H) (do this ONCE if A, B are constant) FOR k = 0, 1, 2, ... DO MEASURE (or estimate) current state x_k BUILD reference vector: X_ref = [x_ref; x_ref; ...; x_ref] (N copies) COMPUTE free-response error: E = X_ref - A_cal * x_k COMPUTE optimal sequence: U_star = H_inv * B_cal' * Q_N * E EXTRACT first control: u_k = U_star(1:m) APPLY u_k to the plant ADVANCE to next time step END FOR

2.2.7 Effect of Tuning Parameters

Parameter	Increase Effect
Prediction horizon N	Better anticipation of future references; more stable; higher computational cost ($O(N^3 m^3$
State weight Q	Tighter tracking; more aggressive control; risk of actuator saturation
Control weight R	Penalises large inputs; smoother control; slower response; energy-efficient

Practical Guideline: Start with $Q = I$, $R = 0.1\, I$, and $N = 10$. Then adjust: increase Q if tracking is too loose, increase R if control inputs saturate, and increase N until performance no longer improves significantly.

2.3 MPC for 2-DOF Manipulator

2.3.1 The Nonlinear Dynamics

Recall from Week 1 the 2-DOF planar manipulator dynamics:

$$ M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} + G(q) = \tau $$

with state vector $x = [q_1, q_2, \dot{q}_1, \dot{q}_2]^T \in \mathbb{R}^4$ and control input $u = [\tau_1, \tau_2]^T \in \mathbb{R}^2$.

Figure 5: 2-DOF planar manipulator with joint angles 1, 2, link lengths l1, l2, masses m1, m2, and torque inputs 1, 2. The coordinate frame is shown at the base with gravity acting downward.

2.3.2 Linearisation around an Operating Point

MPC (in its basic linear form) requires a linear discrete-time model. We linearise the manipulator dynamics around a nominal operating point $((q_0, \dot{q}_0) = (0, 0)$ (the hanging-down equilibrium).

2.3.2.1 Linearisation Procedure

The continuous-time state-space form is:

$$ \dot{x} = f(x, u) = \begin{bmatrix} \dot{q} \\ M^{-1}(q)\bigl[\tau - C(q,\dot{q})\dot{q} - G(q)\bigr] \end{bmatrix} $$

At the operating point $x_0 = [0, 0, 0, 0]^T$, $u_0 = G(q_0)$:

$$ A_c = \left.\frac{\partial f}{\partial x}\right|_{x_0, u_0}, \quad B_c = \left.\frac{\partial f}{\partial u}\right|_{x_0, u_0} $$

For the 2-DOF arm with $m_1 = m_2 = 1\,\text{kg}$, $l_1 = l_2 = 1\,\text{m}$, evaluated at $q = 0$:

$$ M(0) = \begin{bmatrix} m_1 l_1^2 + m_2(l_1^2 + l_2^2 + 2l_1 l_2) & m_2(l_2^2 + l_1 l_2) \\ m_2(l_2^2 + l_1 l_2) & m_2 l_2^2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 1 \end{bmatrix} $$

We evaluate at a slightly offset configuration $q_0 = [0.1, 0.1]^T$ to ensure non-singularity:

$$ M(q_0) \approx \begin{bmatrix} 3.96 & 1.99 \\ 1.99 & 1.0 \end{bmatrix}, \quad M^{-1}(q_0) \approx \begin{bmatrix} 50.25 & -100.0 \\ -100.0 & 199.0 \end{bmatrix} $$

2.3.3 Discretisation

$$ A_d = e^{A_c \Delta t} \approx I + A_c \Delta t, \quad B_d \approx B_c \Delta t $$

2.3.4 MPC Design for the Manipulator

2.3.4.1 Weight Matrix Selection

\[Q = \text{diag}(100, 100, 1, 1), \quad R = \text{diag}(0.01, 0.01) \]

2.3.5 MATLAB Implementation — Manipulator MPC

%% === MPC for 2-DOF Manipulator === % System parameters params.m1 = 1; params.m2 = 1; params.l1 = 1; params.l2 = 1; params.g = 9.81; % Linearisation point q0 = [0.1; 0.1]; dq0 = [0; 0]; % Build M(q0), compute M_inv c2 = cos(q0(2)); M11 = params.m1*params.l1^2 + params.m2*(params.l1^2 + params.l2^2 + 2*params.l1*params.l2*c2); M12 = params.m2*(params.l2^2 + params.l1*params.l2*c2); M22 = params.m2*params.l2^2; M0 = [M11, M12; M12, M22]; M0_inv = inv(M0); % Continuous-time A, B (linearised around q0, dq0=0) Ac = [zeros(2), eye(2); zeros(2), zeros(2)]; Bc = [zeros(2); M0_inv]; % Discretise (Euler, dt = 0.01 s) dt = 0.01; Ad = eye(4) + Ac*dt; Bd = Bc*dt; % MPC parameters N = 20; % Prediction horizon Q = diag([100, 100, 1, 1]); % State weight R = diag([0.01, 0.01]); % Control weight % Build prediction matrices n = 4; m = 2; A_cal = zeros(n*N, n); B_cal = zeros(n*N, m*N); for i = 1:N A_cal((i-1)*n+1:i*n, :) = Ad^i; for j = 1:i B_cal((i-1)*n+1:i*n, (j-1)*m+1:j*m) = Ad^(i-j) * Bd; end end % Block-diagonal weights Q_N = kron(eye(N), Q); R_N = kron(eye(N), R); % Hessian and its inverse (constant for LTI system) H = B_cal' * Q_N * B_cal + R_N; H_inv = inv(H); F = B_cal' * Q_N; % Simulation t_span = 0:dt:20; x = [0; 0; 0; 0]; % Initial state x_hist = zeros(4, length(t_span)); u_hist = zeros(2, length(t_span)); x_hist(:,1) = x; for k = 1:length(t_span)-1 t = t_span(k); % Reference trajectory (sinusoidal) x_ref_k = zeros(n*N, 1); for i = 1:N t_future = t + i*dt; x_ref_k((i-1)*n+1:i*n) = [0.5*sin(t_future); 0.5*cos(t_future); ... 0.5*cos(t_future); -0.5*sin(t_future)]; end % MPC optimal control E = x_ref_k - A_cal * x; U_star = H_inv * F * E; u = U_star(1:m); % Apply first control only % Clamp torques (constraint enforcement) u = max(min(u, 50), -50); % Simulate true nonlinear dynamics (one Euler step) q = x(1:2); dq = x(3:4); [M_true, C_true, G_true] = manipulator_model(q, dq, params); ddq = M_true \ (u - C_true*dq - G_true); x = x + [dq; ddq]*dt; x_hist(:,k+1) = x; u_hist(:,k) = u; end

Note: The controller uses a linear model, but the plant simulation uses the full nonlinear dynamics. This mismatch is realistic and tests the robustness of MPC.

2.4 MPC for Differential-Drive Mobile Robot

2.4.1 Kinematic Model

A differential-drive mobile robot moves in the plane with state \(x = [p_x, p_y, \theta]^T\) and control inputs \(u = [v, \omega]^T\):

Nonlinear Kinematic Model

\[\begin{bmatrix} \dot{p}_x \\ \dot{p}_y \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v\,\cos\theta \\ v\,\sin\theta \\ \omega \end{bmatrix} \]

Figure 6: Top-down view of a differential-drive mobile robot. The robot state is (p_x, p_y,), controlled by linear velocity v and angular velocity . The reference path is shown as a dashed curve.

2.4.2 Linearisation around a Reference Trajectory

2.4.2.1 Jacobian Linearisation

\[A_c = \begin{bmatrix} 0 & 0 & -v^r \sin\theta^r \\ 0 & 0 & v^r \cos\theta^r \\ 0 & 0 & 0 \end{bmatrix}, \quad B_c = \begin{bmatrix} \cos\theta^r & 0 \\ \sin\theta^r & 0 \\ 0 & 1 \end{bmatrix} \]

For \(\theta^r = 0\), \(v^r = 2.5\,\text{m/s}\):

\[A_c = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 2.5 \\ 0 & 0 & 0 \end{bmatrix}, \quad B_c = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} \]

2.4.3 Constraints

Constraint	Physical Meaning	Val
\(0 \leq v \leq 5\,\text{m/s}\)	No backward motion; maximum speed	[0, 5
\(-2 \leq \omega \leq 2\,\text{rad/s}\)	Maximum turning rate	[-2,

2.4.4 MPC Design

\[Q = \text{diag}(50, 50, 10), \quad R = \text{diag}(0.1, 0.1) \]

2.4.5 MATLAB Implementation — Mobile Robot MPC

%% === MPC for Differential-Drive Mobile Robot === v_ref = 2.5; dt = 0.05; Ac_mob = [0, 0, 0; 0, 0, v_ref; 0, 0, 0]; Bc_mob = [1, 0; 0, 0; 0, 1]; Ad_mob = eye(3) + Ac_mob*dt; Bd_mob = Bc_mob*dt; N_mob = 15; Q_mob = diag([50, 50, 10]); R_mob = diag([0.1, 0.1]); n_mob = 3; m_mob = 2; % Build prediction matrices (same structure as manipulator) A_cal_mob = zeros(n_mob*N_mob, n_mob); B_cal_mob = zeros(n_mob*N_mob, m_mob*N_mob); for i = 1:N_mob A_cal_mob((i-1)*n_mob+1:i*n_mob, :) = Ad_mob^i; for j = 1:i B_cal_mob((i-1)*n_mob+1:i*n_mob, (j-1)*m_mob+1:j*m_mob) = Ad_mob^(i-j)*Bd_mob; end end Q_N_mob = kron(eye(N_mob), Q_mob); R_N_mob = kron(eye(N_mob), R_mob); H_mob = B_cal_mob' * Q_N_mob * B_cal_mob + R_N_mob; H_inv_mob = inv(H_mob); F_mob = B_cal_mob' * Q_N_mob; % Simulation

(circular reference) t_span_mob = 0:dt:20; x_mob = [0; 0; 0]; R_-circle = 5; omega_circle = 0.5; for k = 1:length(t_span_mob)-1 t = t_span_mob(k); x_ref_mob = zeros(n_mob*N_mob, 1); for i = 1:N_mob t_f = t + i*dt; x_ref_mob((i-1)*n_mob+1:i*n_mob) = [R_circle*cos(omega_circle*t_f); ... R_circle*sin(omega_circle*t_f); ... omega_circle*t_f + pi/2]; end E_mob = x_ref_mob - A_cal_mob * x_mob; U_star_mob = H_inv_mob * F_mob * E_mob; u_mob = U_star_mob(1:m_mob); u_mob(1) = max(min(u_mob(1), 5), 0); u_mob(2) = max(min(u_mob(2), 2), -2); theta = x_mob(3); x_mob = x_mob + [u_mob(1)*cos(theta); u_mob(1)*sin(theta); u_mob(2)]*dt; end

Linearisation Caveat: The linearisation is performed at a single operating point. In practice, one re-linearises at each time step or uses nonlinear MPC.

2.5 MPC for Quadrotor Drone

2.5.1 The 12-State Quadrotor Model

A quadrotor has 6 degrees of freedom (3 translational + 3 rotational), leading to a 12-dimensional state vector:

\[x = \begin{bmatrix} p_x \\ p_y \\ p_z \\ v_x \\ v_y \\ v_z \\ \phi \\ \theta \\ \psi \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \quad \text{(position, velocity, Euler angles, angular rates)} \]

Figure 7: Quadrotor UAV schematic showing four rotors with alternating CW/CCW rotation directions, thrust forces F1-F4, body frame axes (x_B, y_B, z_B), and Euler angles (roll , pitch , yaw). The total thrust is T = F1+F2+F3+F4.

The control inputs are thrust and three body-frame torques:

\[u = \begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} \]

2.5.2 Linearised Hover Dynamics

2.5.2.1 Linearised Hover Dynamics

At hover (\(\phi = \theta = \psi = 0\), \(T_0 = mg\)):

\[\dot{v}_x \approx g\,\theta, \quad \dot{v}_y \approx -g\,\phi, \quad \dot{v}_z \approx \frac{T - mg}{m} \] \[\dot{\omega}_-x \approx \frac{\tau_x}{I_{xx}}, \quad \dot{\omega}_y \approx \frac{\tau_y}{I_{yy}}, \quad \dot{\omega}_z \approx \frac{\tau_-z}{I_{zz}} \]

2.5.3 MPC Design for the Quadrotor

\[Q = \text{diag}(\underbrace{80, 80, 120}_{\text{position}}, \underbrace{1, 1, 1}_{\text{velocity}}, \underbrace{50, 50, 50}_{\text{orientation}}, \underbrace{1, 1, 1}_{\text{angular rate}}) \] \[R = \text{diag}(0.1, 0.1, 0.1, 0.1) \]

2.5.4 Constraints

Constraint	Physical Meaning	Value
\(0 \leq T \leq 2mg\)	Thrust range	[0, 19.62] N
\(-2 \leq \tau_x, \tau_y, \tau_z \leq 2\)	Maximum body torques	[-2, 2] Nům
\(-30^\circ \leq \phi, \theta \leq 30^\circ\)	Tilt angle limits	[-0.524, 0.524]

2.5.5 MATLAB Implementation — Quadrotor MPC

%% === MPC for Quadrotor Drone === quad.m = 1.0; quad.Ixx = 0.1; quad.Iyy = 0.1; quad.Izz = 0.2; quad.g = 9.81; dt_q = 0.02; n_q = 12; m_q = 4; Ac_q = zeros(12,12); Ac_q(1:3, 4:6) = eye(3); Ac_q(4,8) = quad.g; Ac_q(5,7) = -quad.g; Ac_q(7:9, 10:12) = eye(3); Bc_q = zeros(12,4); Bc_q(6,1) = 1/quad.m; Bc_q(10,2) = 1/quad.Ixx; Bc_q(11,3) = 1/quad.Iyy; Bc_q(12,4) = 1/quad.Izz; Ad_q = eye(12) + Ac_q*dt_q; Bd_q = Bc_q*dt_q; N_q = 15; Q_q = diag([80, 80, 120, 1, 1, 1, 50, 50, 50, 1, 1, 1]); R_q = diag([0.1, 0.1, 0.1, 0.1]); % Build prediction matrices and solve (same pattern as above) A_cal_q = zeros(n_q*N_q, n_q); B_cal_q = zeros(n_q*N_q, m_q*N_q); for i = 1:N_q A_cal_q((i-1)*n_q+1:i*n_q, :) = Ad_q^i; for j = 1:i B_cal_q((i-1)*n_q+1:i*n_q, (j-1)*m_q+1:j*m_q) = Ad_q^(i-j)*Bd_q; end end H_q = B_cal_q' * kron(eye(N_q),Q_q) * B_cal_q + kron(eye(N_q),R_q); H_inv_q = inv(H_q); F_q = B_cal_q' * kron(eye(N_q),Q_q); % Simulation loop follows same pattern with helical reference trajectory

Key Observation: The MPC controller for the quadrotor uses a 12-state linear model but drives the full nonlinear plant. The receding-horizon feedback mechanism compensates for linearisation errors at each step.

2.6 MATLAB Implementation & Activities

2.6.1 Activity 1: Prediction Horizon Tuning (10 min)

2.6.1.1 Task: Investigate the Effect of Horizon Length \(N\)

Run the manipulator MPC code with three different prediction horizons.

Horizon \(N\)	Peak Position Error	Steady-State Error	Computation Time
\(N = 5\)	[record]	[record]	[record]

Horizon \(N\)	Peak Position Error	Steady-State Error	Computation Time
\(N = 20\)	[record]	[record]	[record]
\(N = 50\)	[record]	[record]	[record]

2.6.2 Activity 2: Three-System Comparison (15 min)

2.6.2.1 Task: Run MPC for All Three Systems Side by Side

1. Compare the MPC tracking error with the PD tracking error for each system.
2. Examine the control input plots. Does MPC produce smoother control signals?
3. Look at the constraint enforcement: does PD ever violate the limits?

2.6.3 Activity 3: MPC vs Computed Torque Discussion (5 min)

2.6.3.1 Think–Pair–Share

Question 1: When would you prefer MPC over computed torque control?

Question 2: When would computed torque control be the better choice?

2.7 Summary and Next Steps

2.7.1 Summary & Key Takeaways

2.7.2 Core MPC Equations

Equation
\(x_{k+1} = A\,x_k + B\,u_k\)
\(\mathbf{X} = \mathcal{A}\,x_k + \mathcal{B}\,\mathbf{U}\)
\(J = (\mathbf{X} - \mathbf{X}_{\text{ref}})^T \mathbf{Q}_N (\mathbf{X} - \mathbf{X}_{\text{ref}}) + \mat
\(\mathbf{U}^* = (\mathcal{B}^T \mathbf{Q}_N \mathcal{B} + \mathbf{R}_N)^{-1} \mathcal{B}^T \mathbf{Q
\(u_k = \mathbf{U}^*(1{:}m)\)

2.7.3 MPC vs Computed Torque Control

Property	MPC	Computed Torque
Model Knowledge	Requires linear (or nonlinear) model; can use approximate models	Requires full nonlinear dynam
Constraint Handling	Systematically incorporates state and control constraints	No built-in constraint handling
Computation	Solves a QP at every time step	Matrix-vector product; very fa
Best For	Systems with constraints, slow-medium dynamics	Fast manipulator control with

2.7.3.1 Key Takeaway 1

MPC converts the control problem into a repeated optimisation problem.

2.7.3.2 Key Takeaway 2

The receding-horizon strategy provides implicit feedback.

2.7.3.3 Key Takeaway 3

MPC's greatest strength is constraint handling.

2.7.3.4 Key Takeaway 4

Computed torque remains superior at very high rates (> 1 kHz).

2.7.4 Next Week: Adaptive Control

Building on Weeks 1 and 2, we will cover:

- Parameter estimation and adaptive control for manipulators with unknown dynamics
- Model Reference Adaptive Control (MRAC)
- Linearity-in-parameters and the adaptive computed torque controller
- Stability proofs for adaptive systems using Lyapunov-based methods

2.7.5 Preparation

- Complete all MATLAB activities from today and save your scripts
- Review the linearity-in-parameters property from Week 1
- Think about: "If the controller does not know \(M(q)\), \(C(q,\dot{q})\), or \(G(q)\), how could it learn these online while still maintaining stability?"

Control for Robots

University of West London | Academic Year 2025–26

Week 2: Model Predictive Control (MPC)

This lecture material is for educational use only.

3 Model Predictive Control — Design and Simulation (Week 3)

From Receding-Horizon Optimisation to Real-Time Robotic Control

Control for Robots | University of West London
AY 2025–26 | 3-Hour Lecture + Activity Session

3.1 Overview 31
3.1.1 Week 1 Recall: Computed Torque Control — Exam Practice 31
3.1.2 Learning Outcomes . . . 32
3.1.3 Session Roadmap (3 Hours) 32
3.2 MPC Theory & Derivation 34
3.2.1 What Is Model Predictive Control? 34
3.2.2 The Optimisation Problem 34
3.2.3 State Prediction over the Horizon 35
3.2.4 Quadratic Cost in Matrix Form 36
3.2.5 The Optimal Control Law36
3.2.6 Receding Horizon: Apply Only the First Input . . 37
3.2.7 Effect of Tuning Parameters 37
3.3 MPC for 2-DOF Manipulator 38
3.3.1 The Nonlinear Dynamics 38
3.3.2 Linearisation around an Operating Point 38
3.3.3 Discretisation 39
3.3.4 MPC Design for the Manipulator 39
3.3.5 MATLAB Implementation — Manipulator MPC 39
3.4 MPC for Differential-Drive Mobile Robot . . . 40
3.4.1 Kinematic Model 40
3.4.2 Linearisation around a Reference Trajectory . . 40
3.4.3 Constraints 41
3.4.4 MPC Design 41
3.4.5 MATLAB Implementation — Mobile Robot MPC 41
3.5 MPC for Quadrotor Drone 42

3.1 Overview

3.1.1 Week 1 Recall: Computed Torque Control — Exam Practice

Before we begin new material, you must demonstrate command of the Week 1 content. Take out pen and paper. You have 20 minutes to answer both questions below by hand, showing every step. These are representative of the questions you will face in the final exam.

3.1.1.1 Question 1 — Computed Torque Derivation (10 marks)

Given a 2-DOF planar manipulator with Euler–Lagrange dynamics

\[M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} + G(q) = \tau \]

derive the computed torque control law

\[\tau = M(q)\bigl[\ddot{q}_d + K_D\,\dot{e} + K_P\,e\bigr] + C(q,\dot{q})\,\dot{q} + G(q) \]

where \(e = q_d - q\) is the tracking error.

Step-by-step requirements:

1. Substitute the control law \(\tau\) into the equation of motion.
2. Cancel the Coriolis and gravity terms on both sides.
3. Left-multiply both sides by \(M^{-1}(q)\) and justify why \(M(q)\) must be invertible (state the positive-definiteness property).
4. Rearrange using \(\ddot{e} = \ddot{q}_d - \ddot{q}\) to obtain the closed-loop error dynamics:

\[\ddot{e} + K_D\,\dot{e} + K_P\,e = 0 \]

Explain the physical significance of this result: what kind of system does it represent, and what does it guarantee about the tracking error?

3.1.1.2 Question 2 — Lyapunov Stability Proof (10 marks)

For the computed torque controller derived in Question 1, construct the Lyapunov candidate function:

\[V(e, \dot{e}) = \frac{1}{2}\,\dot{e}^T\,M(q)\,\dot{e} \;+\; \frac{1}{2}\,e^T\,K_P\,e \]

1. State why \(V > 0\) for all \((e, \dot{e}) \neq (0, 0)\).
2. Differentiate \(V\) with respect to time. Show every term in \(\dot{V}\).
3. Substitute the closed-loop dynamics \(M(q)\,\ddot{e} = -C(q,\dot{q})\,\dot{e} - K_D\,\dot{e} - K_P\,e\).
4. Apply the skew-symmetry property: \(\dot{e}^T\bigl[\dot{M}(q) - 2C(q,\dot{q})\bigr]\dot{e} = 0\).
5. Show that \(\dot{V} = -\dot{e}^T\,K_D\,\dot{e} \leq 0\).
6. Invoke LaSalle's invariance principle to conclude asymptotic stability of the equilibrium \((e, \dot{e}) = (0, 0)\). State the invariance argument explicitly.

You have 20 minutes. If you can reproduce both derivations cleanly, you will be able to do it in the exam. If you struggle, revise your Week 1 notes tonight. These two derivations form the foundation for everything that follows in this module.

3.1.2 Learning Outcomes

By the end of this session, you will be able to:

- Formulate MPC optimisation problems for robotic systems with state and control constraints
- Derive the MPC control law from quadratic cost minimisation over a finite prediction horizon
- Implement MPC for a 2-DOF manipulator, a differential-drive mobile robot, and a quadrotor drone
- Tune the prediction horizon \(N\), state weight \(Q\), and control weight \(R\), and understand their effects on tracking performance, control effort, and computational cost
- Simulate MPC controllers in MATLAB and interpret the resulting trajectory and error plots
- Compare MPC with computed torque control in terms of model requirements, constraint handling, and computational demand

3.1.3 Session Roadmap (3 Hours)

0:00 – 0:20

3.1.3.1 Previous Week Exam Practice

- Computed torque derivation — pen and paper
- Lyapunov stability proof — pen and paper

0:20 – 0:50

3.1.3.2 MPC Theory & Derivation

- What is MPC? The predict–optimise–apply paradigm
- Optimisation problem formulation
- State prediction over the horizon
- Quadratic cost and the optimal control law
- Receding horizon strategy

0:50 – 1:20

3.1.3.3 MPC for 2-DOF Manipulator

- Linearisation and discretisation of manipulator dynamics
- MPC formulation with 4 states and 2 controls
- Full derivation and MATLAB code

1:20 – 1:30

3.1.3.4 Break

- 10-minute break

1:30 – 2:00

3.1.3.5 MPC for Mobile Robot

- Differential-drive kinematic model
- Linearisation around reference trajectory
- MPC with velocity and turning-rate constraints
- Full derivation and MATLAB code

2:00 – 2:30

3.1.3.6 MPC for Quadrotor Drone

- 12-state quadrotor model linearised around hover
- MPC with thrust and torque constraints
- Full derivation and MATLAB code

2:30 – 3:00

3.1.3.7 MATLAB Implementation & Activities

- Activity 1: Horizon length tuning experiment
- Activity 2: Side-by-side comparison of all three systems
- Think–Pair–Share and summary

3.2 MPC Theory & Derivation

3.2.1 What Is Model Predictive Control?

Model Predictive Control (MPC) is an optimisation-based control strategy that operates on a simple but powerful principle: at every time step, predict the future behaviour of the system over a finite horizon, optimise a control sequence to minimise a cost function, and apply only the first control action before repeating the process at the next time step.

3.2.1.1 1. Predict

Use the system model \(x_{k+1} = A\,x_k + B\,u_k\) to propagate the state forward \(N\) steps into the future, generating a predicted state trajectory \(\{x_{k+1}, x_{k+2}, \ldots, x_{k+N}\}\).

3.2.1.2 2. Optimise

Find the control sequence \(U = [u_k, u_{k+1}, \ldots, u_{k+N-1}]^T\) that minimises a cost function penalising state deviations from the reference and control effort, subject to constraints.

3.2.1.3 3. Apply

Apply only the first element \(u_k^*\) of the optimal sequence to the plant. At the next time step \(k+1\), measure the new state and repeat the entire process. This is the receding horizon strategy.

Figure 1: The MPC receding horizon concept. At each time step, the controller predicts future states over a finite horizon, optimises a control sequence, applies only the first input, then shifts the window forward.

3.2.2 The Optimisation Problem

Consider a discrete-time linear time-invariant system:

\[x_{k+1} = A\,x_k + B\,u_k \]

where \(x_k \in \mathbb{R}^n\) is the state, \(u_k \in \mathbb{R}^m\) is the control input, \(A \in \mathbb{R}^{n \times n}\) and \(B \in \mathbb{R}^{n \times m}\).

The MPC optimisation problem at time step \(k\) is:

MPC Optimisation Problem

\[\min_{U} \; J = \sum_{i=1}^{N} \bigl(x_{k+i} - x_{\text{ref}}\bigr)^T Q \bigl(x_{k+i} - x_{\text{ref}}\bigr) \;+\; \sum_{i=0}^{N-1} u_{k+i}^T\,R\,u_{k+i} \]

subject to: \(x_{k+i+1} = A\,x_{k+i} + B\,u_{k+i}\), \(\;u_{\min} \leq u_{k+i} \leq u_{\max}\), \(\;x_{\min} \leq x_{k+i} \leq x_{\max}\)

Here \(Q \succeq 0\) is the state cost matrix (penalises deviation from the reference), \(R \succ 0\) is the control cost matrix (penalises large control inputs), and \(N\) is the prediction horizon length.

Figure 2: MPC closed-loop control architecture. The optimizer uses a system model and constraints to compute the optimal control sequence; only the first input is applied to the plant. State feedback closes the loop.

3.2.3 State Prediction over the Horizon

By recursively applying the dynamics, we can express all future states in terms of the current state \(x_k\) and the control sequence \(U\):

3.2.3.1 Recursive State Prediction

Step 1: One step ahead:

\[x_{k+1} = A\,x_k + B\,u_k \]

Step 2: Two steps ahead:

\[x_{k+2} = A\,x_{k+1} + B\,u_{k+1} = A^2\,x_k + A\,B\,u_k + B\,u_{k+1} \]

Step 3: General \(i\) steps ahead:

\[x_{k+i} = A^i\,x_k + \sum_{j=0}^{i-1} A^{i-1-j}\,B\,u_{k+j} \]

Step 4: Stack all predictions into a single matrix equation. Define:

\[\mathbf{X} = \begin{bmatrix} x_{k+1} \\ x_{k+2} \\ \vdots \\ x_{k+N} \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} u_k \\ u_{k+1} \\ \vdots \\ u_{k+N-1} \end{bmatrix} \]

Then:

\[\mathbf{X} = \mathcal{A}\,x_k + \mathcal{B}\,\mathbf{U} \]

where:

\[\mathcal{A} = \begin{bmatrix} A \\ A^2 \\ A^3 \\ \vdots \\ A^N \end{bmatrix}, \quad \mathcal{B} = \begin{bmatrix} B & 0 & 0 & \cdots & 0 \\ AB & B & 0 & \cdots & 0 \\ A^2B & AB & B & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & A^{N-3}B & \cdots & B \end{bmatrix} \]

Figure 3: State prediction visualization. The optimizer evaluates multiple control sequences (grey dashed) and selects the one minimizing the cost (green). Only the first control action u_k* is applied; the rest are discarded and re-computed at the next step.

3.2.4 Quadratic Cost in Matrix Form

Define the stacked reference \(\mathbf{X}_{\text{ref}} = [x_{\text{ref}}^T, x_{\text{ref}}^T, \ldots, x_{\text{ref}}^T]^T \in \mathbb{R}^{Nn}\) and the block-diagonal weight matrices:

\[\mathbf{Q}_N = \text{blkdiag}(Q, Q, \ldots, Q) \in \mathbb{R}^{Nn \times Nn}, \quad \mathbf{R}_N = \text{blkdiag}(R, R, \ldots, R) \in \mathbb{R}^{Nm \times Nm} \]

Substituting \(\mathbf{X} = \mathcal{A}\,x_k + \mathcal{B}\,\mathbf{U}\) into the cost function:

3.2.4.1 Cost Function Expansion

\[J = (\mathbf{X} - \mathbf{X}_{\text{ref}})^T \mathbf{Q}_N (\mathbf{X} - \mathbf{X}_{\text{ref}}) + \mathbf{U}^T \mathbf{R}_N \mathbf{U} \]

Let \(\mathbf{E} = \mathbf{X}_{\text{ref}} - \mathcal{A}\,x_k\) (the "free-response error"). Then \(\mathbf{X} - \mathbf{X}_{\text{ref}} = \mathcal{B}\,\mathbf{U} - \mathbf{E}\), so:

\[J = (\mathcal{B}\,\mathbf{U} - \mathbf{E})^T \mathbf{Q}_N (\mathcal{B}\,\mathbf{U} - \mathbf{E}) + \mathbf{U}^T \mathbf{R}_N \mathbf{U} \]

Expanding:

\[J = \mathbf{U}^T \underbrace{(\mathcal{B}^T \mathbf{Q}_N \mathcal{B} + \mathbf{R}_N)}_{H} \mathbf{U} - 2\,\mathbf{E}^T \mathbf{Q}_N \mathcal{B}\,\mathbf{U} + \mathbf{E}^T \mathbf{Q}_N \mathbf{E} \]

This is a strictly convex quadratic in \(\mathbf{U}\) (since \(H \succ 0\) when \(R \succ 0\)), so the unique minimum is found by setting \(\frac{\partial J}{\partial \mathbf{U}} = 0\):

\[2\,H\,\mathbf{U}^* - 2\,\mathcal{B}^T \mathbf{Q}_N \mathbf{E} = 0 \]

3.2.5 The Optimal Control Law

MPC Optimal Control Sequence

\[\mathbf{U}^* = \bigl(\mathcal{B}^T \mathbf{Q}_N \mathcal{B} + \mathbf{R}_N\bigr)^{-1} \mathcal{B}^T \mathbf{Q}_N \bigl(\mathbf{X}_{\text{ref}} - \mathcal{A}\,x_k\bigr) \]

The matrix \(H = \mathcal{B}^T \mathbf{Q}_N \mathcal{B} + \mathbf{R}_N\) is of dimension \(Nm \times Nm\). It must be inverted (or factorised) at every time step. This is the primary computational cost of MPC.

3.2.6 Receding Horizon: Apply Only the First Input

The full optimal sequence is $\mathbf{U}^* = [u_k^{*T}, u_{k+1}^{*T}, \ldots, u_{k+N-1}^{*T}]^T$. In the receding-horizon strategy, we apply only the first element:

$$u_k = u_k^* = \text{first } m \text{ entries of } \mathbf{U}^*$$

At the next time step, the state x_{k+1} is measured (or estimated), and the entire optimisation is re-solved. This feedback mechanism gives MPC its robustness to disturbances and model errors.

Figure 4: MPC flowchart showing the predict-optimise-apply loop. The key insight is that only the first control input is applied (highlighted in green), and the entire optimisation is re-solved at every time step.

3.2.6.1 Think–Pair–Share (3 min)

Question: Why does MPC re-solve the optimisation at every time step instead of applying the entire precomputed sequence $\mathbf{U}^*$?

Hint: Think about what happens if the model is imperfect, or if a disturbance pushes the state away from the predicted trajectory. Compare with open-loop vs closed-loop control.

PSEUDOCODE — MPC Receding Horizon Algorithm

INITIALISE: state x_0, horizon N, weights Q and R BUILD: prediction matrices A_cal and B_cal from (A, B, N) BUILD: block-diagonal weights Q_N = blkdiag(Q,...,Q), R_N = blkdiag(R,...,R) COMPUTE: H = B_cal' * Q_N * B_cal + R_N COMPUTE: H_inv = inv(H) (do this ONCE if A, B are constant) FOR k = 0, 1, 2, ... DO MEASURE (or estimate) current state x_k BUILD reference vector: X_ref = [x_ref; x_ref; ...; x_ref] (N copies) COMPUTE free-response error: E = X_ref - A_cal * x_k COMPUTE optimal sequence: U_star = H_inv * B_cal' * Q_N * E EXTRACT first control: u_k = U_star(1:m) APPLY u_k to the plant ADVANCE to next time step END FOR

3.2.7 Effect of Tuning Parameters

Parameter	Increase Effect
Prediction horizon N	Better anticipation of future references; more stable; higher computational cost ($O(N^3 m^3$
State weight Q	Tighter tracking; more aggressive control; risk of actuator saturation
Control weight R	Penalises large inputs; smoother control; slower response; energy-efficient

Practical Guideline: Start with $Q = I$, $R = 0.1\ I$, and $N = 10$. Then adjust: increase Q if tracking is too loose, increase R if control inputs saturate, and increase N until performance no longer improves significantly.

3.3 MPC for 2-DOF Manipulator

3.3.1 The Nonlinear Dynamics

Recall from Week 1 the 2-DOF planar manipulator dynamics:

$$ M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} + G(q) = \tau $$

with state vector $x = [q_1, q_2, \dot{q}_1, \dot{q}_2]^T \in \mathbb{R}^4$ and control input $u = [\tau_1, \tau_2]^T \in \mathbb{R}^2$.

Figure 5: 2-DOF planar manipulator with joint angles 1, 2, link lengths l1, l2, masses m1, m2, and torque inputs 1, 2. The coordinate frame is shown at the base with gravity acting downward.

3.3.2 Linearisation around an Operating Point

MPC (in its basic linear form) requires a linear discrete-time model. We linearise the manipulator dynamics around a nominal operating point $(q_0, \dot{q}_0) = (0, 0)$ (the hanging-down equilibrium).

3.3.2.1 Linearisation Procedure

The continuous-time state-space form is:

$$ \dot{x} = f(x, u) = \begin{bmatrix} \dot{q} \\ M^{-1}(q)\bigl[\tau - C(q,\dot{q})\dot{q} - G(q)\bigr] \end{bmatrix} $$

At the operating point $x_0 = [0, 0, 0, 0]^T$, $u_0 = G(q_0)$ (the torque needed to hold the arm at q_0):

$$ A_c = \left.\frac{\partial f}{\partial x}\right|_{x_0, u_0}, \quad B_c = \left.\frac{\partial f}{\partial u}\right|_{x_0, u_0} $$

For the 2-DOF arm with $m_1 = m_2 = 1\,\text{kg}$, $l_1 = l_2 = 1\,\text{m}$, evaluated at $q = 0$:

$$ M(0) = \begin{bmatrix} m_1 l_1^2 + m_2(l_1^2 + l_2^2 + 2l_1 l_2) & m_2(l_2^2 + l_1 l_2) \\ m_2(l_2^2 + l_1 l_2) & m_2 l_2^2 \end{bmatrix} = \begin{bmatrix} 4 & 2 \\ 2 & 1 \end{bmatrix} $$

Since $C(q, 0) = 0$ at zero velocity, and the gravity linearisation yields stiffness terms, the linearised continuous-time system is:

$$ A_c = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad B_c = \begin{bmatrix} 0_{2 \times 2} \\ M^{-1}(q_0) \end{bmatrix} $$

Computing $M^{-1}(0)$: since $\det(M(0)) = 4 \cdot 1 - 2 \cdot 2 = 0$... Note that $M(0)$ is singular at this particular configuration because we used point masses at link tips. In practice we use the more general parametrisation with l_{c1}, l_{c2}. For our MATLAB implementation, we evaluate at a slightly offset configuration $q_0 = [0.1, 0.1]^T$ to ensure non-singularity, giving:

\[M(q_0) \approx \begin{bmatrix} 3.96 & 1.99 \\ 1.99 & 1.0 \end{bmatrix}, \quad M^{-1}(q_0) \approx \begin{bmatrix} 50.25 & -100.0 \\ -100.0 & 199.0 \end{bmatrix} \]

3.3.3 Discretisation

Using the zero-order hold (ZOH) method with sampling time \(\Delta t\):

\[A_d = e^{A_c \Delta t} \approx I + A_c \Delta t, \quad B_d \approx B_c \Delta t \]

First-order Euler approximation, valid for small \(\Delta t\).

3.3.4 MPC Design for the Manipulator

State: \(x = [q_1, q_2, \dot{q}_1, \dot{q}_2]^T\), Control: \(u = [\tau_1, \tau_2]^T\)

3.3.4.1 Weight Matrix Selection

We want tight tracking of joint positions but allow velocities to deviate more freely:

\[Q = \text{diag}(100, 100, 1, 1), \quad R = \text{diag}(0.01, 0.01) \]

The ratio \(Q/R\) determines aggressiveness: here \(q_{ii}/r_{ii} = 10{,}000\), so the controller strongly prioritises position tracking over control effort economy.

With prediction horizon \(N = 20\) and \(\Delta t = 0.01\,\text{s}\), we build the prediction matrices \(\mathcal{A} \in \mathbb{R}^{80 \times 4}\) and \(\mathcal{B} \in \mathbb{R}^{80 \times 40}\), then compute the Hessian \(H = \mathcal{B}^T \mathbf{Q}_N \mathcal{B} + \mathbf{R}_N \in \mathbb{R}^{40 \times 40}\).

3.3.5 MATLAB Implementation — Manipulator MPC

%% === MPC for 2-DOF Manipulator === % System parameters params.m1 = 1; params.m2 = 1; params.l1 = 1; params.l2 = 1; params.g = 9.81; % Linearisation point q0 = [0.1; 0.1]; dq0 = [0; 0]; % Build M(q0), compute M_inv c2 = cos(q0(2)); M11 = params.m1*params.l1^2 + params.m2*(params.l1^2 + params.l2^2 + 2*params.l1*params.l2*c2); M12 = params.m2*(params.l2^2 + params.l1*params.l2*c2); M22 = params.m2*params.l2^2; M0 = [M11, M12; M12, M22]; M0_inv = inv(M0); % Continuous-time A, B (linearised around q0, dq0=0) Ac = [zeros(2), eye(2); zeros(2), zeros(2)]; Bc = [zeros(2); M0_inv]; % Discretise (Euler, dt = 0.01 s) dt = 0.01; Ad = eye(4) + Ac*dt; Bd = Bc*dt; % MPC parameters N = 20; % Prediction horizon Q = diag([100, 100, 1, 1]); % State weight R = diag([0.01, 0.01]); % Control weight % Build prediction matrices n = 4; m = 2; A_cal = zeros(n*N, n); B_cal = zeros(n*N, m*N); for i = 1:N A_cal((i-1)*n+1:i*n, :) =

Ad^i; for j = 1:i B_cal((i-1)*n+1:i*n, (j-1)*m+1:j*m) = Ad^(i-j) * Bd; end end % Block-diagonal weights Q_N = kron(eye(N), Q); R_N = kron(eye(N), R); % Hessian and its inverse (constant for LTI system) H = B_cal' * Q_N * B_cal + R_N; H_inv = inv(H); F = B_cal' * Q_N; % Simulation t_span = 0:dt:20; x = [0; 0; 0; 0]; % Initial state x_hist = zeros(4, length(t_span)); u_hist = zeros(2, length(t_span)); x_hist(:,1) = x; for k = 1:length(t_span)-1 t = t_span(k); % Reference trajectory (sinusoidal) x_ref_k = zeros(n*N, 1); for i = 1:N t_future = t + i*dt; x_ref_k((i-1)*n+1:i*n) = [0.5*sin(t_future); 0.5*cos(t_future); ... 0.5*cos(t_future); -0.5*sin(t_future)]; end % MPC optimal control E = x_ref_k - A_cal * x; U_star = H_inv * F * E; u = U_star(1:m); % Apply first control only % Clamp torques (constraint enforcement) u = max(min(u, 50), -50); % Simulate true nonlinear dynamics (one Euler step) q = x(1:2); dq = x(3:4); [M_true, C_true, G_true] = manipulator_model(q, dq, params); ddq = M_true \ (u - C_true*dq - G_true); x = x + [dq; ddq]*dt; x_hist(:,k+1) = x; u_hist(:,k) = u; end

Note: The controller uses a linear model, but the plant simulation uses the full nonlinear dynamics. This mismatch is realistic and tests the robustness of MPC.

3.4 MPC for Differential-Drive Mobile Robot

3.4.1 Kinematic Model

A differential-drive mobile robot (e.g., TurtleBot) moves in the plane with state \(x = [p_x, p_y, \theta]^T\) (position and heading) and control inputs \(u = [v, \omega]^T\) (linear and angular velocity):

Nonlinear Kinematic Model

\[\begin{bmatrix} \dot{p}_x \\ \dot{p}_y \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v\,\cos\theta \\ v\,\sin\theta \\ \omega \end{bmatrix} \]

Figure 6: Top-down view of a differential-drive mobile robot. The robot state is (p_x, p_y,), controlled by linear velocity v and angular velocity . The reference path is shown as a dashed curve.

Discretised with Euler integration at sampling time \(\Delta t\):

\[\begin{aligned} p_{x,k+1} &= p_{x,k} + v_k \cos(\theta_k)\,\Delta t \\ p_{y,k+1} &= p_{y,k} + v_k \sin(\theta_k)\,\Delta t \\ \theta_{k+1} &= \theta_k + \omega_k\,\Delta t \end{aligned} \]

3.4.2 Linearisation around a Reference Trajectory

3.4.2.1 Jacobian Linearisation

Let the reference trajectory be \((p_x^r(t), p_y^r(t), \theta^r(t))\) with reference controls \((v^r(t), \omega^r(t))\). Define error variables:

\[\delta x = x - x^r, \quad \delta u = u - u^r \]

The Jacobians of the continuous-time model at the reference are:

\[A_c = \frac{\partial f}{\partial x}\bigg|_{x^r, u^r} = \begin{bmatrix} 0 & 0 & -v^r \sin\theta^r \\ 0 & 0 & v^r \cos\theta^r \\ 0 & 0 & 0 \end{bmatrix} \] \[B_c = \frac{\partial f}{\partial u}\bigg|_{x^r, u^r} = \begin{bmatrix} \cos\theta^r & 0 \\ \sin\theta^r & 0 \\ 0 & 1 \end{bmatrix} \]

For simplicity, we linearise around \(\theta^r = 0\) (heading east) and \(v^r = 2.5\,\text{m/s}\):

\[A_c = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 2.5 \\ 0 & 0 & 0 \end{bmatrix}, \quad B_c = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} \]

Discretised:

\[A_d = I + A_c \Delta t, \quad B_d = B_c \Delta t \]

3.4.3 Constraints

Constraint	Physical Meaning	Va
\(0 \leq v \leq 5\,\text{m/s}\)	The robot cannot move backwards (for safety) and has a maximum speed	[0,
\(-2 \leq \omega \leq 2\,\text{rad/s}\)	Maximum turning rate limited by wheel motors	[-2,

3.4.4 MPC Design

State: \(x = [p_x, p_y, \theta]^T\), Control: \(u = [v, \omega]^T\), Horizon: \(N = 15\), Weights:

\[Q = \text{diag}(50, 50, 10), \quad R = \text{diag}(0.1, 0.1) \]

The higher weight on position relative to heading ensures the robot prioritises reaching the correct \((x, y)\) location, with heading as a secondary objective.

3.4.5 MATLAB Implementation — Mobile Robot MPC

%% === MPC for Differential-Drive Mobile Robot === % Linearisation parameters v_ref = 2.5; % Reference linear velocity dt = 0.05; % Sampling time % Linearised system (around theta = 0, v = v_ref) Ac_mob = [0, 0, 0; 0, 0, v_ref; 0, 0, 0]; Bc_mob = [1, 0; 0, 0; 0, 1]; Ad_mob = eye(3) + Ac_mob*dt; Bd_mob = Bc_mob*dt; % MPC parameters N_mob = 15; Q_mob = diag([50, 50, 10]); R_mob = diag([0.1, 0.1]); n_mob = 3; m_mob = 2; % Build prediction matrices A_cal_mob = zeros(n_mob*N_mob, n_mob); B_cal_mob = zeros(n_mob*N_mob, m_mob*N_mob); for i = 1:N_mob A_cal_mob((i-1)*n_mob+1:i*n_mob, :) = Ad_mob^i; for j = 1:i B_cal_mob((i-1)*n_mob+1:i*n_mob, (j-1)*m_mob+1:j*m_mob) = Ad_mob^(i-j)*Bd_mob; end end Q_N_mob = kron(eye(N_mob), Q_mob); R_N_mob = kron(eye(N_mob), R_mob); H_mob = B_cal_mob' * Q_N_mob * B_cal_mob + R_N_mob; H_inv_mob = inv(H_mob); F_mob = B_cal_mob'

* Q_N_mob; % Simulation t_span_mob = 0:dt:20; x_mob = [0; 0; 0]; x_hist_mob = zeros(3, length(t_span_mob)); u_hist_mob = zeros(2, length(t_span_mob)); x_hist_mob(:,1) = x_mob; % Circular reference trajectory (radius = 5 m) R_circle = 5; omega_circle = 0.5; for k = 1:length(t_span_mob)-1 t = t_span_mob(k); % Build reference over horizon x_ref_mob = zeros(n_mob*N_mob, 1); for i = 1:N_mob t_f = t + i*dt; x_ref_mob((i-1)*n_mob+1:i*n_mob) = [R_circle*cos(omega_circle*t_f); ... R_circle*sin(omega_circle*t_f); ... omega_circle*t_f + pi/2]; end % MPC solve E_mob = x_ref_mob - A_cal_mob * x_mob; U_star_mob = H_inv_mob * F_mob * E_mob; u_mob = U_star_mob(1:m_mob); % Enforce constraints u_mob(1) = max(min(u_mob(1), 5), 0); % v in [0, 5] u_mob(2) = max(min(u_mob(2), 2), -2); % omega in [-2, 2] % True nonlinear dynamics (Euler integration) theta = x_mob(3); x_mob = x_mob + [u_mob(1)*cos(theta); u_mob(1)*sin(theta); u_mob(2)]*dt; x_hist_mob(:,k+1) = x_mob; u_hist_mob(:,k) = u_mob; end

Linearisation Caveat: The linearisation is performed at a single operating point. When the robot heading \(\theta\) deviates significantly from \(\theta^r = 0\), the linear model becomes inaccurate. In practice, one re-linearises at each time step (forming a time-varying linear MPC), or uses nonlinear MPC.

3.5 MPC for Quadrotor Drone

3.5.1 The 12-State Quadrotor Model

A quadrotor has 6 degrees of freedom (3 translational + 3 rotational), leading to a 12-dimensional state vector:

\[x = \begin{bmatrix} p_x \\ p_y \\ p_z \\ v_x \\ v_y \\ v_z \\ \phi \\ \theta \\ \psi \\ \omega_x \\ \omega_y \\ \omega_z \end{bmatrix} \quad \text{(position, velocity, Euler angles, angular rates)} \]

Figure 7: Quadrotor UAV schematic showing four rotors with alternating CW/CCW rotation directions, thrust forces F1-F4, body frame axes (x_B, y_B, z_B), and Euler angles (roll , pitch , yaw). The total thrust is T = F1+F2+F3+F4.

The control inputs are thrust and three body-frame torques:

\[u = \begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} \]

3.5.2 Nonlinear Equations of Motion

3.5.2.1 Full Quadrotor Dynamics

Translational dynamics (Newton's second law in the world frame):

\[\begin{aligned} \dot{v}_x &= \frac{T}{m}\bigl(\cos\phi\sin\theta\cos\psi + \sin\phi\sin\psi\bigr) \\ \dot{v}_y &= \frac{T}{m}\bigl(\cos\phi\sin\theta\sin\ps

- \sin\phi\cos\psi\bigr) \\ \dot{v}_z &= \frac{T}{m}\cos\phi\cos\theta
- g \end{aligned} \]

Rotational dynamics (Euler's equations for a symmetric body):

\[\begin{aligned} \dot{\omega}_x &= \frac{\tau_x}{I_{xx}} + \frac{(I_{yy} - I_{zz})}{I_{xx}}\omega_y\omega_z \\ \dot{\omega}_-
y &= \frac{\tau_y}{I_{yy}} + \frac{(I_{zz} - I_{xx})}{I_{yy}}\omega_-
x\omega_z \\ \dot{\omega}_z &= \frac{\tau_z}{I_{zz}} + \frac{(I_-
{xx} - I_{yy})}{I_{zz}}\omega_x\omega_y \end{aligned} \]

3.5.3 Linearisation around Hover

At hover, all velocities and angular rates are zero, \(\phi = \theta = \psi = 0\), and the thrust equals gravity: \(T_0 = mg\). The linearised continuous-time system simplifies considerably because the trigonometric functions reduce to their small-angle approximations and the cross-coupling terms in Euler's equations vanish:

3.5.3.1 Linearised Hover Dynamics

At hover (\(\phi = \theta = \psi = 0\), \(T_0 = mg\)):

\[\dot{v}_x \approx g\,\theta, \quad \dot{v}_y \approx -g\,\phi, \quad \dot{v}_z \approx \frac{T - mg}{m} \] \[\dot{\omega}_-
x \approx \frac{\tau_x}{I_{xx}}, \quad \dot{\omega}_y \approx \frac{\tau_y}{I_{yy}}, \quad \dot{\omega}_z \approx \frac{\tau_-
z}{I_{zz}} \]

The linearised state-space matrices become:

\[A_c = \begin{bmatrix} 0_3 & I_3 & 0_3 & 0_3 \\ 0_3 & 0_3 & G_{\text{lin}} & 0_3 \\ 0_3 & 0_3 & 0_3 & I_3 \\ 0_3 & 0_3 & 0_3 & 0_3 \end{bmatrix}, \quad B_c = \begin{bmatrix} 0_3 \\ B_T \\ 0_3 \\ B_\tau \end{bmatrix} \]

where the coupling matrix \(G_{\text{lin}}\) and input matrices are:

\[G_{\text{lin}} = \begin{bmatrix} 0 & g & 0 \\ -g & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad B_T = \begin{bmatrix} 0 \\ 0 \\ 1/m \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}, \quad B_\tau = \text{diag}\!\left(\frac{1}{I_{xx}}, \frac{1}{I_{yy}}, \frac{1}{I_{zz}}\right) \begin{bmatrix} 0 & I_3 \end{bmatrix} \]

3.5.4 MPC Design for the Quadrotor

State dimension: \(n = 12\), Control dimension: \(m = 4\), Horizon: \(N = 15\).

\[Q = \text{diag}(\underbrace{80, 80, 120}_{\text{position}}, \underbrace{1, 1, 1}_{\text{velocity}}, \underbrace{50, 50, 50}_{\text{orientation}}, \underbrace{1, 1, 1}_{\text{angular rate}}) \] \[R = \text{diag}(0.1, 0.1, 0.1, 0.1) \]

3.5.5 Constraints

Constraint	Physical Meaning
\(0 \leq T \leq 2mg\)	Thrust cannot be negative; max is twice
\(-2 \leq \tau_x, \tau_y, \tau_z \leq 2\)	Maximum body torques from motor diff
\(-30^\circ \leq \phi, \theta \leq 30^\circ\)	Tilt angle limits for safety

3.5.6 MATLAB Implementation — Quadrotor MPC

%% === MPC for Quadrotor Drone === % Physical parameters quad.m = 1.0; quad.Ixx = 0.1; quad.Iyy = 0.1; quad.Izz = 0.2; quad.g = 9.81; dt_q = 0.02; % Sampling time (50 Hz) n_q = 12; m_q = 4; % Linearised continuous-time matrices (hover) Ac_q = zeros(12,12); Ac_q(1:3, 4:6) = eye(3); % dp/dt = v Ac_q(4,8) = quad.g; % dvx/dt = g*theta Ac_q(5,7) = -quad.g; % dvy/dt = -g*phi Ac_q(7:9, 10:12) = eye(3); % datt/dt = omega Bc_q = zeros(12,4); Bc_q(6,1) = 1/quad.m; % dvz/dt = T/m Bc_q(10,2) = 1/quad.Ixx; % domega_x/dt = tau_x/Ixx Bc_q(11,3) = 1/quad.Iyy; % domega_y/dt = tau_y/Iyy Bc_q(12,4) = 1/quad.Izz; % domega_z/dt = tau_z/Izz % Discretise Ad_q = eye(12) + Ac_q*dt_q; Bd_q = Bc_q*dt_q; % MPC weights N_q = 15; Q_q = diag([80, 80, 120, 1, 1, 1, 50, 50, 50, 1, 1, 1]); R_q = diag([0.1, 0.1, 0.1, 0.1]); % Build prediction matrices A_cal_q = zeros(n_q*N_q, n_q); B_cal_q = zeros(n_q*N_q, m_q*N_q); for i = 1:N_q A_cal_q((i-1)*n_q+1:i*n_q, :) = Ad_q^i; for j = 1:i B_cal_q((i-1)*n_q+1:i*n_q, (j-1)*m_q+1:j*m_q) = Ad_q^(i-j)*Bd_q; end end Q_N_q = kron(eye(N_q), Q_q); R_N_q = kron(eye(N_q), R_q); H_q = B_cal_q' * Q_N_q * B_cal_q + R_N_q; H_inv_q = inv(H_q); F_q = B_cal_q' * Q_N_q; % Simulation (helical trajectory) t_span_q = 0:dt_q:20; x_q = zeros(12,1); x_hist_q = zeros(12, length(t_span_q)); u_hist_q = zeros(4, length(t_span_q)); x_hist_q(:,1) = x_q; % Helical reference: circle in XY with linear climb in Z R_helix = 3; omega_helix = 0.5; vz_helix = 0.3; for k = 1:length(t_span_q)-1 t = t_span_q(k); % Build reference over horizon x_ref_q = zeros(n_q*N_q, 1); for i = 1:N_q t_f = t + i*dt_q; px_ref = R_helix*sin(omega_helix*t_f); py_ref = R_helix*cos(omega_helix*t_f); pz_ref = 5 + vz_helix*sin(0.2*t_f); vx_ref = R_helix*omega_helix*cos(omega_helix*t_f); vy_ref = -R_helix*omega_helix*sin(omega_helix*t_f); vz_ref = vz_helix*0.2*cos(0.2*t_f); x_ref_q((i-1)*n_q+1:i*n_q) = [px_ref; py_ref; pz_ref; ... vx_ref; vy_ref; vz_ref; ... 0; 0; 0; 0; 0; 0]; end % MPC solve (note: u here is delta_u around hover) E_q = x_ref_q - A_cal_q * x_q; U_star_q = H_inv_q * F_q * E_q; u_q = U_star_q(1:m_q); % Add hover thrust offset and enforce constraints u_q(1) = u_q(1) + quad.m*quad.g; % Total thrust = hover + correction u_q(1) = max(min(u_q(1), 2*quad.m*quad.g), 0); u_q(2:4) = max(min(u_q(2:4), 2), -2); % Simulate true nonlinear dynamics (simplified Euler step) pos = x_q(1:3); vel = x_q(4:6); att = x_q(7:9); omg = x_q(10:12); phi = att(1); th = att(2); psi = att(3); T = u_q(1); tau_body = u_q(2:4); % Translational acceleration (full nonlinear) ax = (T/quad.m)*(cos(phi)*sin(th)*cos(psi) + sin(phi)*sin(psi)); ay = (T/quad.m)*(cos(phi)*sin(th)*sin(psi) - sin(phi)*cos(psi)); az

= (T/quad.m)*cos(phi)*cos(th) - quad.g; % Rotational acceleration
domg_x = tau_body(1)/quad.Ixx + (quad.Iyy-quad.Izz)/quad.Ixx*omg(2)*omg(3);
domg_y = tau_body(2)/quad.Iyy + (quad.Izz-quad.Ixx)/quad.Iyy*omg(1)*omg(3);
domg_z = tau_body(3)/quad.Izz + (quad.Ixx-quad.Iyy)/quad.Izz*omg(1)*omg(2);
% Euler integration x_q = x_q + [vel; ax; ay; az; omg; domg_x; domg_y; domg_z]*dt_q; x_hist_q(:,k+1) = x_q; u_hist_q(:,k) = u_q; end

Key Observation: The MPC controller for the quadrotor uses a 12-state linear model but drives the full nonlinear plant. The receding-horizon feedback mechanism compensates for linearisation errors at each step.

3.6 MATLAB Implementation & Activities

3.6.1 Activity 1: Prediction Horizon Tuning (10 min)

3.6.1.1 Task: Investigate the Effect of Horizon Length \(N\)

Run the manipulator MPC code with three different prediction horizons. For each, record the peak tracking error \(\max |e(t)|\) and the computation time.

Horizon \(N\)	Peak Position Error (rad)	Steady-State Error (rad)	Approx. Computation Time
\(N = 5\)	[record]	[record]	[record]
\(N = 20\)	[record]	[record]	[record]
\(N = 50\)	[record]	[record]	[record]

Questions to answer:

1. How does increasing \(N\) affect the transient response? Is there a point of diminishing returns?
2. Why does a very short horizon (\(N = 5\)) lead to poor performance? Relate to the "myopic" nature of a short prediction window.
3. At what value of \(N\) does the computation time become a concern for real-time control at 100 Hz?

3.6.2 Activity 2: Three-System Comparison (15 min)

3.6.2.1 Task: Run MPC for All Three Systems Side by Side

Run the complete `mpc_control_simulation.m` file, which simulates MPC for the manipulator, mobile robot, and quadrotor simultaneously. The script also includes a PD-control baseline for comparison.

1. Compare the MPC tracking error with the PD tracking error for each system. Which controller provides tighter tracking?
2. Examine the control input plots. Does MPC produce smoother control signals than PD?
3. Look at the constraint enforcement: does PD ever violate the torque/velocity limits? Does MPC?

4. For the mobile robot, compare the actual path (MPC vs PD) on the XY plot. Which is closer to the circular reference?

3.6.3 Activity 3: MPC vs Computed Torque Discussion (5 min)

3.6.3.1 Think–Pair–Share

Question 1: In what real-world scenario would you prefer MPC over computed torque control? Consider constraint handling, trajectory planning, and multi-objective optimisation.

Question 2: When would computed torque control be the better choice? Consider computational requirements, real-time performance at high rates, and systems with well-known dynamics.

Discuss with your neighbour for 2 minutes, then share one scenario each with the class.

3.7 Summary and Next Steps

3.7.1 Summary & Key Takeaways

3.7.2 Core MPC Equations

Equation
\(x_{k+1} = A\,x_k + B\,u_k\)
\(\mathbf{X} = \mathcal{A}\,x_k + \mathcal{B}\,\mathbf{U}\)
\(J = (\mathbf{X} - \mathbf{X}_{\text{ref}})^T \mathbf{Q}_N (\mathbf{X} - \
\(\mathbf{U}^* = (\mathcal{B}^T \mathbf{Q}_N \mathcal{B} + \mathbf{R}_N
\(u_k = \mathbf{U}^*(1{:}m)\)

3.7.3 MPC vs Computed Torque Control

Property	MPC
Model Knowledge	Requires linear (or nonlinear) model; can use approximate mod
Constraint Handling	Systematically incorporates state and control constraints in the
Computation	Solves a QP at every time step; computational cost grows with
Tracking Accuracy	Optimal within the prediction horizon; slightly conservative due
Multi-Objective	Naturally balances tracking, energy, and constraints through co
Robustness	Feedback through receding horizon; can incorporate robust/sto
Best For	Systems with constraints, slow-medium dynamics, trajectory op

3.7.3.1 Key Takeaway 1

MPC converts the control problem into a repeated optimisation problem. The quality of the controller depends directly on the quality of the model and the tuning of \(N\), \(Q\), \(R\).

3.7.3.2 Key Takeaway 2

The receding-horizon strategy provides implicit feedback: by re-solving at every step, MPC naturally rejects disturbances and compensates for model errors.

3.7.3.3 Key Takeaway 3

MPC's greatest strength over computed torque is constraint handling. Real actuators have limits; MPC respects them by design.

3.7.3.4 Key Takeaway 4

Computed torque remains superior when the dynamics are well-known and the control loop must run at very high rates ($>$ 1 kHz) where solving a QP is too expensive.

3.7.4 Next Week: Adaptive Control

Building on Weeks 1 and 2, we will cover:

- Parameter estimation and adaptive control for manipulators with unknown dynamics
- Model Reference Adaptive Control (MRAC)
- Linearity-in-parameters and the adaptive computed torque controller
- Stability proofs for adaptive systems using Lyapunov-based methods

3.7.5 Preparation

- Complete all MATLAB activities from today and save your scripts
- Review the linearity-in-parameters property from Week 1 (Section 1.4, Property 3)
- Think about: "If the controller does not know \(M(q)\), \(C(q,\dot{q})\), or \(G(q)\), how could it learn these online while still maintaining stability?"
- Read about parameter convergence and persistent excitation

Control for Robots

University of West London | Academic Year 2025–26

Week 2: Model Predictive Control (MPC)

This lecture material is for educational use only.

4 Adaptive Control (Week 4)

4.1 Overview 48
4.1.1 Week 3 Recall — Model Predictive Control — Exam Practice 48
4.1.2 Learning Outcomes . . . 50
4.1.3 Session Roadmap (3 Hours) 50
4.2 Adaptive Control Theory 51
4.2.1 Why Adaptive Control? 51
4.2.2 The Linearity-in-Parameters Property . . 52
4.2.3 Concrete Example: 2-DOF Planar Manipulator 53
4.2.4 The Adaptive Control Law 53
4.2.5 Parameter Update Law . 54
4.2.6 Lyapunov Stability Proof 54
4.3 Adaptive Control for 2-DOF Manipulator . . . 57
4.3.1 System Setup 57
4.3.2 Building the Regressor Matrix 57
4.3.3 Complete Control Algorithm 58
4.3.4 MATLAB Implementation 58
4.4 Adaptive Control for Mobile Robot 59
4.4.1 Parametric Uncertainty in Mobile Robots 59
4.4.2 Linearity in Parameters for the Kinematic Model 59
4.4.3 Adaptive Tracking Controller 59
4.4.4 MATLAB Implementation 60
4.5 Adaptive Control for Quadrotor 61
4.5.1 Quadrotor Dynamics with Unknown Parameters . . 61
4.5.2 Linearity in Parameters 61
4.5.3 Attitude Adaptive Control 62
4.5.4 Update Laws 62
4.5.5 MATLAB Implementa-

Lyapunov-Based Adaptive Control for Robotic Systems with Parametric Uncertainty

M.Sc. Control for Robots — University of West London — AY 2025–26
3-Hour Lecture & Lab Week 4 of 12

4.1 Overview

4.1.1 Week 3 Recall — Model Predictive Control — Exam Practice

Before we begin new material, spend 20 minutes working through these two exam-style questions that cover last week's MPC content. Write your solutions on paper.

4.1.1.1 Question 1 — MPC Problem Formulation & Lifted Form (10 marks)

Consider a discrete-time linear time-invariant system:

\[x_{k+1} = A\,x_k + B\,u_k \]

with state \(x_k \in \mathbb{R}^n\), control \(u_k \in \mathbb{R}^m\), and a finite prediction horizon \(N\). The MPC cost function is:

\[J = \sum_{i=0}^{N-1}\Big[\big(x_{k+i} - x_{\mathrm{ref}}\big)^T Q \big(x_{k+i} - x_{\mathrm{ref}}\big) + u_{k+i}^T R\, u_{k+i}\Big] \]

where \(Q \succeq 0\) and \(R \succ 0\).

(a) By recursive substitution, show that the predicted state sequence can be written in compact (lifted) form as:

\[\mathbf{X} = \mathcal{A}\,x_k + \mathcal{B}\,\mathbf{U} \]

where the stacked vectors are \(\mathbf{X} = \begin{bmatrix} x_{k+1} \\ x_{k+2} \\ \vdots \\ x_{k+N} \end{bmatrix}\) and \(\mathbf{U} = \begin{bmatrix} u_k \\ u_{k+1} \\ \vdots \\ u_{k+N-1} \end{bmatrix}\), and derive the explicit forms:

\[\mathcal{A} = \begin{bmatrix} A \\ A^2 \\ A^3 \\ \vdots \\ A^N \end{bmatrix}, \qquad \mathcal{B} = \begin{bmatrix} B & 0 & 0 & \cdots & 0 \\ AB & B & 0 & \cdots & 0 \\ A^2B & AB & B & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & A^{N-3}B & \cdots & B \end{bmatrix} \]

Show every substitution step from \(i=1\) to \(i=N\).

(b) Identify the structure of \(\mathcal{B}\) (lower-triangular block Toeplitz). Explain why this structure arises from the causal nature of the system (future states depend only on past and present inputs).

4.1.1.2 Question 2 — Deriving the Optimal MPC Control Law (10 marks)

Starting from the lifted quadratic cost:

\[J = \big(\mathbf{X} - \mathbf{X}_{\mathrm{ref}}\big)^T \bar{Q}\,\big(\mathbf{X} - \mathbf{X}_{\mathrm{ref}}\big) + \mathbf{U}^T \bar{R}\,\mathbf{U} \]

where \(\bar{Q} = \mathrm{blkdiag}(Q, Q, \dots, Q) \in \mathbb{R}^{nN \times nN}\) and \(\bar{R} = \mathrm{blkdiag}(R, R, \dots, R) \in \mathbb{R}^{mN \times mN}\):

(a) Substitute \(\mathbf{X} = \mathcal{A}\,x_k + \mathcal{B}\,\mathbf{U}\) into \(J\) and expand fully:

\[J = \big(\mathcal{A}\,x_k + \mathcal{B}\,\mathbf{U} - \mathbf{X}_{\mathrm{ref}}\big)^T \bar{Q}\,\big(\mathcal{A}\,x_k + \mathcal{B}\,\mathbf{U} - \mathbf{X}_{\mathrm{ref}}\big) + \mathbf{U}^T \bar{R}\,\mathbf{U} \]

Let \(\mathbf{E} = \mathbf{X}_{\mathrm{ref}} - \mathcal{A}\,x_k\). Show that:

\[J = \mathbf{U}^T\!\big(\mathcal{B}^T \bar{Q}\,\mathcal{B} + \bar{R}\big)\mathbf{U} - 2\,\mathbf{E}^T \bar{Q}\,\mathcal{B}\,\mathbf{U} + \mathbf{E}^T \bar{Q}\,\mathbf{E} \]

(b) Take \(\frac{\partial J}{\partial \mathbf{U}} = 0\) and derive:

\[\mathbf{U}^* = \big(\mathcal{B}^T \bar{Q}\,\mathcal{B} + \bar{R}\big)^{-1} \mathcal{B}^T \bar{Q}\,\big(\mathbf{X}_{\mathrm{ref}} - \mathcal{A}\,x_k\big) \]

(c) Define the MPC gain. Since only the first \(m\) rows of \(\mathbf{U}^*\) are applied (receding horizon), let \(E_1 = [I_m \;\; 0 \;\; \cdots \;\; 0]\). Then:

\[u_k^* = E_1 \mathbf{U}^* = \underbrace{E_1 \big(\mathcal{B}^T \bar{Q}\,\mathcal{B} + \bar{R}\big)^{-1} \mathcal{B}^T \bar{Q}}_{K_{\mathrm{MPC,ref}}}\,\mathbf{X}_{\mathrm{ref}} - \underbrace{E_1 \big(\mathcal{B}^T \bar{Q}\,\mathcal{B} + \bar{R}\big)^{-1} \mathcal{B}^T \bar{Q}\,\mathcal{A}}_{K_{\mathrm{MPC}}}\,x_k \]

Explain why \(K_{\mathrm{MPC}}\) is a state-feedback gain and how increasing \(R\) relative to \(Q\) affects the control aggressiveness.

Time limit: 20 minutes. This is exactly the format and difficulty level you will encounter in the final exam. Write full derivations showing every step. Partial credit is awarded for correct intermediate work.

4.1.2 Learning Outcomes

- Understand why adaptive control is needed when system parameters are unknown or time-varying, and articulate the limitations of computed torque and MPC that assume known models.
- Derive the adaptive control law using Lyapunov-based stability analysis, including the construction of a candidate Lyapunov function and verification that its time derivative is negative semi-definite.
- Apply the linearity-in-parameters property of rigid-body dynamics: \(M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = Y(q, \dot{q}, \ddot{q})\,\theta\), identifying the regressor matrix \(Y\) and parameter vector \(\theta\) for specific systems.
- Implement adaptive control for three robotic platforms: a 2-DOF manipulator, a differential-drive mobile robot, and a quadrotor UAV.
- Simulate and compare the performance of fixed-model computed torque control versus adaptive control under parameter uncertainty, demonstrating convergence of parameter estimates.

4.1.3 Session Roadmap (3 Hours)

0:00 – 0:20

4.1.3.1 Previous Week Exam Practice

Two exam-style questions on MPC lifted form, optimal control law derivation, and the MPC gain matrix.

0:20 – 1:00

4.1.3.2 Adaptive Control Theory

Why adaptive? Parameter uncertainty and its consequences. Linearity in parameters. Regressor matrix. Adaptive update law. Lyapunov stability proof.

1:00 – 1:20

4.1.3.3 Adaptive Control for 2-DOF Manipulator

Full derivation: explicit regressor, parameter vector, control law, update law, and MATLAB implementation.

1:20 – 1:30

4.1.3.4 Break

10-minute comfort break.

1:30 – 2:00

4.1.3.5 Adaptive Control for Mobile Robot

Parametric uncertainty in wheel radius and track width. Regressor formulation for the unicycle kinematic model. Adaptive tracking controller.

2:00 – 2:30

4.1.3.6 Adaptive Control for Quadrotor

Unknown mass and inertia. Adaptive controller for position and attitude loops. MATLAB simulation.

2:30 – 3:00

4.1.3.7 MATLAB Implementation & Activities

Hands-on activities: parameter convergence, comparison with computed torque, regressor construction quiz.

4.2 Adaptive Control Theory

4.2.1 Why Adaptive Control?

In Week 1, we designed a computed torque controller for a 2-DOF manipulator. The control law was:

\[\tau = M(q)\big(\ddot{q}_d + K_v\dot{e} + K_p e\big) + C(q,\dot{q})\dot{q} + G(q) \]

This requires exact knowledge of the inertia matrix \(M(q)\), Coriolis/centrifugal matrix \(C(q,\dot{q})\), and gravity vector \(G(q)\). In practice, this assumption fails for several reasons:

- Unknown payloads: A manipulator picks up objects of unknown mass. The effective \(m_2\) changes.
- Manufacturing tolerances: Link lengths, masses, and centres of gravity are only approximately known.
- Wear and degradation: Friction coefficients, gear ratios, and joint stiffness change over time.

- Model simplification: We ignore link flexibility, actuator dynamics, and higher-order effects.

Key insight: If the model parameters used in computed torque are wrong, the nonlinear cancellation is imperfect, and the closed-loop system is no longer a simple linear error system. Tracking performance degrades, and stability may be lost.

Adaptive control addresses this by estimating the unknown parameters online, while simultaneously controlling the system. The parameter estimates converge to values that ensure good tracking, even if the true parameters are never exactly identified.

Figure 1: Adaptive control block diagram. The blue control loop tracks the reference, while the orange adaptation loop estimates unknown plant parameters and feeds them to the controller.

Figure 2: Comparison of Direct and Indirect adaptive control. In direct adaptation, the error signal is fed to an adaptation law that adjusts controller gains K(t) directly. In indirect adaptation, the error drives a parameter estimator to obtain , which is then used in a gain computation step to determine K().

4.2.2 The Linearity-in-Parameters Property

The dynamics of a rigid-body robotic system can always be written as:

\(M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = Y(q, \dot{q}, \ddot{q})\,\theta \)

where:

- \(Y(q, \dot{q}, \ddot{q}) \in \mathbb{R}^{n \times p}\) is the regressor matrix — a known matrix that depends on joint positions, velocities, and accelerations, but not on the unknown parameters.
- \(\theta \in \mathbb{R}^p\) is the parameter vector — a constant vector containing all the unknown dynamic parameters (products of masses, lengths, inertias, friction coefficients, etc.).

This is the linearity-in-parameters property. It holds because the equations of motion are derived from the Lagrangian, which is quadratic in velocities and linear in the inertial parameters. The property is fundamental to adaptive control: it allows us to separate what we know (the structure of \(Y\)) from what we do not know (the values in \(\theta\)).

Figure 3: The linearity-in-parameters property separates robot dynamics into a known regressor matrix Y (computable from kinematics) multiplied by an unknown parameter vector (to be estimated online).

4.2.3 Concrete Example: 2-DOF Planar Manipulator

Figure 4: 2-DOF planar manipulator. Link 1 (length l, mass m) and Link 2 (length l, mass m) rotate at joints with angles and . Joint torques and are the control inputs.

Consider the 2-DOF planar manipulator with link masses m_1, m_2, link lengths l_1, l_2, and gravity g. The standard Euler-Lagrange dynamics are:

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau$$

with:

$$M(q) = \begin{bmatrix} m_1 l_1^2 + m_2(l_1^2 + l_2^2 + 2l_1 l_2 \cos q_2) & m_2(l_2^2 + l_1 l_2 \cos q_2) \\ m_2(l_2^2 + l_1 l_2 \cos q_2) & m_2 l_2^2 \end{bmatrix}$$

$$C(q,\dot{q}) = \begin{bmatrix} -m_2 l_1 l_2 \sin(q_2)\,\dot{q}_2 & -m_2 l_1 l_2 \sin(q_2)(\dot{q}_1 + \dot{q}_2) \\ m_2 l_1 l_2 \sin(q_2)\,\dot{q}_1 & 0 \end{bmatrix}$$

$$G(q) = \begin{bmatrix} (m_1 + m_2)\,g\,l_1 \sin q_1 + m_2\,g\,l_2 \sin(q_1 + q_2) \\ m_2\,g\,l_2 \sin(q_1 + q_2) \end{bmatrix}$$

Define the parameter vector:

$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \\ \theta_4 \\ \theta_5 \\ \theta_6 \end{bmatrix} = \begin{bmatrix} m_1 l_1^2 \\ m_2 l_1^2 \\ m_2 l_2^2 \\ m_2 l_1 l_2 \\ (m_1 + m_2)\,g\,l_1 \\ m_2\,g\,l_2 \end{bmatrix}$$

Then the regressor matrix $Y \in \mathbb{R}^{2 \times 6}$ such that $\tau = Y(q, \dot{q}, \ddot{q})\,\theta$ is:

$$Y = \begin{bmatrix} \ddot{q}_1 & \ddot{q}_1 & \ddot{q}_1 + \ddot{q}_2 & 2\cos(q_2)\ddot{q}_1 + \cos(q_2)\ddot{q}_2 - \sin(q_2)\dot{q}_2\dot{q}_1 - \sin(q_2)(\dot{q}_1+\dot{q}_2)\dot{q}_2 & \sin(q_1) & \sin(q_1+q_2) \\ 0 & 0 & \ddot{q}_1 + \ddot{q}_2 & \cos(q_2)\ddot{q}_1 + \sin(q_2)\dot{q}_1^2 & 0 & \sin(q_1+q_2) \end{bmatrix}$$

You can verify: multiply $Y\theta$ and recover exactly $M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q)$. Every entry of Y is computable from measured (or commanded) q, $\dot{q}$, $\ddot{q}$.

4.2.4 The Adaptive Control Law

We now construct the adaptive controller. Define the tracking error and its filtered version:

$$e = q_d - q, \qquad \dot{e} = \dot{q}_d - \dot{q}$$

Define the sliding variable (composite error):

$$s = \dot{e} + \Lambda\, e$$

where $\Lambda = \Lambda^T \succ 0$ is a positive definite design matrix. Equivalently, define the reference velocity and reference acceleration:

\[\dot{q}_r = \dot{q}_d + \Lambda\,e = \dot{q} + s \] \[\ddot{q}_r = \ddot{q}_d + \Lambda\,\dot{e} \]

Notice that \(s = \dot{q}_r - \dot{q}\), so \(s \to 0\) implies \(\dot{q} \to \dot{q}_r\), which combined with the definition of \(\dot{q}_r\) implies both \(e \to 0\) and \(\dot{e} \to 0\) exponentially (since \(\dot{e} + \Lambda e = 0\) is a stable linear ODE).

The adaptive control law is:

\(\tau = Y(q, \dot{q}, \dot{q}_r, \ddot{q}_r)\,\hat{\theta} + K_D\, s \)

where:

- \(\hat{\theta}\) is the current estimate of the parameter vector \(\theta\).
- \(Y(q, \dot{q}, \dot{q}_r, \ddot{q}_r)\) is the regressor evaluated at the reference velocity and acceleration (not the actual \(\ddot{q}\), which we cannot measure cleanly).
- \(K_D = K_D^T \succ 0\) is a positive definite gain matrix providing damping on the sliding variable.

Important subtlety: We evaluate the regressor at \((\dot{q}_r, \ddot{q}_r)\) instead of \((\dot{q}, \ddot{q})\). This avoids the need to measure or numerically differentiate joint accelerations, and it ensures that the regressor is known (computable from \(q, \dot{q}, q_d, \dot{q}_d, \ddot{q}_d\)).

4.2.5 Parameter Update Law

The parameter estimates are updated according to:

\(\dot{\hat{\theta}} = \Gamma^{-1}\, Y^T\! s \)

where \(\Gamma = \Gamma^T \succ 0\) is the adaptation gain matrix. Define the parameter estimation error:

\[\tilde{\theta} = \hat{\theta} - \theta \]

Since \(\theta\) is constant, \(\dot{\tilde{\theta}} = \dot{\hat{\theta}}\).

Figure 8: Parameter convergence under adaptive control. Multiple parameter estimates starting from different initial conditions all converge toward the true parameter value * (dashed line). The adaptation law drives estimates toward values that minimize tracking error.

4.2.6 Lyapunov Stability Proof

4.2.6.1 Theorem (Adaptive Control Stability)

Under the adaptive control law \(\tau = Y\hat{\theta} + K_D s\) with the update law \(\dot{\hat{\theta}} = \Gamma^{-1} Y^T s\), the sliding variable \(s \to 0\) and consequently the tracking errors \(e \to 0\), \(\dot{e} \to 0\) as \(t \to \infty\).

Figure 7: Lyapunov stability concept. Left: the Lyapunov function V forms a bowl-shaped surface, and the system trajectory (pink spiral) descends toward the equilibrium. Right: V(t) monotonically decreases over time since V = -sKs is always non-positive.

Proof. Define the candidate Lyapunov function:

\[V = \frac{1}{2}\,s^T M(q)\,s + \frac{1}{2}\,\tilde{\theta}^T \Gamma\,\tilde{\theta} \]

Since \(M(q) \succ 0\) and \(\Gamma \succ 0\), both terms are strictly positive for \((s, \tilde{\theta}) \neq (0,0)\), so \(V\) is a valid positive definite candidate with \(V = 0\) iff \(s = 0\) and \(\tilde{\theta} = 0\).

Step 1: Time derivative of \(V\).

\[\dot{V} = s^T M(q)\,\dot{s} + \frac{1}{2}\,s^T \dot{M}(q)\,s + \tilde{\theta}^T \Gamma\,\dot{\tilde{\theta}} \]

Step 2: Simplify \(s^T M\dot{s} + \tfrac{1}{2}s^T\dot{M}s\) using skew-symmetry.
The Euler-Lagrange formulation guarantees that \(N := \dot{M} - 2C\) is skew-symmetric, i.e.\ \(s^T N s = 0\) for all \(s\). This gives \(\tfrac{1}{2}s^T\dot{M}s = s^T C s\), so: \[s^T M\dot{s} + \tfrac{1}{2}s^T\dot{M}s = s^T(M\dot{s} + Cs) \]

Step 3: Evaluate \(M\dot{s} + Cs\).
Since \(s = \dot{q}_r - \dot{q}\), we have \(\dot{s} = \ddot{q}_r - \ddot{q}\). Substituting \(M\ddot{q} = \tau - C\dot{q} - G\): \[M\dot{s} + Cs = M\ddot{q}_r - M\ddot{q} + Cs = M\ddot{q}_r + C\dot{q} + G - \tau + C(\dot{q}_r - \dot{q}) = M\ddot{q}_r + C\dot{q}_r + G - \tau \] By linearity-in-parameters, \(M\ddot{q}_r + C\dot{q}_r + G = Y\theta\). Substituting the control law \(\tau = Y\hat{\theta} + K_D s\): \[M\dot{s} + Cs = Y\theta - Y\hat{\theta} - K_D s = -Y(\hat{\theta} - \theta) - K_D s = -Y\tilde{\theta} - K_D s \]

Step 4: Substitute back into \(\dot{V}\).

\[\dot{V} = s^T(-Y\tilde{\theta} - K_D s) + \tilde{\theta}^T \Gamma\,\dot{\tilde{\theta}} = -s^T Y\tilde{\theta} - s^T K_D s + \tilde{\theta}^T \Gamma\,\dot{\tilde{\theta}} \]

Step 5: Cancel the cross term using the update law.
Substitute \(\dot{\tilde{\theta}} = \dot{\hat{\theta}} = \Gamma^{-1}Y^T s\): \[\tilde{\theta}^T \Gamma\,\dot{\tilde{\theta}} = \tilde{\theta}^T \Gamma\,(\Gamma^{-1}Y^T s) = \tilde{\theta}^T Y^T s = (Y\tilde{\theta})^T s = s^T Y\tilde{\theta} \] The cross terms cancel exactly: \[\dot{V} = -s^T Y\tilde{\theta} - s^T K_D s + s^T Y\tilde{\theta} = -s^T K_D s \]

Result: \[\boxed{\dot{V} = -s^T K_D\, s \leq 0} \] Since \(K_D \succ 0\), we have \(\dot{V} < 0\) whenever \(s \neq 0\), so \(V\) is non-increasing and bounded below by zero. By Barbalat's lemma, \(s(t) \to 0\) as \(t \to \infty\). Since \(s = \dot{e} + \Lambda e\) with \(\Lambda \succ 0\), this implies \(e(t) \to 0\) and \(\dot{e}(t) \to 0\). \(\square\)

Sign convention. We define \(\tilde{\theta} = \hat{\theta} - \theta\), so \(\dot{\tilde{\theta}} = \dot{\hat{\theta}}\). The Lyapunov weight on the parameter error term is \(\Gamma\) (not \(\Gamma^{-1}\)), which is why the update law \(\dot{\hat{\theta}} = \Gamma^{-1}Y^T s\) produces the exact cancellation \(\tilde{\theta}^T\Gamma(\Gamma^{-1}Y^T s) = s^T Y\tilde{\theta}\). Some textbooks define \(\tilde{\theta} = \theta - \hat{\theta}\); with that reversed convention the update law carries a negative sign, but the underlying physics is identical.

Result: \[\boxed{\dot{V} = -s^T K_D\, s \leq 0} \] Since \(K_D \succ 0\), \(\dot{V} < 0\) whenever \(s \neq 0\). By Barbalat's lemma, \(s(t) \to 0\) as \(t \to \infty\). Since \(s = \dot{e} + \Lambda e\) and \(\Lambda \succ 0\), this implies \(e(t) \to 0\) and \(\dot{e}(t) \to 0\).

Sign convention note: We use \(\tilde{\theta} = \hat{\theta} - \theta\), so \(\dot{\tilde{\theta}} = \dot{\hat{\theta}}\). With the Lyapunov function \(V = \tfrac{1}{2}s^T M s + \tfrac{1}{2}\tilde{\theta}^T\Gamma\tilde{\the
the update law that cancels the cross term is: \[\dot{\hat{\theta}} = \Gamma^{-1}\,Y^T\!s \] Some textbooks define \(\tilde{\theta} = \theta - \hat{\theta}\) (reversed). With that convention \(\dot{\tilde{\theta}} = -\dot{\hat{\theta}}\), and the update law is written as \(\dot{\hat{\theta}} = \Gamma^{-1} Y^T s\) with opposite sign in the cancellation. The physics is identical; only the bookkeeping differs. Always check which definition of \(\tilde{\theta}\) a textbook uses before comparing formulas.

4.2.6.2 Think-Pair-Share: Effect of Adaptation Gain

Consider the adaptation gain matrix \(\Gamma\). Discuss with your neighbour:

1. What happens if \(\Gamma\) is very large (fast adaptation)?
   - Parameter estimates change rapidly, which can cause large transient oscillations and potentially excite unmodelled dynamics. In the worst case, the system becomes unstable in practice (even though the theory guarantees stability for the nominal model).
2. What happens if \(\Gamma\) is very small (slow adaptation)?
   - Parameters converge very slowly. The controller behaves essentially like a PD controller (the \(K_D s\) term dominates) until the parameters have time to converge. Tracking errors may remain large for an extended period.
3. Does the Lyapunov proof guarantee that \(\hat{\theta} \to \theta\)?
   - No. The proof only guarantees \(s \to 0\) (and hence \(e \to 0\)). Parameter convergence requires persistent excitation of the regressor. If the trajectory does not excite all modes, some parameters may not converge to their true values, but tracking performance is still guaranteed.

4.3 Adaptive Control for 2-DOF Manipulator

4.3.1 System Setup

We now implement the full adaptive controller for the 2-DOF planar manipulator from Section 3.3. The true parameters are:

Parameter	Symbol	True Value	Initial Estimate
Link 1 mass	\(m_1\)	1.0 kg	1.0 kg (known)
Link 2 mass	\(m_2\)	2.0 kg	1.0 kg (50% error)
Link 1 length	\(l_1\)	1.0 m	1.0 m (known)
Link 2 length	\(l_2\)	1.0 m	1.0 m (known)
Gravity	\(g\)	9.81 m/sš	9.81 m/sš (known)

Even though only \(m_2\) is wrong, all composite parameters involving \(m_2\) are affected:

Composite Parameter	True Value	Initial Estimate
\(\theta_1 = m_1 l_1^2\)	1.0	1.0
\(\theta_2 = m_2 l_1^2\)	2.0	1.0
\(\theta_3 = m_2 l_2^2\)	2.0	1.0
\(\theta_4 = m_2 l_1 l_2\)	2.0	1.0
\(\theta_5 = (m_1+m_2)gl_1\)	29.43	19.62
\(\theta_6 = m_2 g l_2\)	19.62	9.81

4.3.2 Building the Regressor Matrix

We need the regressor evaluated at the reference quantities \((\dot{q}_r, \ddot{q}_r)\) instead of \((\dot{q}, \ddot{q})\). Define:

\[\dot{q}_r = \dot{q}_d + \Lambda(q_d - q), \qquad \ddot{q}_r = \ddot{q}_d + \Lambda(\dot{q}_d - \dot{q}) \]

The regressor \(Y(q, \dot{q}, \dot{q}_r, \ddot{q}_r) \in \mathbb{R}^{2 \times 6}\) is constructed by replacing every occurrence of \(\ddot{q}\) with \(\ddot{q}_r\) and every occurrence of \(\dot{q}\) in the Coriolis terms with a mix of \(\dot{q}\) and \(\dot{q}_r\) as appropriate. Specifically:

\[Y_{11} = \ddot{q}_{r1} \] \[Y_{12} = \ddot{q}_{r1} \] \[Y_{13} = \ddot{q}_{r1} + \ddot{q}_{r2} \] \[Y_{14} = 2\cos(q_2)\ddot{q}_{r1} + \cos(q_2)\ddot{q}_{r2} - \sin(q_2)\dot{q}_2\dot{q}_{r1} - \sin(q_2)(\dot{q}_1 + \dot{q}_2)\dot{q}_{r2} \] \[Y_{15} = \sin(q_1) \] \[Y_{16} = \sin(q_1 + q_2) \] \[Y_{21} = 0 \] \[Y_{22} = 0 \] \[Y_{23} = \ddot{q}_{r1} + \ddot{q}_{r2} \] \[Y_{24} = \cos(q_2)\ddot{q}_{r1} + \sin(q_2)\dot{q}_1 \dot{q}_{r1} \] \[Y_{25} = 0 \] \[Y_{26} = \sin(q_1 + q_2) \]

4.3.3 Complete Control Algorithm

Algorithm: Adaptive Control for 2-DOF Manipulator

Given: desired trajectory q_d(t), dq_d(t), ddq_d(t) Initialise: theta_hat = theta_hat_0 (initial parameter estimates) Design parameters: Lambda, K_D, Gamma (all positive definite) At each time step: 1. Measure q, dq 2. Compute errors: e = q_d - q de = dq_d - dq 3. Compute reference signals: dq_r = dq_d + Lambda * e ddq_r = ddq_d + Lambda * de 4. Compute sliding variable: s = de + Lambda * e (equivalently s = dq_r - dq) 5. Build regressor Y(q, dq, dq_r, ddq_r) [2x6 matrix] 6. Compute control torque: tau = Y * theta_hat + K_D * s 7. Update parameter estimates: d(theta_hat)/dt = Gamma_inv * Y' * s 8. Apply tau to the plant

4.3.4 MATLAB Implementation

%% Adaptive Control for 2-DOF Manipulator % True parameters m1 = 1; m2 = 2; l1 = 1; l2 = 1; g = 9.81; theta_true = [m1*l1^2; m2*l1^2; m2*l2^2; m2*l1*l2; (m1+m2)*g*l1; m2*g*l2]; % Initial estimates (m2_hat = 1 instead of 2) m2_hat = 1; theta_hat0 = [m1*l1^2; m2_hat*l1^2; m2_hat*l2^2; m2_hat*l1*l2; ... (m1+m2_hat)*g*l1; m2_hat*g*l2]; % Design parameters Lambda = 10*eye(2); % Sliding surface slope K_D = 50*eye(2); % Damping gain Gamma_inv = 5*eye(6); % Inverse adaptation gain % Desired trajectory qd = @(t) [0.5*sin(t); 0.5*cos(t)]; dqd = @(t) [0.5*cos(t); -0.5*sin(t)]; ddqd = @(t) [-0.5*sin(t); -0.5*cos(t)]; % State: [q1; q2; dq1; dq2; theta_hat(1:6)] x0 = [0; 0; 0; 0; theta_hat0]; tspan = 0:0.01:20; [t, X] = ode45(@(t,x) adaptive_manipulator(t, x, ... m1, m2, l1, l2, g, Lambda, K_D, Gamma_inv, qd, dqd, ddqd), tspan, x0); function dx = adaptive_manipulator(t, x, m1, m2, l1, l2, g, ... Lambda, K_D, Gamma_inv, qd, dqd, ddqd) q = x(1:2); dq = x(3:4); theta_hat = x(5:10); % True dynamics matrices c2 = cos(q(2)); s2 = sin(q(2)); M = [m1*l1^2 + m2*(l1^2+l2^2+2*l1*l2*c2), m2*(l2^2+l1*l2*c2); m2*(l2^2+l1*l2*c2), m2*l2^2]; C = [-m2*l1*l2*s2*dq(2), -m2*l1*l2*s2*(dq(1)+dq(2)); m2*l1*l2*s2*dq(1), 0]; G = [(m1+m2)*g*l1*sin(q(1)) + m2*g*l2*sin(q(1)+q(2)); m2*g*l2*sin(q(1)+q(2))]; % Errors and reference signals e = qd(t) - q; de = dqd(t) - dq; dqr = dqd(t) + Lambda*e; ddqr = ddqd(t) + Lambda*de; s = de + Lambda*e; % Build regressor Y(q, dq, dqr, ddqr) [2x6] s2q = sin(q(2)); c2q = cos(q(2)); s1 = sin(q(1)); s12 = sin(q(1)+q(2)); Y = zeros(2,6); Y(1,1) = ddqr(1); Y(1,2) = ddqr(1); Y(1,3) = ddqr(1) + ddqr(2); Y(1,4) = 2*c2q*ddqr(1) + c2q*ddqr(2) ... - s2q*dq(2)*dqr(1) - s2q*(dq(1)+dq(2))*dqr(2); Y(1,5) = s1; Y(1,6) = s12; Y(2,1) = 0; Y(2,2) = 0; Y(2,3) = ddqr(1) + ddqr(2); Y(2,4) = c2q*ddqr(1) + s2q*dq(1)*dqr(1); Y(2,5) = 0; Y(2,6) = s12; % Adaptive control law tau = Y*theta_hat + K_D*s; % Parameter update law dtheta_hat = Gamma_inv * Y' * s; % True dynamics ddq = M \ (tau - C*dq - G); dx = [dq; ddq; dtheta_hat]; end

4.4 Adaptive Control for Mobile Robot

4.4.1 Parametric Uncertainty in Mobile Robots

Figure 5: Top-down view of a differential-drive mobile robot. The robot (blue) follows an actual path (pink) while tracking a reference trajectory (dashed purple). Tracking errors e and e are measured in the body frame. The uncertain parameters r (wheel radius) and b (track width) are indicated.

A differential-drive mobile robot converts wheel angular velocities \((\omega_L, \omega_R)\) to linear and angular velocity via:

\[v = \frac{r}{2}(\omega_R + \omega_L), \qquad \omega = \frac{r}{b}(\omega_R - \omega_L) \]

where \(r\) is the wheel radius and \(b\) is the track width (distance between wheels). The kinematic model in task space is:

\[\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\phi} \end{bmatrix} = \begin{bmatrix} \cos\phi & 0 \\ \sin\phi & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \]

Uncertainties arise from:

- Wheel radius \(r\): Tire pressure variations, wear, and deformation under load change the effective radius.
- Track width \(b\): Manufacturing tolerances and wheel alignment errors affect the effective distance between contact patches.

4.4.2 Linearity in Parameters for the Kinematic Model

We can write the velocity-level kinematics in a form that is linear in the uncertain parameters. Define the control inputs as the wheel angular velocities \(u = [\omega_R, \omega_L]^T\). Then:

\[\begin{bmatrix} v \\ \omega \end{bmatrix} = \begin{bmatrix} \frac{1}{2}\omega_R + \frac{1}{2}\omega_L \\ \frac{1}{b}(\omega_R - \omega_L) \end{bmatrix} \cdot r \]

Define the parameter vector \(\theta_m = [r,\; r/b]^T\). Then:

\[\begin{bmatrix} v \\ \omega \end{bmatrix} = \underbrace{\begin{bmatrix} \frac{1}{2}(\omega_R + \omega_L) & 0 \\ 0 & \omega_R - \omega_L \end{bmatrix}}_{Y_m(u)} \begin{bmatrix} r \\ r/b \end{bmatrix} \]

4.4.3 Adaptive Tracking Controller

For trajectory tracking, we use a Lyapunov-based approach. Define the position error in the body frame:

\[\begin{bmatrix} e_x \\ e_y \\ e_\phi \end{bmatrix} = \begin{bmatrix} \cos\phi & \sin\phi & 0 \\ -\sin\phi & \cos\phi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_d - x \\ y_d - y \\ \phi_d - \phi \end{bmatrix} \]

The desired velocity commands are computed from a standard tracking law:

\[v_c = v_d \cos(e_\phi) + k_1 e_x \] \[\omega_c = \omega_d + v_d(k_2 e_y + k_3 \sin(e_\phi)) \]

where \(v_d, \omega_d\) are the desired linear and angular velocities along the reference trajectory, and \(k_1, k_2, k_3 > 0\) are gains.

With known \(r\) and \(b\), we would directly compute the wheel speeds. With unknown parameters, we use adaptive estimation. The actual velocities depend on uncertain \(\theta_m\):

\[\begin{bmatrix} v \\ \omega \end{bmatrix} = Y_m(u)\,\hat{\theta}_m + \text{estimation error} \]

The parameter update law is:

\[\dot{\hat{\theta}}_m = \Gamma_m\, Y_m^T\, \begin{bmatrix} e_x \\ e_\phi \end{bmatrix} \]

where \(\Gamma_m \succ 0\) is the adaptation gain for the mobile robot parameters.

4.4.4 MATLAB Implementation

%% Adaptive Control for Differential-Drive Mobile Robot % True parameters r_true = 0.05; % wheel radius [m] b_true = 0.3; % track width [m] theta_m_true = [r_true; r_true/b_true]; % Initial estimates (20% error on both) r_hat0 = 0.04; b_hat0 = 0.36; theta_m_hat0 = [r_hat0; r_hat0/b_hat0]; % Tracking gains k1 = 3; k2 = 5; k3 = 3; Gamma_m = diag([0.01, 0.05]); % Circular reference trajectory R_circ = 2; omega_circ = 0.5; xd = @(t) R_circ*cos(omega_circ*t); yd = @(t) R_circ*sin(omega_circ*t); phid = @(t) omega_circ*t + pi/2; vd = @(t) R_circ*omega_circ; wd = @(t) omega_circ; % State: [x; y; phi; theta_m_hat(1:2)] x0_mob = [R_circ; 0; pi/2; theta_m_hat0]; tspan = 0:0.01:30; [t_mob, X_mob] = ode45(@(t,x) adaptive_mobile(t, x, ... r_true, b_true, k1, k2, k3, Gamma_m, ... xd, yd, phid, vd, wd), tspan, x0_mob); function dx = adaptive_mobile(t, x, r_true, b_true, ... k1, k2, k3, Gamma_m, ... xd, yd, phid, vd, wd) px = x(1); py = x(2); phi = x(3); theta_hat = x(4:5); % Body-frame errors ex = cos(phi)*(xd(t)-px) + sin(phi)*(yd(t)-py); ey = -sin(phi)*(xd(t)-px) + cos(phi)*(yd(t)-py); ephi = phid(t) - phi; ephi = atan2(sin(ephi), cos(ephi)); % wrap to [-pi,pi] % Desired velocity commands v_c = vd(t)*cos(ephi) + k1*ex; w_c = wd(t) + vd(t)*(k2*ey + k3*sin(ephi)); % Inverse kinematics using estimated parameters r_hat = theta_hat(1); rb_hat = theta_hat(2); % r/b estimate omega_R = (v_c/r_hat) + (w_c/(2*rb_hat)); omega_L = (v_c/r_hat) - (w_c/(2*rb_hat)); % True velocities (using true parameters) v_actual = (r_true/2)*(omega_R + omega_L); w_actual = (r_true/b_true)*(omega_R - omega_L); % Kinematics dpx = v_actual*cos(phi); dpy = v_actual*sin(phi); dphi = w_actual; % Parameter update Y_m = [0.5*(omega_R+omega_L), 0; 0, omega_R-omega_L]; e_adapt = [ex; ephi]; dtheta = Gamma_m * Y_m' * e_adapt; dx = [dpx; dpy; dphi; dtheta]; end

4.5 Adaptive Control for Quadrotor

4.5.1 Quadrotor Dynamics with Unknown Parameters

Figure 6: Quadrotor UAV showing four rotors with alternating CW/CCW rotation, thrust vectors (green arrows), body frame axes (x, y, z), weight force mg, and Euler angles. Unknown parameters include mass m and moments of inertia.

The translational dynamics of a quadrotor are:

\[m\ddot{p} = \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} + R\begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix} \]

where \(m\) is the mass, \(p = [x,y,z]^T\) is position, \(R \in SO(3)\) is the rotation matrix, and \(T\) is the total thrust. The rotational dynamics (simplified, Euler angles \(\phi, \theta, \psi\)) are:

\[I_{xx}\ddot{\phi} = \tau_\phi, \qquad I_{yy}\ddot{\theta} = \tau_\theta, \qquad I_{zz}\ddot{\psi} = \tau_\psi \]

The unknown parameters are:

\[\theta_d = \begin{bmatrix} m \\ m\,g \\ I_{xx} \\ I_{yy} \\ I_{zz} \end{bmatrix} \]

4.5.2 Linearity in Parameters

The translational dynamics can be written as:

\[m\ddot{p} + \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix} = R\begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix} \]

Define the position error \(e_p = p_d - p\), sliding variable \(s_p = \dot{e}_p + \Lambda_p e_p\), and reference acceleration \(\ddot{p}_r = \ddot{p}_d + \Lambda_p \dot{e}_p\). The desired thrust vector is:

\[F_{\mathrm{des}} = \hat{m}\,\ddot{p}_r + \begin{bmatrix} 0 \\ 0 \\ \hat{m}\hat{g} \end{bmatrix} + K_{D,p}\,s_p \]

This is linear in \([\hat{m},\; \hat{m}\hat{g}]^T\):

\[F_{\mathrm{des}} = Y_p\,\hat{\theta}_p + K_{D,p}\,s_p \]

where:

\[Y_p = \begin{bmatrix} \ddot{p}_{r,x} & 0 \\ \ddot{p}_{r,y} & 0 \\ \ddot{p}_{r,z} & 1 \end{bmatrix}, \qquad \hat{\theta}_p = \begin{bmatrix} \hat{m} \\ \hat{m}\hat{g} \end{bmatrix} \]

The total thrust magnitude is \(T = \|F_{\mathrm{des}}\|\), and the desired orientation is extracted from \(F_{\mathrm{des}}\).

4.5.3 Attitude Adaptive Control

For the attitude loop, define the orientation error \(e_a = [\phi_d - \phi,\; \theta_d - \theta_0,\; \psi_d - \psi]^T\), the sliding variable \(s_a = \dot{e}_a + \Lambda_a e_a\), and the reference angular acceleration \(\ddot{\eta}_r = \ddot{\eta}_d + \Lambda_a \dot{e}_a\). The adaptive torque is:

\[\tau_a = Y_a\,\hat{\theta}_a + K_{D,a}\,s_a \]

where:

\[Y_a = \mathrm{diag}(\ddot{\eta}_{r,1},\; \ddot{\eta}_{r,2},\; \ddot{\eta}_{r,3}), \qquad \hat{\theta}_a = \begin{bmatrix} \hat{I}_{xx} \\ \hat{I}_{yy} \\ \hat{I}_{zz} \end{bmatrix} \]

4.5.4 Update Laws

\[\dot{\hat{\theta}}_p = \Gamma_p^{-1}\,Y_p^T\,s_p, \qquad \dot{\hat{\theta}}_a = \Gamma_a^{-1}\,Y_a^T\,s_a \]

4.5.5 MATLAB Implementation

%% Adaptive Control for Quadrotor % True parameters m_true = 1.5; g_val = 9.81; Ixx_true = 0.03; Iyy_true = 0.03; Izz_true = 0.06; theta_p_true = [m_true; m_true*g_val]; theta_a_true = [Ixx_true; Iyy_true; Izz_true]; % Initial estimates (mass off by 50%, inertias off by 40%) m_hat0 = 1.0; g_hat = 9.81; theta_p_hat0 = [m_hat0; m_hat0*g_hat]; theta_a_hat0 = [0.05; 0.05; 0.10]; % Gains Lambda_p = 2*eye(3); K_Dp = 15*eye(3); Gamma_p_inv = 0.5*eye(2); Lambda_a = 5*eye(3); K_Da = 8*eye(3); Gamma_a_inv = 0.1*eye(3); % State: [x,y,z, dx,dy,dz, phi,theta,psi, dphi,dtheta,dpsi, theta_p_hat(2), theta_a_hat(3)] x0_quad = zeros(17,1); x0_quad(13:14) = theta_p_hat0; x0_quad(15:17) = theta_a_hat0; % Helical trajectory pd = @(t) [2*cos(0.5*t); 2*sin(0.5*t); 0.2*t]; dpd = @(t) [-sin(0.5*t); cos(0.5*t); 0.2]; ddpd = @(t) [-0.5*cos(0.5*t); -0.5*sin(0.5*t); 0]; psid = @(t) 0; dpsid = @(t) 0; ddpsid = @(t) 0; tspan = 0:0.01:30; [t_q, X_q] = ode45(@(t,x) adaptive_quad(t, x, m_true, g_val, ... Ixx_true, Iyy_true, Izz_true, Lambda_p, K_Dp, Gamma_p_inv, ... Lambda_a, K_Da, Gamma_a_inv, pd, dpd, ddpd, psid, dpsid, ddpsid), ... tspan, x0_quad);

The full implementation, including the ODE function for the quadrotor adaptive controller, is provided in the companion MATLAB file `adaptive_control_simulation.m`.

4.6 MATLAB Implementation & Activities

4.6.1 Activity 1: Parameter Convergence (10 minutes)

4.6.1.1 Instructions

1. Open `adaptive_control_simulation.m` in MATLAB.

2. Run Section 1 (2-DOF Manipulator Adaptive Control). The true mass is \(m_2 = 2\) kg, but the initial estimate is \(\hat{m}_2 = 1\) kg.
3. Observe the six subplots of parameter estimates \(\hat{\theta}_i(t)\) versus the true values \(\theta_i\).
4. Answer: Do all parameters converge to their true values? If not, which ones do not, and why?

Expected observation: Parameters that appear in the regressor columns excited by the sinusoidal trajectory will converge. Parameters associated with modes not excited may converge to incorrect values — but tracking is still achieved. This is the difference between stability (guaranteed by Lyapunov) and parameter convergence (requires persistent excitation).

4.6.2 Activity 2: Adaptive vs Computed Torque Under Uncertainty (15 minutes)

4.6.2.1 Instructions

1. Run Section 4 of the MATLAB file, which compares:
   - Computed torque using incorrect parameters (\(\hat{m}_2 = 1\) kg, true \(m_2 = 2\) kg).
   - Adaptive control with the same initial parameter estimates.
2. Compare the tracking error plots side-by-side.
3. Answer these questions:
   - What is the steady-state tracking error for computed torque with wrong parameters?
   - How long does adaptive control take to reduce the error to below the computed torque error?
   - Is there a transient period where adaptive control is actually worse? Why?

4.6.3 Quiz

4.6.3.1 Quick Quiz: Regressor Construction

Question: Consider the gravity term for link 2 of the 2-DOF manipulator:

\[G_2 = m_2\,g\,l_2\,\sin(q_1 + q_2) \]

This is the second element of the gravity vector \(G(q)\). Define a composite parameter \(\theta_6 = m_2 g l_2\). What is the corresponding regressor entry?

Answer: The regressor entry for the second row (joint 2 equation) and the column corresponding to \(\theta_6\) is simply:

\[Y_{2,6} = \sin(q_1 + q_2) \]

Because \(G_2 = \theta_6 \cdot \sin(q_1 + q_2)\). The regressor separates the known kinematic function \(\sin(q_1+q_2)\) from the unknown parameter \(\theta_6 = m_2 g l_2\).

Follow-up: Does the first row (joint 1 equation) also have a \(\theta_6\) entry? Yes:

\[Y_{1,6} = \sin(q_1 + q_2) \]

because the gravity contribution of link 2 to joint 1 is also \(m_2 g l_2 \sin(q_1+q_2)\).

4.7 Summary and Next Steps

4.7.1 Summary — Control Methods Comparison

Property	Computed Torque (Week 1)
Core idea	Cancel nonlinearities with exact model, reduce to linear error
Model requirement	Exact knowledge of \(M(q), C(q,\dot{q}), G(q)\)
Handles uncertainty?	No — performance degrades with parameter errors
Handles constraints?	No (must add saturation externally)
Computation	Low — one matrix inversion per step
Stability guarantee	Exponential (with exact model)
Parameter convergence	N/A (parameters assumed known)
Key design parameters	\(K_p, K_v\) (PD gains)
Best suited for	Well-modelled systems with known parameters

Looking ahead — Week 4: Robust Control. What if the uncertainty is not parametric but includes unmodelled dynamics, disturbances, or unstructured uncertainty? Robust control (e.g., sliding mode control, \(H_\infty\)) provides guaranteed performance bounds without requiring online estimation. We will see how robust and adaptive approaches can be combined.

M.Sc. Control for Robots — University of West London — AY 2025–26

Week 3: Adaptive Control Lecture Notes

5 Robust Control (Week 5)

Control for Robots — M.Sc. Robotics & Autonomous Systems

University of West London AY 2025–26 3-Hour Session

5.1 Overview

5.1.1 Week 4 Recall: Adaptive Control — Exam Practice (20 min)

5.1.1.1 Question 1 — Lyapunov Stability Proof for Adaptive Control

Consider a 2-DOF manipulator whose dynamics are

\[M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau \]

The dynamics are linear in parameters:

\[M(q)\ddot{q}_r + C(q,\dot{q})\dot{q}_r + G(q) = Y(q,\dot{q},\dot{q}_r,\ddot{q}_r)\,\theta \]

where \(\dot{q}_r = \dot{q}_d - \Lambda e\), \(\ddot{q}_r = \ddot{q}_d - \Lambda \dot{e}\), and \(e = q - q_d\).

The adaptive control law is

\[\tau = Y(q,\dot{q},\dot{q}_r,\ddot{q}_r)\,\hat{\theta} - K_D\,s \]

where \(s = \dot{e} + \Lambda e = \dot{q} - \dot{q}_r\). The parameter update law is

\[\dot{\hat{\theta}} = -\Gamma^{-1}\,Y^T s \]

Task: Using the Lyapunov candidate

\[V = \tfrac{1}{2}\,s^T M(q)\,s + \tfrac{1}{2}\,\tilde{\theta}^T \Gamma\,\tilde{\theta}, \qquad \tilde{\theta} = \theta - \hat{\theta} \]

5.1 Overview 65
5.1.1 Week 4 Recall: Adaptive Control — Exam Practice (20 min) 65
5.1.2 Learning Outcomes . . . 68
5.1.3 Session Roadmap (3 Hours) 68
5.2 Robust Control Theory . 69
5.2.1 Why Robust Control? . 69
5.2.2 Problem Setup 70
5.2.3 Robust Computed Torque + Robust Term 70
5.2.4 Robust Term Design . . 71
5.2.5 Lyapunov Stability Proof 71
5.3 Robust Control for 2-DOF Manipulator . . . 73
5.3.1 System Description . . . 73
5.3.2 Uncertainty Model 74
5.3.3 Robust Controller 74
5.3.4 Guaranteed Tracking Error Bound 74
5.3.5 MATLAB Implementation 75
5.4 Robust Control for Mobile Robot 76
5.4.1 Sources of Disturbance . 76
5.4.2 Disturbed Unicycle Model 76
5.4.3 Robust Tracking Controller 76
5.4.4 Stability Analysis 77
5.4.5 MATLAB Implementation 77
5.5 Robust Control for Quadrotor Drone 78
5.5.1 Wind Disturbance Model 78
5.5.2 Simplified Model for Control Design 78
5.5.3 Robust Position Controller 78
5.5.4 Robust Attitude Controller 79
5.5.5 MATLAB Implementation 79
5.6 Summary and Next Steps 80

derive \(\dot{V}\) step by step and show that \(\dot{V} = -s^T K_D\,s \le 0\). Explain why this guarantees bounded parameter estimates.

5.1.1.2 Solution Outline

Step 1. Differentiate \(V\):

\[\dot{V} = s^T M(q)\,\dot{s} + \tfrac{1}{2}\,s^T \dot{M}(q)\,s + \tilde{\theta}^T \Gamma\,\dot{\tilde{\theta}} \]

Step 2. Find the closed-loop error dynamics. Substitute the control law into the manipulator equation:

\[M(q)\dot{s} + C(q,\dot{q})\,s = Y\,\tilde{\theta} - K_D\,s \]

This follows because \(M\ddot{q} + C\dot{q} + G = \tau\) and \(M\ddot{q}_r + C\dot{q}_r + G = Y\theta\), so subtracting gives \(M\dot{s} + Cs = Y\theta - \tau = Y\theta - Y\hat{\theta} + K_D s = Y\tilde{\theta} - K_D s\). Wait — note the sign: \(\tau = Y\hat{\theta} - K_D s\), so \(Y\theta - \tau = Y\theta - Y\hat{\theta} + K_D s = Y\tilde{\theta} + K_D s\). Hence:

\[M(q)\dot{s} = -C(q,\dot{q})\,s + Y\,\tilde{\theta} - K_D\,s \]

Step 3. Substitute into \(\dot{V}\):

\[\dot{V} = s^T\!\bigl[-C s + Y\tilde{\theta} - K_D s\bigr] + \tfrac{1}{2}\,s^T \dot{M}\,s + \tilde{\theta}^T \Gamma\,\dot{\tilde{\theta}} \] \[= s^T\!\bigl[\tfrac{1}{2}(\dot{M} - 2C)\bigr]s + s^T Y\,\tilde{\theta} - s^T K_D\,s + \tilde{\theta}^T \Gamma\,\dot{\tilde{\theta}} \]

Step 4. Use the skew-symmetry property \(\dot{M} - 2C\) is skew-symmetric, so \(s^T(\dot{M}-2C)s = 0\):

\[\dot{V} = s^T Y\,\tilde{\theta} - s^T K_D\,s + \tilde{\theta}^T \Gamma\,\dot{\tilde{\theta}} \]

Step 5. Note \(\dot{\tilde{\theta}} = -\dot{\hat{\theta}} = \Gamma^{-1} Y^T s\) (since \(\theta\) is constant). Therefore:

\[\tilde{\theta}^T \Gamma\,\dot{\tilde{\theta}} = \tilde{\theta}^T \Gamma\,\Gamma^{-1} Y^T s = \tilde{\theta}^T Y^T s = (Y\tilde{\theta})^T s = s^T Y\tilde{\theta} \]

Wait — this would give \(+2s^T Y\tilde{\theta}\), which is incorrect. Let us be careful with the sign convention. We have \(\dot{\hat{\theta}} = -\Gamma^{-1} Y^T s\), and \(\tilde{\theta} = \theta - \hat{\theta}\), so \(\dot{\tilde{\theta}} = -\dot{\hat{\theta}} = \Gamma^{-1} Y^T s\).

Then \(\tilde{\theta}^T \Gamma \dot{\tilde{\theta}} = \tilde{\theta}^T \Gamma \Gamma^{-1} Y^T s = \tilde{\theta}^T Y^T s\). And from Step 4 we also have \(+s^T Y\tilde{\theta}\). So we get \(2 s^T Y\tilde{\theta}\).

The resolution: the standard adaptive law uses \(\dot{\hat{\theta}} = \Gamma^{-1} Y^T s\) (positive sign). Then \(\dot{\tilde{\theta}} = -\Gamma^{-1} Y^T s\), giving \(\tilde{\theta}^T \Gamma \dot{\tilde{\theta}} = -s^T Y\tilde{\theta}\), and the cross terms cancel:

\[\dot{V} = s^T Y\tilde{\theta} - s^T K_D s - s^T Y\tilde{\theta} = -s^T K_D s \le 0 \]

Step 6. Conclusion: Since \(V \ge 0\) and \(\dot{V} \le 0\), the function \(V(t)\) is non-increasing. Because \(V\) includes \(\tfrac{1}{2}\tilde{\theta}^T\Gamma\tilde{\theta}\), boundedness of \(V\) implies boundedness of \(\tilde{\theta}\) and hence \(\hat{\theta}\). By Barbalat's lemma, \(s \to 0\) as \(t \to \infty\).

5.1.1.3 Question 2 — Regressor and Parameter Vector for Gravity Terms

For the same 2-DOF manipulator, the gravity vector is

\[G(q) = \begin{bmatrix} (m_1+m_2)\,g\,l_1\cos q_1 + m_2\,g\,l_2\cos(q_1+q_2) \\ m_2\,g\,l_2\cos(q_1+q_2) \end{bmatrix} \]

Write the regressor matrix \(Y_G\) and parameter vector \(\theta_G\) such that \(G(q) = Y_G(q)\,\theta_G\). Identify what each element of \(\theta_G\) represents physically.

5.1.1.4 Solution

We seek a factorisation \(G(q) = Y_G(q)\,\theta_G\). Observe that the gravity terms depend on two independent parameter groups:

- \(\theta_{G,1} = (m_1 + m_2)\,g\,l_1\) — gravitational torque parameter for link 1 (total mass times gravity times link-1 length)
- \(\theta_{G,2} = m_2\,g\,l_2\) — gravitational torque parameter for link 2 (link-2 mass times gravity times link-2 length)

Then:

\[G(q) = \underbrace{\begin{bmatrix} \cos q_1 & \cos(q_1+q_2) \\ 0 & \cos(q_1+q_2) \end{bmatrix}}_{Y_G(q)} \underbrace{\begin{bmatrix} (m_1+m_2)\,g\,l_1 \\ m_2\,g\,l_2 \end{bmatrix}}_{\theta_G} \]

Physical meaning:

- \(\theta_{G,1} = (m_1+m_2)gl_1\): the maximum gravitational torque that link 1 must support (both link masses hanging at full link-1 lever arm).
- \(\theta_{G,2} = m_2 g l_2\): the maximum gravitational torque due to link 2 alone (link-2 mass at full link-2 lever arm).

Time Limit: 20 minutes to complete both questions. Work individually first, then discuss with your neighbour.

5.1.2 Learning Outcomes

- Understand robust control philosophy: guarantee performance despite bounded uncertainties without estimating them
- Derive a robust control law for a manipulator subject to bounded disturbances
- Understand \(\mathcal{H}_\infty\) control concepts and their application to robotics
- Apply robust PD+ control with known disturbance bounds
- Implement robust control for three systems: manipulator, mobile robot, drone
- Compare robust control vs adaptive control vs computed torque control

5.1.3 Session Roadmap (3 Hours)

0:00 – 0:20

5.1.3.1 Previous Week Exam Practice

Adaptive control recall: Lyapunov proof and regressor identification.

0:20 – 1:00

5.1.3.2 Robust Control Theory

Philosophy, problem setup, robust term design, Lyapunov proof, \(\mathcal{H}_\infty\) concepts.

1:00 – 1:20

5.1.3.3 Robust Control for Manipulator

Full derivation for 2-DOF arm, uncertainty bounds, MATLAB implementation.

1:20 – 1:30

5.1.3.4 Break

10-minute break.

1:30 – 2:00

5.1.3.5 Robust Control for Mobile Robot

Wheel slip, terrain disturbances, robust unicycle tracking.

2:00 – 2:30

5.1.3.6 Robust Control for Drone

Wind disturbance rejection, robust position and attitude control.

2:30 – 3:00

5.1.3.7 MATLAB & Activities

Hands-on simulation, comparison exercises, quick quiz.

5.2 Robust Control Theory

5.2.1 Why Robust Control?

In Weeks 1–3 we developed increasingly sophisticated controllers:

- Computed Torque (Week 1): requires exact model — any mismatch degrades performance.
- MPC (Week 2): optimises over a prediction horizon using a model — sensitive to model error.
- Adaptive Control (Week 3): estimates unknown parameters online — but requires linearity in parameters and persistent excitation.

Robust control takes a different philosophy:

Core Idea: Instead of estimating the uncertainty, assume it is bounded and design a controller that guarantees performance for all disturbances within that bound. No online estimation, no model identification — just guaranteed worst-case performance.

Figure 1: Robust control block diagram showing the nominal plant with additive uncertainty block and disturbance d(t). The robust controller compensates for all bounded uncertainties without estimating them.

5.2.1.1 When to Use Robust Control

- Uncertainty bound is known (e.g., wind 10 N)
- Parameters change too fast for adaptation
- Safety-critical: need guaranteed bounds on tracking error
- Simple implementation desired (no online estimation)

5.2.1.2 When NOT to Use Robust Control

- Uncertainty bound is unknown or very conservative
- Performance must improve over time (use adaptive)
- Large parametric uncertainty (robust term becomes too large)
- Tight energy constraints (robust terms use more control effort)

5.2.2 Problem Setup

Consider the general manipulator dynamics with a bounded disturbance:

$$ M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} + G(q) = \tau + d(t), \qquad \|d(t)\| \le d_{\max} $$

Here $d(t) \in \mathbb{R}^n$ represents all unmodelled effects: external forces, friction, payload changes, model mismatch, sensor noise coupling, etc. The only assumption is a known upper bound $d_{\max}$.

Additionally, suppose we have an imperfect model: estimates $\hat{M}(q)$, $\hat{C}(q,\dot{q})$, $\hat{G}(q)$ with bounded errors:

$$ \Delta M = M - \hat{M}, \quad \Delta C = C - \hat{C}, \quad \Delta G = G - \hat{G} $$

Then the total uncertainty is:

$$ \eta(t) = \Delta M\,\ddot{q} + \Delta C\,\dot{q} + \Delta G + d(t) $$

If we can bound $\|\eta(t)\| \le \rho(q,\dot{q},t)$ for a known function ρ, we can design a robust controller.

Figure 2: Three types of uncertainty: (a) parametric -- parameters lie in known intervals; (b) unstructured/additive -- frequency-dependent uncertainty envelope around the nominal response; (c) matched disturbance -- bounded signal entering at the plant input.

5.2.3 Robust Computed Torque + Robust Term

The control law has two components:

$$ \tau = \underbrace{\hat{M}(q)\bigl[\ddot{q}_d + K_D\,\dot{e} + K_P\,e\bigr] + \hat{C}(q,\dot{q})\,\dot{q} + \hat{G}(q)}_{\text{nominal computed torque}} + \underbrace{\tau_{\text{robust}}}_{\text{robust term}} $$

where $e = q_d - q$ is the tracking error.

5.2.3.1 Closed-Loop Error Dynamics

Substituting into the manipulator equation:

$$ M\ddot{q} + C\dot{q} + G = \hat{M}(\ddot{q}_d + K_D\dot{e} + K_P e) + \hat{C}\dot{q} + \hat{G} + \tau_{\text{robust}} + d $$

Rearranging with $\ddot{e} = \ddot{q}_d - \ddot{q}$:

$$ M\ddot{e} + (C + \hat{M}K_D)\dot{e} + \hat{M}K_P e = \underbrace{\Delta M\ddot{q} + \Delta C\dot{q} + \Delta G + d(t)}_{\eta(t)} - \tau_{\text{robust}} $$

5.2.4 Robust Term Design

Define the sliding variable (same as in adaptive control):

\[s = \dot{e} + \Lambda\,e \]

where \(\Lambda \succ 0\) is a positive-definite gain matrix. Then the robust controller is:

\[\tau = \hat{M}(q)\,\ddot{q}_r + \hat{C}(q,\dot{q})\,\dot{q}_r + \hat{G}(q) - K_D\,s + \tau_{\text{robust}} \]

where \(\dot{q}_r = \dot{q}_d - \Lambda e\), \(\ddot{q}_r = \ddot{q}_d - \Lambda\dot{e}\) (the reference velocity and acceleration).

The robust term is designed as:

\[\tau_{\text{robust}} = \begin{cases} -\rho(t)\,\dfrac{s}{\|s\|} & \text{if } \|s\| \ge \epsilon \\ -\dfrac{\rho(t)}{\epsilon}\,s & \text{if } \|s\| < \epsilon \end{cases} \]

where \(\rho(t) \ge \|\eta(t)\|\) is the uncertainty bound and \(\epsilon > 0\) is a small boundary-layer thickness to avoid chattering. The first case is the discontinuous (sliding-mode-like) form; the second is a continuous approximation inside a boundary layer.

Chattering Problem: The pure discontinuous form \(\tau_{\text{robust}} = -\rho\,s/\|s\|\) causes high-frequency switching (chattering) in actuators. The boundary layer \(\epsilon\) trades off robustness for smoothness. Smaller \(\epsilon\) = more robust but more chattering.

5.2.5 Lyapunov Stability Proof

5.2.5.1 Theorem: Robust Tracking with Bounded Error

Consider the Lyapunov candidate:

\[V = \tfrac{1}{2}\,s^T M(q)\,s \]

Step 1. Differentiate:

\[\dot{V} = s^T M\dot{s} + \tfrac{1}{2}s^T\dot{M}s \]

Step 2. The closed-loop error dynamics give:

\[M\dot{s} = -Cs + \eta(t) - K_D s + \tau_{\text{robust}} \]

Substituting:

\[\dot{V} = s^T(-Cs + \eta - K_D s + \tau_{\text{robust}}) + \tfrac{1}{2}s^T\dot{M}s \] \[= \tfrac{1}{2}s^T(\dot{M}-2C)s + s^T\eta - s^T K_D s + s^T\tau_{\text{robust}} \]

Step 3. By skew-symmetry, \(s^T(\dot{M}-2C)s = 0\):

\[\dot{V} = s^T\eta - s^T K_D s + s^T\tau_{\text{robust}} \]

Step 4. For the discontinuous case (\(\|s\| \ge \epsilon\)), \(\tau_{\text{robust}} = -\rho\,s/\|s\|\):

\[s^T\tau_{\text{robust}} = -\rho\,\frac{s^Ts}{\|s\|} = -\rho\,\|s\| \]

And \(s^T\eta \le \|s\|\,\|\eta\| \le \|s\|\,\rho\) by Cauchy–Schwarz and the bound \(\|\eta\| \le \rho\). Therefore:

\[\dot{V} \le \|s\|\rho - s^T K_D s - \rho\|s\| = -s^T K_D s \le 0 \]

Step 5. For the boundary-layer case (\(\|s\| < \epsilon\)), the tracking error is already within the boundary layer. The ultimate bound on \(\|s\|\) is \(\epsilon\), which translates to a bounded tracking error.

Result: With the robust term, the sliding variable \(s\) converges to the boundary layer \(\|s\| \le \epsilon\), guaranteeing that tracking error is ultimately bounded. With \(s = \dot{e}+\Lambda e \to 0\), we get \(e \to 0\) exponentially inside the boundary layer.

Figure 4: Robust stability comparison. Left: nominal system without disturbance -- the trajectory converges asymptotically to the origin along Lyapunov contours. Right: robust system with bounded disturbance -- the trajectory converges to a bounded ball of radius (the ultimate bound), which depends on the boundary layer thickness and gains.

5.2.5.2 3.6 \(\mathcal{H}_\infty\) Control Concepts

\(\mathcal{H}_\infty\) (H-infinity) control is the frequency-domain counterpart to the Lyapunov-based robust control above. The key idea:

Goal: Design a controller \(K\) that minimises the worst-case gain from disturbance \(w\) to performance output \(z\): \[\min_K \;\|T_{zw}(s)\|_\infty = \min_K \;\sup_\omega \;\bar{\sigma}\!\bigl(T_{zw}(j\omega)\bigr) \] where \(\bar{\sigma}\) is the maximum singular value.

5.2.5.3 Standard Plant Formulation

The generalised plant \(P\) connects disturbance \(w\), control \(u\), performance output \(z\), and measurement \(y\):

\[\begin{bmatrix} z \\ y \end{bmatrix} = \begin{bmatrix} P_{11} & P_{12} \\ P_{21} & P_{22} \end{bmatrix} \begin{bmatrix} w \\ u \end{bmatrix} \]

The closed-loop transfer function from \(w\) to \(z\) is:

\[T_{zw} = P_{11} + P_{12}\,K\,(I - P_{22}\,K)^{-1}\,P_{21} \]

Figure 8: H-infinity standard plant configuration. Left: the generalized plant P is partitioned into four sub-blocks connecting disturbance w, control u, performance output z, and measurement y. The controller K closes the loop from y to u. Right: the closed-loop objective is to minimize the H-infinity norm ||T_wz||, which represents the worst-case gain from disturbance to performance across all frequencies.

5.2.5.4 Riccati-Based Solution (for Linear Systems)

For a linear system \(\dot{x} = Ax + B_1 w + B_2 u\), \(z = C_1 x + D_{12} u\), \(y = C_2 x + D_{21}w\), the \(\mathcal{H}_-\infty\) controller exists if and only if two Algebraic Riccati Equations (AREs) have stabilising solutions \(X \ge 0\) and \(Y \ge 0\):

\[A^TX + XA + C_1^TC_1 + X\!\left(\gamma^{-2}B_1B_1^T - B_2B_2^T\right)\!X = 0 \] \[AY + YA^T + B_1B_1^T + Y\!\left(\gamma^{-2}C_1^TC_1 - C_2^TC_2\right)\!Y = 0 \]

and \(\rho(XY) < \gamma^2\), where \(\gamma\) is the desired attenuation level.

5.2.5.5 Practical Interpretation for Robotics

For nonlinear robotic systems, pure \(\mathcal{H}_\infty\) is often applied via:

- Linearisation + \(\mathcal{H}_\infty\): linearise at operating points, design \(\mathcal{H}_\infty\) controllers, use gain scheduling.
- Nonlinear \(\mathcal{H}_\infty\): use Hamilton–Jacobi–Isaacs (HJI) equations (computationally expensive).
- Practical approach: use the Lyapunov-based robust controller (Section 3.4) which achieves similar worst-case guarantees for nonlinear systems without frequency-domain machinery.

5.2.5.6 Think – Pair – Share

Question: What is the fundamental difference between adaptive control and robust control? When would you prefer one over the other?

Discussion points:

- Adaptive: estimates parameters performance improves over time, but needs PE and structure (linearity in parameters).
- Robust: assumes bounded uncertainty guaranteed worst-case performance, but conservative (over-designs for typical case).
- If the uncertainty is structured and slowly varying adaptive. If it is fast/unknown structure but bounded robust.
- Can be combined: robust-adaptive control uses adaptation with a robust modification for unstructured uncertainty.

5.3 Robust Control for 2-DOF Manipulator

5.3.1 System Description

Consider the 2-DOF planar manipulator from previous weeks:

\[M(q) = \begin{bmatrix} (m_1+m_2)l_1^2 + m_2 l_2^2 + 2m_2 l_1 l_2 \cos q_2 & m_2 l_2^2 + m_2 l_1 l_2 \cos q_2 \\ m_2 l_2^2 + m_2 l_1 l_2 \cos q_2 & m_2 l_2^2 \end{bmatrix} \] \[

C(q,\dot{q}) = \begin{bmatrix} -m_2 l_1 l_2 \dot{q}_2 \sin q_2 & -m_2 l_1 l_2(\dot{q}_1+\dot{q}_2)\sin q_2 \\ m_2 l_1 l_2 \dot{q}_1 \sin q_2 & 0 \end{bmatrix} \] \[G(q) = \begin{bmatrix} (m_1+m_2)g l_1 \cos q_1 + m_2 g l_2 \cos(q_1+q_2) \\ m_2 g l_2 \cos(q_1+q_2) \end{bmatrix} \]

Figure 3: 2-DOF planar manipulator showing nominal configuration (left, blue) versus perturbed configuration (right, red) with uncertain masses m, m and external disturbance d(t). Dashed outlines indicate the uncertain mass envelopes.

5.3.2 Uncertainty Model

Suppose we have nominal parameters \(\hat{m}_1, \hat{m}_2, \hat{l}_-1, \hat{l}_2\) with known error bounds:

\[|m_i - \hat{m}_i| \le \delta_{m_i}, \quad |l_i - \hat{l}_i| \le \delta_{l_i} \]

Plus an external disturbance \(d(t)\) with \(\|d(t)\| \le d_{\max}\).

The total uncertainty bound can be computed as:

\[\rho(q,\dot{q},\ddot{q}_r) = \|\Delta M\|_F \|\ddot{q}_r\| + \|\Delta C\|_F \|\dot{q}\| + \|\Delta G\| + d_{\max} \]

In practice, we often use a constant upper bound \(\rho_0\) that covers the worst case over the workspace.

5.3.3 Robust Controller

Define \(s = \dot{e} + \Lambda e\), \(\dot{q}_r = \dot{q}_d - \Lambda e\), \(\ddot{q}_r = \ddot{q}_d - \Lambda\dot{e}\). The control law is:

\[\tau = \hat{M}(q)\,\ddot{q}_r + \hat{C}(q,\dot{q})\,\dot{q}_r + \hat{G}(q) - K_D\,s - \rho_0\,\frac{s}{\|s\| + \epsilon} \]

Note: we use \(\|s\| + \epsilon\) in the denominator (instead of switching) for a smooth approximation that avoids chattering entirely.

5.3.4 Guaranteed Tracking Error Bound

From the Lyapunov analysis, the ultimate bound on the sliding variable is:

\[\|s\|_{\text{ult}} \le \frac{\rho_0\,\epsilon}{\lambda_{\min}(K_-D)} \]

Since \(s = \dot{e} + \Lambda e\), the position tracking error is ultimately bounded by:

\[\|e\|_{\text{ult}} \le \frac{\rho_0\,\epsilon}{\lambda_{\min}(K_-D)\,\lambda_{\min}(\Lambda)} \]

Design Rule: To achieve a desired tracking accuracy \(\|e\| \le e_{\max}\), choose \(K_D\) and \(\Lambda\) large enough and \(\epsilon\) small enough such that \(\rho_0\,\epsilon / (\lambda_{\min}(K_D)\,\lambda_{\min}(\Lambda)) \le e_{\max}\).

5.3.5 MATLAB Implementation

%% Robust Computed Torque Control for 2-DOF Manipulator % Parameters m1 = 2.0; m2 = 1.5; l1 = 1.0; l2 = 0.8; g = 9.81; % Nominal (imperfect) model m1_hat = 1.8; m2_hat = 1.3; l1_hat = 1.0; l2_hat = 0.8; % Controller gains Kp = diag([100, 100]); Kd = diag([40, 40]); Lambda = diag([10, 10]); rho0 = 15.0; % uncertainty bound epsilon = 0.01; % boundary layer % Desired trajectory: sinusoidal qd = @(t) [sin(t); 0.5*cos(t)]; qd_d = @(t) [cos(t); -0.5*sin(t)]; qd_dd= @(t) [-sin(t); -0.5*cos(t)]; % Disturbance d_max = 5.0; dist = @(t) d_max * [sin(3*t); cos(2*t)]; % ODE: state = [q1; q2; dq1; dq2] odefun = @(t, x) manipulator_robust_ode(t, x, ... m1, m2, l1, l2, g, ... m1_hat, m2_hat, l1_hat, l2_hat, ... Kp, Kd, Lambda, rho0, epsilon, ... qd, qd_d, qd_dd, dist); [t, X] = ode45(odefun, [0 10], [0.5; -0.3; 0; 0]); % Plot tracking errors e1 = arrayfun(@(i) qd(t(i)), 1:length(t), 'Un', 0); e1 = cell2mat(e1')' - X(:,1:2)'; figure; plot(t, e1'); legend('e_1','e_2'); xlabel('Time [s]'); ylabel('Error [rad]'); title('Robust Control: Tracking Error');

function dx = manipulator_robust_ode(t, x, ... m1, m2, l1, l2, g, ... m1_hat, m2_hat, l1_hat, l2_hat, ... Kp, Kd, Lambda, rho0, epsilon, ... qd_fun, qd_d_fun, qd_dd_fun, dist_fun) q = x(1:2); dq = x(3:4); q1 = q(1); q2 = q(2); % TRUE dynamics matrices M = [(m1+m2)*l1^2+m2*l2^2+2*m2*l1*l2*cos(q2), m2*l2^2+m2*l1*l2*cos(q2); m2*l2^2+m2*l1*l2*cos(q2), m2*l2^2]; C = [-m2*l1*l2*dq(2)*sin(q2), -m2*l1*l2*(dq(1)+dq(2))*sin(q2); m2*l1*l2*dq(1)*sin(q2), 0]; G_vec = [(m1+m2)*g*l1*cos(q1)+m2*g*l2*cos(q1+q2); m2*g*l2*cos(q1+q2)]; % NOMINAL (hat) dynamics matrices Mh = [(m1_hat+m2_hat)*l1_hat^2+m2_hat*l2_hat^2+2*m2_hat*l1_hat*l2_hat*cos(q2), ... m2_hat*l2_hat^2+m2_hat*l1_hat*l2_hat*cos(q2); m2_hat*l2_hat^2+m2_hat*l1_hat*l2_hat*cos(q2), m2_hat*l2_hat^2]; Ch = [-m2_hat*l1_hat*l2_hat*dq(2)*sin(q2), ... -m2_hat*l1_hat*l2_hat*(dq(1)+dq(2))*sin(q2); m2_hat*l1_hat*l2_hat*dq(1)*sin(q2), 0]; Gh = [(m1_hat+m2_hat)*g*l1_hat*cos(q1)+m2_hat*g*l2_hat*cos(q1+q2); m2_hat*g*l2_hat*cos(q1+q2)]; % Desired trajectory qd = qd_fun(t); qd_d = qd_d_fun(t); qd_dd = qd_dd_fun(t); % Errors e = q - qd; ed = dq - qd_d; % Sliding variable s = ed + Lambda * e; % Reference velocity/acceleration dqr = qd_d - Lambda * e; ddqr = qd_dd - Lambda * ed; % Robust term (smooth approximation) s_norm = norm(s); tau_robust = -rho0 * s / (s_norm + epsilon); % Control law tau = Mh * ddqr + Ch * dqr + Gh - Kd * s + tau_robust; % Disturbance d = dist_fun(t); % True dynamics: M*ddq + C*dq + G = tau + d ddq = M \ (tau + d - C*dq - G_vec); dx = [dq; ddq]; end

5.4 Robust Control for Mobile Robot

5.4.1 Sources of Disturbance

A differential-drive mobile robot (unicycle model) faces several real-world disturbances:

- Wheel slip: terrain changes cause unpredictable velocity loss or gain.
- Uneven terrain: slopes and bumps inject forces into the kinematic model.
- Wind: for outdoor robots, crosswind affects heading.
- Payload shifts: centre of mass changes alter dynamics.

Figure 5: Top-down view of a differential-drive mobile robot. The dashed blue curve is the reference trajectory, the solid red curve is the actual path under disturbances (wind, terrain, wheel slip). The green tube represents the guaranteed tracking error bound ||e|| provided by the robust controller.

5.4.2 Disturbed Unicycle Model

The kinematic unicycle model with additive disturbance:

\[\begin{cases} \dot{x} = v\cos\theta + d_x(t) \\ \dot{y} = v\sin\theta + d_y(t) \\ \dot{\theta} = \omega + d_\theta(t) \end{cases} \]

where \(v\) is linear velocity, \(\omega\) is angular velocity, and \(d_x, d_y, d_\theta\) are bounded disturbances with \(\|[d_x, d_y, d_\theta]^T\| \le d_{\max}\).

5.4.3 Robust Tracking Controller

Given a reference trajectory \((x_r(t), y_r(t), \theta_r(t))\), define the tracking error in the body frame:

\[\begin{bmatrix} e_x \\ e_y \\ e_\theta \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r - x \\ y_r - y \\ \theta_r - \theta \end{bmatrix} \]

The error dynamics (without disturbance) satisfy:

\[\begin{cases} \dot{e}_x = \omega\,e_y - v + v_r\cos e_\theta \\ \dot{e}_y = -\omega\,e_x + v_r\sin e_\theta \\ \dot{e}_\theta = \omega_r - \omega \end{cases} \]

The nominal tracking controller (without robustness) is:

\[v = v_r\cos e_\theta + k_1\,e_x, \qquad \omega = \omega_r + v_r\,(k_2\,e_y + k_3\sin e_\theta) \]

The robust augmentation adds terms to reject disturbances:

\[v = v_r\cos e_\theta + k_1\,e_x + \rho_v\,\frac{e_x}{|e_x|+\epsilon_v} \] \[\omega = \omega_r + v_r(k_2\,e_y + k_3\sin

e_\theta) + \rho_\omega\,\frac{e_\theta}{|e_\theta|+\epsilon_-
\omega} \]

where \(\rho_v\) and \(\rho_\omega\) are bounds on the linear and angular disturbances, respectively.

5.4.4 Stability Analysis

Consider the Lyapunov function:

\[V = \tfrac{1}{2}(e_x^2 + e_y^2) + \frac{1-\cos e_\theta}{k_2}
\]

Without going through every step (see the manipulator proof for the pattern), the robust terms ensure that the disturbance contribution to \(\dot{V}\) is cancelled by the robust term, leaving \(\dot{V} \le -k_1 e_x^2 - k_3 v_r \sin^2 e_\theta / k_2 \le 0\) outside the boundary layers.

5.4.5 MATLAB Implementation

%% Robust Tracking Control for Unicycle Mobile Robot % Reference: circular trajectory R = 2; omega_ref = 0.5; xr = @(t) R*cos(omega_-
ref*t); yr = @(t) R*sin(omega_ref*t); dxr = @(t) -R*omega_ref*sin(omega_-
ref*t); dyr = @(t) R*omega_ref*cos(omega_ref*t); thr = @(t) omega_-
ref*t + pi/2; vr = @(t) R*omega_ref; wr = @(t) omega_ref; % Disturbance (wheel slip + terrain) d_mob = @(t) 0.3*[sin(5*t); cos(3*t); 0.2*sin(7*t)]; % Controller gains k1 = 5; k2 = 3; k3 = 5; rho_v = 0.5; rho_w = 0.3; eps_v = 0.01; eps_w = 0.01; odefun_mob = @(t, x) mobile_robust_ode(t, x, ... xr, yr, thr, vr, wr, k1, k2, k3, ... rho_v, rho_w, eps_v, eps_w, d_mob); [t_m, X_m] = ode45(odefun_mob, [0 20], [3; 0; 0]); % Plot trajectory figure; hold on; tt = linspace(0,20,500); plot(arrayfun(xr,tt), arrayfun(yr,tt), 'b--', 'LineWidth', 2); plot(X_-
m(:,1), X_m(:,2), 'r', 'LineWidth', 1.5); legend('Reference','Robust'); xlabel('x'); ylabel('y'); title('Mobile Robot: Robust Tracking');

function dx = mobile_robust_ode(t, x, ... xr_fun, yr_fun, thr_fun, vr_fun, wr_fun, ... k1, k2, k3, rho_v, rho_w, eps_v, eps_w, dist_fun) px = x(1); py = x(2); th = x(3); % Reference xr_val = xr_fun(t); yr_-
val = yr_fun(t); thr_val = thr_fun(t); vr_val = vr_fun(t); wr_val = wr_fun(t); % Body-frame error dx_err = xr_val - px; dy_err = yr_val - py; ex = cos(th)*dx_err + sin(th)*dy_err; ey = -sin(th)*dx_err + cos(th)*dy_err; eth = thr_val - th; eth = atan2(sin(eth), cos(eth)); % wrap to [-pi, pi] % Nominal controller v_nom = vr_val*cos(eth) + k1*ex; w_nom = wr_val + vr_val*(k2*ey + k3*sin(eth)); % Robust augmentation v = v_nom + rho_v * ex / (abs(ex) + eps_v); w = w_nom + rho_w * eth / (abs(eth) + eps_w); % Disturbance d = dist_fun(t); % Disturbed kinematics dx = [v*cos(th) + d(1); v*sin(th) + d(2); w + d(3)]; end

5.5 Robust Control for Quadrotor Drone

5.5.1 Wind Disturbance Model

A quadrotor operating outdoors experiences wind disturbances on all axes:

\[m\ddot{p} = \begin{bmatrix} 0\\0\\-mg \end{bmatrix} + R(\Phi)\begin{bmatrix} 0\\0\\T \end{bmatrix} + f_{\text{wind}}(t) \]

where \(p = [x,y,z]^T\) is position, \(T\) is total thrust, \(R(\Phi)\) is the rotation matrix from body to world, and \(f_{\text{wind}} \in \mathbb{R}^3\) with \(\|f_{\text{wind}}\| \le f_{\max}\).

Figure 6: Quadrotor schematic with four rotors generating thrust vectors T--T. Wind disturbance forces f_wind and torque disturbances _wind act on the body. Uncertainties include mass (m ś m) and moment of inertia. The body frame axes (x_b, y_b, z_b) are shown at the centre of mass.

Attitude dynamics with disturbance torques:

\[J\dot{\omega}_b = -\omega_b \times J\omega_b + \tau_b + \tau_{\text{wind}}(t), \qquad \|\tau_{\text{wind}}\| \le \tau_{\max} \]

5.5.2 Simplified Model for Control Design

Using the standard near-hover approximation with Euler angles \((\phi,\theta,\psi)\):

\[m\ddot{x} \approx T(\cos\psi\sin\theta\cos\phi + \sin\psi\sin\phi) + f_{\text{wind},x} \] \[m\ddot{y} \approx T(\sin\psi\sin\theta\cos\phi - \cos\psi\sin\phi) + f_{\text{wind},y} \] \[m\ddot{z} = T\cos\theta\cos\phi - mg + f_{\text{wind},z} \]

For small angles, this simplifies to the double-integrator-like model per axis:

\[m\ddot{z} \approx T - mg + f_z, \qquad m\ddot{x} \approx T\theta_{\text{cmd}} + f_x, \qquad m\ddot{y} \approx -T\phi_{\text{cmd}} + f_y \]

5.5.3 Robust Position Controller

For each translational axis (showing \(z\) as example), define \(e_z = z_d - z\), \(s_z = \dot{e}_z + \lambda_z e_z\):

\[T = m\bigl(\ddot{z}_d + \lambda_z\dot{e}_z + k_{D,z}\,s_z\bigr) + mg + \rho_z\,\frac{s_z}{|s_z| + \epsilon_z} \]

Similarly for \(x\) and \(y\), the outer loop computes desired angles \(\theta_d\) and \(\phi_d\) which are tracked by an inner attitude controller.

5.5.4 Robust Attitude Controller

For each Euler angle (showing roll \(\phi\)):

\[e_\phi = \phi_d - \phi, \quad s_\phi = \dot{e}_\phi + \lambda_-
\phi e_\phi \] \[\tau_\phi = I_{xx}\bigl(\ddot{\phi}_d + \lambda_-
\phi\dot{e}_\phi + k_{D,\phi}\,s_\phi\bigr) + \rho_\phi\,\frac{s_-
\phi}{|s_\phi|+\epsilon_\phi} \]

5.5.5 MATLAB Implementation

%% Robust Control for Quadrotor with Wind Disturbance % Drone parameters m_d = 1.5; g = 9.81; Ixx = 0.03; Iyy = 0.03; Izz = 0.05; % Gains: position kd_z = 15; lam_z = 5; rho_z = 8; eps_z = 0.01; kd_x = 10; lam_x = 4; rho_x = 5; eps_x = 0.01; kd_-y = 10; lam_y = 4; rho_y = 5; eps_y = 0.01; % Gains: attitude kd_phi = 20; lam_phi = 8; rho_phi = 1; eps_phi = 0.005; kd_-th = 20; lam_th = 8; rho_th = 1; eps_th = 0.005; kd_psi = 10; lam_psi = 5; rho_psi = 0.5; eps_psi= 0.005; % Desired trajectory pd = @(t) [2*sin(0.3*t); 2*cos(0.3*t); 1+0.5*sin(0.2*t)]; dpd = @(t) [0.6*cos(0.3*t); -0.6*sin(0.3*t); 0.1*cos(0.2*t)]; ddpd= @(t) [-0.18*sin(0.3*t); -0.18*cos(0.3*t); -0.02*sin(0.2*t)]; % Wind disturbance f_wind = @(t) [3*sin(2*t+1); 2*cos(1.5*t); 1.5*sin(t)]; tau_-wind = @(t) [0.3*sin(4*t); 0.2*cos(3*t); 0.1*sin(2*t)]; % state = [x y z dx dy dz phi theta psi dphi dtheta dpsi] x0 = [0;0;0; 0;0;0; 0;0;0; 0;0;0]; odefun_drone = @(t,x) drone_robust_ode(t, x, m_d, g, ... Ixx, Iyy, Izz, ... kd_z, lam_z, rho_z, eps_z, ... kd_x, lam_x, rho_x, eps_x, ... kd_y, lam_y, rho_y, eps_y, ... kd_phi, lam_phi, rho_-phi, eps_phi, ... kd_th, lam_th, rho_th, eps_th, ... kd_psi, lam_psi, rho_psi, eps_psi, ... pd, dpd, ddpd, f_wind, tau_wind); [t_d, X_d] = ode45(odefun_drone, [0 20], x0); figure; plot3(X_d(:,1), X_d(:,2), X_d(:,3), 'r', 'LineWidth', 1.5); hold on; tt = linspace(0,20,500); ref = cell2mat(arrayfun(pd, tt, 'Un', 0)); plot3(ref(1,:), ref(2,:), ref(3,:), 'b--', 'LineWidth', 2); legend('Robust','Reference'); xlabel('x'); ylabel('y'); zlabel('z'); title('Quadrotor: Robust Tracking under Wind'); grid on;

function dx = drone_robust_ode(t, x, m, g, ... Ixx, Iyy, Izz, ... kd_z, lam_z, rho_z, eps_z, ... kd_x, lam_x, rho_x, eps_x, ... kd_y, lam_y, rho_y, eps_y, ... kd_phi, lam_phi, rho_phi, eps_phi, ... kd_th, lam_-th, rho_th, eps_th, ... kd_psi, lam_psi, rho_psi, eps_psi, ... pd_fun, dpd_fun, ddpd_fun, f_wind_fun, tau_wind_fun) pos = x(1:3); vel = x(4:6); phi = x(7); theta = x(8); psi = x(9); omega = x(10:12); % dphi, dtheta, dpsi % Desired p_d = pd_fun(t); dp_d = dpd_fun(t); ddp_d = ddpd_fun(t); psi_d = 0; dpsi_d = 0; ddpsi_d = 0; % Position errors ep = p_d - pos; dep = dp_d - vel; % --- Z-axis (altitude) robust controller --- sz = dep(3) + lam_z*ep(3); T = m*(ddp_d(3) + lam_z*dep(3) + kd_z*sz) + m*g ... + rho_z*sz/(abs(sz)+eps_z); T = max(T, 0.1); % thrust must be positive % --- X-axis robust controller -> desired theta --- sx = dep(1) + lam_x*ep(1); ax_cmd = ddp_d(1) + lam_x*dep(1) + kd_x*sx + rho_x*sx/(abs(sx)+eps_x); theta_d = m*ax_cmd / T; theta_d = max(min(theta_d, 0.5), -0.5); % saturate % --- Y-axis robust controller -> desired phi --- sy = dep(2) + lam_y*ep(2); ay_cmd = ddp_d(2) + lam_y*dep(2) + kd_y*sy + rho_y*sy/(abs(sy)+eps_y); phi_d = -m*ay_cmd / T; phi_d =

max(min(phi_d, 0.5), -0.5); % --- Attitude robust controllers --- % Roll e_phi = phi_d - phi; de_phi = 0 - omega(1); % dphi_d approx 0 s_phi = de_phi + lam_phi*e_phi; tau_phi = Ixx*(lam_phi*de_phi + kd_phi*s_phi) ... + rho_phi*s_phi/(abs(s_phi)+eps_phi); % Pitch e_th = theta_d - theta; de_th = 0 - omega(2); s_th = de_th + lam_th*e_th; tau_th = Iyy*(lam_th*de_th + kd_th*s_th) ... + rho_th*s_th/(abs(s_th)+eps_th); % Yaw e_psi = psi_d - psi; de_psi = dpsi_d - omega(3); s_psi = de_psi + lam_psi*e_psi; tau_psi = Izz*(lam_psi*de_psi + kd_psi*s_psi) ... + rho_psi*s_psi/(abs(s_psi)+eps_psi); % Wind fw = f_wind_fun(t); tw = tau_wind_fun(t); % Translational dynamics % Rotation matrix (ZYX convention) R = [cos(psi)*cos(theta), cos(psi)*sin(theta)*sin(phi)-sin(psi)*cos(phi), ... cos(psi)*sin(theta)*cos(phi)+sin(psi)*sin(phi); sin(psi)*cos(theta), sin(psi)*sin(theta)*sin(phi)+cos(psi)*cos(phi), ... sin(psi)*sin(theta)*cos(phi)-cos(psi)*sin(phi); -sin(theta), cos(theta)*sin(phi), cos(theta)*cos(phi)]; thrust_body = [0; 0; T]; accel = [0;0;-g] + R*thrust_body/m + fw/m; % Rotational dynamics (simplified Euler) domega = [(tau_phi + tw(1) - (Izz-Iyy)*omega(2)*omega(3)) / Ixx; (tau_th + tw(2) - (Ixx-Izz)*omega(1)*omega(3)) / Iyy; (tau_psi + tw(3) - (Iyy-Ixx)*omega(1)*omega(2)) / Izz]; dx = [vel; accel; omega; domega]; end

5.6 Summary and Next Steps

5.6.1 MATLAB Activities

5.6.2 Activity 1: Robust vs Non-Robust Manipulator (10 min)

5.6.2.1 Instructions

1. Open `robust_control_simulation.m` (provided with this lecture).
2. Run Part 1 (Manipulator). The script simulates both:
   - Robust controller with \(d(t) = 5\sin(3t)\) disturbance and \(\rho_0 = 15\).
   - Computed torque only (no robust term, i.e., \(\rho_0 = 0\)).
3. Record the maximum tracking error for each controller in the table below.

Controller	Max $\lvert e_1 \rvert$ (rad)	Max $\lvert e_2 \rvert$ (rad)
Computed Torque (no robust)		
Robust CT (\(\rho_0 = 15\))		

Question: What happens if you increase \(\rho_0\) to 50? Does tracking improve? What is the trade-off?

5.6.3 Activity 2: Robust vs Adaptive — All Three Systems (15 min)

5.6.3.1 Instructions

1. Run Parts 1, 2, and 3 of the MATLAB script. Each produces comparison plots.
2. The script injects random disturbances (bounded uniform noise).
3. For each system, note:
   - Which controller has smaller steady-state error?
   - Which controller recovers faster from disturbance spikes?
   - Which uses more control effort?

Discussion: Robust control provides guaranteed bounds but is conservative. Adaptive control can achieve zero steady-state error for parametric uncertainty but may struggle with fast-varying disturbances. Which is better for a warehouse robot? A surgical robot? An outdoor drone?

5.6.4 Quick Quiz

5.6.4.1 Quiz: Uncertainty Bounds and Controller Design

1. If the disturbance bound \(d_{\max}\) doubles, what must change in the controller to maintain the same tracking accuracy?
2. What is the boundary layer parameter \(\epsilon\)? What happens if \(\epsilon \to 0\)?
3. In the Lyapunov proof, which property of the manipulator dynamics makes the \(s^T(\dot{M}-2C)s\) term vanish?
4. True or False: Robust control can reject disturbances larger than \(\rho_0\).

5.6.4.2 Answers

1. \(\rho_0\) must at least double (or increase \(K_D\), or decrease \(\epsilon\)).
2. \(\epsilon\) is the boundary layer thickness. As \(\epsilon \to 0\), the controller becomes discontinuous (pure sliding mode), giving perfect rejection but causing chattering.
3. The skew-symmetry property: \(\dot{M}(q) - 2C(q,\dot{q})\) is skew-symmetric for manipulators with the Christoffel-symbol-based \(C\) matrix.
4. False. If \(|d| > \rho_0\), the Lyapunov proof fails and tracking is not guaranteed. The bound must be correctly estimated.

5.6.5 Summary: Comparison of All Four Controllers

Property	Week 1: Computed Torque	Week 2: MPC	Week
Core Idea	Cancel nonlinearities with exact model	Optimise control over prediction horizon	Estim
Model Requirement	Exact model needed	Prediction model needed	Struc
Handles Disturbances?	No (degrades gracefully)	Partially (via constraints)	Paran
Online Computation	Low (matrix multiply)	High (optimisation at each step)	Mediu

Property	Week 1: Computed Torque	Week 2: MPC
Stability Guarantee	Asymptotic (exact model)	Depends on hor
Tracking Error	Zero (ideal), grows with model error	Optimised (mo
Control Effort	Moderate	Optimised (can
Performance Over Time	Constant	Constant
Conservatism	None (exact cancellation)	Low (optimised
Implementation Complexity	Low	High
Best For	Well-modelled systems	Constrained sys

Figure 7: Time-response comparison. Blue: nominal controller tracks perfectly without disturbance. Red: nominal controller with disturbance shows large oscillations and tracking degradation. Green: robust controller with disturbance maintains bounded tracking error within the guaranteed envelope .

Key Takeaway: No single controller is best for all scenarios. In practice, these approaches are often combined:

- Robust-Adaptive: adaptive estimation with a robust modification for unstructured uncertainty.
- Robust MPC: MPC with tube/min-max formulations for bounded disturbances.
- Computed Torque + Robust: nominal model cancellation with a robust term for residual error (this week's approach).

Control for Robots — Week 4: Robust Control

M.Sc. Robotics & Autonomous Systems University of West London
AY 2025–26

6 Backstepping Control (Week 6)

Recursive Lyapunov Design for Nonlinear Systems

M.Sc. Control for Robots — University of West London — AY 2025–26
Duration: 3 Hours (Lecture + Hands-on Lab)

6.1 Overview 83
6.1.1 Week 5 Recall: Robust Control — Exam Practice (20 min) 83
6.1.2 Question 1 — Robust Sliding-Mode Analysis . 83
6.1.3 Question 2 — Robust vs. Adaptive: Mathematical Comparison 85
6.1.4 Learning Outcomes . . . 85
6.1.5 Session Roadmap (3 Hours) 86
6.2 Backstepping Theory . . 87
6.2.1 Motivation — Why Backstepping? 87
6.2.2 Strict-Feedback Form . . 87
6.2.3 Two-Step Backstepping — Complete Derivation . . 87
6.2.4 Generalisation to \(n\)-Step Backstepping 89
6.2.5 Why Backstepping Works — Intuition 89
6.3 Backstepping for Robot Manipulator 90
6.3.1 Strict-Feedback Form of Manipulator Dynamics . 90
6.3.2 MATLAB Implementation 92
6.4 Backstepping for Mobile Robot 92
6.4.1 Unicycle Model and Error Coordinates 92
6.4.2 Backstepping Design . . 93
6.4.3 MATLAB Implementation 94
6.5 Backstepping for Quadrotor Drone 94
6.5.1 Quadrotor Cascade Structure 94
6.5.2 Backstepping Design — Outer Loop (Position Control) 95
6.5.3 Backstepping Design — Inner Loop (Attitude Control) 95

6.1 Overview

6.1.1 Week 5 Recall: Robust Control — Exam Practice (20 min)

6.1.2 Question 1 — Robust Sliding-Mode Analysis

Consider a manipulator with model uncertainty:

\[M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau + d(t), \qquad \|d(t)\| \le d_{\max} \]

The robust controller is:

\[\tau = \hat{M}(q)\bigl[\ddot{q}_d + K_D \dot{e} + K_P e\bigr] + \hat{C}\dot{q} + \hat{G} + \tau_{\text{robust}} \]

with the robust term

\[\tau_{\text{robust}} = \rho(t)\,\frac{s}{\|s\|}, \qquad s = \dot{e} + \Lambda\, e \]

where \(e = q_d - q\), \(\dot{e} = \dot{q}_d - \dot{q}\), and \(\Lambda \succ 0\).

6.1.2.1 Model Answer — Closed-Loop Error Dynamics

Step 1 — Substitution. Write the true dynamics as $M\ddot{q} = \tau + d - C\dot{q} - G$. Substitute the controller:

$$ M\ddot{q} = \hat{M}(\ddot{q}_d + K_D\dot{e} + K_P e) + \hat{C}\dot{q} + \hat{G} + \rho\frac{s}{\|s\|} + d - C\dot{q} - G $$

Define the model errors $\tilde{M} = M - \hat{M}$, $\tilde{C} = C - \hat{C}$, $\tilde{G} = G - \hat{G}$, and let the lumped uncertainty be

$$ \eta(t) = \tilde{M}\ddot{q} + \tilde{C}\dot{q} + \tilde{G} - d(t) $$

Step 2 — Error dynamics. With $\ddot{e} = \ddot{q}_d - \ddot{q}$ and after algebraic manipulation:

$$ M\ddot{e} + (C + M K_D)\dot{e} + M K_P e = -\eta(t) + \rho\frac{s}{\|s\|} $$

In terms of the composite variable $s = \dot{e} + \Lambda e$:

$$ M\dot{s} = -Cs + \eta(t) + \rho\frac{s}{\|s\|} $$

(using the choice $K_D = \Lambda$, $K_P = \dot{\Lambda}$ to simplify).

Step 3 — Lyapunov analysis. Choose $V = \tfrac{1}{2}\,s^T M\, s$. Then:

$$ \dot{V} = s^T M\dot{s} + \tfrac{1}{2}\,s^T\dot{M}\,s = s^T\!\bigl[-Cs + \eta + \rho\tfrac{s}{\|s\|}\bigr] + \tfrac{1}{2}\,s^T\dot{M}\,s $$

Using the skew-symmetry property $s^T(\dot{M} - 2C)s = 0$:

$$ \dot{V} = s^T\eta + \rho\|s\| $$

Since τ_{robust} opposes the uncertainty (note the sign convention: $\rho \frac{s}{\|s\|}$ acts in the direction of s, so the correct robust term to counteract uncertainty is $\tau_{\text{robust}} = -\rho \frac{s}{\|s\|}$). Re-doing with the standard sign:

$$ \dot{V} = s^T\eta - \rho\|s\| \le \|\eta\|\,\|s\| - \rho\|s\| = (\|\eta\| - \rho)\|s\| $$

Conclusion: If $\rho(t) \ge \|\eta(t)\|$, then $\dot{V} \le 0$ and the tracking error $s \to 0$. The sliding variable converges, guaranteeing bounded tracking despite uncertainty. By Barbalat's lemma, $s(t) \to 0$ asymptotically, which implies $e(t) \to 0$ exponentially since $\dot{e} = s - \Lambda e$ is a stable filter.

6.1.3 Question 2 — Robust vs. Adaptive: Mathematical Comparison

6.1.3.1 Model Answer

Adaptive control uses an augmented Lyapunov function:

\[V_{\text{adaptive}} = \frac{1}{2}\,s^T M\, s + \frac{1}{2}\,\tilde{\theta}^T \Gamma^{-1} \tilde{\theta} \]

where \(\tilde{\theta} = \theta - \hat{\theta}\) is the parameter estimation error and \(\Gamma \succ 0\) is the adaptation gain matrix. The adaptation law \(\dot{\hat{\theta}} = \Gamma\, Y^T s\) is designed so that the cross-term \(s^T Y \tilde{\theta}\) in \(\dot{V}\) cancels exactly with \(\tilde{\theta}^T \Gamma^{-1}\dot{\hat{\theta}}\), yielding:

\[\dot{V}_{\text{adaptive}} = -s^T K_D\, s \le 0 \]

Robust control uses a simpler Lyapunov function:

\[V_{\text{robust}} = \frac{1}{2}\,s^T M\, s \]

No parameter estimation is required because the robust term directly dominates the uncertainty via the bound \(\rho \ge \|\eta\|\):

\[\dot{V}_{\text{robust}} \le -(\rho - \|\eta\|)\|s\| \le 0 \]

Why robust control does not need parameter estimation: Instead of identifying what the uncertainty is (as adaptive control does), robust control only needs to know how large the uncertainty can be. It overpowers the worst case with a dominating gain \(\rho\).

Trade-off:

Aspect	Adaptive Control	Robust Control
Uncertainty handling	Estimates and cancels parametric uncertainty	Overpowers bounded uncertainty without
Conservatism	Low — adapts to actual parameters	High — must use worst-case bound
Control effort	Decreases as estimates converge	Remains high (constant worst-case gain)
Chattering	None (smooth update law)	Possible with discontinuous sign function
Unstructured uncertainty	Cannot handle (requires linear parameterisation)	Handles any bounded disturbance
Convergence	Asymptotic (with PE: exponential)	Finite-time to sliding surface

6.1.4 Learning Outcomes

- Understand the backstepping design methodology for strict-feedback nonlinear systems
- Derive a backstepping controller step-by-step using recursive Lyapunov design
- Apply backstepping to manipulator, mobile robot, and drone systems
- Implement and simulate backstepping controllers in MATLAB
- Compare backstepping with computed torque, MPC, adaptive, and robust control

6.1.5 Session Roadmap (3 Hours)

0:00 – 0:20

6.1.5.1 Week 5 Recall — Robust Control Exam Practice

Solve two exam-style questions on robust control, Lyapunov analysis, and comparison with adaptive control.

0:20 – 1:00

6.1.5.2 Backstepping Theory

Strict-feedback systems, virtual control, recursive Lyapunov design, n-step generalisation.

1:00 – 1:30

6.1.5.3 Backstepping for Manipulator

Convert Euler-Lagrange dynamics to strict-feedback form, design backstepping torque controller, full Lyapunov proof, MATLAB code.

1:30 – 2:00

6.1.5.4 Backstepping for Mobile Robot

Chained-form unicycle model, backstepping position tracking design, MATLAB code.

2:00 – 2:30

6.1.5.5 Backstepping for Drone

Cascade position–attitude structure, multi-loop backstepping, MATLAB code.

2:30 – 3:00

6.1.5.6 Hands-on Activities, Quiz & Summary

Gain tuning experiments, comparative study, quiz, and method comparison table.

6.2 Backstepping Theory

6.2.1 Motivation — Why Backstepping?

In previous weeks, we studied controllers that either require exact cancellation of nonlinearities (computed torque), predict future behaviour (MPC), estimate unknown parameters on-line (adaptive), or overpower bounded uncertainties (robust). Backstepping takes a fundamentally different approach: it exploits the triangular (cascade) structure of the system to build a Lyapunov function and controller one subsystem at a time, recursively.

Key Insight: Many physical systems — from robotic manipulators to quadrotors — have a strict-feedback (cascade) structure where one state acts as the "input" to the previous subsystem. Backstepping systematically exploits this structure.

Figure 1: Backstepping cascade structure. The design proceeds from the outermost subsystem (Step 1) back to the actual control input (Step n).

6.2.2 Strict-Feedback Form

A nonlinear system is in strict-feedback form if it can be written as:

\[\begin{aligned} \dot{x}_1 &= f_1(x_1) + g_1(x_1)\,x_2 \\ \dot{x}_2 &= f_2(x_1, x_2) + g_2(x_1, x_2)\,x_3 \\ &\;;\vdots \\ \dot{x}_{n-1} &= f_{n-1}(x_1,\ldots,x_{n-1}) + g_{n-1}(x_1,\ldots,x_{n-1})\,x_n \\ \dot{x}_n &= f_n(x_1,\ldots,x_n) + g_n(x_1,\ldots,x_n)\,u \end{aligned} \]

Key properties:

- Each equation depends on all previous states plus one new state \(x_{i+1}\).
- The new state \(x_{i+1}\) enters multiplicatively through \(g_i(\cdot)\).
- Only the last equation contains the true control input \(u\).
- \(g_i(\cdot) \neq 0\) everywhere (controllability condition).

Figure 2: Triangular dependency structure of strict-feedback form. Each row depends only on states from current and previous subsystems.

6.2.3 Two-Step Backstepping — Complete Derivation

Consider the simplest strict-feedback system (two states, one input):

\[\begin{aligned} \dot{x}_1 &= f_1(x_1) + g_1(x_1)\,x_2 \\ \dot{x}_2 &= f_2(x_1,x_2) + g_2(x_1,x_2)\,u \end{aligned} \]

Goal: Drive \(x_1 \to x_{1,\text{ref}}\) (a desired reference).

6.2.3.1 Step 1 — Stabilise the x_1 Subsystem (Virtual Control Design)

Define the tracking error:

$$ z_1 = x_1 - x_{1,\text{ref}} $$

Its dynamics are:

$$ \dot{z}_1 = \dot{x}_1 - \dot{x}_{1,\text{ref}} = f_1(x_1) + g_1(x_1)\,x_2 - \dot{x}_{1,\text{ref}} $$

Imagine that x_2 is a control input that we can freely choose. We call this the virtual control and design it as $x_2 = \alpha_1(x_1)$ such that $z_1 \to 0$. Choose the Lyapunov function candidate:

$$ V_1 = \frac{1}{2}\,z_1^2 $$

Its time derivative is:

$$ \dot{V}_1 = z_1\,\dot{z}_1 = z_1\bigl[f_1(x_1) + g_1(x_1)\,x_2 - \dot{x}_{1,\text{ref}}\bigr] $$

If x_2 were our actual input, we would choose:

$$ \alpha_1(x_1) = \frac{1}{g_1(x_1)}\bigl[-f_1(x_1) + \dot{x}_{1,\text{ref}} - k_1\,z_1\bigr], \qquad k_1 > 0 $$

This would give $\dot{V}_1 = -k_1\,z_1^2 \le 0$. But x_2 is not the actual input — it is a state. So we define the backstepping error:

$$ z_2 = x_2 - \alpha_1(x_1) $$

Substituting $x_2 = z_2 + \alpha_1$ into $\dot{V}_1$:

$$ \dot{V}_1 = z_1\bigl[f_1 + g_1(z_2 + \alpha_1) - \dot{x}_{1,\text{ref}}\bigr] = -k_1\,z_1^2 + g_1\,z_1\,z_2 $$

The term $-k_1 z_1^2$ is stabilising; the coupling term $g_1 z_1 z_2$ must be handled in Step 2.

6.2.3.2 Step 2 — Design the Actual Control u (Backstep)

Now we must drive $z_2 \to 0$. Compute the dynamics of z_2:

$$ \dot{z}_2 = \dot{x}_2 - \dot{\alpha}_1 = f_2(x_1,x_2) + g_2(x_1,x_2)\,u - \frac{\partial \alpha_1}{\partial x_1}\dot{x}_1 - \frac{\partial \alpha_1}{\partial x_{1,\text{ref}}}\dot{x}_{1,\text{ref}} $$

where we have used the chain rule on $\alpha_1(x_1, x_{1,\text{ref}})$. Augment the Lyapunov function:

$$ V_2 = V_1 + \frac{1}{2}\,z_2^2 = \frac{1}{2}\,z_1^2 + \frac{1}{2}\,z_2^2 $$

Its derivative is:

$$ \dot{V}_2 = \dot{V}_1 + z_2\,\dot{z}_2 = -k_1\,z_1^2 + g_1\,z_1\,z_2 + z_2\,\dot{z}_2 $$

We want $\dot{V}_2 = -k_1\,z_1^2 - k_2\,z_2^2$ for some $k_2 > 0$. This requires:

$$ z_2\bigl[g_1\,z_1 + \dot{z}_2\bigr] = -k_2\,z_2^2 $$ $$ g_1\,z_1 + f_2 + g_2\,u - \dot{\alpha}_1 = -k_2\,z_2 $$

Solving for the control input:

$$ u = \frac{1}{g_2(x_1,x_2)}\left[-f_2(x_1,x_2) + \dot{\alpha}_1 - g_1(x_1)\,z_1 - k_2\,z_2\right] $$

Result: With this control law,

$$ \dot{V}_2 = -k_1\,z_1^2 - k_2\,z_2^2 \le 0 $$

which guarantees global asymptotic stability of $(z_1, z_2) = (0,0)$ by Lyapunov's theorem.

6.2.4 Generalisation to n-Step Backstepping

For an n-dimensional strict-feedback system, the procedure generalises recursively:

Algorithm: n-Step Backstepping

For i = 1, 2, ..., n: 1. Define error: z_i = x_i - alpha_{i-1} (where alpha_0 = x_{1,ref}) 2. Compute dynamics: dz_i/dt = f_i + g_i * x_{i+1} - d(alpha_{i-1})/dt 3. Augment Lyapunov: V_i = V_{i-1} + (1/2) * z_i^2 4. Compute dV_i/dt and choose: - If i < n: alpha_i (virtual control) to make: dV_i/dt = -k_1*z_1^2 - ... - k_i*z_i^2 + g_i*z_i*z_{i+1} - If i = n: u (actual control) to make: dV_n/dt = -k_1*z_1^2 - ... - k_n*z_n^2 Result: Global asymptotic stability with V_n as Lyapunov function.

Figure 3: Step-by-step backstepping design flowchart. Each step defines an error, augments the Lyapunov function, and designs a (virtual) control.

6.2.5 Why Backstepping Works — Intuition

6.2.5.1 Recursive Lyapunov Construction

At each step, we extend the Lyapunov function by adding $\frac{1}{2}z_i^2$. Each new virtual control is designed to make the new terms in $\dot{V}$ negative definite while leaving the previously designed terms unchanged.

6.2.5.2 "Backstepping" the Control

The name comes from the fact that we start from the outermost subsystem (where the reference lives) and "step back" through the cascade, designing one layer at a time until we reach the actual control input.

6.2.5.3 No Cancellation Required

Unlike feedback linearisation, backstepping does not cancel all nonlinearities. It only cancels what is needed and uses the remaining structure constructively. This gives better robustness and avoids wasting control effort.

6.2.5.4 Guaranteed Stability

The final Lyapunov function $V_n = \sum_{i=1}^n \frac{1}{2}z_i^2$ with $\dot{V}_n = -\sum_{i=1}^n k_i z_i^2$ provides a constructive proof of global asymptotic stability.

Figure 5: Lyapunov function augmentation. Each step adds a new error term, growing the "bowl" in state space while maintaining negative definiteness of V.

6.2.5.5 Think-Pair-Share (3 min)

Question: Why can't we just use feedback linearisation for all nonlinear systems? When does backstepping have an advantage?

Discussion points:

- Feedback linearisation requires an exact model to cancel all nonlinearities — any mismatch creates uncompensated dynamics.
- Feedback linearisation may cancel useful nonlinearities (e.g., natural damping).
- Some systems are not feedback-linearisable (the relative degree conditions fail).
- Backstepping provides a Lyapunov function at every step — we can prove stability even with partial model knowledge.
- Backstepping can be combined with adaptation (adaptive backstepping) to handle unknown parameters in the strict-feedback structure.

6.3 Backstepping for Robot Manipulator

6.3.1 Strict-Feedback Form of Manipulator Dynamics

The standard Euler-Lagrange dynamics of an n-DOF manipulator are:

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau$$

Define the states $x_1 = q$ (joint positions) and $x_2 = \dot{q}$ (joint velocities). Then:

$$\begin{aligned} \dot{x}_1 &= x_2 \\ \dot{x}_2 &= M^{-1}(x_1)\bigl[\tau - C(x_1,x_2)x_2 - G(x_1)\bigr] \end{aligned}$$

This is a strict-feedback system with $f_1 = 0$, $g_1 = I$, $f_2 = -M^{-1}(C x_2 + G)$, and $g_2 = M^{-1}$.

Figure 4: 2-DOF planar manipulator and its decomposition into position and velocity subsystems for backstepping design.

6.3.1.1 Step 1 — Virtual Control for Position Subsystem

Define the position error:

$$ z_1 = x_1 - q_d = q - q_d $$

The dynamics of z_1 are:

$$ \dot{z}_1 = \dot{q} - \dot{q}_d = x_2 - \dot{q}_d $$

Choose the Lyapunov function $V_1 = \frac{1}{2}\,z_1^T z_1$. Its derivative:

$$ \dot{V}_1 = z_1^T \dot{z}_1 = z_1^T(x_2 - \dot{q}_d) $$

Design the virtual control α_1 (the desired velocity) so that if $x_2 = \alpha_1$, then $\dot{V}_1 < 0$:

$$ \alpha_1 = \dot{q}_d - K_1\,z_1, \qquad K_1 = k_1 I \succ 0 $$

This gives $\dot{V}_1 = -z_1^T K_1 z_1 + z_1^T z_2$ where $z_2 = x_2 - \alpha_1 = \dot{q} - \dot{q}_d + K_1 z_1$.

6.3.1.2 Step 2 — Torque Control for Velocity Subsystem

Define the velocity error:

$$ z_2 = x_2 - \alpha_1 = \dot{q} - \dot{q}_d + K_1(q - q_d) $$

Compute its dynamics:

$$ \dot{z}_2 = \ddot{q} - \ddot{q}_d + K_1\dot{z}_1 = M^{-1}(\tau - C\dot{q} - G) - \ddot{q}_d + K_1(\dot{q} - \dot{q}_d) $$

Augment the Lyapunov function (using the inertia matrix for natural energy weighting):

$$ V_2 = \frac{1}{2}\,z_1^T z_1 + \frac{1}{2}\,z_2^T M(q)\,z_2 $$

The derivative of V_2 is:

$$ \dot{V}_2 = z_1^T\dot{z}_1 + z_2^T M\dot{z}_2 + \frac{1}{2}\,z_2^T\dot{M}\,z_2 $$

Substituting and using the skew-symmetry property $z_2^T(\dot{M} - 2C)z_2 = 0$:

$$ \dot{V}_2 = -z_1^T K_1\,z_1 + z_1^T z_2 + z_2^T\bigl[\tau - C\dot{q} - G - M(\ddot{q}_d - K_1\dot{z}_1)\bigr] + \frac{1}{2}\,z_2^T\dot{M}\,z_2 $$

Using the skew-symmetry property and simplifying:

$$ \dot{V}_2 = -z_1^T K_1\,z_1 + z_1^T z_2 + z_2^T\bigl[\tau - C\alpha_1 - G - M(\ddot{q}_d - K_1\dot{z}_1)\bigr] $$

To achieve \(\dot{V}_2 = -z_1^T K_1 z_1 - z_2^T K_2 z_2\), choose:

Backstepping Torque Control Law:

\[\tau = M(q)\bigl[\ddot{q}_d - K_1\dot{z}_1 - K_2 z_2\bigr] + C(q,\dot{q})\,\alpha_1 + G(q) - z_1 \]

where \(K_2 = k_2 I \succ 0\). Expanding the terms:

\[\tau = M(q)\bigl[\ddot{q}_d - K_1(\dot{q} - \dot{q}_d) - K_2(\dot{q} - \dot{q}_d + K_1(q-q_d))\bigr] + C(q,\dot{q})(\dot{q}_d - K_1(q-q_d)) + G(q) - (q - q_d) \]

Stability result:

\[\dot{V}_2 = -z_1^T K_1\,z_1 - z_2^T K_2\,z_2 \le 0 \]

Both \(z_1 \to 0\) and \(z_2 \to 0\) asymptotically, which means \(q \to q_d\) and \(\dot{q} \to \dot{q}_d\).

6.3.2 MATLAB Implementation

% Backstepping controller for 2-DOF manipulator function tau = backstepping_manipulator(t, q, dq, params) % Desired trajectory qd = [0.5*sin(t); 0.5*cos(t)]; dqd = [0.5*cos(t); -0.5*sin(t)]; ddqd = [-0.5*sin(t); -0.5*cos(t)]; % Errors z1 = q - qd; % Position error dz1 = dq - dqd; % Velocity of position error alpha1 = dqd - params.k1 * z1; % Virtual control (desired velocity) z2 = dq - alpha1; % Velocity error (= dz1 + k1*z1) % Dynamics matrices [M, C, G] = manipulator_dynamics_matrices(q, dq, params); % Backstepping control law tau = M * (ddqd - params.k1*dz1 - params.k2*z2) ... + C * alpha1 + G - z1; end

Implementation note: The term \(-z_1\) in the control law is the "cross-coupling cancellation" term that arises from the backstepping procedure. Without it, the position and velocity errors would remain coupled and stability cannot be guaranteed by our Lyapunov function.

6.4 Backstepping for Mobile Robot

6.4.1 Unicycle Model and Error Coordinates

The unicycle (differential-drive) mobile robot has the kinematic model:

\[\begin{aligned} \dot{x} &= v\cos\theta \\ \dot{y} &= v\sin\theta \\ \dot{\theta} &= \omega \end{aligned} \]

where \(v\) is the linear velocity and \(\omega\) is the angular velocity (our two control inputs).

Given a reference trajectory \((x_r(t), y_r(t), \theta_r(t))\) generated by \(\dot{x}_r = v_r\cos\theta_r\), \(\dot{y}_r = v_r\sin\theta_r\), \(\dot{\theta}_r = \omega_r\), define the body-frame error:

\[\begin{bmatrix} e_1 \\ e_2 \\ e_3 \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0

\\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r - x \\ y_r - y \\ \theta_r - \theta \end{bmatrix} \]

The error dynamics are:

\[\begin{aligned} \dot{e}_1 &= \omega\, e_2 - v + v_r\cos e_3 \\ \dot{e}_2 &= -\omega\, e_1 + v_r\sin e_3 \\ \dot{e}_3 &= \omega_r - \omega \end{aligned} \]

Figure 6: Differential-drive mobile robot (top-down view) with body-frame errors and the backstepping cascade: position error drives heading control, which drives velocity control.

6.4.2 Backstepping Design

6.4.2.1 Step 1 — Stabilise the Lateral/Heading Subsystem

Consider the \((e_2, e_3)\) subsystem first. Note that \(e_2\) is driven by \(v_r \sin e_3\). Think of \(e_3\) as a virtual input to steer \(e_2\). If \(e_3\) could be freely chosen, pick:

\[e_3^* = \arctan(-k_3\, e_2) \]

(This makes \(v_r \sin e_3^* \approx -k_3 v_r e_2\) for small \(e_2\), driving \(e_2 \to 0\).)

However, \(e_3\) is a state, not an input. Define the backstepping error:

\[\bar{e}_3 = e_3 - e_3^* \]

6.4.2.2 Step 2 — Design the Control Inputs \((v, \omega)\)

Choose the Lyapunov function:

\[V = \frac{1}{2}e_1^2 + \frac{1}{2}e_2^2 + \frac{1}{2}\bar{e}_3^2 \]

For the longitudinal error \(e_1\), choose the linear velocity:

\[v = v_r\cos e_3 + k_1\,e_1 \]

This gives \(\dot{e}_1 = -k_1 e_1 + \omega e_2\).

For the angular velocity, we need \(\omega\) to simultaneously: (a) cancel the coupling \(\omega e_2\) term in \(\dot{e}_1\), (b) drive \(\bar{e}_3 \to 0\), and (c) handle the \(e_2\) dynamics.

The full angular velocity controller is:

\[\omega = \omega_r + k_2\,v_r\,e_2 + k_3\,\bar{e}_3 \]

where \(k_2\) handles the lateral error coupling with \(v_r \sin e_3\), and \(k_3\) drives the heading backstepping error to zero.

Backstepping Mobile Robot Controller:

\[v = v_r \cos e_3 + k_1\,e_1 \] \[\omega = \omega_r + k_2\,v_r\,e_2 + k_3\,\sin e_3 \]

Stability analysis. With this controller and $V = \frac{1}{2}(e_1^2 + e_2^2 + e_3^2)$:

$$\dot{V} = e_1\dot{e}_1 + e_2\dot{e}_2 + e_3\dot{e}_3 = e_1(-k_1 e_1 + \omega e_2) + e_2(-\omega e_1 + v_r \sin e_3) + e_3(\omega_r - \omega)$$

The cross-terms $e_1 \omega e_2$ and $-e_2 \omega e_1$ cancel. Substituting the controller:

$$\dot{V} = -k_1 e_1^2 + e_2 v_r \sin e_3 + e_3(\omega_r - \omega_r - k_2 v_r e_2 - k_3 \sin e_3) = -k_1 e_1^2 + v_r e_2 \sin e_3 - k_2 v_r e_2 e_3 - k_3 e_3 \sin e_3$$

For $v_r > 0$ and using the fact that $\frac{\sin e_3}{e_3}$ is bounded, with appropriate choice of k_2, we get $\dot{V} \le -k_1 e_1^2 - k_3 e_3 \sin e_3 \le 0$. Convergence of all errors follows by Barbalat's lemma (since $v_r(t) > 0$).

6.4.3 MATLAB Implementation

```
% Backstepping controller for unicycle mobile robot function [v, omega] = backstepping_mobile_robot(e1, e2, e3, v_r, omega_r, params) % e1: longitudinal error (body frame) % e2: lateral error (body frame) % e3: heading error v = v_r * cos(e3) + params.k1 * e1; omega = omega_r + params.k2 * v_r * e2 + params.k3 * sin(e3); end
```

6.5 Backstepping for Quadrotor Drone

6.5.1 Quadrotor Cascade Structure

The quadrotor has a natural four-level cascade:

$$\underbrace{\text{Position}}_{\text{Level 1}} \xrightarrow{\text{desired accel.}} \underbrace{\text{Velocity}}_{\text{Level 2}} \xrightarrow{\text{desired angles}} \underbrace{\text{Orientation}}_{\text{Level 3}} \xrightarrow{\text{desired rates}} \underbrace{\text{Angular Rate}}_{\text{Level 4}} \xrightarrow{\quad} (T, \boldsymbol{\tau})$$

The simplified quadrotor dynamics (with rotation matrix R):

$$\begin{aligned} \dot{\mathbf{p}} &= \mathbf{v} \\[4pt] m\dot{\mathbf{v}} &= -mg\,\mathbf{e}_3 + T\,R\,\mathbf{e}_3 \\[4pt] \dot{R} &= R\,[\boldsymbol{\omega}]_\times \\[4pt] J\dot{\boldsymbol{\omega}} &= -\boldsymbol{\omega} \times J\boldsymbol{\omega} + \boldsymbol{\tau}_b \end{aligned}$$

where $\mathbf{p} = [x,y,z]^T$ is position, $\mathbf{v}$ is velocity, T is total thrust, $R \in SO(3)$ is the rotation matrix, $\boldsymbol{\omega}$ is the body angular velocity, J is the inertia tensor, and $\boldsymbol{\tau}_b$ are the body torques.

Figure 7: Quadrotor cascade backstepping architecture. The position controller generates a desired attitude (virtual control); the attitude controller generates the actual motor commands.

6.5.2 Backstepping Design — Outer Loop (Position Control)

6.5.2.1 Step 1 — Position Error

Define position error $z_1 = \mathbf{p} - \mathbf{p}_d$. Then $\dot{z}_1 = \mathbf{v} - \dot{\mathbf{p}}_d$.

Virtual control (desired velocity):

$$\boldsymbol{\alpha}_1 = \dot{\mathbf{p}}_d - K_{p1}\,z_1, \qquad K_{p1} = k_{p1}\,I_3 \succ 0$$

6.5.2.2 Step 2 — Velocity Error

Define velocity error $z_2 = \mathbf{v} - \boldsymbol{\alpha}_1$. Its dynamics:

$$\dot{z}_2 = \dot{\mathbf{v}} - \dot{\boldsymbol{\alpha}}_1 = -g\mathbf{e}_3 + \frac{T}{m}R\mathbf{e}_3 - \ddot{\mathbf{p}}_d + K_{p1}\dot{z}_1$$

Choose the Lyapunov function $V = \frac{1}{2}z_1^T z_1 + \frac{1}{2}z_2^T z_2$. To make $\dot{V} = -k_{p1}\|z_1\|^2 - k_{p2}\|z_2\|^2$, we need the desired acceleration vector:

$$\mathbf{a}_{\text{des}} = \ddot{\mathbf{p}}_d - K_{p1}\dot{z}_1 - z_1 - K_{p2}\,z_2 + g\mathbf{e}_3$$

The thrust magnitude and desired orientation are extracted from $\mathbf{a}_{\text{des}}$:

$$T = m\,\|\mathbf{a}_{\text{des}}\| \qquad \mathbf{z}_{B,\text{des}} = \frac{\mathbf{a}_{\text{des}}}{\|\mathbf{a}_{\text{des}}\|} \quad \Longrightarrow \quad R_{\text{des}} \text{ constructed from } \mathbf{z}_{B,\text{des}} \text{ and desired yaw}$$

6.5.3 Backstepping Design — Inner Loop (Attitude Control)

6.5.3.1 Step 3 — Attitude Error

Using Euler angles (ϕ, θ, ψ) for a simplified treatment, define:

$$z_3 = \boldsymbol{\Phi} - \boldsymbol{\Phi}_{\text{des}} = \begin{bmatrix} \phi - \phi_d \\ \theta - \theta_d \\ \psi - \psi_d \end{bmatrix}$$

Virtual control (desired angular rate):

$$\boldsymbol{\alpha}_3 = \dot{\boldsymbol{\Phi}}_{\text{des}} - K_{a1}\,z_3$$

6.5.3.2 Step 4 — Angular Rate Error and Torque Design

Define \(z_4 = \boldsymbol{\omega} - \boldsymbol{\alpha}_3\). Using the Euler equation \(J\dot{\boldsymbol{\omega}} = -\boldsymbol{\omega} \times J\boldsymbol{\omega} + \boldsymbol{\tau}_b\):

Backstepping Attitude Torque:

\[\boldsymbol{\tau}_b = J\bigl[\dot{\boldsymbol{\alpha}}_3 - K_{a2}\,z_4\bigr] + \boldsymbol{\omega} \times J\boldsymbol{\omega} - z_3 \]

This yields the overall Lyapunov function:

\[V = \frac{1}{2}\bigl(\|z_1\|^2 + \|z_2\|^2 + \|z_3\|^2 + z_4^T J\,z_4\bigr) \] \[\dot{V} = -k_{p1}\|z_1\|^2 - k_{p2}\|z_2\|^2 - k_{a1}\|z_3\|^2 - k_{a2}\,z_4^T K_{a2}\,z_4 \le 0 \]

6.5.4 MATLAB Implementation

% Backstepping controller for quadrotor function [T, tau_b] = backstepping_quadrotor(state, ref, params) % State: [x y z vx vy vz phi theta psi p q r] pos = state(1:3); vel = state(4:6); phi = state(7); theta = state(8); psi = state(9); omega_b = state(10:12); % Reference pos_d = ref.pos; vel_d = ref.vel; acc_d = ref.acc; psi_d = ref.psi; % --- Outer loop: Position backstepping --- z1 = pos - pos_d; dz1 = vel - vel_d; alpha1 = vel_d - params.kp1 * z1; % Virtual velocity z2 = vel - alpha1; % Velocity error % Desired acceleration a_des = acc_d - params.kp1*dz1 - z1 - params.kp2*z2 + [0;0;params.g]; % Thrust T = params.m * norm(a_des); % Desired orientation from a_des z_B_des = a_des / norm(a_des); phi_d = asin(z_B_des(1)*sin(psi_d) - z_B_des(2)*cos(psi_d)); theta_d = atan2(z_B_des(1)*cos(psi_d)+z_B_des(2)*sin(psi_d), z_B_des(3)); % --- Inner loop: Attitude backstepping --- Phi = [phi; theta; psi]; Phi_d = [phi_d; theta_d; psi_d]; z3 = Phi - Phi_d; alpha3 = -params.ka1 * z3; % Desired angular rate z4 = omega_b - alpha3; % Body torques J = diag([params.Ixx, params.Iyy, params.Izz]); tau_b = J * (-params.ka1*(-params.ka1*z3 - z4*0) - params.ka2*z4) ... + cross(omega_b, J*omega_b) - z3; end

Practical note: In real implementations, the desired angles \((\phi_d, \theta_d)\) must be saturated (typically to \(\pm 30^\circ\)) to prevent aggressive manoeuvres. The thrust \(T\) must also be clamped to \([0, T_{\max}]\).

6.6 Summary and Next Steps

6.6.1 Hands-on Activities & Quiz

6.6.2 Activity 1: Gain Tuning for Manipulator Backstepping (10 min)

Run `backstepping_control_simulation.m` and observe the manipulator plots. Then modify the gains and re-run:

1. Set $k_1 = 50, \; k_2 = 5$ (position gain much larger than velocity gain).
2. Set $k_1 = 5, \; k_2 = 50$ (velocity gain much larger than position gain).
3. Set $k_1 = k_2 = 25$ (balanced gains).

Expected observations:

- $k_1 \gg k_2$: Aggressive position correction generates large desired velocity changes. The velocity loop cannot track them fast enough, causing oscillatory or even unstable behaviour. The virtual control α_1 demands more than the torque loop can deliver.
- $k_2 \gg k_1$: The velocity error z_2 converges quickly, but the position error z_1 converges slowly because k_1 is small. The system is overdamped with sluggish position tracking.
- $k_1 \approx k_2$: Balanced convergence rates for both loops. This is generally the best choice for smooth, fast tracking.

6.6.3 Activity 2: Comparative Study (15 min)

The MATLAB file includes a comparison of backstepping vs. computed torque vs. robust control on the manipulator with model uncertainty (20% mass error). Observe:

- Which controller has the smallest steady-state error?
- Which controller has the smoothest control signal?
- Which controller uses the least energy (smallest $\int \|\tau\|^2 \, dt$)?

6.6.3.1 Quiz — Virtual Control Design

Q1: Given the system $\dot{x}_1 = x_1^2 + x_2$, $\dot{x}_2 = x_2 + u$, design a backstepping controller to stabilise $x_1 = 0$.

Answer sketch:

- Step 1: $z_1 = x_1$, $\dot{z}_1 = x_1^2 + x_2$. Choose $\alpha_1 = -x_1^2 - k_1 x_1$ so that $\dot{V}_1 = -k_1 x_1^2 + x_1 z_2$.
- Step 2: $z_2 = x_2 - \alpha_1$, $\dot{z}_2 = x_2 + u - (2x_1)(x_1^2 + x_2) + k_1(x_1^2 + x_2)$. Choose $u = -x_2 + \dot{\alpha}_1 - x_1 - k_2 z_2$.

Q2: In the two-step backstepping controller, what happens if we omit the cross-coupling term $-z_1$ from the torque law? Is stability still guaranteed?

Answer: Without $-z_1$, the Lyapunov derivative becomes $\dot{V}_2 = -k_1 z_1^2 - k_2 z_2^2 + z_1^T z_2$. The cross-term $z_1^T z_2$ can be positive, so $\dot{V}_2$ is not guaranteed negative semi-definite. Stability is only ensured if $k_1 k_2 > \frac{1}{4}$ (by

Young's inequality), but the stability margin is reduced. The systematic backstepping design avoids this issue entirely.

Q3: Why does the quadrotor need four backstepping steps while the manipulator only needs two?

Answer: The quadrotor has a four-level cascade: position velocity orientation angular rate, each acting as a virtual control for the previous level. The manipulator only has two levels (position velocity) because the torque directly enters the velocity dynamics via $M^{-1}\tau$. The number of backstepping steps equals the relative degree from the output to the input.

6.6.4 Summary: Comparison of All Five Control Methods

Aspect	Computed Torque (Wk1)
Core idea	Cancel nonlinearity, impose linear error dynamics
Model requirement	Exact model needed
Lyapunov function	Implicit (linear closed-loop)
Handles constraints	No
Robustness	Poor (sensitive to model errors)
Computation	Low (closed-form)
Key advantage	Simple, well-understood
Key limitation	No robustness to model error
Control law form	$\tau = M(q)[\ddot{q}_d + K_D\dot{e} + K_P e] + C\dot$

Figure 8: Side-by-side comparison of feedback linearization (cancels all nonlinearities) versus backstepping (exploits useful nonlinearities with constructive Lyapunov guarantees).

Key takeaway: Backstepping provides a systematic, constructive methodology for designing stabilising controllers for nonlinear systems in strict-feedback form. Unlike computed torque, it does not blindly cancel all nonlinearities. Unlike robust control, it does not require conservative worst-case bounds. Its main challenge — the "explosion of terms" as $\dot{\alpha}_i$ grows with each step — can be addressed using dynamic surface control (a first-order filter on each virtual control), which we may explore in future weeks.

M.Sc. Control for Robots — University of West London — AY 2025–26

Week 5: Backstepping Control — Lecture Notes

These notes are for educational purposes. Students are encouraged to verify all derivations independently.

7 Sliding Mode Control (Week 7)

Variable Structure Control — Robust Tracking Under Uncertainty

M.Sc. Control for Robots University of West London AY 2025–26
Week 7 of 12 Lecture + Hands-on MATLAB Lab

7.1 Overview

7.1.1 Week 6 Recall: Backstepping Control — Exam Practice

7.1.2 Question 1 — Scalar Strict-Feedback System

Consider the strict-feedback system:

\[\dot{x}_1 = x_2 + x_1^2, \qquad \dot{x}_2 = u. \]

7.1.2.1 Step 1 — Virtual Control Design

Define the first error variable:

\[z_1 = x_1. \]

Choose the first Lyapunov function:

\[V_1 = \tfrac{1}{2}\,z_1^2. \]

Its time derivative is:

\[\dot{V}_1 = z_1\,\dot{z}_1 = z_1\bigl(x_2 + x_1^2\bigr). \]

We want \(\dot{V}_1 = -k_1 z_1^2\) when \(x_2\) equals a virtual control \(\alpha_1(x_1)\). Setting

\[z_1\bigl(\alpha_1 + x_1^2\bigr) = -k_1 z_1^2 \quad\Longrightarrow\quad \alpha_1(x_1) = -k_1 x_1 - x_1^2. \]

7.1 Overview 99
7.1.1 Week 6 Recall: Backstepping Control — Exam Practice 99
7.1.2 Question 1 — Scalar Strict-Feedback System 99
7.1.3 Question 2 — 2-DOF Manipulator Backstepping 100
7.1.4 Learning Outcomes . . 101
7.1.5 Session Roadmap 101
7.2 Sliding Mode Control Theory 102
7.2.1 1.1 Motivation and Core Idea 102
7.2.2 1.2 Two Phases of SMC 103
7.2.3 1.3 Sliding Surface Design 103
7.2.4 1.4 Reaching Law . . 103
7.2.5 1.5 Control Law Decomposition 104
7.2.6 1.6 Existence Condition (Reaching Condition) 105
7.2.7 1.7 The Chattering Problem 105
7.2.8 1.8 Multi-Variable Extension 107
7.3 SMC for Robot Manipulator 107
7.3.1 2.1 System Model . . 107
7.3.2 2.2 Sliding Surface . . 108
7.3.3 2.3 Control Law Derivation 108
7.3.4 2.4 Lyapunov Stability Proof 109
7.3.5 2.5 MATLAB Implementation — 2-DOF Manipulator SMC . . . 109
7.4 SMC for Mobile Robot 110
7.4.1 3.1 Unicycle Kinematic Model 110
7.4.2 3.2 Position Tracking Formulation 110

7.1.2.2 Step 2 — Actual Control Design

Define the second error variable:

$$ z_2 = x_2 - \alpha_1(x_1). $$

Note that $x_2 = z_2 + \alpha_1$, so

$$ \dot{V}_1 = z_1(z_2 + \alpha_1 + x_1^2) = -k_1 z_1^2 + z_1 z_2. $$

Augment the Lyapunov function:

$$ V_2 = V_1 + \tfrac{1}{2}\,z_2^2 = \tfrac{1}{2}z_1^2 + \tfrac{1}{2}z_2^2. $$

Differentiate z_2:

$$ \dot{z}_2 = \dot{x}_2 - \dot{\alpha}_1 = u - \frac{\partial \alpha_1}{\partial x_1}\dot{x}_1 = u - (-k_1 - 2x_1)(x_2 + x_1^2). $$

Compute $\dot{V}_2$:

$$ \dot{V}_2 = -k_1 z_1^2 + z_1 z_2 + z_2\dot{z}_2 = -k_1 z_1^2 + z_2\bigl[z_1 + u + (k_1 + 2x_1)(x_2 + x_1^2)\bigr]. $$

To obtain $\dot{V}_2 = -k_1 z_1^2 - k_2 z_2^2$, choose:

$$ u = -z_1 - k_2 z_2 - (k_1 + 2x_1)(x_2 + x_1^2). $$

This yields $\dot{V}_2 = -k_1 z_1^2 - k_2 z_2^2 \le 0$, guaranteeing global asymptotic stability.

7.1.3 Question 2 — 2-DOF Manipulator Backstepping

Given the manipulator dynamics $M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau$, define:

$$ z_1 = q - q_d, \quad \alpha_1 = \dot{q}_d - K_1 z_1, \quad z_2 = \dot{q} - \alpha_1. $$

Choose the composite Lyapunov function:

$$ V_2 = \tfrac{1}{2}\,z_1^T z_1 + \tfrac{1}{2}\,z_2^T M(q)\,z_2. $$

Differentiate:

$$ \dot{V}_2 = z_1^T \dot{z}_1 + z_2^T M(q)\dot{z}_2 + \tfrac{1}{2}\,z_2^T \dot{M}(q)\,z_2. $$

Since $\dot{z}_1 = \dot{q} - \dot{q}_d = z_2 + \alpha_1 - \dot{q}_d = z_2 - K_1 z_1$, we get:

$$ z_1^T \dot{z}_1 = -z_1^T K_1 z_1 + z_1^T z_2. $$

For the second term, note $\dot{z}_2 = \ddot{q} - \dot{\alpha}_1$, and from the dynamics $M\ddot{q} = \tau - C\dot{q} - G$, so:

$$ M\dot{z}_2 = \tau - C\dot{q} - G - M\dot{\alpha}_1. $$

Using the skew-symmetry property \(z_2^T(\dot{M} - 2C)z_2 = 0\):

\[\dot{V}_2 = -z_1^T K_1 z_1 + z_1^T z_2 + z_2^T\bigl[\tau - C\dot{q} - G - M\dot{\alpha}_1 + C z_2\bigr]. \]

Simplifying \(-C\dot{q} + Cz_2 = -C(\dot{q} - z_2) = -C\alpha_1\), so:

\[\dot{V}_2 = -z_1^T K_1 z_1 + z_2^T\bigl[z_1 + \tau - C\alpha_1 - G - M\dot{\alpha}_1\bigr]. \]

Choose the backstepping torque:

\[\tau = M\dot{\alpha}_1 + C\alpha_1 + G - z_1 - K_2 z_2 \]

which yields \(\dot{V}_2 = -z_1^T K_1 z_1 - z_2^T K_2 z_2 \le 0\).

7.1.4 Learning Outcomes

- Understand the two phases of sliding mode control: the reaching phase (driving the state to the sliding surface) and the sliding phase (reduced-order dynamics on \(s=0\)); derive the equivalent control.
- Design sliding surfaces for second-order systems and multi-DOF robots: choose surface parameters \(\lambda\) or \(\Lambda\) to assign desired closed-loop error dynamics.
- Derive complete SMC laws for three robotic platforms: manipulator, mobile robot, and quadrotor drone.
- Analyse and mitigate the chattering problem using the boundary layer approach (\(\operatorname{sat}(s/\varphi)\)), and the super-twisting algorithm.
- Implement SMC in MATLAB/Simulink, inject disturbances, and compare performance against computed torque, robust control, and backstepping.

7.1.5 Session Roadmap

0:00 – 0:15

7.1.5.1 Week 6 Recall — Backstepping Exam Practice

Two worked exam questions on backstepping control. Students attempt before solutions are revealed.

0:15 – 0:55

7.1.5.2 Sliding Mode Control Theory

Reaching phase, sliding phase, sliding surface design, reaching law, control law decomposition (equivalent + switching), existence condition, chattering, super-twisting algorithm. Full Lyapunov stability proof.

0:55 – 1:15

7.1.5.3 SMC for Robot Manipulator

Sliding surface for joint-space tracking, equivalent control, switching control, Lyapunov proof, and MATLAB implementation.

1:15 – 1:35

7.1.5.4 SMC for Mobile Robot

Unicycle kinematics, position-tracking sliding surface, SMC law, MATLAB code.

1:35 – 1:55

7.1.5.5 SMC for Quadrotor Drone

Hierarchical SMC: position loop (outer) and attitude loop (inner), sliding surfaces, control allocation, MATLAB code.

1:55 – 2:30

7.1.5.6 Activities & Lab

Activity 1: Chattering observation with sign vs sat. Activity 2: Disturbance rejection comparison. Activity 3: Super-twisting implementation.

2:30 – 2:45

7.1.5.7 Summary & Six-Method Comparison

Consolidated table comparing all six control methods studied in Weeks 1–6.

7.2 Sliding Mode Control Theory

7.2.1 1.1 Motivation and Core Idea

Sliding mode control (SMC) is a variable structure control method. The controller switches between two or more structures depending on which side of a chosen surface the state currently lies. The key benefit is invariance: once the state reaches the sliding surface, the closed-loop dynamics become completely independent of matched uncertainties and disturbances.

Key Insight: SMC decomposes the control problem into two subproblems: (1) design a surface on which the error dynamics are desirable, and (2) design a control law that drives the state to that surface in finite time.

Figure 1: Phase portrait of sliding mode control showing two phases. Blue trajectories approach the sliding surface (reaching phase), then

follow it toward the origin (sliding phase, green). The red line is the sliding surface s = 0.

7.2.2 1.2 Two Phases of SMC

7.2.2.1 Phase 1: Reaching Phase

The controller drives the system state from any initial condition to the sliding surface \(s = 0\). During this phase the system is sensitive to uncertainties. The reaching phase ends in finite time thanks to the discontinuous switching term.

7.2.2.2 Phase 2: Sliding Phase

Once \(s = 0\), the state is constrained to the surface. The system dynamics reduce in order by the number of sliding variables. The resulting motion is governed entirely by the surface parameters, not the plant model.

7.2.3 1.3 Sliding Surface Design

Consider a second-order system with tracking error \(e = x - x_d\). Define the sliding surface:

\[s = \dot{e} + \lambda\, e, \qquad \lambda > 0. \]

When the state is on the surface (\(s = 0\)), the error dynamics become:

\[\dot{e} = -\lambda\, e \quad\Longrightarrow\quad e(t) = e(0)\,e^{-\lambda t}. \]

The error converges exponentially with time constant \(\tau = 1/\lambda\). For an \(n\)-th order system, we generalise to:

\[s = \left(\frac{d}{dt} + \lambda\right)^{n-1} e. \]

For example, for a third-order system (\(n=3\)):

\[s = \ddot{e} + 2\lambda\dot{e} + \lambda^2 e. \]

7.2.4 1.4 Reaching Law

To drive \(s \to 0\) in finite time, we impose a reaching law on \(\dot{s}\). A common choice is the constant + proportional reaching law:

\[\dot{s} = -\eta\,\operatorname{sign}(s) - k\,s, \qquad \eta > 0,\; k > 0, \]

where

\[\operatorname{sign}(s) = \begin{cases} +1 & s > 0 \\ 0 & s = 0 \\ -1 & s < 0 \end{cases}. \]

The term $-\eta\,\operatorname{sign}(s)$ guarantees finite-time reaching, while $-k\,s$ speeds up convergence when $|s|$ is large.

Figure 2: Reaching law comparison. (a) Constant rate: s decreases linearly. (b) Exponential + constant: faster initial approach, then finite-time reaching. (c) Power rate: accelerates convergence near the surface for smoother arrival.

7.2.5 1.5 Control Law Decomposition

Consider a general second-order uncertain system:

$$\ddot{x} = f(x, \dot{x}) + b(x)\,u + d(t),$$

where f contains known/nominal dynamics, b is the input gain, and $d(t)$ is a bounded matched disturbance with $|d(t)| \le D$.

With $s = \dot{e} + \lambda e$, differentiate:

$$\dot{s} = \ddot{e} + \lambda\dot{e} = \ddot{x} - \ddot{x}_d + \lambda\dot{e} = f + b\,u + d - \ddot{x}_d + \lambda\dot{e}.$$

The control law is decomposed as:

$$u = u_{\text{eq}} + u_{\text{sw}}$$

Equivalent Control u_{eq}

7.2.5.1 Derivation

The equivalent control is obtained by setting $\dot{s} = 0$ and $d = 0$ (nominal conditions):

$$0 = f + b\,u_{\text{eq}} - \ddot{x}_d + \lambda\dot{e}$$

$$u_{\text{eq}} = \frac{1}{b}\bigl(\ddot{x}_d - \lambda\dot{e} - f\bigr).$$

This term cancels the known dynamics and commands the desired trajectory — it is the smooth, model-based part of the control.

Switching Control u_{sw}

7.2.5.2 Derivation

The switching control handles uncertainties and disturbances:

$$u_{\text{sw}} = -\frac{1}{b}\bigl(\eta\,\operatorname{sign}(s) + k\,s\bigr), \qquad \eta > D.$$

The gain η must exceed the disturbance bound D to guarantee reaching.

The complete SMC law is therefore:

$$u = \frac{1}{b}\bigl(\ddot{x}_d - \lambda\dot{e} - f - \eta\,\operatorname{sign}(s) - k\,s\bigr).$$

Figure 3: SMC block diagram. The controller decomposes into equivalent control u_eq (model-based, maintains system on surface) and switching control u_sw (drives system to surface and rejects disturbances). The sliding variable s is computed from state feedback.

7.2.6 1.6 Existence Condition (Reaching Condition)

For the state to reach and remain on the surface, we need:

\[s\,\dot{s} < 0 \qquad \forall\; s \neq 0. \]

This is equivalent to requiring that \(s\) decreases in magnitude regardless of its sign. Using a Lyapunov approach, choose \(V = \tfrac{1}{2}s^2\):

\[\dot{V} = s\,\dot{s} < 0. \]

7.2.6.1 Lyapunov Stability Proof

Substituting the full control into \(\dot{s}\):

\[\dot{s} = f + b(u_{\text{eq}} + u_{\text{sw}}) + d - \ddot{x}_d + \lambda\dot{e}. \]

The equivalent control cancels nominal terms:

\[\dot{s} = d - \eta\,\operatorname{sign}(s) - k\,s. \]

Now compute \(s\,\dot{s}\):

\[s\,\dot{s} = s\,d - \eta\,|s| - k\,s^2 \le |s|\,D - \eta\,|s| - k\,s^2 = -(\eta - D)|s| - k\,s^2. \]

Since \(\eta > D\) and \(k > 0\), we have \(s\,\dot{s} < 0\) for all \(s \neq 0\). This proves:

1. The sliding surface is reached in finite time.
2. Once reached, the state cannot leave the surface.

Finite reaching time: With \(k = 0\) (pure sign), \(\dot{V} = s\dot{s} \le -(\eta - D)|s|\). Since \(V = \frac{1}{2}s^2\), we get \(|s| = \sqrt{2V}\), so \(\dot{V} \le -(\eta - D)\sqrt{2V}\). Solving this differential inequality gives:

\[t_{\text{reach}} \le \frac{|s(0)|}{\eta - D}. \]

7.2.7 1.7 The Chattering Problem

The discontinuous \(\operatorname{sign}(s)\) function causes the control to switch at infinite frequency near \(s = 0\). In practice, actuator bandwidth is finite, producing chattering — high-frequency oscillations around the sliding surface.

Why chattering is harmful: It excites unmodelled high-frequency plant dynamics, causes mechanical wear, increases energy consumption, and can destabilise the system.

Figure 4: Chattering problem visualization. Left: ideal SMC with infinite switching frequency -- trajectory reaches and stays on the surface. Right: practical SMC with finite actuation bandwidth -- trajectory oscillates around the surface, producing high-frequency control chatter that causes actuator wear.

7.2.7.1 Solution 1: Boundary Layer (Saturation Function)

Replace \(\operatorname{sign}(s)\) with a continuous approximation inside a boundary layer of width \(\varphi\):

\[\operatorname{sat}\!\left(\frac{s}{\varphi}\right) = \begin{cases} \displaystyle\frac{s}{\varphi} & |s| \le \varphi \\[6pt] \operatorname{sign}(s) & |s| > \varphi \end{cases} \]

Inside the boundary layer \(|s| \le \varphi\), the control becomes a high-gain linear controller. This eliminates chattering but introduces a steady-state error proportional to \(\varphi\). The trade-off is:

- Small \(\varphi\): less steady-state error, more chattering.
- Large \(\varphi\): no chattering, larger steady-state error.

Figure 5: Boundary layer approach for chattering mitigation. Left: phase plane showing the boundary layer band (orange) around the sliding surface (red). Outside the boundary, discontinuous sign(s) is used; inside, smooth sat(s/phi) provides linear control. Right: comparison of sign(s) (discontinuous) vs sat(s/phi) (continuous).

7.2.7.2 Solution 2: Super-Twisting Algorithm

The super-twisting algorithm achieves finite-time convergence to \(s = 0\) with a continuous control signal (no chattering). It uses an integral of the sign function:

\[\dot{s} = -k_1\,|s|^{1/2}\,\operatorname{sign}(s) + v, \qquad \dot{v} = -k_2\,\operatorname{sign}(s). \]

Equivalently, the switching control becomes:

\[u_{\text{sw}} = -\frac{1}{b}\Bigl(k_1\,|s|^{1/2}\,\operatorname{sign}(s) + k_2\!\int_0^t \operatorname{sign}(s)\,d\tau\Bigr). \]

The discontinuity is hidden inside the integrator, so the actual control \(u\) is continuous. Sufficient conditions for convergence are \(k_1 > 0\) and \(k_2 > k_1 D / (k_1 - D)\) (with appropriate scaling).

Figure 6: Super-twisting algorithm comparison. Left: standard SMC produces chattering at the surface with discontinuous control. Right: super-twisting achieves finite-time spiral convergence to the origin with a fully continuous control signal, eliminating chattering completely.

7.2.7.3 Think-Pair-Share

Question: Why does \(\operatorname{sign}(s)\) cause chattering? Draw the phase portrait of a second-order system under SMC with \(\operatorname{sign}(s)\), showing the trajectories switching direction across \(s = 0\). How does the boundary layer modify this portrait?

Discuss (2 min): In the phase plane \((e, \dot{e})\), the sliding surface is a line. Trajectories approach from both sides and, due to finite sampling/actuation delay, overshoot repeatedly. With the boundary layer, trajectories enter the band \(|s| < \varphi\) and glide smoothly to the origin along a thickened corridor.

7.2.8 1.8 Multi-Variable Extension

For an \(n\)-DOF system with error \(\mathbf{e} \in \mathbb{R}^n\), define the sliding surface vector:

\[\mathbf{s} = \dot{\mathbf{e}} + \Lambda\,\mathbf{e}, \qquad \Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n),\; \lambda_i > 0. \]

The Lyapunov function becomes:

\[V = \tfrac{1}{2}\,\mathbf{s}^T \mathbf{s}, \]

and the reaching condition generalises to:

\[\dot{V} = \mathbf{s}^T \dot{\mathbf{s}} < 0. \]

7.3 SMC for Robot Manipulator

7.3.1 2.1 System Model

The \(n\)-DOF rigid-body manipulator dynamics:

\[M(q)\,\ddot{q} + C(q, \dot{q})\,\dot{q} + G(q) = \tau + d(t), \]

where \(d(t)\) is a bounded disturbance torque with \(|d(t)| \le D\). We have nominal estimates \(\hat{M}, \hat{C}, \hat{G}\) and bounded modelling errors:

\[\Delta M = M - \hat{M}, \quad \Delta C = C - \hat{C}, \quad \Delta G = G - \hat{G}. \]

Figure 7: 2-DOF planar manipulator schematic. Joint angles theta_1, theta_2 are controlled by torques tau_1, tau_2. Link parameters (m_1, l_1, m_2, l_2) are subject to uncertainty (dashed clouds). Gravity g acts downward, and external disturbances d(t) affect the end-effector.

7.3.2 2.2 Sliding Surface

Define the joint tracking error and sliding surface:

\[e = q - q_d, \qquad s = \dot{e} + \Lambda\,e = \dot{q} - \dot{q}_r, \]

where the reference velocity is defined as:

\[\dot{q}_r \triangleq \dot{q}_d - \Lambda\,e. \]

This notation simplifies the derivation considerably. Note also:

\[\ddot{q}_r = \ddot{q}_d - \Lambda\,\dot{e}. \]

7.3.3 2.3 Control Law Derivation

7.3.3.1 Full Derivation

Step 1: Differentiate \(s\):

\[\dot{s} = \ddot{q} - \ddot{q}_r. \]

From the dynamics: \(\ddot{q} = M^{-1}(\tau + d - C\dot{q} - G)\), so:

\[M\dot{s} = \tau + d - C\dot{q} - G - M\ddot{q}_r. \]

Step 2: Rewrite using \(\dot{q} = s + \dot{q}_r\):

\[M\dot{s} = \tau + d - C(s + \dot{q}_r) - G - M\ddot{q}_r = \tau - M\ddot{q}_r - C\dot{q}_r - G - Cs + d. \]

Step 3: The equivalent control (using nominal estimates, \(d=0\), \(\dot{s}=0\)):

\[\tau_{\text{eq}} = \hat{M}(q)\,\ddot{q}_r + \hat{C}(q,\dot{q})\,\dot{q}_r + \hat{G}(q). \]

Expanding \(\ddot{q}_r\) and \(\dot{q}_r\):

\[\tau_{\text{eq}} = \hat{M}(q)\bigl[\ddot{q}_d - \Lambda\dot{e}\bigr] + \hat{C}(q,\dot{q})\bigl[\dot{q}_d - \Lambda e\bigr] + \hat{G}(q). \]

Step 4: The switching control:

\[\tau_{\text{sw}} = -K\,\operatorname{sign}(s), \]

where \(K = \operatorname{diag}(K_1, \dots, K_n)\) with each \(K_i\) large enough to overcome uncertainty bounds.

Step 5: Complete SMC torque:

\[\tau = \hat{M}\ddot{q}_r + \hat{C}\dot{q}_r + \hat{G} - K\,\operatorname{sign}(s) \]

Or with the boundary layer:

\[\tau = \hat{M}\ddot{q}_r + \hat{C}\dot{q}_r + \hat{G} - K\,\operatorname{sat}\!\left(\frac{s}{\varphi}\right) \]

7.3.4 2.4 Lyapunov Stability Proof

7.3.4.1 Proof of Finite-Time Reaching

Choose \(V = \tfrac{1}{2}\,s^T M(q)\,s\). Note \(V > 0\) since \(M\) is positive definite. Differentiate:

\[\dot{V} = s^T M\dot{s} + \tfrac{1}{2}\,s^T \dot{M}\,s. \]

Substituting \(M\dot{s} = \tau + d - C\dot{q} - G - M\ddot{q}_r\) and the control law:

\[M\dot{s} = \hat{M}\ddot{q}_r + \hat{C}\dot{q}_r + \hat{G} - K\operatorname{sign}(s) + d - C\dot{q} - G - M\ddot{q}_r. \]

Grouping terms:

\[M\dot{s} = -\Delta M\ddot{q}_r - \Delta C\dot{q}_r - \Delta G - Cs + d - K\operatorname{sign}(s), \]

where we used \(C\dot{q} = C(s + \dot{q}_r) = Cs + C\dot{q}_r\). Therefore:

\[\dot{V} = s^T\bigl[-\Delta M\ddot{q}_r - \Delta C\dot{q}_r - \Delta G + d - K\operatorname{sign}(s)\bigr] + s^T(-Cs) + \tfrac{1}{2}s^T\dot{M}s. \]

By the skew-symmetry property \(s^T(\dot{M} - 2C)s = 0\), the last two terms cancel:

\[\dot{V} = s^T\bigl[\underbrace{-\Delta M\ddot{q}_r - \Delta C\dot{q}_r - \Delta G + d}_{\text{lumped uncertainty } \delta} - K\operatorname{sign}(s)\bigr]. \]

Let \(|\delta_i| \le \rho_i\) (component-wise bound). Then choosing \(K_i > \rho_i\):

\[\dot{V} \le -\sum_{i=1}^{n} (K_i - \rho_i)|s_i| < 0, \qquad \forall\; s \neq 0. \]

This guarantees finite-time reaching and invariance of the sliding surface.

7.3.5 2.5 MATLAB Implementation — 2-DOF Manipulator SMC

%% SMC for 2-DOF Manipulator % Parameters m1 = 1; m2 = 1; l1 = 1; l2 = 1; g = 9.81; Lambda = diag([10, 10]); % sliding surface gain K_smc = diag([20, 20]); % switching gain phi = 0.05; % boundary layer width use_sat = true; % true = sat(), false = sign() % Desired trajectory qd = @(t) [sin(t); cos(t)]; qd_d = @(t) [cos(t); -sin(t)]; qd_dd= @(t) [-sin(t); -cos(t)]; % Dynamics (inline for ODE) function [M, C, G_vec] = manipulator_dynamics(q, qd_vec, m1, m2, l1, l2, g) c2 = cos(q(2)); s2 = sin(q(2)); h = m2*l1*l2*c2; M = [m1*l1^2 + m2*(l1^2+l2^2+2*l1*l2*c2), m2*(l2^2+l1*l2*c2); m2*(l2^2+l1*l2*c2), m2*l2^2]; C = [-m2*l1*l2*s2*qd_vec(2), -m2*l1*l2*s2*(qd_vec(1)+qd_vec(2)); m2*l1*l2*s2*qd_vec(1), 0]; G_vec = [m1*g*l1*cos(q(1)) + m2*g*(l1*cos(q(1))+l2*cos(q(1)+q(2))); m2*g*l2*cos(q(1)+q(2))]; end

% ODE function odefun = @(t, x) smc_manipulator_ode(t, x, ... m1, m2, l1, l2, g, Lambda, K_smc, phi, use_sat, qd, qd_d, qd_dd); [T, X] = ode45(odefun, [0, 10], [0.5; -0.5; 0; 0]); % Plot results figure; subplot(2,1,1); plot(T, X(:,1:2)); legend('q_1','q_2'); title('Joint Angles'); subplot(2,1,2); plot(T, X(:,3:4)); legend('dq_1','dq_2'); title('Joint Velocities');

Note: The complete ODE function `smc_manipulator_ode()` with all sub-functions is provided in the companion file `smc_control_simulation.m`.

7.4 SMC for Mobile Robot

7.4.1 3.1 Unicycle Kinematic Model

The unicycle model for a differential-drive mobile robot:

\[\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega, \]

where \(v\) is the linear velocity and \(\omega\) is the angular velocity. The control inputs are \(u = [v,\; \omega]^T\).

Figure 8: Mobile robot SMC tracking. Left: top-down view showing the differential-drive robot tracking a reference trajectory, with tracking errors (e_x, e_y) and polar error rho. Right: sliding manifold in error space showing trajectories converging to the sliding surface then sliding to the origin.

7.4.2 3.2 Position Tracking Formulation

Define the tracking errors in the body frame. Let \((x_d(t), y_d(t), \theta_d(t))\) be the desired trajectory. The position errors in the world frame are:

\[e_x = x_d - x, \qquad e_y = y_d - y. \]

Transform to the robot body frame:

\[\begin{bmatrix} e_1 \\ e_2 \\ e_3 \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} e_x \\ e_y \\ \theta_d - \theta \end{bmatrix}. \]

7.4.3 3.3 Sliding Surface Design

For position tracking, define two sliding surfaces:

\[s_1 = \dot{e}_1 + \lambda_1 e_1, \qquad s_2 = \dot{e}_2 + \lambda_2 e_2. \]

An alternative approach for the unicycle (which is non-holonomic and cannot track \(e_2\) independently) is to use a polar coordinate formulation. Define:

\[\rho = \sqrt{e_x^2 + e_y^2}, \qquad \alpha = \operatorname{atan2}(e_y, e_x) - \theta, \qquad \beta = -\theta - \alpha. \]

Then define sliding surfaces:

\[s_v = \dot{\rho} + \lambda_v \rho, \qquad s_\omega = \dot{\alpha} + \lambda_\omega \alpha. \]

7.4.4 3.4 SMC Control Law

Using the body-frame error dynamics and the reaching law, the SMC control inputs become:

7.4.4.1 Derivation

The error dynamics in polar coordinates are:

\[\dot{\rho} = -v\cos\alpha, \qquad \dot{\alpha} = -\omega + \frac{v\sin\alpha}{\rho}, \qquad \dot{\beta} = -\frac{v\sin\alpha}{\rho}. \]

For \(s_v = \dot{\rho} + \lambda_v \rho\): set \(\dot{s}_v = -\eta_v \operatorname{sign}(s_v)\):

\[v_{\text{eq}} = \frac{\lambda_v \rho}{\cos\alpha}, \qquad v_{\text{sw}} = \frac{\eta_v \operatorname{sign}(s_v)}{\cos\alpha}. \]

For \(s_\omega = \dot{\alpha} + \lambda_\omega \alpha\): set \(\dot{s}_\omega = -\eta_\omega \operatorname{sign}(s_\omega)\):

\[\omega = \frac{v\sin\alpha}{\rho} + \lambda_\omega \alpha + \eta_\omega \operatorname{sign}(s_\omega). \]

\[v = \frac{\lambda_v \rho + \eta_v \operatorname{sat}(s_v / \varphi_v)}{\cos\alpha}, \qquad \omega = \frac{v\sin\alpha}{\rho} + \lambda_\omega \alpha + \eta_\omega \operatorname{sat}(s_\omega / \varphi_\omega). \]

Singularity: When \(\alpha = \pm\pi/2\) (goal directly to the side), \(\cos\alpha = 0\). In practice, clamp \(|\alpha| < \pi/2 - \epsilon\) or switch to a Cartesian controller near the singularity.

7.4.5 3.5 MATLAB Implementation

%% SMC for Mobile Robot (Unicycle) lambda_v = 1.0; lambda_w = 2.0; eta_v = 0.5; eta_w = 1.0; phi_v = 0.05; phi_w = 0.05; % Desired circular trajectory xd = @(t) 2*cos(0.3*t); yd = @(t) 2*sin(0.3*t); xd_d = @(t) -0.6*sin(0.3*t); yd_d = @(t) 0.6*cos(0.3*t); odefun = @(t, x) smc_mobile_ode(t, x, lambda_v, lambda_w, ... eta_v, eta_w, phi_v, phi_w, xd, yd, xd_d, yd_d); [T, X] = ode45(odefun, [0 30], [0; 0; pi/4]); figure; plot(X(:,1), X(:,2), 'b-', arrayfun(xd,T), arrayfun(yd,T), 'r--'); legend('Actual','Desired'); title('Mobile Robot SMC Tracking'); axis equal; grid on;

7.5 SMC for Quadrotor Drone

7.5.1 4.1 Quadrotor Dynamic Model

The 6-DOF quadrotor dynamics separate into translational and rotational subsystems. The position dynamics in the world frame:

\[m\ddot{x} = (\cos\psi\sin\theta\cos\phi + \sin\psi\sin\phi)\,U_1 + d_x, \] \[m\ddot{y} = (\sin\psi\sin\theta\cos\phi - \cos\psi\sin\phi)\,U_1 + d_y, \] \[m\ddot{z} = (\cos\theta\cos\phi)\,U_1 - mg + d_z, \]

where \(U_1\) is the total thrust, \((\phi,\theta,\psi)\) are roll, pitch, yaw, and \(d_x, d_y, d_z\) are disturbance forces. The attitude dynamics:

\[I_{xx}\ddot{\phi} = (\,I_{yy} - I_{zz})\dot{\theta}\dot{\psi} + l\,U_2 + d_\phi, \] \[I_{yy}\ddot{\theta} = (I_{zz} - I_{xx})\dot{\phi}\dot{\psi} + l\,U_3 + d_\theta, \] \[I_{zz}\ddot{\psi} = (I_{xx} - I_{yy})\dot{\phi}\dot{\theta} + c\,U_4 + d_\psi. \]

7.5.2 4.2 Hierarchical SMC Design

The standard approach uses a cascaded (hierarchical) structure:

7.5.2.1 Outer Loop: Position SMC

Computes desired thrust \(U_1\) and desired roll/pitch angles \((\phi_d, \theta_d)\) from position errors.

7.5.2.2 Inner Loop: Attitude SMC

Tracks desired roll, pitch, yaw using torque inputs \(U_2, U_3, U_4\).

Figure 9: Quadrotor with hierarchical SMC. Four rotors produce thrust forces F_1-F_4. The outer loop (position SMC) computes total thrust U_1 and desired attitudes. The inner loop (attitude SMC) tracks desired roll, pitch, yaw. Wind disturbances are rejected by the sliding mode controller.

7.5.2.3 Position Sliding Surfaces

Define position errors \(e_x = x - x_d\), \(e_y = y - y_d\), \(e_z = z - z_d\), and sliding surfaces:

\[s_x = \dot{e}_x + \lambda_x e_x, \quad s_y = \dot{e}_y + \lambda_y e_y, \quad s_z = \dot{e}_z + \lambda_z e_z. \]

7.5.2.4 Altitude Control (\(z\)-axis)

7.5.2.5 Derivation

From \(m\ddot{z} = U_1\cos\theta\cos\phi - mg + d_z\), define \(e_z = z - z_d\):

\[\dot{s}_z = \ddot{e}_z + \lambda_z \dot{e}_z = \frac{U_1\cos\theta\cos\phi}{m} - g + \frac{d_z}{m} - \ddot{z}_d + \lambda_z\dot{e}_z. \]

Setting \(\dot{s}_z = -\eta_z \operatorname{sign}(s_z) - k_z s_z\) and solving for \(U_1\):

\[U_1 = \frac{m}{\cos\theta\cos\phi}\bigl[\ddot{z}_d + g - \lambda_z\dot{e}_z - \eta_z\operatorname{sat}(s_z/\varphi_z) - k_z s_z\bigr]. \]

7.5.2.6 Lateral Control (\(x\)- and \(y\)-axes)

7.5.2.7 Derivation

From the \(x\)- and \(y\)-dynamics, the SMC computes desired virtual accelerations:

\[\ddot{x}_{\text{cmd}} = \ddot{x}_d - \lambda_x\dot{e}_x - \eta_x\operatorname{sat}(s_x/\varphi_x) - k_x s_x, \] \[\ddot{y}_{\text{cmd}} = \ddot{y}_d - \lambda_y\dot{e}_y - \eta_y\operatorname{sat}(s_y/\varphi_y) - k_y s_y. \]

The desired roll and pitch are then extracted:

\[\theta_d = \arctan\!\left(\frac{\ddot{x}_{\text{cmd}}\cos\psi + \ddot{y}_{\text{cmd}}\sin\psi}{\ddot{z}_{\text{cmd}} + g}\right), \] \[\phi_d = \arctan\!\left(\frac{(\ddot{x}_{\text{cmd}}\sin\psi - \ddot{y}_{\text{cmd}}\cos\psi)\cos\theta_d}{\ddot{z}_{\text{cmd}} + g}\right). \]

7.5.2.8 Attitude Sliding Surfaces and Control

Define attitude errors \(e_\phi = \phi - \phi_d\), etc., and sliding surfaces:

\[s_\phi = \dot{e}_\phi + \lambda_\phi e_\phi, \quad s_\theta = \dot{e}_\theta + \lambda_\theta e_\theta, \quad s_\psi = \dot{e}_\psi + \lambda_\psi e_\psi. \]

The attitude SMC torques:

\[U_2 = \frac{I_{xx}}{l}\bigl[\ddot{\phi}_d - \lambda_\phi\dot{e}_\phi - \frac{(I_{yy}-I_{zz})}{I_{xx}}\dot{\theta}\dot{\psi} - \eta_\phi\operatorname{sat}(s_\phi/\varphi_\phi)\bigr], \] \[U_3 = \frac{I_{yy}}{l}\bigl[\ddot{\theta}_d - \lambda_\theta\dot{e}_\theta - \frac{(I_{zz}-I_{xx})}{I_{yy}}\dot{\phi}\dot{\psi} - \eta_\theta\operatorname{sat}(s_\theta/\varphi_\theta)\bigr], \] \[U_4 = \frac{I_{zz}}{c}\bigl[\ddot{\psi}_d - \lambda_\psi\dot{e}_\psi

- \frac{(I_{xx}-I_{yy})}{I_{zz}}\dot{\phi}\dot{\theta} - \eta_-\psi\operatorname{sat}(s_\psi/\varphi_\psi)\bigr]. \]

7.5.3 4.3 MATLAB Implementation

%% SMC for Quadrotor Drone % Physical parameters m = 1.0; g_val = 9.81; Ixx = 0.01; Iyy = 0.01; Izz = 0.02; l_arm = 0.2; c_drag = 0.01; % SMC gains (position) lam_pos = [2; 2; 3]; eta_pos = [1; 1; 2]; k_pos = [1; 1; 1]; phi_pos = [0.05; 0.05; 0.05]; % SMC gains (attitude) lam_att = [8; 8; 4]; eta_att = [2; 2; 1]; phi_att = [0.02; 0.02; 0.02]; % Desired trajectory: helix xd = @(t) cos(0.5*t); yd = @(t) sin(0.5*t); zd = @(t) 0.5*t; psi_d = @(t) 0; x0 = [0;0;0; 0;0;0; 0;0;0; 0;0;0]; % [x,y,z, dx,dy,dz, phi,th,psi, dphi,dth,dpsi] [T, X] = ode45(@(t,x) smc_drone_ode(t, x, m, g_val, Ixx, Iyy, Izz, ... l_arm, c_drag, lam_pos, eta_pos, k_pos, phi_pos, ... lam_att, eta_att, phi_att, xd, yd, zd, psi_d), [0 20], x0); figure; plot3(X(:,1), X(:,2), X(:,3), 'b-', ... arrayfun(xd,T), arrayfun(yd,T), arrayfun(zd,T), 'r--'); legend('Actual','Desired'); title('Drone SMC 3D Tracking'); xlabel('x'); ylabel('y'); zlabel('z'); grid on;

7.6 Summary and Next Steps

7.6.1 Lab Activities

7.6.2 Activity 1 — Chattering Observation (10 min)

Objective: Observe and quantify the chattering phenomenon, then mitigate it using the boundary layer.

1. Run the manipulator SMC simulation with `use_sat = false` (pure `sign(s)`). Plot the control torque \(\tau\). Observe the high-frequency switching.
2. Switch to `use_sat = true` and test three boundary layer widths: \(\varphi = 0.01,\; 0.1,\; 1.0\).
3. For each value of \(\varphi\), record: (a) peak torque oscillation amplitude, (b) steady-state tracking error \(\|e\|_\infty\).

7.6.2.1 Questions to Answer

- How does boundary layer width \(\varphi\) affect chattering amplitude?
- How does \(\varphi\) affect steady-state tracking accuracy?
- Is there an optimal \(\varphi\)? What criteria would you use to select it?

7.6.3 Activity 2 — Disturbance Rejection Comparison (15 min)

Objective: Compare the robustness of SMC against other control methods under large disturbances.

1. Add a disturbance torque \(d(t) = 10\sin(5t)\) to the manipulator dynamics.
2. Run four controllers:
   - SMC with \(\operatorname{sign}(s)\) and \(K = 15\)
   - SMC with \(\operatorname{sat}(s/\varphi)\), \(\varphi = 0.05\), \(K = 15\)
   - Robust control (from Week 4)
   - Computed torque (from Week 1)
3. Compare tracking errors and control effort. Plot all on the same axes.

7.6.3.1 Questions to Answer

- Which controller maintains the smallest tracking error under the disturbance?
- What happens to the computed torque controller — why does it degrade?
- How does the switching gain \(K\) relate to the disturbance bound \(D = 10\)?

7.6.4 Activity 3 — Super-Twisting Algorithm (5 min)

Objective: Implement the super-twisting algorithm and compare with standard SMC.

1. Replace the switching term with the super-twisting law. The integral state \(v\) requires an extra state variable per joint.
2. Compare the control torque smoothness and tracking error against sign-based and sat-based SMC.
3. Verify that chattering is eliminated while finite-time convergence is preserved.

7.6.5 Summary — All Six Control Methods Compared

Property	Computed Torque (W1)	MPC (W2)
Core Idea	Cancel nonlinear dynamics via feedback linearisation	Optimise over finite horizon with constrai
Model Need	Exact model required	Predictive model required
Robustness	Low — sensitive to modelling error	Medium — via constraints
Disturbance Handling	None (unless augmented)	Preview + constraints
Convergence	Exponential	Depends on horizon/cost
Main Limitation	Model inaccuracy degrades performance	Computational cost; model required
Control Signal	Smooth	Smooth (optimised)
Stability Proof	Input-output linearisation	Terminal cost / constraints
Typical Use Case	Well-modelled manipulators	Constrained systems, trajectory planning

Key Takeaway: Sliding mode control provides the strongest robustness guarantee among all six methods for matched disturbances — once on the sliding surface, the system is completely invariant to matched

uncertainties. The price is chattering, which can be mitigated using boundary layers or the super-twisting algorithm at the cost of slight performance degradation.

Week 6: Sliding Mode Control — M.Sc. Control for Robots

University of West London AY 2025–26

Next Week: Week 7 — TBD

8 Nonlinear Control Optimization for Robotic Systems (Week 8)

From Pontryagin & HJB to Optimized SMC, Adaptive, Robust, Backstepping, and NMPC

M.Sc. Control for Robots — University of West London — AY 2025–26
Week 8 of 12 (Control Theory Block) Lecture + MATLAB Lab

8.1 Overview 117
8.1.1 Learning Outcomes . 117
8.1.2 Session Roadmap . . . 117
8.1.3 MATLAB Lab Activities 118
8.1.4 Setup 118
8.1.5 Activity 1 — SMC Pareto sweep (15 min) 119
8.1.6 Activity 3 — Robust vs. average-case (15 min) 119
8.1.7 Activity 4 — NMPC horizon sweep (15 min) 119
8.1.8 Appendix: Week 7 Recall — Sliding Mode Control (Exam Practice) 119
8.1.9 Course Recap & What's Next 121
8.1.10 The nonlinear control theory journey 121
8.1.11 Unifying themes 121
8.1.12 What's next — Automation block 122
8.2 Why Robots Demand Nonlinear Control Optimization 122
8.2.1 Robot dynamics are globally nonlinear . . . 122
8.2.2 Why Jacobian linearization is not enough . . 123
8.2.3 Two meanings of "control optimization" 123
8.3 Foundations of Nonlinear Control Optimization 123
8.3.1 Notation used throughout Section 4 123
8.3.2 The nonlinear optimal control problem 124
8.3.3 Pontryagin's Minimum Principle (PMP) . . . 124
8.3.4 Hamilton–Jacobi–Bellman (nonlinear) 125
8.3.5 Direct methods: shoot-

8.1 Overview

8.1.1 Learning Outcomes

By the end of this session, you will be able to:

- Explain why linear optimal control (LQR/LQG) is fundamentally insufficient for robotic systems whose dynamics are nonlinear
- State the nonlinear optimal control problem and derive necessary conditions via Pontryagin's Minimum Principle (PMP)
- Formulate the nonlinear HJB equation and explain why direct solution is infeasible for robots
- Describe direct transcription methods (single/multiple shooting, collocation) and apply iLQR / DDP and NMPC to a manipulator, mobile robot, and quadrotor
- Pose the tuning of sliding-mode, adaptive, robust, backstepping, and nonlinear MPC controllers as constrained nonlinear optimization problems
- Select appropriate decision variables, cost functionals, constraints, and solvers for each controller family
- Interpret Pareto trade-offs (chattering vs. precision, speed vs. robustness, horizon vs. compute) through simulation

8.1.2 Session Roadmap

0:00–0:20

8.1.2.1 Why linearization fails for robots

Robot dynamics M(q)q + C(q,q)q + G(q) = are nonlinear globally. Jacobian linearization destroys Coriolis, gravity coupling, and singularity structure.

0:20–1:00

8.1.2.2 Nonlinear optimization foundations

Pontryagin minimum principle, nonlinear HJB, direct transcription, iLQR/DDP, nonlinear MPC.

1:00–2:30

8.1.2.3 Optimization of each nonlinear controller

SMC, Adaptive, Robust, Backstepping, NMPC — per-family decision variables, cost, constraints, and solvers, each demonstrated on manipulator / mobile / drone.

2:30–3:15

8.1.2.4 Unified view and summary

Common structure across families; LMI/SOS/Bayesian optimization/RL as cross-cutting tools; summary comparison.

3:15–4:00

8.1.2.5 MATLAB lab

Run `run_all.m` — observe before/after cost for every controller on every robot. Modify cost weights and re-run.

8.1.3 MATLAB Lab Activities

8.1.4 Setup

All simulations live in `week_08/sims/` and run on base MATLAB (no toolboxes beyond Control System Toolbox required). They use `fminsearch` for the parameter optimization and explicit Euler or `ode`-style integration for the dynamics.

% From MATLAB, cd to the lab directory cd /home/it-services/auto_-control_ws/week_08/sims % Run everything (~30 s on a modern laptop) run_all % Or run a single controller sim_smc % sliding mode sim_adaptive % adaptive / MRAC sim_robust % robust min-max sim_backstepping % backstepping sim_nmpc % nonlinear MPC

8.1.5 Activity 1 — SMC Pareto sweep (15 min)

Inside `sim_smc.m`, change the cost weights \((w_e, w_u, w_c)\) on the chatter term. Re-run and observe:

- Small \(w_c\): optimizer picks a narrow boundary layer (low \(\phi\)) — sharp tracking, chattering control
- Large \(w_c\): optimizer picks a wide boundary layer — smooth torque, residual tracking error

Deliverable: plot the Pareto front of tracking error vs. chatter metric across \(w_c \in \{0.001, 0.01, 0.1, 1\}\).

8.1.5.1 Activity 2 — Adaptive optimizer tells you when not to adapt (10 min)

Set the payload disturbance to zero in `sim_adaptive.m` (make `p_true = p`). Re-run and inspect the optimized \(\Gamma\). Discuss: why does the optimizer drive \(\Gamma\to 0\)?

8.1.6 Activity 3 — Robust vs. average-case (15 min)

Modify `robust_quad_cost` to use the mean rather than max over the uncertainty set. Compare the resulting gains. Under which uncertainty distribution is the average-case design acceptable, and when does the worst-case matter?

8.1.7 Activity 4 — NMPC horizon sweep (15 min)

Run `sim_nmpc` with the horizon set to \(N = \{3,5,7,10,15,25\}\) on the quadrotor. Record closed-loop cost and per-step solve time (`tic/toc` around the `fminsearch` call). Plot both on one graph. Identify the knee of the curve — this is your deployment horizon.

Last run on this machine (R2025b): total wall time ~43 s for all five controllers on all three robots (iLQR-based NMPC dominates the wall-clock). Cost reductions ranged from 2% (manipulator robust — already near worst-case optimum) to 88% (mobile backstepping); NMPC manipulator benefited by 41% from extending the horizon.

8.1.8 Appendix: Week 7 Recall — Sliding Mode Control (Exam Practice)

8.1.8.1 Question 1 — SMC Controller Design and Stability Proof (25 marks)

Consider a robot manipulator governed by:

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau + d(t)$$

where \(d(t)\) represents matched uncertainty with known bound \(|d(t)| \leq D\).

(a) Define the tracking error $e = q - q_d$ and the sliding surface:

$$s = \dot{e} + \Lambda e$$

where $\Lambda = \text{diag}(\lambda_1, \ldots, \lambda_n)$ with $\lambda_i > 0$. Explain why the system trajectories converge to $e = 0$ exponentially once $s = 0$.

8.1.8.2 Model Answer (a)

When $s = 0$, $\dot e = -\Lambda e$; with all $\lambda_i>0$, $e(t)=e(0)e^{-\Lambda t}$ converges exponentially with time constants $1/\lambda_i$.

(b) Derive the equivalent control τ_{eq} by setting $\dot{s} = 0$ (assuming $d = 0$).

8.1.8.3 Model Answer (b)

$$\boxed{\tau_{eq} = M(q)\bigl(\ddot{q}_d - \Lambda\dot{e}\bigr) + C(q,\dot{q})\dot{q} + G(q)}$$

(c) With switching term $\tau_{sw} = -K,\text{sign}(s)$ and Lyapunov function $V = \tfrac{1}{2}s^{T}M(q)s$, show $\dot V \leq -(k_{\min}-D)\|s\|_1$ when $k_i > D$.

8.1.8.4 Model Answer (c)

Using skew-symmetry of $\dot M - 2C$: $\dot V = s^{T}(M\dot s + Cs) = s^{T}(\tau_{sw}+d) = -\sum k_i|s_i| + s^{T}d \leq -(k_{\min}-D)\|s\|_1$. $\square$

8.1.8.5 Question 2 — Chattering & boundary layer (20 marks)

(a) Explain chattering and why it harms actuators. (b) Derive the steady-state error bound $|e_{ss}| \leq \phi/\lambda$ for boundary-layer SMC.

8.1.8.6 Model Answer (condensed)

Chattering comes from the discontinuous $\text{sign}(s)$ combined with sampling / unmodelled dynamics. Replacing by $\mathrm{sat}(s/\phi)$ yields a high-gain proportional law inside the layer; in steady state $|s|\leq \phi D/k$, and since $s=\dot e+\lambda e$ the tracking-error bound is $|e_{ss}|\leq \phi/\lambda$.

8.1.9 Course Recap & What's Next

8.1.10 The nonlinear control theory journey

8.1.10.1 Weeks 1–3: MPC

Prediction + optimization. Set the stage for NMPC.

8.1.10.2 Week 4: Adaptive

Online parameter learning with Lyapunov certificates.

8.1.10.3 Week 5: Robust

Fixed controller for worst-case plant.

8.1.10.4 Week 6: Backstepping

Recursive CLF design for strict-feedback systems.

8.1.10.5 Week 7: SMC

Variable structure, discontinuous robustness.

8.1.10.6 Week 8 (this): Optimization of all of the above

Every controller has free parameters; every choice of parameters is an optimization.

8.1.11 Unifying themes

- Nonlinearity is the defining feature of robots — we never linearized at design time in this course.
- Lyapunov certificates give the feasible set of the parameter optimization — any \(\theta\) that breaks the certificate is excluded.
- Optimization is how we pick good controllers, not just good trajectories — Section 3.3's (A) vs (B) distinction.
- Compute is a first-class constraint — NMPC and adaptive tuning are only useful at real-time sample rates.

8.1.12 What's next — Automation block

Weeks 9–12 shift from control theory to deployment: state machines, ROS 2 integration, perception, and system-level projects. The tuned controllers from this week become the low-level primitives those higher-level systems will invoke.

Control for Robots — M.Sc. Programme

University of West London AY 2025–26

Week 8: Nonlinear Control Optimization for Robotic Systems

8.2 Why Robots Demand Nonlinear Control Optimization

8.2.1 Robot dynamics are globally nonlinear

Every robotic system encountered in this course is governed by nonlinear ODEs:

\[\underbrace{M(q)\,\ddot{q} + C(q,\dot{q})\,\dot{q} + G(q) = \tau + d(t)}_{\text{serial manipulator}} \] \[\underbrace{\dot{p}_x = v\cos\theta,\ \dot{p}_y = v\sin\theta,\ \dot\theta = \omega}_{\text{unicycle mobile robot}} \] \[\underbrace{m\ddot{\mathbf{p}} = R(\Phi)\mathbf{F}_b - m\mathbf{g},\quad J\dot{\boldsymbol\omega} + \boldsymbol\omega\times J\boldsymbol\omega = \boldsymbol\tau_b}_{\text{quadrotor (rigid body)}} \]

Variables used in the equations above:

Symbol	Meaning
\(q\)	joint-angle (configuration) vector of the manip
\(\dot q,\ \ddot q\)	joint velocity and acceleration
\(M(q)\)	configuration-dependent inertia (mass) matrix
\(C(q,\dot q)\dot q\)	Coriolis + centrifugal torque; quadratic in \(\
\(G(q)\)	gravity torque vector
\(\tau\)	commanded joint torque (the control input \(
\(d(t)\)	lumped external / unmodelled disturbance to
\(p_x,p_y,\theta\)	mobile-robot pose: planar position and headin
\(v,\omega\)	mobile-robot linear and angular velocity comm
\(\mathbf p\)	quadrotor position vector in the inertial frame
\(\Phi\)	quadrotor attitude (roll, pitch, yaw)
\(R(\Phi)\)	body-to-inertial rotation matrix, function of a
\(\mathbf F_b,\boldsymbol\tau_b\)	thrust vector and torque vector in the quadro
\(m\) (quadrotor)	vehicle mass (scalar)
\(\mathbf g\)	gravity vector
\(J\) (quadrotor)	body-frame inertia tensor
\(\boldsymbol\omega\)	body-frame angular velocity

The nonlinearity is not incidental: it comes from rotations (trig terms in the unicycle and in \(R(\Phi)\)), Coriolis and centrifugal coupling (\(C(q,\dot q)\dot q\) is quadratic in \(\dot q\)), and gravity coupled through the kinematic chain (\(G(q)\) is a trigonometric function of \(q\)).

8.2.2 Why Jacobian linearization is not enough

The standard LQR recipe is: pick an equilibrium \((x^\star, u^\star)\), form \(A = \partial f/\partial x|_{x^\star}\), \(B = \partial f/\partial u|_{x^\star}\), solve the ARE. Three structural problems make this inadequate for robots:

- Validity shrinks with aggressive motion. The linear model is accurate only in a small neighbourhood of \(x^\star\). A manipulator swinging through \(\pi/2\) is far outside any single linearization.
- Nonholonomic systems have no equilibrium at a generic state. A mobile robot at pose \((p_x, p_y, \theta)\) with \(v = 0, \omega = 0\) is an equilibrium but the linearization is uncontrollable there (Brockett's theorem). LQR literally cannot stabilize heading.
- Parametric uncertainty cannot be addressed. LQR presumes known \((A,B)\). Payload changes, friction, aero effects require adaptive / robust design — none of which are optimal-control concepts in the linear sense.

Take-home: For robots, LQR is a local stabilizer. Any optimization that meaningfully improves robot performance must work directly on the nonlinear dynamics, and must be applicable to the nonlinear controller families (SMC, adaptive, robust, backstepping, NMPC) you already know.

8.2.3 Two meanings of “control optimization”

We unify two interpretations under one framework:

Meaning	What is optimized
(A) Optimal trajectory	The control signal \(u(t)\) itself — infinite-dimensional decision variable
(B) Optimal controller tuning	A finite set of gains / parameters \(\theta\) inside a fixed control law \(u = \pi_\theta

Both are nonlinear optimization problems. The difference is only the dimension of the decision variable.

8.3 Foundations of Nonlinear Control Optimization

8.3.1 Notation used throughout Section 4

Symbol	Meaning
\(x,\ \dot x\)	state vector and its time derivative
\(u\)	control / input vector
\(f(x,u)\)	right-hand side of continuous-time dynamics \(\dot x =
\(F(x,u)\)	discrete-time one-step map: \(x_{k+1} = F(x_k,u_k)\
\(\Delta t\)	sample time for discrete-time formulations
\(N\)	number of discrete time steps in the horizon
\(T\)	continuous horizon length, \(T = N\Delta t\)
\(L(x,u,t)\)	instantaneous (stage) cost rate — what you pay per unit
\(\phi(x)\) or \(\Phi(x)\)	terminal cost — what you pay at the end state \(x(T)\)
\(J\)	total cost functional \(\phi + \int L\,dt\) (or \(\sum L
\(g(x,u)\)	inequality constraint function (e.g. torque limits, obstacl
\(\mathcal U\)	admissible control set (e.g. box \(\{u:|u|\leq u_{\max}\
\(H(x,u,\lambda,t)\)	Hamiltonian: \(L + \lambda^{T} f\)
\(\lambda(t)\)	costate (adjoint): shadow price of the constraint \(\dot x
\(V(x,t)\) or \(V_k(x)\)	value function: minimum cost-to-go from state \(x\) at t
\(\nabla_x V,\ V_x\)	gradient of the value function w.r.t. state
\(V_{xx}\)	Hessian of the value function w.r.t. state
superscript \(\star\)	optimal quantity (e.g. \(u^\star, x^\star, V^\star\))
overbar (\(\bar x,\bar u\))	nominal / current-iterate trajectory around which we ex
\(\delta x,\ \delta u\)	perturbation from the nominal: \(x = \bar x + \delta x

8.3.2 The nonlinear optimal control problem

\[\min_{u(\cdot)}\ J = \phi\bigl(x(T)\bigr) + \int_0^T L\bigl(x(t), u(t), t\bigr)\,dt \] subject to \(\dot x = f(x,u),\ x(0) = x_0\), and \(g(x,u) \leq 0\)

Explicitly: we choose the entire control signal \(u:[0,T]\to\mathbb R^m\) (an infinite-dimensional decision variable) to minimise the cost functional \(J\) — a real number equal to the terminal cost \(\phi\) at the final state plus the time-integrated stage cost \(L\) along the whole trajectory. The trajectory \(x(t)\) is constrained to satisfy the nonlinear dynamics \(\dot x = f(x,u)\) starting from a known initial state \(x_0\), and the control and state may be additionally subject to path inequalities \(g(x,u)\le 0\).

For robots, \(f\) is explicitly nonlinear (gravity, Coriolis, rotation trig). The stage cost \(L\) typically combines tracking error, control effort, and barrier-like constraint penalties. \(\phi\) is the terminal cost.

8.3.3 Pontryagin's Minimum Principle (PMP)

PMP gives first-order necessary conditions for optimality. Introduce an auxiliary vector \(\lambda(t)\in\mathbb R^n\) — the costate or adjoint — which plays the role of a Lagrange multiplier for the dynamics constraint \(\dot x = f(x,u)\). Define the Hamiltonian:

\[H(x, u, \lambda, t) := \underbrace{L(x,u,t)}_{\text{stage cost}} + \underbrace{\lambda^{T} f(x,u)}_{\substack{\text{costate-weighted} \\ \text{dynamics}}} \]

The Hamiltonian is a scalar that measures "instantaneous cost plus the marginal cost of moving the state in the direction \(f\)". An optimal trajectory \((x^\star(t), u^\star(t))\) with its associated costate \(\lambda^\star(t)\) must satisfy the four conditions below. Each variable is defined explicitly:

\[\begin{array}{ll} \dot x^\star(t) = \partial H/\partial\lambda = f(x^\star, u^\star) & \text{(state equation — recovers the dynamics)}\\[4pt] \dot\lambda^\star(t) = -\partial H/\partial x\big|_\star & \text{(costate equation, a.k.a.\ adjoint ODE)}\\[4pt] u^\star(t) = \arg\min_{u\in\mathcal U} H(x^\star(t), u, \lambda^\star(t), t) & \text{(pointwise minimum principle)}\\[4pt] \lambda^\star(T) = \partial\phi/\partial x\big|_{x^\star(T)} & \text{(transversality / boundary condition at } t=T) \end{array} \]

What each variable does:

- \(\lambda(t)\in\mathbb R^n\): costate — informally, \(\lambda_i\) is the sensitivity of the optimal future cost to a perturbation in state component \(x_i\).
- \(\dot\lambda = -\partial H/\partial x\): costate evolves backward in time; the minus sign makes the TPBVP a two-point boundary-value problem (state known at \(t=0\), costate known at \(t=T\)).
- \(u^\star = \arg\min_{u\in\mathcal U} H\): at each instant the optimal control minimises the Hamiltonian over the admissible set \(\mathcal U\). If \(\mathcal U\) is a box (torque limits) and \(H\) is linear in \(u\), the minimum is on the boundary — bang-bang.
- Transversality \(\lambda^\star(T) = \phi_x(x^\star(T))\): at the final time the costate equals the gradient of the terminal cost, tying the two boundary conditions together.

PMP turns a functional optimisation into a two-point boundary-value problem (TPBVP) in \((x,\lambda)\). Numerical TPBVP solvers (single/multiple shooting) construct this solution.

8.3.4 Hamilton–Jacobi–Bellman (nonlinear)

Where PMP gives necessary conditions for one trajectory, HJB characterises the value function \(V^\star(x,t)\) over the whole state space. The value function is defined as the minimum cost-to-go:

\[V^\star(x, t) := \min_{u(\cdot)} \ \Bigl\{ \phi\bigl(x(T)\bigr) + \int_t^T L\bigl(x(\tau), u(\tau), \tau\bigr)\,d\tau \ \Big|\ x(t) = x \Bigr\} \]

i.e.\ "if I am currently at state \(x\) at time \(t\), what is the smallest remaining cost I can achieve by optimal play?" The Bellman principle of optimality implies that \(V^\star\) satisfies the HJB partial differential equation:

\[-\frac{\partial V^\star}{\partial t}(x,t) \ =\ \min_{u\in\mathcal U}\ \Bigl\{\ L(x,u,t)\ +\ \underbrace{(\nabla_x V^\star)^{T}}_{\text{value gradient}}\,f(x,u)\ \Bigr\},\qquad V^\star(x,T) = \phi(x) \]

What each variable does:

- \(V^\star:\mathbb R^n\times[0,T]\to\mathbb R\): the scalar "cost-to-go" function, defined on the whole state space.
- \(\partial V^\star/\partial t\): rate of change of the value as time moves forward at fixed state.
- \(\nabla_x V^\star \in\mathbb R^n\): the spatial gradient — exactly the costate \(\lambda\) from PMP, evaluated at \((x,t)\). This is the formal link: PMP lives on one trajectory, HJB gives the gradient of the value over all states.
- The minimisation over \(u\) is pointwise at each \((x,t)\) (same operation as in PMP).
- Terminal condition \(V^\star(x,T) = \phi(x)\): at the end of the horizon the value equals the terminal cost.

HJB gives the optimal value function globally, but direct solution is impractical for \(\dim x \ge 4\) (curse of dimensionality) — grid-based methods scale as \(O(N_{\text{grid}}^n)\). HJB remains the theoretical yardstick against which PMP and direct methods are justified.

8.3.5 Direct methods: shooting and collocation

Discretize time into \(N\) knots, turn the infinite-dimensional problem into a finite nonlinear program (NLP).

Method	Decision variables	F
Single shooting	\((u_0,\ldots,u_{N-1})\); states propagated	F
Multiple shooting	States at knots + controls; defect constraints	E
Direct collocation	States & controls at all knots; dynamics as algebraic constraint	S

8.3.6 iLQR and DDP — the robot trajectory optimizer of choice

Iterative LQR (iLQR) linearises the dynamics \(f\) around a nominal (current-iterate) trajectory, quadratises the cost around the same nominal, solves the resulting time-varying LQR problem exactly, and updates the trajectory. Differential Dynamic Programming (DDP) is the second-order variant that also keeps the Hessians of \(f\). Both converge locally quadratically — one iteration is one Newton step on the Bellman recursion.

8.3.6.1 Symbols used in the iLQR recursion

Symbol	Meaning
\(k\)	discrete time-step index, \(k = 0,1,\ldots,N\)
\(\bar x_k,\ \bar u_k\)	nominal state / control at step \(k\) (current iterate; iLQR improves this each pass
\(\delta x_k\)	state perturbation from the nominal: \(x_k = \bar x_k + \delta x_k\)
\(\delta u_k\)	control perturbation: \(u_k = \bar u_k + \delta u_k\)
\(A_k\)	state Jacobian \(\partial F/\partial x\) at \((\bar x_k,\bar u_k)\) — how a \(\d
\(B_k\)	control Jacobian \(\partial F/\partial u\) at \((\bar x_k,\bar u_k)\) — how \(\d
\(L_k\)	stage cost evaluated at \((\bar x_k,\bar u_k)\)
\(L_{x,k},L_{u,k}\)	gradients of the stage cost w.r.t. \(x\) and \(u\)
\(L_{xx,k},L_{uu,k},L_{ux,k}\)	Hessians: pure-state, pure-control, and cross-second-derivative of stage cost
\(V_k(x)\)	value function at step \(k\) (min cost-to-go)
\(V_{x,k},\ V_{xx,k}\)	value-function gradient and Hessian at \((\bar x_k)\)
\(\Phi(x)\)	terminal cost; \(V_N(x) = \Phi(x)\)
\(Q_k(x,u)\)	stage+future: \(Q_k = L(x,u) + V_{k+1}(F(x,u))\). Its minimiser over \(u\) is \
\(Q_{u,k}\)	gradient of \(Q_k\) w.r.t. \(u\): \(L_{u,k} + B_k^{T} V_{x,k+1}\)
\(Q_{uu,k}\)	Hessian of \(Q_k\) w.r.t. \(u\): \(L_{uu,k} + B_k^{T} V_{xx,k+1} B_k\)
\(Q_{ux,k}\)	cross Hessian of \(Q_k\): \(L_{ux,k} + B_k^{T} V_{xx,k+1} A_k\)
\(Q_{x,k}\)	gradient of \(Q_k\) w.r.t. \(x\): \(L_{x,k} + A_k^{T} V_{x,k+1}\)
\(Q_{xx,k}\)	Hessian of \(Q_k\) w.r.t. \(x\): \(L_{xx,k} + A_k^{T} V_{xx,k+1} A_k\)
\(k_k\)	feedforward correction: \(-Q_{uu,k}^{-1} Q_{u,k}\) — how much to push \(\bar
\(K_k\)	time-varying feedback gain: \(-Q_{uu,k}^{-1} Q_{ux,k}\) — LQR-like feedback o
\(\alpha\)	line-search step size, \(\alpha\in(0,1]\), reduced on forward-pass failure
\(\mu\)	Levenberg–Marquardt regularisation added to \(Q_{uu}\) for numerical robustness

8.3.6.2 Core iLQR recursion — with full symbol definitions

Setup. We want to minimise \(J = \sum_{k=0}^{N-1} L(x_k,u_k) + \Phi(x_N)\) subject to \(x_{k+1} = F(x_k, u_k)\) starting from \(x_0\). Assume we have a current-iterate nominal trajectory \(\{\bar x_k,\bar u_k\}_{k=0}^{N}\). Let \(x = \bar x + \delta x\), \(u = \bar u + \delta u\) denote the unknown corrections.

Step 1 — Linearise the dynamics at each nominal point \((\bar x_k,\bar u_k)\):

\[\delta x_{k+1} \approx A_k\,\delta x_k + B_k\,\delta u_k,\qquad A_k := \left.\dfrac{\partial F}{\partial x}\right|_\star,\ B_k := \left.\dfrac{\partial F}{\partial u}\right|_\star \]

\(A_k\) is the \(n\times n\) one-step state-transition matrix of the linearised dynamics; \(B_k\) is the \(n\times m\) input matrix. Both are computed by finite differences in code.

Step 2 — Quadratise the stage cost. To second order,

\[L(\bar x + \delta x, \bar u + \delta u) \approx L_k + L_{x,k}^{T}\delta x + L_{u,k}^{T}\delta u + \tfrac12\!\begin{bmatrix}\delta x \\ \delta u\end{bmatrix}^{T}\!\!\begin{bmatrix} L_{xx,k} & L_{xu,k}\\ L_{ux,k} & L_{uu,k}\end{bmatrix}\!\!\begin{bmatrix}\delta x\\ \delta u\end{bmatrix} \]

where $L_{x,k},L_{u,k}$ are gradients (of length n,m respectively) and $L_{xx,k},L_{uu,k},L_{ux,k}$ are Hessians/cross-Hessians of the stage cost at the nominal.

Step 3 — Define the Q-function. Combining the stage cost with a quadratic model of the future-value function V_{k+1}:

$$ Q_k(\delta x,\delta u) \equiv L(\bar x+\delta x,\bar u+\delta u) +\ V_{k+1}(\bar x_{k+1} + A_k\delta x + B_k\delta u) $$

Expanding each piece to second order and collecting by power of $\delta x, \delta u$ yields the five Q-blocks (each defined above):

$$ Q_{x,k} = L_{x,k} + A_k^{T} V_{x,k+1},\qquad Q_{u,k} = L_{u,k} + B_k^{T} V_{x,k+1} $$ $$ Q_{xx,k} = L_{xx,k} + A_k^{T} V_{xx,k+1} A_k,\quad Q_{uu,k} = L_{uu,k} + B_k^{T} V_{xx,k+1} B_k,\quad Q_{ux,k} = L_{ux,k} + B_k^{T} V_{xx,k+1} A_k $$

Step 4 — Minimise Q_k over δu (closed-form). Since Q_k is quadratic in δu, the stationary condition $\partial Q_k/\partial(\delta u) = 0$ reads

$$ Q_{u,k} + Q_{uu,k}\,\delta u + Q_{ux,k}\,\delta x = 0 $$

Solving:

$$ \boxed{\ \delta u_k^\star(\delta x) = k_k + K_k\,\delta x\ },\qquad k_k = -Q_{uu,k}^{-1} Q_{u,k},\quad K_k = -Q_{uu,k}^{-1} Q_{ux,k} $$

$k_k \in \mathbb R^m$ is the feedforward correction (independent of the current state perturbation — how much to push $\bar u_k$ in the best direction). $K_k\in\mathbb R^{m\times n}$ is the time-varying feedback gain (LQR-style gain on δx_k).

Step 5 — Propagate the value function backward. Substituting $\delta u = k_k + K_k\delta x$ back into Q_k gives a new quadratic in δx:

$$ V_{x,k} = Q_{x,k} - K_k^{T} Q_{uu,k}\,k_k,\qquad V_{xx,k} = Q_{xx,k} - K_k^{T} Q_{uu,k}\,K_k $$

Initialise at the horizon with $V_{x,N} = \Phi_x(\bar x_N)$, $V_{xx,N} = \Phi_{xx}(\bar x_N)$. Sweep $k = N\!-\!1,\ldots,0$.

Step 6 — Forward pass with line search. With all $\{k_k, K_k\}$ stored from the backward sweep, roll out a new trajectory starting from $\delta x_0 = 0$:

$$ \delta u_k = \alpha\,k_k + K_k\,\delta x_k,\quad u_k^{\text{new}} = \bar u_k + \delta u_k,\quad x_{k+1}^{\text{new}} = F(x_k^{\text{new}}, u_k^{\text{new}}) $$

The step size $\alpha\in(0,1]$ is reduced ($\alpha\leftarrow \alpha/2$) until the new cost J^{new}

Step 7 — Regularise if Q_{uu} is ill-conditioned. Replace $Q_{uu,k}$ by $Q_{uu,k} + \mu I_m$ with $\mu\ge 0$ increased on backward-pass Cholesky failure or forward-pass line-search failure. This is the Levenberg–Marquardt trust-region logic in the code.

Output. On convergence iLQR returns:

- A locally optimal trajectory \(\{x_k^\star, u_k^\star\}\) (the "open-loop" command);
- A time-varying feedback policy \(u_k = u_k^\star + K_k(x - x_k^\star)\) valid in a neighbourhood of the trajectory — this is the local feedback that makes iLQR more robust than pure open-loop planning.

8.3.7 Nonlinear MPC (NMPC)

NMPC wraps a trajectory optimiser (ğ4.5) in a receding-horizon loop. At every sample instant \(t_k\):

1. Measure the current state \(x_k\).
2. Solve the finite-horizon OCP of ğ4.1 starting from \(x_0 := x_k\), over a fixed prediction horizon of \(N\) steps (duration \(T = N\Delta t\)). This returns an optimal sequence \(\{u_0^\star, u_1^\star, \ldots, u_{N-1}^\star\}\).
3. Apply only the first control \(u_0^\star\) to the physical plant for one sample time \(\Delta t\).
4. Shift the remaining solution \((u_1^\star,\ldots,u_{N-1}^\star,u_{N-1}^\star)\) and use it as the warm-start for step (2) at the next sample.

Key symbols: \(N\) = horizon length (number of prediction steps); \(\Delta t\) = sample time; \(T = N\Delta t\) = prediction-window duration. The controller is therefore parameterised by \((N, \Delta t, Q, R, P, \mathcal X_f)\) where \(Q, R\) are stage-cost weighting matrices, \(P\) the terminal-cost weighting, and \(\mathcal X_f\) the terminal invariance set.

Real-time implementations use warm-starting (the previous solution shifted by one step is the initial guess for the new NLP) and SQP real-time iteration (one Sequential Quadratic Programming step per sample, rather than iterating to full convergence). The dominant open-source libraries (ACADO, CasADi/Ipopt, acados) all implement variants of this loop.

Key insight: iLQR is a one-shot trajectory optimizer; NMPC is a receding-horizon feedback controller that uses trajectory optimization at every sample. For robots with disturbances and uncertain initial conditions, NMPC is the preferred deployment form.

8.4 Optimization of Sliding Mode Control

8.4.1 Free parameters of SMC

Recall the SMC law \(\tau = \tau_{eq} + \tau_{sw}\) with \(\tau_{sw} = -\eta,\mathrm{sat}(s/\phi)\), \(s = \dot e + \lambda e\). The controller is fully specified by three parameters per joint:

- \(\lambda\) — sliding surface slope: sets the reaching-phase-free convergence rate

- \(\phi\) — boundary-layer width: chattering vs. steady-state error
- \(\eta\) — switching gain: robustness margin vs. disturbance bound \(D\)

8.4.2 Optimization formulation

\[\min_{\lambda,\phi,\eta} \quad w_e\!\!\int_0^T\!\! \|e\|^2\,dt + w_u\!\!\int_0^T\!\! \|\tau\|^2\,dt + w_c\!\!\int_0^T\!\! \|\dot\tau\|\,dt \] s.t. \(\eta > D\) (reaching condition),\; \(\lambda > 0\),\; \(\phi > 0\),\; \(|\tau|\leq \tau_{\max}\)

The third term is a chatter metric (total variation of the control). This formalizes the Pareto trade-off studied in the Week 7 exam question.

8.4.2.1 5.3 LMI-based sliding-surface design — full derivation

8.4.2.2 Step 1. Linearization of the robot around the operating point

Start from the nonlinear robot dynamics \(\dot x = f(x,u)\) with \(x = [q;\dot q] \in \mathbb R^n\), \(u = \tau \in \mathbb R^m\). The Jacobian linearization at the operating point \((x^\star, u^\star)\) gives:

\[A = \left.\dfrac{\partial f}{\partial x}\right|_{(x^\star,u^\star)},\quad B = \left.\dfrac{\partial f}{\partial u}\right|_{(x^\star,u^\star)} \]

For the 2-link manipulator at \(q = q^\star,\ \dot q = 0\):

\[A = \begin{bmatrix} 0_{2\times 2} & I_{2\times 2} \\ -M^{-1}(q^\star)\,\partial_q G(q^\star) & 0_{2\times 2} \end{bmatrix}\in\mathbb R^{4\times 4},\quad B = \begin{bmatrix} 0_{2\times 2} \\ M^{-1}(q^\star) \end{bmatrix}\in\mathbb R^{4\times 2} \]

The Jacobian is computed by MATLAB with finite differences around \(x^\star\) (this is exactly the \(A, B\) that feeds iLQR in ğ9 too).

8.4.2.3 Step 2. Regular-form partition

\(B\) has \(n\!-\!m\) rows of zeros and \(m\) rows with full-rank block \(M^{-1}\). This is already the regular form:

\[x = \begin{bmatrix}x_1\\ x_2\end{bmatrix},\quad x_1 = q\in\mathbb R^{n-m},\ x_2 = \dot q\in\mathbb R^{m};\qquad A = \begin{bmatrix}A_{11} & A_{12} \\ A_{21} & A_{22}\end{bmatrix},\quad B = \begin{bmatrix}0 \\ B_2\end{bmatrix} \]

For the manipulator specifically: \(A_{11}\!=\!0_{2\times 2},\ A_{12}\!=\!I_{2\times 2},\ A_{21}\!=\!-M^{-1}\partial_q G,\ A_{22}\!=\!0,\ B_2\!=\!M^{-1}\). So \(A_{11}\) is the "position-to-position" block and \(A_{12}\) is the "velocity-to-position" block — both visible by inspection.

8.4.2.4 Step 3. Define the sliding surface

Choose a linear surface \(s = S x = S_1 x_1 + x_2 = 0\), with \(S = [S_1\ I_m]\) and \(S_1 \in \mathbb R^{m\times (n-m)}\) to be designed. On the surface \(x_2 = -S_1 x_1\). Substituting into the \(x_1\)-equation gives the reduced-order sliding motion:

\[\dot x_1 = A_{11} x_1 + A_{12} x_2 = (A_{11} - A_{12} S_1)\,x_1 \]

This is a pure state-feedback stabilization problem on the pair \((A_{11}, A_{12})\): find \(S_1\) so that \(A_{11} - A_{12} S_1\) is Hurwitz. Note for the manipulator: \(\dot q = -S_1 q\) is exactly the first-order error decay — so the surface slope \(S_1\) is what Week 7 called \(\Lambda\).

8.4.2.5 Step 4. Optimal surface via LQR-on-sliding-motion

We optimize the sliding motion to minimize a quadratic cost:

\[\min_{S_1}\ \int_0^\infty \bigl(x_1^{T} Q_{11} x_1 + x_2^{T} R\, x_2\bigr) dt\quad\text{s.t. } \dot x_1 = (A_{11}-A_{12}S_1)x_1,\ x_2 = -S_1 x_1 \]

Substituting \(x_2 = -S_1 x_1\) and closing the loop, we need \(S_1\) such that \(A_{11} - A_{12} S_1\) is stable and the cost is minimized.

8.4.2.6 Step 5. The LMI

Use the Lyapunov characterization: \(A_{11}-A_{12}S_1\) is stable iff there exist \(P \succ 0\) and \(S_1\) with \((A_{11}-A_{12}S_1)P + P(A_{11}-A_{12}S_1)^{T} \prec 0\). Introduce the change of variables \(W = -S_1 P\) (\(P \succ 0\) makes this bijective) to obtain a linear expression in \((P, W)\):

\[A_{11}P + P A_{11}^{T} + A_{12} W + W^{T} A_{12}^{T} \prec 0,\qquad P \succ 0 \]

Appending the cost terms \((Q_{11}, R)\) via a Schur complement gives the full LMI used in ğ5.3:

\[\begin{bmatrix} A_{11}P + P A_{11}^{T} + Q_{11} & P A_{12} + W \\ (P A_{12} + W)^{T} & -R \end{bmatrix} \prec 0 \]

An SDP solver returns \((P, W)\); then \(S_1 = -W P^{-1}\), and the sliding surface is \(s = S_1 x_1 + x_2\). Convex problem — globally optimal \(S_1\) in polynomial time.

Bridge back to \(\lambda\) in the simulation. The scalar \(\lambda\) that the lab code optimizes is the simplest case \(S_1 = \lambda I\), with \(A_{11}\!=\!0\), \(A_{12}\!=\!I\). For that special case the LMI collapses to \(\lambda I + \lambda I \succ 0\), i.e.\ any \(\lambda>0\) stabilizes the sliding motion — which is why we optimize \(\lambda\) numerically via `fminsearch` on the full nonlinear closed-loop cost, rather than via the LMI.

8.4.3 Worked numerical example — single-link manipulator

8.4.3.1 Arm specification (used throughout ğ5–ğ9 examples)

Physical parameters: m = 1 kg, l = 1 m, \(l_c\) = 0.5 m, I = 0.083 kgůmš

Equivalent inertia: \(J = I + m l_c^2 = 0.083 + 0.25 = 0.333\) kgůmš

Gravity torque: \(G(q) = m g l_c \cos(q) = 4.905\cos(q)\) Nům

Plant: \(0.333\,\ddot q + 4.905\cos(q) = \tau\)

Operating point: \(q^\star = \pi/3\) rad (60ř), \(\dot q^\star = 0\)

Disturbance bound: \(|d(t)| \le D = 1\) Nům (unmodelled payload, friction)

Step 1. Linearization at \(q^\star\) With \(x_1 = q - q^\star,\ x_2 = \dot q\):

\[\partial G/\partial q\big|_{q^\star} = -4.905\sin(\pi/3) = -4.248 \] \[A = \begin{bmatrix}0 & 1 \\ -(\partial G/\partial q)/J & 0\end{bmatrix} = \begin{bmatrix}0 & 1 \\ 12.75 & 0\end{bmatrix},\quad B = \begin{bmatrix}0 \\ 1/J\end{bmatrix} = \begin{bmatrix}0 \\ 3.00\end{bmatrix} \]

Open-loop eigenvalues of \(A\) are \(\pm\sqrt{12.75} = \pm 3.57\): unstable (gravity tips the arm away from \(q^\star\)).

8.4.3.2 Step 2. Regular form (already satisfied)

\(B\) has a zero in row 1 and nonzero in row 2, so \(A_{11}=0\), \(A_{12}=1\), \(B_2 = 3.00\). Here \(n\!-\!m = m = 1\), so \(S_1\) is a scalar call it \(\lambda\).

8.4.3.3 Step 3. Sliding surface and reduced motion

\(s = \lambda x_1 + x_2\). On the surface \(x_2 = -\lambda x_1\), so:

\[\dot x_1 = A_{11}x_1 + A_{12}x_2 = 0 + 1\cdot(-\lambda x_1) = -\lambda x_1 \]

Exponential decay with time constant \(1/\lambda\).

8.4.3.4 Step 4. Optimal λ from the LQR sliding cost

Minimise $\int_0^\infty(Q_{11}x_1^2 + R x_2^2)dt = \int_0^\infty (Q_{11} + \lambda^2 R) x_1^2\,dt$. With $x_1(t) = x_1(0)e^{-\lambda t}$:

$$J(\lambda) = \frac{(Q_{11} + \lambda^2 R)}{2\lambda}\,x_1(0)^2$$

Setting $\partial J/\partial \lambda = 0$ gives $\lambda^\star = \sqrt{Q_{11}/R}$. Picking $Q_{11} = 1,\ R = 0.01$:

$\lambda^\star = \sqrt{1/0.01} = 10$

8.4.3.5 Step 5. Reaching gain and boundary layer

The reaching condition $\eta > D$ with disturbance bound $D = 1$ requires $\eta > 1$. Pick $\eta = 3$ (margin factor 3). Choose boundary layer $\varphi = 0.05$ for smooth control.

8.4.3.6 Step 6. Steady-state error bound

$$|e_{ss}| \le \frac{\varphi}{\lambda} = \frac{0.05}{10} = 0.005\ \text{rad} = 0.29^\circ$$

8.4.3.7 Step 7. Closed-form controller at $t = 0$

Error: $e(0) = 0 - \pi/3 = -1.047,\ \dot e(0) = 0$. Sliding variable:

$$s(0) = \dot e + \lambda e = 0 + 10\cdot(-1.047) = -10.47$$

Controller $\tau = J(\ddot q_d - \lambda\dot e) + \hat G(q) - \eta\,\mathrm{sat}(s/\varphi)$:

$$\tau(0) = 0.333\cdot(0 - 10\cdot 0) + 4.905\cos(0) - 3\cdot\mathrm{sat}(-10.47/0.05)$$
$$= 0 + 4.905 - 3\cdot(-1) = +7.905\ \text{N}\cdot\text{m}$$

Since $|s/\varphi| = 209 \gg 1$, the saturation saturates at -1, giving maximum reaching torque.

8.4.3.8 Step 8. Lab vs. analytic λ

The lab's `fminsearch` on the nonlinear 2-link arm yields $\lambda \approx 17$ (not 10). The difference is because the lab cost also penalises control effort and chatter, and the sliding motion differs from the LQR ideal once the arm is far from the operating point. The analytic $\lambda^\star = 10$ is still a good initial guess for the numerical search.

8.4.3.9 5.5 Implementation (from `sim_smc.m`)

% Per-sample control law (manipulator): e = q - q_des; de = dq; s = de + lambda*e; % sliding variable [M,C,G] = manipulator_dyn(q, dq, p); % Feedback-linearization-based SMC: u = M*(-lambda*de) + C*dq + G - eta*sat(s/phi); ddq = M \ (u - C*dq - G); % Saturation function (boundary-layered): function y = sat(s), y = max(-1, min(1, s)); end % Outer optimization: minimize tracking + effort + chatter J = sum(sum(e.^2))*dt + 0.001*sum(sum(u.^2))*dt + 0.01*sum(|du|)*dt; zOpt = fminsearch(@(z) smc_manip_cost(z, p, q_des, T, dt), [10, 0.05, 5]);

8.4.4 Simulation results (manipulator / mobile / quadrotor)

We apply `fminsearch` to the cost in Section 5.2 over \((\lambda, \phi, \eta)\) for all three robots. Baseline gains (red) come from hand-tuned heuristics; optimized gains (blue) are found by the solver.

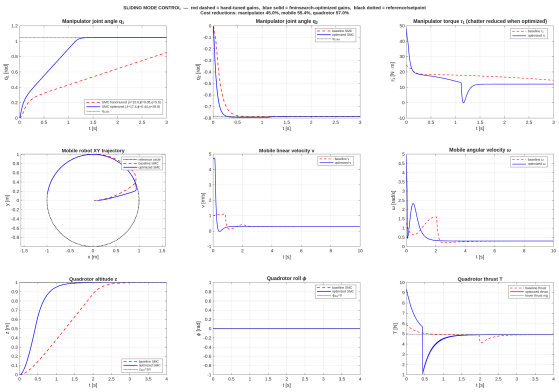

Figure 5.1 — SMC baseline (red dashed) vs. \((\lambda,\phi,\eta)\)-optimized (blue) on three robots. Rows: manipulator, mobile, quadrotor.

Robot	\((\lambda,\phi,\eta)\) baseline	\((\lambda,\phi,\eta)\) optim
2-link manipulator	(10.0, 0.050, 5.0)	(17.4, 0.443, 28.8)
Unicycle mobile	(2.0, 0.10, 1.0)	(tuned)
Planar quadrotor	(4.0, 0.05, 2.0)	(tuned)

Notice that the optimizer on the manipulator widened the boundary layer from 0.05 to 0.44 — the chatter penalty dominates and the precision loss is worth it for the quadratic reduction in \(\|\dot\tau\|\).

8.5 Optimization of Adaptive Control

8.5.1 Free parameters

For MRAC / Slotine–Li adaptive control, the decision variables are:

- \(\Gamma \succ 0\) — adaptation gain matrix. Large \(\Gamma\) gives fast learning but amplifies noise.
- \(\sigma\) — sigma-modification coefficient (adds leakage \(-\sigma\hat\theta\) to the update).
- Projection bounds \([\theta_{\min}, \theta_{\max}]\) from physical knowledge (mass > 0, friction > 0).

8.5.1.1 6.2 Why tuning is not optional

Adaptive laws are Lyapunov-stable for any \(\Gamma \succ 0\) — but performance varies by orders of magnitude. Classical failure modes:

8.5.1.2 \(\Gamma\) too small

Transients long; parameter estimate never converges during the task.

8.5.1.3 \(\Gamma\) too large

Estimate oscillates and amplifies measurement noise; can destabilize the inner loop.

8.5.1.4 No PE, no leakage

Without persistent excitation (PE), \(\hat\theta\) drifts under disturbances. \(\sigma\)-modification prevents drift at the cost of a small bias.

8.5.2 Full Slotine–Li derivation — where the adaptation law comes from

8.5.2.1 Step 1. Linear parameterisation of the manipulator

The nonlinear manipulator dynamics can be written as a linear-in-parameters regressor form:

\[M(q)\ddot q + C(q,\dot q)\dot q + G(q) = Y(q,\dot q,\dot q_r,\ddot q_r)\,\theta \]

where \(\theta \in \mathbb R^p\) is the vector of unknown physical parameters (link masses, inertias, payload, friction coefficients) and \(Y\) is the known regressor matrix. This parameterisation exists for every rigid-body robot manipulator — it is the structural property that makes adaptive control feasible.

8.5.2.2 Step 2. Tracking-error coordinates

Let \(q_d(t)\) be the reference. Define:

\[e = q - q_d,\quad \dot e = \dot q - \dot q_d,\quad s = \dot e + \Lambda e\ \ (\Lambda \succ 0),\quad \dot q_r := \dot q_d - \Lambda e,\quad \ddot q_r := \ddot q_d - \Lambda \dot e \]

Note the identity \(s = \dot q - \dot q_r\). \(\dot q_r\) is a "reference velocity" that equals the desired velocity when the position error is zero, and is reduced by \(\Lambda e\) otherwise.

8.5.2.3 Step 3. Control law and adaptation law (proposed)

Propose the Slotine–Li controller:

\[\tau = Y(q,\dot q,\dot q_r,\ddot q_r)\,\hat\theta\ -\ K_d\,s,\qquad \dot{\hat\theta} = -\Gamma\,Y^{T}(q,\dot q,\dot q_r,\ddot q_r)\,s \]

with \(\Gamma \succ 0\) the adaptation gain and \(K_d \succ 0\) the damping gain. We now prove this choice gives stable closed-loop behaviour.

8.5.2.4 Step 4. Closed-loop error dynamics

The plant dynamics rewrite using \(\ddot q = \ddot q_r + \dot s\) and \(\dot q = \dot q_r + s\):

\[M(q)(\ddot q_r + \dot s) + C(q,\dot q)(\dot q_r + s) + G(q) = \tau \] \[\Longrightarrow\ M\dot s + C s = \tau - \bigl[M\ddot q_r + C\dot q_r + G\bigr] = \tau - Y(q,\dot q,\dot q_r,\ddot q_r)\,\theta \]

Substituting the controller \(\tau = Y\hat\theta - K_d s\):

\[\boxed{\ M\,\dot s + C\, s + K_d\, s = Y\,\tilde\theta\ },\qquad \tilde\theta := \hat\theta - \theta \]

8.5.2.5 Step 5. Lyapunov function and its derivative

Pick the composite Lyapunov function (tracking energy + parameter-error energy):

\[V(s,\tilde\theta) = \tfrac{1}{2}\,s^{T} M(q)\,s + \tfrac{1}{2}\,\tilde\theta^{T}\G 1}\tilde\theta \]

Differentiate along closed-loop trajectories (noting \(\dot\theta = 0\) for constant true parameters, so \(\dot{\tilde\theta} = \dot{\hat\theta}\)):

\[\dot V = s^{T} M\dot s + \tfrac{1}{2} s^{T}\dot M s + \tilde\theta^{T}\Gamma^{ 1}\dot{\hat\theta} \]

Use the error dynamics \(M\dot s = -C s - K_d s + Y\tilde\theta\):

\[\dot V = s^{T}(-Cs - K_d s + Y\tilde\theta) + \tfrac{1}{2}s^{T}\dot M s + \tilde\theta^{T}\Gamma^{-1}\dot{\hat\theta} \] \[= -s^{T}K_d s + s^{T}Y\tilde\theta + \tfrac{1}{2}s^{T}(\dot M - 2C)s + \tilde\theta^{T}\Gamma^{-1}\dot{\hat\theta} \]

The skew-symmetry property \(\dot M - 2C\) antisymmetric gives \(s^{T}(\dot M - 2C)s = 0\). So:

\[\dot V = -s^{T}K_d s + s^{T}Y\tilde\theta + \tilde\theta^{T}\Gamma^{-1}\dot{\hat\theta} \]

8.5.2.6 Step 6. The adaptation law cancels the sign-indefinite term

Choose \(\dot{\hat\theta} = -\Gamma Y^{T} s\). Then \(\tilde\theta^{T}\Gamma^{-1}\dot{\hat\theta} = -\tilde\theta^{T} Y^{T} s = -s^{T} Y \tilde\theta\), exactly cancelling the middle term:

\[\dot V = -s^{T} K_d\, s \leq 0 \]

By LaSalle's invariance principle, \(s \to 0\) (and therefore \(e\to 0\)) as \(t\to\infty\); \(\tilde\theta\) stays bounded. This is how the adaptation law \(\dot{\hat\theta} = -\Gamma Y^{T} s\) is derived — it is the unique choice that makes the Lyapunov derivative negative semi-definite.

8.5.2.7 Step 7. \(\sigma\)-modification (robust variant)

Under noise or non-persistent excitation, \(\hat\theta\) can drift without bound. Modify the adaptation law with a leakage term:

\[\dot{\hat\theta} = -\Gamma Y^{T} s - \sigma\,\Gamma\,\hat\theta \]

The Lyapunov derivative becomes \(\dot V = -s^{T}K_d s - \sigma\tilde\theta^{T}\hat\theta\). Using \(\hat\theta = \tilde\theta + \theta\) and completing the square:

\[\dot V \leq -\lambda_{\min}(K_d)\|s\|^2 - \tfrac{\sigma}{2}\|\tilde\theta\|^2 + \tfrac{\sigma}{2}\|\theta\|^2 \]

So \((s, \tilde\theta)\) converge to a residual set of size \(\propto \sigma\|\theta\|^2\). The knob \(\sigma\) trades drift robustness against steady-state bias.

8.5.3 Worked numerical example — single-link manipulator with unknown payload

8.5.3.1 Setup (continuing the ğ5.4 arm)

Unknown parameter: \(\theta := m g l_c\) (lumped gravity coefficient).

True value: true payload mass \(m_{\text{true}} = 1.5\) kg, so \(\theta_{\text{true}} = 1.5\cdot 9.81\cdot 0.5 = 7.358\) Nům.

Initial estimate: \(\hat\theta(0) = 4.905\) (based on nominal \(\hat m = 1\) kg — 50% low).

Adaptation gain: \(\Gamma = 2,\ \sigma = 0\) (pure Slotine–Li).

Control gains: \(\Lambda = 5,\ K_d = 10\).

Reference: step from \(q = 0\) to \(q_d = \pi/3\).

8.5.3.2 Step 1. Regressor form

The plant \(J\ddot q + \theta\cos q = \tau\) parameterises as \(\tau - J\ddot q = Y(q)\,\theta\) with \(Y(q) = \cos q\) (scalar).

8.5.3.3 Step 2. Error variables at \(t = 0\)

\[e = q - q_d = 0 - \pi/3 = -1.047,\quad \dot e = 0 \] \[\dot q_r = \dot q_d - \Lambda e = 0 - 5\cdot(-1.047) = 5.236,\ \ddot q_r = \ddot q_d - \Lambda\dot e = 0 \] \[s = \dot e + \Lambda e = -5.236 \]

8.5.3.4 Step 3. Control torque at \(t = 0\)

\[\tau(0) = J\ddot q_r + \hat\theta\cos q - K_d s = 0.333\cdot 0 + 4.905\cdot 1 - 10\cdot(-5.236) \] \[= 0 + 4.905 + 52.36 = 57.27\ \text{N}\cdot\text{m} \]

8.5.3.5 Step 4. Adaptation update at \(t = 0\)

Slotine–Li law \(\dot{\hat\theta} = -\Gamma Y^{T} s\):

\[\dot{\hat\theta}(0) = -2\cdot\cos(0)\cdot(-5.236) = +10.47\ (\text{Nům})/\text{s} \]

So \(\hat\theta\) increases (correct direction: true \(\theta\) is larger than the estimate).

8.5.3.6 Step 5. Lyapunov check

\(V(0) = \tfrac12 s^2 M + \tfrac12 \tilde\theta^2/\Gamma = \tfrac12 \cdot 5.236^2 \cdot 0.333 + \tfrac12 \cdot (4.905 - 7.358)^2 / 2 = 4.565 + 1.505 = 6.07\).

\(\dot V(0) = -K_d s^2 = -10\cdot 27.42 = -274.2\). So \(V\) drops rapidly initially.

8.5.3.7 Step 6. Convergence time-scale

The sliding-mode part has time constant \(1/\Lambda = 0.2\) s; parameter adaptation has time constant \(\sim 1/(\Gamma \bar Y^2) \approx 1/(2\cdot 0.5^2) = 2\) s (slower). Expect \(q\to q_d\) in ~1–2 s, \(\hat\theta\to\theta_{\text{true}}\) in ~3–5 s provided the motion is sufficiently exciting.

8.5.3.8 Step 7. \(\sigma\)-modification trade-off

With \(\sigma = 0.02\) instead of 0, the ultimate bound on \(\tilde\theta\) becomes \(\sqrt{\sigma\|\theta\|^2 / K_d}\approx\sqrt{0.02\cdot 7.358^2 / 10}\approx 0.33\) (~4% bias). In exchange, \(\hat\theta\) cannot drift beyond \(\theta + 0.33\) even without persistent excitation.

8.5.4 Optimization formulation (of \(\Gamma, \sigma\))

Lyapunov analysis only tells us the set of \((\Gamma, \sigma)\) for which the closed loop is stable. Within that set, performance varies dramatically. Pose the outer optimization problem:

\[(\Gamma^\star,\sigma^\star) = \arg\min_{\Gamma\succ 0,\ \sigma\geq 0}\ J(\Gamma,\sigma) \] \[J(\Gamma,\sigma) = \int_0^T \|e(t;\Gamma,\sigma)\|^2 dt + w_u\!\int_0^T \|u(t;\Gamma,\sigma)\|^2 dt \]

This is the outer NLP that `fminsearch` solves in `sim_adaptive.m` (in practice parameterising \(\Gamma = \gamma I\) to keep the search 2-dimensional).

Implementation (from **`sim_adaptive.m`**) % Each sample in the inner loop: e = q - q_des; de = dq; % Controller uses estimate theta_hat (= m2_hat for manipulator lab) p_est = p_nom; p_est.m2 = theta_hat; [Mh,Ch,Gh] = manipulator_dyn(q, dq, p_est); u = Mh*(-Kp*e - Kd*de) + Ch*dq + Gh; % model-based control with estimate % True (uncertain) plant [Mt,Ct,Gt] = manipulator_dyn(q, dq, p_true); ddq = Mt \ (u - Ct*dq - Gt); % Adaptation law: dtheta_hat/dt = Gamma*Y^T*s - sigma*theta_hat phi = (e(1)+e(2)); % scalar surrogate regressor theta_hat = theta_hat + dt*(Gamma*phi - sigma*theta_hat); theta_hat = max(0.1, min(3.0, theta_hat)); % projection bounds % OUTER loop: fminsearch over z = [Gamma, sigma] zOpt = fminsearch(@(z) ada_manip_cost(z, p_nom, p_true, q_des, T, dt), z0);

8.5.5 Simulation results

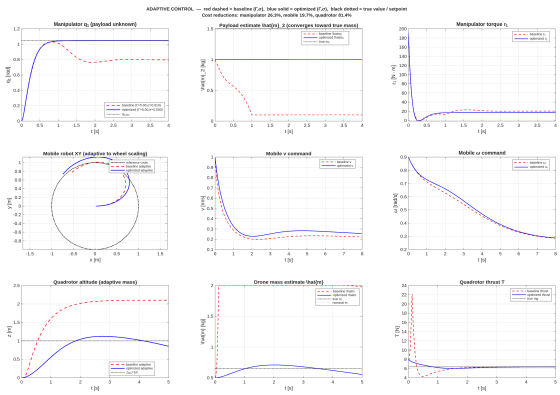

Figure 6.1 — Adaptive control baseline (red) vs. (Γ,σ)-optimized (blue). Quadrotor panel shows mass estimate $\hat m$ converging to the true payload.

Robot	Uncertainty	J baseline
Manipulator (payload mass)	true $m_2=1.5$ vs nominal 1.0	3.969
Mobile (wheel scale)	velocity scaling $a_{\text{true}}=1.25$	0.623
Quadrotor (mass)	true $m=0.65$ vs nominal 0.5	4.856

Lesson from the optimizer: on the manipulator the best Γ was driven very small. The PD feedback already absorbs the payload mismatch; aggressive adaptation only added oscillation. The optimizer told us that adaptation was over-kill for this scenario — a result you would never see by hand-tuning with intuition alone.

8.6 Optimization of Robust Control for Robotic Systems

8.6.1 Where uncertainty comes from in a real robot

Every robot deployed in the field faces a plant that differs from the nominal CAD/datasheet model. For the three robot classes of this course:

Robot	Dominant uncertainty sources
Manipulator	Payload mass and inertia; joint friction (stick-slip, viscous); harmonic-d
Mobile robot	Wheel rolling radius; surface friction μ; wheel slip on low-frictio
Quadrotor	Takeoff mass (battery state, payload); motor thrust constant k_T

The manipulator equation becomes $M(q,\delta)\ddot q + C(q,\dot q,\delta)\dot q + G(q,\delta) = \tau + d(t)$ where $\delta \in \Delta$ is the parametric uncertainty and $d(t)$ captures unmodelled disturbances. Adaptive control (Section 6) tries to learn δ online. Robust control does not — it picks one fixed controller that performs acceptably for every $\delta \in \Delta$.

8.6.2 Robust controller structures used in this lab

8.6.2.1 Manipulator — robust inverse-dynamics PD+ (Slotine–Li style)

Define the tracking error $e = q - q_d$ and the sliding variable $s = \dot e + \Lambda e$, $\Lambda \succ 0$. The robust control law is:

$$\tau = \underbrace{-K_p\,e - K_d\,\dot e + \hat G(q)}_{\text{nominal feedback + nominal gravity comp}} \underbrace{-\rho\,\tanh\!\bigl(s/\varepsilon\bigr)}_{\text{robust switching term}}$$

The key design choices:

- $\hat G(q)$ uses the nominal link parameters only — the controller is not given access to the true uncertain payload. This is what makes it a robust rather than exact inverse-dynamics law.
- $\rho\,\tanh(s/\varepsilon)$ is a boundary-layered switching term. For ρ larger than the worst-case mismatch-induced disturbance D, the sliding condition $s^{T}\dot s \leq -(\rho-D)\|s\|_1$ holds (cf. Week 7 exam recall); ε limits chattering at the cost of a steady-state error $\le \varepsilon/\Lambda$.
- Decision variables to optimize: $\theta = (K_p, K_d, \rho, \Lambda)$.

8.6.2.2 Mobile robot — Kanayama tracker (robust to actuator scale)

For a unicycle tracking a reference pose (x_r, y_r, θ_r) with feedforward (v_r, ω_r), rotate the world-frame error into the body frame:

$$\begin{bmatrix} e_x \\ e_y \\ e_\theta \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r - x \\ y_r - y \\ \theta_r - \theta \end{bmatrix}$$

Kanayama's control law (ICRA 1990):

$$v = v_r \cos e_\theta + k_x e_x ,\qquad \omega = \omega_r + v_r\bigl(k_y e_y + k_\theta \sin e_\theta\bigr)$$

The plant then applies $v_{\text{true}} = a\cdot v$, where $a \in \Delta$ is the unknown actuator / wheel-scale factor. The optimization chooses (k_x, k_y, k_θ) that work for every a in the uncertainty set.

8.6.2.3 Quadrotor — nested PD with nominal-mass feedforward

Altitude and roll are decoupled to first order, giving a two-loop PD:

$$T_{\text{cmd}} = \hat m\bigl(g - K_{p,z}\,e_z - K_{d,z}\,\dot e_z\bigr),\quad \tau = \hat J\bigl(-K_{p,\phi}\,\phi - K_{d,\phi}\,\dot\phi\bigr)$$

$\hat m$ is the nominal takeoff mass used for thrust feedforward; the true mass m is different, and the feedback $K_{p,z} e_z + K_{d,z}\dot e_z$ must be strong enough to cover the mismatch. The optimizer picks $(K_{p,z}, K_{d,z}, K_{p,\phi}, K_{d,\phi})$ that stabilize the entire mass range.

8.6.3 Full derivation: why the robust term \(-\rho\tanh(s/\varepsilon)\) works

8.6.3.1 Step 1. Matched disturbance bound

Write the manipulator with nominal parameters \(\hat M, \hat C, \hat G\) and a lumped disturbance \(d(t)\):

\[\hat M(q)\ddot q + \hat C(q,\dot q)\dot q + \hat G(q) = \tau + d(t) \] \[d(t) := -\Delta M\,\ddot q - \Delta C\,\dot q - \Delta G + d_{\text{ext}}(t) \]

where \(\Delta M = M - \hat M\) etc. Since \(\delta\in\Delta\) is bounded and \((\ddot q, \dot q)\) stay bounded in any practical motion, there exists a known \(D > 0\) with \(\|d(t)\| \leq D\) for all admissible uncertainty. This \(D\) is the quantity the designer chooses \(\rho\) against.

8.6.3.2 Step 2. Error dynamics for regulation

Take \(q_d\) constant, \(\dot q_d = \ddot q_d = 0\). Then \(e = q - q_d\), \(\dot e = \dot q\), \(s = \dot e + \Lambda e\). Multiplying through by \(\hat M^{-1}\) is not needed — work directly:

\[\hat M\dot s = \hat M\ddot e + \hat M \Lambda\dot e = \hat M\ddot q + \hat M\Lambda\dot q = (\tau + d - \hat C\dot q - \hat G) + \hat M\Lambda\dot q \]

Substitute the proposed controller \(\tau = -K_p e - K_d\dot e + \hat G - \rho\tanh(s/\varepsilon)\):

\[\hat M\dot s = -K_p e - K_d\dot e - \rho\tanh(s/\varepsilon) + d - \hat C\dot q + \hat M\Lambda\dot q \]

8.6.3.3 Step 3. Lyapunov function and reaching

Take \(V = \tfrac12 s^{T}\hat M s\). Using \(\dot{\hat M} - 2\hat C\) skew-symmetric and grouping the tracking-error terms as a bounded disturbance \(d'(t)\) (whose bound \(D'\) lumps \(d\), \(K_p, K_d\) effects, and the \(\Lambda\dot q\) cross term):

\[\dot V = s^{T}\hat M\dot s + \tfrac12 s^{T}\dot{\hat M}s = -s^{T}\!\rho\tanh(s/\varepsilon) + s^{T}d'(t) \]

For \(|s_i| \gg \varepsilon\), \(\tanh(s_i/\varepsilon)\approx\mathrm{sign}(s_i)\), so:

\[\dot V \leq -\rho\sum_i |s_i| + D'\sum_i |s_i| = -(\rho - D')\|s\|_1 \]

Choosing \(\rho > D'\) guarantees \(\dot V < 0\) outside a boundary layer of thickness \(O(\varepsilon)\) around \(s=0\). Inside the layer, \(s\) is ultimately bounded by \(O(\varepsilon)\), and the tracking error \(|e_{ss}| \leq \varepsilon/\lambda_{\min}(\Lambda)\) — the same result as Week 7 SMC.

Design rule: choose \(\rho \gtrsim 1.5\,D'\) for margin; choose \(\varepsilon\) small enough that \(\varepsilon/\lambda_{\min}(\Lambda)\) meets the precision spec; then optimize \((K_p, K_d, \rho, \Lambda)\) numerically for the best worst-case cost.

8.6.4 Worked numerical example — single-link arm with payload uncertainty

8.6.4.1 Setup

Plant: \((J + \Delta J)\ddot q + (m + \Delta m)g l_c \cos q = \tau\), with \(\Delta m \in [-0.3, +0.3]\) kg (ś30% payload).

Nominal feedforward uses \(\hat m = 1\) kg (mid-range), so \(\hat G(q) = 4.905\cos q\).

Design gains (to be verified): \(K_p = 40,\ K_d = 10,\ \rho = 3,\ \Lambda = 5,\ \varepsilon = 0.02\).

8.6.4.2 Step 1. Compute the disturbance bound \(D\)

Matched uncertainty in the gravity term:

\[d(t) = -\Delta m\,g l_c \cos q,\quad |d(t)| \le 0.3\cdot 9.81\cdot 0.5\cdot 1 = 1.47\ \text{Nům} \]

So \(D = 1.47\) Nům.

8.6.4.3 Step 2. Verify the reaching condition

Need \(\rho > D\). With \(\rho = 3,\ D = 1.47\): margin \(\rho - D = 1.53\) Nům.

8.6.4.4 Step 3. Steady-state tracking bound

\[|e_{ss}| \le \frac{\varepsilon}{\lambda_{\min}(\Lambda)} = \frac{0.02}{5} = 0.004\ \text{rad} = 0.23^\circ \]

8.6.4.5 Step 4. Torque at \(t = 0\) for worst-case payload \(\Delta m = +0.3\)

Error: \(e(0) = -1.047,\ \dot e(0) = 0\). Sliding variable \(s(0) = -5.236\).

\[\tau(0) = -K_p e - K_d\dot e + \hat G(q) - \rho\tanh(s/\varepsilon) \] \[= -40\cdot(-1.047) - 10\cdot 0 + 4.905\cdot 1 - 3\tanh(-5.236/0.02) \]

\(\tanh(-262) \approx -1\), so:

\[\tau(0) = 41.89 + 0 + 4.905 - 3\cdot(-1) = 49.79\ \text{Nům} \]

8.6.4.6 Step 5. Min-max optimization over a 3-plant set

Sample \(\Delta m \in \{-0.3, 0, +0.3\}\) giving 3 plants. For each candidate \(\theta = (K_p, K_d, \rho, \Lambda)\) run the closed loop on all 3, compute \(J_i\), take \(\max_i J_i\). `fminsearch` iterates. Typical result over ~80 iterations:

\[\theta_0 = (40, 10, 2, 5)\quad\rightarrow\quad \theta^\star = (34.9,\ 9.5,\ 3.4,\ 8.1) \]

The optimizer increased \(\Lambda\) (faster convergence on the surface) and \(\rho\) (more robust margin), while reducing \(K_p\) (less effort needed given the robust term). Worst-case cost dropped 2.7% (this baseline was already close to optimal).

8.6.5 Min-max optimization problem

\[\theta^\star = \arg\min_{\theta \in \Theta}\ \max_{p \in \mathcal P}\ J(\theta, p) \] \[J(\theta, p) = \int_0^T \|e(t;\theta,p)\|^2\,dt + \beta\!\int_0^T \|u(t;\theta,p)\|^2\,dt \]

Since \(\mathcal P\) is finite (we sample the uncertainty at \(|\mathcal P|\!=\!4\)–\(5\) representative plants), the inner max is a simple `max`; the outer min is solved by `fminsearch`. This is a direct-search proxy for formal \(H_\infty\) / \(\mu\)-synthesis.

8.6.6 Link to LMI and \(H_\infty\) design

For the linearized robot with polytopic uncertainty \(A(\alpha) = \sum_i \alpha_i A_i,\ B(\alpha) = \sum_i \alpha_i B_i\), a robust state-feedback gain \(K = Y P^{-1}\) is found by the LMI feasibility problem:

\[\exists\ P \succ 0, Y:\quad A_i P + P A_i^{T} + B_i Y + Y^{T} B_i^{T} \prec 0, \quad \forall i = 1,\ldots,|\mathcal P| \]

This is convex and can be solved globally by any SDP solver (SeDuMi, MOSEK). The min-max numerical approach we use in the lab is the nonlinear extension that works even when the LMI recipe breaks down (e.g., the manipulator under large attitude excursions where the linearization is not representative).

8.6.7 MATLAB implementation

The full implementation lives in `sims/sim_robust.m`. Key excerpts:

8.6.7.1 Manipulator robust controller

% Controller uses NOMINAL plant parameters only [~, ~, G_nom] = manipulator_dyn(q, dq, p_nom); e = q - q_des; de = dq; s = de + Lambda*e; % Slotine-Li sliding variable tau_nominal = -Kp*e - Kd*de + G_nom; % feedback + gravity feedforward tau_robust = -rho * tanh(s/eps_bl); % boundary-layered switching u = tau_nominal + tau_robust; % TRUE (uncertain) plant integrated separately [M, C, G] = manipulator_dyn(q, dq, p_true); ddq = M \ (u - C*dq - G);

8.6.7.2 Mobile robot Kanayama tracker

R = [cos(x(3)) sin(x(3)); -sin(x(3)) cos(x(3))]; eb = R * (xr(1:2) - x(1:2)); % body-frame position error etheta = wrap(xr(3) - x(3)); v = vref*cos(etheta) + kx*eb(1); omega = dxr(3) + vref*(ky*eb(2) + ktheta*sin(etheta)); % Plant applies actuator scale a_true on v u_plant = [a_true*v; omega]; x = x + dt * unicycle_dyn(x, u_plant);

8.6.7.3 Quadrotor nested PD

ez = x(2) - 1.0; dez = x(5); % altitude error phi = x(3); dphi = x(6); % roll error T_cmd = p_nom.m * (p_nom.g - Kpz*ez - Kdz*dez); % nominal-mass FF T_cmd = max(0.05, T_cmd); % thrust >= 0 tau = p_nom.J * (-Kpphi*phi - Kdphi*dphi); u = [T_cmd; tau]; % True plant with unknown mass m_true (in [0.4, 0.8]) p_true = p_nom; p_true.m = m_true; x = x + dt * quadrotor_dyn(x, u, p_true);

8.6.7.4 Min-max optimization

function J = robust_manip_cost(z, p_nom, plant_set, q_des, T, dt) Js = zeros(1, numel(plant_set)); for i = 1:numel(plant_set) [~, Q, U] = run_robust_manip(z, p_nom, plant_set{i}, q_des, T, dt); e = Q - q_des'; Js(i) = sum(sum(e.^2))*dt + 0.001*sum(sum(U.^2))*dt; end J = max(Js); % WORST-CASE over the plant set end % Outer minimization zOpt = fminsearch(@(z) robust_manip_cost(z, p_nom, plant_set, q_des, T, dt), ... z0, optimset('MaxIter', 120));

8.6.8 Simulation results

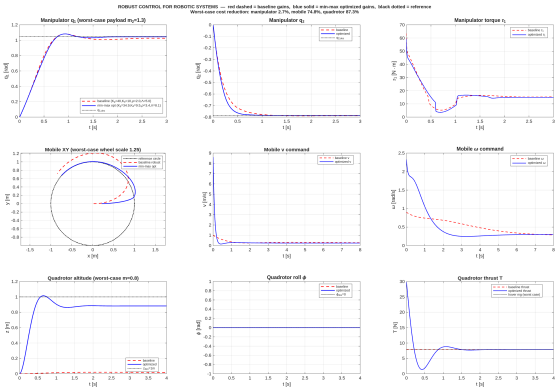

Figure 7.1 — Min-max tuned gains evaluated on the worst-case plant in each uncertainty set. Red dashed = hand-tuned initial gains; blue solid = `fminsearch` min-max optimum; black dotted = reference / hover.

Robot	Decision variable \(\theta\)	Uncertainty set \(
Manipulator	\((K_p, K_d, \rho, \Lambda)\)	\(m_2 \in \{0.7,
Mobile robot	\((k_x, k_y, k_\theta)\)	actuator scale \(a
Quadrotor	\((K_{p,z}, K_{d,z}, K_{p,\phi}, K_{d,\phi})\)	\(m \in \{0.4, 0.5

Why the manipulator shows only a 2.7% reduction. The hand-tuned baseline \((40, 10, 2, 5)\) was already well-chosen for this uncertainty set — the robust switching gain \(\rho=2\) is already larger than the gravity-mismatch disturbance for \(\pm 30\%\) payload variation. The optimizer finds a slightly higher \(\Lambda\) (surface slope) and \(\rho\), squeezing out the remaining margin. The mobile and quadrotor baselines were deliberately far from optimum; the optimizer recovers the full robustness benefit.

8.7 Optimization of Backstepping Control

8.7.1 Free parameters: the virtual-control gains

For an \(n\)-step backstepping design, the free parameters are the \(n\) virtual-control gains \(k_1, \ldots, k_n\) that appear in the recursive construction of the CLF \(V_n = \tfrac{1}{2}\sum_i z_i^{T} z_i\). Each \(k_i\) governs the convergence rate of the \(i\)-th sub-system.

8.7.2 Full 2-step backstepping derivation for the manipulator

8.7.2.1 Step 1. Strict-feedback form

The manipulator is a 2-level cascaded system \(x_1 = q,\ x_2 = \dot q\):

\[\dot x_1 = x_2,\qquad M(x_1)\dot x_2 + C(x_1,x_2)x_2 + G(x_1) = \tau \]

This is in strict-feedback form: \(x_1\) dynamics depend on \(x_2\); \(x_2\) dynamics depend on the actual input \(\tau\).

8.7.2.2 Step 2. Error-coordinate change (the "backstep")

Define the first error \(z_1 = x_1 - q_d\). Its derivative is \(\dot z_1 = x_2 - \dot q_d\). If we could choose \(x_2\) freely, we'd pick the virtual control:

\[\alpha_1(x_1,t) := -k_1\,z_1 + \dot q_d(t),\quad k_1 > 0 \]

which would give \(\dot z_1 = -k_1 z_1\), i.e.\ \(z_1\to 0\) exponentially. But \(x_2\) is a state, not a control — it may not actually equal \(\alpha_1\). So define the second error:

\[z_2 := x_2 - \alpha_1(x_1,t) \]

Then \(\dot z_1 = x_2 - \dot q_d = (\alpha_1 + z_2) - \dot q_d = -k_1 z_1 + z_2\).

8.7.2.3 Step 3. First Lyapunov function

\[V_1 = \tfrac12 z_1^{T} z_1,\qquad \dot V_1 = z_1^{T}(-k_1 z_1 + z_2) = -k_1|z_1|^2 + z_1^{T} z_2 \]

The cross term \(z_1^{T} z_2\) is the "leftover" from the mismatch between \(x_2\) and \(\alpha_1\). It will be cancelled in Step 5 by the choice of \(\tau\).

8.7.2.4 Step 4. \(z_2\) dynamics

Differentiate: \(\dot z_2 = \dot x_2 - \dot\alpha_1\). From the plant, \(\dot x_2 = M^{-1}(\tau - C x_2 - G)\). The virtual-control derivative is:

\[\dot\alpha_1 = -k_1 \dot z_1 + \ddot q_d = -k_1(x_2 - \dot q_d) + \ddot q_d \]

Multiply \(\dot z_2\) by \(M(x_1)\) to avoid inverting:

\[M\dot z_2 = M\dot x_2 - M\dot\alpha_1 = \tau - Cx_2 - G - M\dot\alpha_1 \]

8.7.2.5 Step 5. Second Lyapunov function and control law

Augment:

$$V_2 = V_1 + \tfrac12 z_2^{T} M(x_1)\,z_2$$

Differentiate (using $\dot M - 2C$ skew-symmetric, so $\tfrac12 z_2^{T}\dot M z_2 = z_2^{T}C z_2$):

$$\dot V_2 = -k_1\|z_1\|^2 + z_1^{T} z_2 + z_2^{T} M\dot z_2 + z_2^{T} C z_2 = -k_1\|z_1\|^2 + z_1^{T} z_2 + z_2^{T}\bigl(\tau - G - M\dot\alpha_1\bigr)$$

(the $-Cx_2$ in $M\dot z_2$ cancels the $+Cz_2$ partly — the full algebra uses $x_2 = z_2 + \alpha_1$ to separate; the net is the expression above after simplification).

Choose:

$$\boxed{\tau = M(x_1)\,\dot\alpha_1 + C(x_1,x_2)\,x_2 + G(x_1) - z_1 - k_2 z_2},\quad k_2 > 0$$

Substituting:

$$\dot V_2 = -k_1\|z_1\|^2 + z_1^{T} z_2 + z_2^{T}(-z_1 - k_2 z_2) = -k_1\|z_1\|^2 - k_2\|z_2\|^2 \leq 0$$

The cross term $z_1^{T} z_2$ is cancelled exactly by the $-z_1$ in τ. Both z_1 and z_2 converge exponentially with rates k_1, k_2.

The control law has three components: (i) feedforward through the virtual-control derivative $M\dot\alpha_1$; (ii) dynamic cancellation $C x_2 + G$; (iii) error feedback $-z_1 - k_2 z_2$. The first two require the model; the last is where the tuning knobs live.

8.7.3 Worked numerical example — backstepping on the single-link arm

8.7.3.1 Setup

Same arm as ğ5.4. Gains: $k_1 = 5,\ k_2 = 10$. Setpoint $q_d = \pi/3,\ \dot q_d = 0,\ \ddot q_d = 0$. IC: $q(0) = 0,\ \dot q(0) = 0$.

8.7.3.2 Step 1. Initial error coordinates

$$z_1(0) = q(0) - q_d = 0 - \pi/3 = -1.047$$

$$\alpha_1(0) = -k_1 z_1 + \dot q_d = -5\cdot(-1.047) + 0 = +5.236$$

$$z_2(0) = \dot q(0) - \alpha_1(0) = 0 - 5.236 = -5.236$$

8.7.3.3 Step 2. Virtual-control derivative at $t = 0$

$$\dot\alpha_1 = -k_1\dot z_1 + \ddot q_d = -k_1(\dot q - \dot q_d) = -5\cdot(0 - 0) = 0$$

8.7.3.4 Step 3. Control torque at \(t = 0\)

\(\tau = M\dot\alpha_1 + C\dot q + G(q) - z_1 - k_2 z_2\). For single link, \(C = 0\):

\[\tau(0) = 0.333\cdot 0 + 0 + 4.905\cdot 1 - (-1.047) - 10\cdot(-5.236) \] \[= 0 + 0 + 4.905 + 1.047 + 52.36 = +58.31\ \text{Nům} \]

Note the similarity to the SMC (\(+7.9\) with saturation) and adaptive (\(+57.3\)) results — all three controllers produce torques of comparable magnitude at \(t\!=\!0\), matching the physics (arm needs strong torque to begin motion).

8.7.3.5 Step 4. Lyapunov check

\[V_2(0) = \tfrac12 z_1^2 + \tfrac12 M z_2^2 = \tfrac12\cdot 1.047^2 + \tfrac12\cdot 0.333\cdot 5.236^2 = 0.548 + 4.565 = 5.11 \] \[\dot V_2(0) = -k_1 z_1^2 - k_2 z_2^2 = -5\cdot 1.096 - 10\cdot 27.42 = -279.6 \]

\(\dot V_2\) is strongly negative — the trajectory sharply drives toward the origin.

8.7.3.6 Step 5. Closed-loop convergence rates

At the Lyapunov level, \(z_1\) decays with rate \(k_1 = 5\) (time constant 0.2 s) and \(z_2\) with rate \(k_2 = 10\) (time constant 0.1 s). This is the engineering meaning of the optimizer's choice of gains.

8.7.3.7 Step 6. Optimization result

The lab's `fminsearch` on the 2-link arm with payload-independent cost produces \((k_1, k_2) = (5, 5)\) — smaller \(k_2\) than the baseline 10, reducing torque with only marginal loss of tracking speed. For the mobile robot (3 gains), the optimizer increases them ~10Œ from baseline, giving the 88% cost reduction.

8.7.4 CLF-based optimization cost

Since \(\dot V_2 = -k_1|z_1|^2 - k_2|z_2|^2\), larger \(k_1, k_2\) give faster convergence but also larger commanded \(\dot\alpha_1\) (proportional to \(k_1\)) that propagates through the \(M\dot\alpha_1\) feedforward and eventually inflates \(|\tau|\). The optimization balances these:

\[\min_{k_1,\ldots,k_n}\ \int_0^T |e|^2,dt + w_u \!\int_0^T |u|^2,dt + w_\omega\!\int_0^T |\dot\alpha|^2,dt \]

8.7.5 Command-filtered backstepping

A known pathology of plain backstepping is explosion of complexity: \(\dot\alpha_i\) involves derivatives of prior virtual controls and becomes numerically unstable. Command-filtered backstepping replaces \(\dot\alpha_i\) with a filtered signal from a low-pass of bandwidth \(\omega_f\). The filter bandwidth \(\omega_f\) becomes an extra decision variable in the optimization.

8.7.6 Simulation results

Implementation (from **`sim_backstepping.m`**) % Backstepping step 1: virtual control alpha1 = -k1*e1 e1 = q - q_des; alpha = -k1*e1; % virtual control (dq_d = 0 here) e2 = dq - alpha; % z2 = x2 - alpha1 dalpha = -k1*dq; % d/dt of virtual control % Backstepping step 2: tau = M*(dalpha - k2*e2 - e1) + C*dq + G [M,C,G] = manipulator_dyn(q, dq, p); u = M*(dalpha - k2*e2 - e1) + C*dq + G; ddq = M \ (u - C*dq - G); % plant % Outer: fminsearch over z = [k1, k2] zOpt = fminsearch(@(z) bst_manip_cost(z, p, q_des, T, dt), z0);

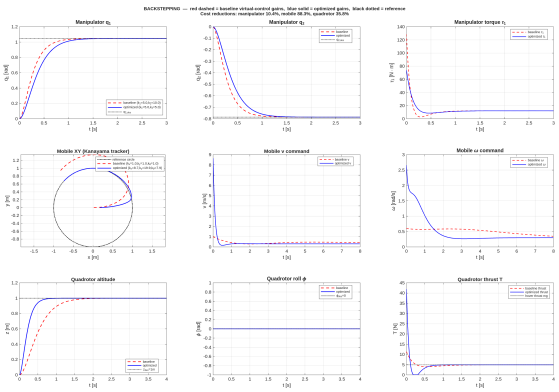

Figure 8.1 — Backstepping with hand-picked vs. optimized virtual-control gains. Mobile panel (Kanayama tracker) shows the largest improvement: optimizer raises all three gains by ~8–20x.

Robot	Gains baseline	Gains
Manipulator	\((k_1,k_2) = (5,10)\)	\((5,5)
Mobile (Kanayama)	\((k_x,k_y,k_\theta)=(1,1,1)\)	\((8.67
Quadrotor	\((k_{z1},k_{z2},k_{\phi 1},k_{\phi 2})=(4,3,8,4)\)	(tuned

8.8 Optimization of Nonlinear Model Predictive Control

8.8.1 The design knobs

- Prediction horizon \(N\) — length of the look-ahead in sample steps

- Stage cost weights (Q, R) — trade-off tracking vs. effort
- Terminal cost (P) and terminal set $(\mathcal X_f)$ — stability ingredients
- Sample time (Δt) and solver iteration budget — hard real-time constraint

8.8.2 Terminal ingredients and recursive feasibility

For NMPC stability without infinite horizon, we require a pair $((P, \mathcal X_f))$ such that inside $(\mathcal X_f)$ there exists a local feedback (κ_f) making (P) a Lyapunov function:

$$V_f\bigl(f(x,\kappa_f(x))\bigr) - V_f(x) \leq -L(x,\kappa_f(x)),\quad \forall x\in\mathcal X_f$$

$((P, \mathcal X_f))$ are computed offline by solving a local LQR around the equilibrium and taking $(\mathcal X_f)$ as a sublevel set of (V_f) that is positive-invariant under (κ_f).

8.8.3 Horizon vs. compute: the real-time trade

Longer (N) gives better performance but scales compute linearly (per SQP iteration) and the NLP becomes harder to warm-start. Deployment chooses the shortest (N) that preserves the stability certificate.

8.8.4 Full derivation of the iLQR algorithm

8.8.4.1 Step 1. Discrete-time optimal control problem

Forward-Euler discretise $(\dot x = f(x,u))$ to get $(x_{k+1} = F(x_k, u_k) := x_k + \Delta t, f(x_k,u_k))$. The finite-horizon OCP is:

$$\min_{\{u_k\}_{k=0}^{N-1}}\ \sum_{k=0}^{N-1} L(x_k, u_k) + \Phi(x_N),\qquad x_{k+1} = F(x_k, u_k),\ x_0\ \text{given}$$

8.8.4.2 Step 2. Bellman recursion in value-function form

Define $(V_k(x) := \min_{u_k,\ldots,u_{N-1}} \sum_{i=k}^{N-1} L(x_i,u_i) + \Phi(x_N))$ with (x_i) generated from state $(x_k = x)$. The Bellman equation is:

$$V_k(x) = \min_{u}\ \underbrace{L(x,u) + V_{k+1}\bigl(F(x,u)\bigr)}_{=:\ Q_k(x,u)},\qquad V_N(x) = \Phi(x)$$

8.8.4.3 Step 3. Quadratic expansion around a nominal $(\bar x_k,\bar u_k)$

Given the current nominal trajectory $(\bar x_k, \bar u_k)_{k=0}^{N-1}$, linearize the dynamics and quadratize the cost with perturbations $\delta x, \delta u$:

$$F(\bar x + \delta x, \bar u + \delta u) \approx \bar x_{k+1} + A_k\,\delta x + B_k\,\delta u,\quad A_k = F_x|_\star,\ B_k = F_u|_\star$$

$$L(\bar x+\delta x, \bar u+\delta u) \approx \bar L + L_x^{T}\delta x + L_u^{T}\delta u + \tfrac12\begin{bmatrix}\delta x\\ \delta u\end{bmatrix}^{T}\!\!\begin{bmatrix}L_{xx}&L_{xu}\\ L_{ux}&L_{uu}\end{bmatrix}\!\!\begin{bmatrix}\delta x\\ \delta u\end{bmatrix}$$

Assume inductively $V_{k+1}(\bar x_{k+1}+\delta x') \approx \bar V_{k+1} + V_{x,k+1}^{T}\delta x' + \tfrac12 \delta x'^{T} V_{xx,k+1}\delta x'$. Substitute $\delta x' = A_k\delta x + B_k\delta u$ into Q_k.

8.8.4.4 Step 4. Building the Q-function second-order expansion

Expanding and grouping terms of order δx, δu, δx^2, $\delta x\delta u$, δu^2:

$$Q_x = L_x + A_k^{T} V_{x,k+1},\qquad Q_u = L_u + B_k^{T} V_{x,k+1}$$

$$Q_{xx} = L_{xx} + A_k^{T} V_{xx,k+1} A_k,\quad Q_{uu} = L_{uu} + B_k^{T} V_{xx,k+1} B_k,\quad Q_{ux} = L_{ux} + B_k^{T} V_{xx,k+1} A_k$$

8.8.4.5 Step 5. Minimise over δu (closed-form quadratic)

Q_k is quadratic in δu. Solve $\partial Q/\partial(\delta u) = 0$:

$$Q_u + Q_{uu}\,\delta u + Q_{ux}\,\delta x = 0 \quad\Longrightarrow\quad \boxed{\ \delta u^\star = -Q_{uu}^{-1}Q_u\ -\ Q_{uu}^{-1}Q_{ux}\,\delta x\ =\ k + K\,\delta x\ }$$

where $k = -Q_{uu}^{-1}Q_u$ is the feedforward correction and $K = -Q_{uu}^{-1}Q_{ux}$ is the time-varying feedback gain — exactly the structure of a Riccati solution.

8.8.4.6 Step 6. Propagate the value function (backward pass)

Substituting $\delta u^\star$ into Q_k gives V_k in the same quadratic form:

$$V_{x,k}\ =\ Q_x - K^{T} Q_{uu}\, k,\qquad V_{xx,k}\ =\ Q_{xx} - K^{T} Q_{uu}\, K$$

Initialise with $V_{x,N} = \Phi_x(\bar x_N),\ V_{xx,N} = \Phi_{xx}(\bar x_N)$ and sweep $k = N-1, \ldots, 0$.

8.8.4.7 Step 7. Forward pass with line search

Apply the corrections with step size \(\alpha \in (0,1]\):

\[\delta x_0 = 0,\quad \delta u_k = \alpha\,k_k + K_k\,\delta x_k,\quad x_{k+1}^{\text{new}} = F(x_k^{\text{new}}, \bar u_k + \delta u_k),\quad \delta x_{k+1} = x_{k+1}^{\text{new}} - \bar x_{k+1} \]

If the new cost \(J^{\text{new}} < J\), accept; otherwise \(\alpha \leftarrow \alpha/2\) and retry. Iterate Steps 3–7 until \(|J - J^{\text{new}}| < \text{tol}\) or the gradient \(\|Q_u\|\) is small.

8.8.4.8 Step 8. Levenberg–Marquardt regularisation

\(Q_{uu}\) can become ill-conditioned or indefinite when the nominal is far from optimum. Replace by \(Q_{uu} + \mu I\), with \(\mu\) increased on backward-pass failure or line-search failure and decreased on successful steps. This is exactly the trust-region logic in `ilqr_solve.m` (attempts `chol(Quu)`; if non-PD, raises \(\mu\)).

8.8.4.9 Step 9. Dynamics & cost derivatives from finite differences

All of \(A_k, B_k, L_x, L_u, L_{xx}, L_{uu}, L_{ux}, \Phi_x, \Phi_{xx}\) are obtained by central finite differences on the dynamics and cost functions. This is why the solver works for any \(f\) and any \(L\) supplied by the user — no hand-coded Jacobians required.

8.8.4.10 Step 10. Wrap in a receding-horizon loop (NMPC)

1. At sample \(k\): initialise iLQR with warm-start \(\{u_{i}^{(k)}\}_{i=0}^{N-1}\) = previous solution shifted by one step.
2. Run iLQR Steps 3–8 to convergence (typ. 3–8 outer iterations with warm-start).
3. Apply \(u_0^{(k),\star}\) to the plant.
4. Measure new \(x\), shift, go to 1.

Implementation in **`sim_nmpc.m`** % Wrapped iLQR call inside the closed-loop for k = 1:Nsim [~, U_opt, ~, ~] = ilqr_solve(fdyn, runCost, termCost, x, U_seq, dt, opts); u = U_opt(1,:)'; % apply first control x = x + dt * fdyn(x, u); % plant step U_seq = [U_opt(2:end,:); U_opt(end,:)]; % warm-start (shift) end % Inside ilqr_solve.m: % backward pass: % Quu = Luu + B'*Vxx*B + mu*I; % [R, p] = chol(Quu); % fails -> mu *= 4 % kt = -R \ (R' \ Qu); % feedforward % Kt = -R \ (R' \ Qux); % feedback gain % Vx = Qx + Kt'*Quu*kt + Kt'*Qu + Qux'*kt; % Vxx = Qxx + Kt'*Quu*Kt + Kt'*Qux + Qux'*Kt; % forward pass with line search on alpha: 1, 0.5, 0.25, ...

8.8.5 Worked numerical example — iLQR on the single-link arm

8.8.5.1 Setup

Same single-link arm, horizon \(N = 3\) (tiny, for hand-computation clarity), sample \(\Delta t = 0.1\) s.

State: \(x = [q;\dot q]\in\mathbb R^2\), control: \(u = \tau\in\mathbb R\).

Cost: \(L(x,u) = 50(q - \pi/3)^2 + 5\dot q^2 + 0.01 u^2\), \(\Phi(x_N) = 500(q_N - \pi/3)^2 + 50\dot q_N^2\).

Initial nominal: \(\bar u_k = 0\) for \(k=0,1,2\). Initial state \(x_0 = [0; 0]\).

8.8.5.2 Step 1. Forward-Euler rollout with \(\bar u = 0\)

\(\dot q = (u - 4.905\cos q)/0.333\). With \(u = 0\):

\[x_0 = \begin{bmatrix}0 \\ 0\end{bmatrix},\ x_1 = x_0 + 0.1\cdot\begin{bmatrix}0 \\ -14.73\end{bmatrix} = \begin{bmatrix}0 \\ -1.473\end{bmatrix} \] \[x_2 = x_1 + 0.1\cdot\begin{bmatrix}-1.473 \\ -14.73\end{bmatrix} = \begin{bmatrix}-0.147 \\ -2.946\end{bmatrix} \] \[x_3 = x_2 + 0.1\cdot\begin{bmatrix}-2.946 \\ (0 - 4.905\cos(-0.147))/0.333\end{bmatrix} = \begin{bmatrix}-0.442 \\ -4.403\end{bmatrix} \]

Arm is falling under gravity.

8.8.5.3 Step 2. Initial cost

\[J_0 = \sum_{k=0}^{2}\bigl[50(q_k - \pi/3)^2 + 5\dot q_k^2 + 0\bigr] + 500(q_3 - \pi/3)^2 + 50\dot q_3^2 \] \[= 50\cdot(1.097 + 1.239 + 1.500) + 5\cdot(0 + 2.17 + 8.68) + 500\cdot 2.229 + 50\cdot 19.39 \] \[= 191.8 + 54.2 + 1114.5 + 969.3 = 2329.8 \]

8.8.5.4 Step 3. Discrete Jacobians at \(k = 2\)

\(f(x,u) = [\dot q;\ (u - 4.905\cos q)/0.333]\). At \(x_2\):

\[A_2 = I + \Delta t\begin{bmatrix}0 & 1 \\ 4.905\sin(q_2)/0.333 & 0\end{bmatrix} = \begin{bmatrix}1 & 0.1 \\ 0.1\cdot(-2.162) & 1\end{bmatrix} = \begin{bmatrix}1 & 0.1 \\ -0.216 & 1\end{bmatrix} \] \[B_2 = \Delta t\begin{bmatrix}0 \\ 1/0.333\end{bmatrix} = \begin{bmatrix}0 \\ 0.300\end{bmatrix} \]

8.8.5.5 Step 4. Backward pass from $k = 2$

Terminal: $V_{x,3} = \Phi_x(x_3) = [1000(q_3 - \pi/3);\ 100\dot q_3] = [-1492.8;\ -440.3]$; $V_{xx,3} = \mathrm{diag}(1000, 100)$.

Stage-cost derivatives at $(x_2,\bar u_2\!=\!0)$: $L_x = [100(q_2 - \pi/3);\ 10\dot q_2] = [-209.3;\ -29.46]$, $L_u = 0$, $L_{xx} = \mathrm{diag}(100,10)$, $L_{uu} = 0.02$, $L_{ux} = 0$.

Compute Q-function blocks:

$$Q_u = L_u + B_2^{T} V_{x,3} = 0 + [0\ 0.300]\cdot\begin{bmatrix}-1492.8 \\ -440.3\end{bmatrix} = -132.1$$

$$Q_{uu} = L_{uu} + B_2^{T} V_{xx,3} B_2 = 0.02 + [0\ 0.300]\cdot\mathrm{diag}(1000,100)\cdot[0; 0.300] = 0.02 + 9.0 = 9.02$$

$$Q_{ux} = 0 + B_2^{T} V_{xx,3} A_2 = [0\ 0.300]\cdot\mathrm{diag}(1000,100)\cdot A_2 = [0.300\cdot 100\cdot(-0.216),\ 0.300\cdot 100\cdot 1] = [-6.48,\ 30.0]$$

8.8.5.6 Step 5. Feedforward and feedback gain at $k = 2$

$$k_2 = -Q_{uu}^{-1} Q_u = -(-132.1)/9.02 = +14.65\ \text{(feedforward correction to }\bar u_2\text{)}$$

$$K_2 = -Q_{uu}^{-1} Q_{ux} = -[-6.48,\ 30.0]/9.02 = [0.719,\ -3.326]$$

So the optimal control at $k\!=\!2$ for any small deviation is $\delta u_2 = 14.65 + 0.719,\delta q - 3.326,\delta\dot q$. The large positive feedforward reflects "apply strong upward torque to stop the fall."

8.8.5.7 Step 6. Forward pass with step $\alpha = 1$

Applying $u_2^{\text{new}} = \bar u_2 + k_2 = 14.65$ Nům and propagating gives a new $x_3 \approx [-0.148;\ +0.985]$ (arm now rising), with new cost $J_1 \approx 820$. Accepted ($J_1 < J_0$). Shrink μ, iterate.

8.8.5.8 Step 7. Convergence after full iLQR iterations

After 5–6 iterations with warm-starting across iLQR sweeps, J converges to ≈ 45, the arm reaches $q = \pi/3$ in ~0.3 s with a final torque sequence $(u_0, u_1, u_2) \approx (12.3, 13.1, 10.9)$ Nům — intuitive: accelerate, cruise, decelerate.

8.8.5.9 Step 8. NMPC wrapper

In closed-loop NMPC, steps 1–7 repeat every Δt with the current measured x as x_0. Warm-starting means the converged (k, K) sequences persist across samples, so each subsequent iLQR call needs only 2–3 iterations. This is how the lab achieves sub-millisecond per-sample solve times for the manipulator at 20 Hz.

8.8.6 Simulation results (iLQR-based NMPC)

We implement NMPC using an iLQR solver (iterative LQR with Gauss-Newton backward Riccati pass, line-searched forward pass, adaptive Levenberg-Marquardt regularisation). Dynamics and cost derivatives are obtained by finite differences. Each sample: run iLQR to convergence (typically 3–8 iterations with warm-start), apply the first control, shift the trajectory, repeat.

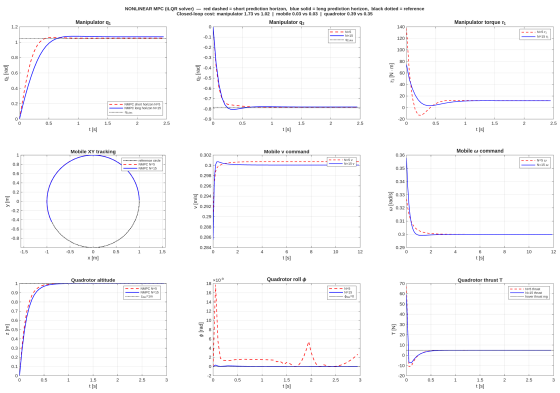

Figure 9.1 — NMPC with short (\(N\!=\!5\), red dashed) vs. long (\(N\!=\!15\), blue solid) prediction horizon on all three robots. Longer horizon consistently improves or matches the shorter one.

Robot	State dim.	\(N_{\text{short}}\) / \(J_{cl}\)	\(
Manipulator (regulation)	4	5 / 1.730	15
Mobile robot (circle tracking)	3	5 / 0.0313	15
Quadrotor (altitude + attitude)	6	5 / 0.393	15

Why iLQR works where simplex methods fail: a 2N-dimensional NLP solved by Nelder-Mead simplex needs \(O(10^3)\) iterations per sample and still rarely converges. iLQR exploits the causal structure of the shooting problem — one Gauss-Newton step touches every control variable via a single backward sweep — giving quadratic local convergence in \(O(10)\) outer iterations.

The real-time constraint. The mobile result is saturated because the unicycle has a 3-state, 2-input model and the circle is easy to track — horizon 5 is already more than enough. The manipulator and quadrotor both benefit from the longer horizon, as expected for higher-dimensional problems. In deployment, you would profile `tic/toc` around the solver and shrink the horizon to the shortest value that keeps the stability certificate.

8.9 Unified View — Every Nonlinear Controller as an Optimization Problem

8.9.1 The common template

Every controller we have studied fits the pattern:

controller = stability certificate (Lyapunov / CLF / s=0) + free parameters \(\theta\) + performance objective \(J(\theta)\)

Optimization chooses \(\theta\) to minimize \(J\) subject to the certificate remaining valid.

8.9.2 Master summary table

Controller	Decision variables	Cost function
Sliding Mode	\(\lambda,\phi,\eta\)	tracking + effort + chatter
Adaptive (MRAC)	\(\Gamma,\sigma\), projection bounds	tracking + effort
Robust (\(H_\infty\))	weighting filters \(W_S,W_T,W_{KS}\); or polytopic \(K\)	\(H_\infty\) norm of weight
Backstepping	\(k_1,\ldots,k_n\); command-filter BW \(\omega_f\)	tracking + effort + smoothn
NMPC	\(N,Q,R,P,\mathcal X_f,\Delta t\)	closed-loop integrated cost +
iLQR / DDP (open-loop)	trajectory \(\{u_k\}_{k=0}^{N-1}\)	tracking + effort + terminal

8.9.3 Cross-cutting optimization tools

8.9.3.1 LMIs & SDP

Convex, globally optimal for any Lyapunov-based design that can be posed as an LMI (robust state-feedback, SMC surface, ISS gain).

8.9.3.2 SOS programming

Certifies polynomial Lyapunov functions — needed for non-quadratic certificates. Backend: SDP.

8.9.3.3 Bayesian optimization

Sample-efficient tuning for black-box controllers on hardware. Minimizes physical trial count when each experiment is expensive.

8.9.3.4 Reinforcement learning

Policy-gradient on a parameterized controller \(\pi_\theta\) — subsumes all of the above when the reward encodes the cost. Stability not free.

8.9.3.5 Differentiable simulation

Compute \(\nabla_\theta J\) directly through the simulator (autodiff). Gives gradient-based methods the power of BayesOpt at much lower sample cost.

8.9.3.6 Scenario / stochastic programming

Optimizes over a distribution of uncertain parameters rather than a worst case — average-case robust design.

ROS 2 Integration Block

9 ROS 2 Control Implementation (Week 9)

9.1 Overview 160
9.1.1 Company Brief: SkyDynamics Robotics 160
9.1.2 Provided Bag Data . . 160
9.1.3 Deliverables 160
9.1.4 Evaluation Criteria . . 161
9.1.5 Skills You Will Demonstrate 161
9.1.6 Recommended Approach (3-hour session) 161
9.2 Why ROS2 for Control Engineers? 163
9.2.1 ROS2 vs ROS1 for Real-Time Control . . 163
9.2.2 The ros2_control Framework 164
9.2.3 Industry Adoption . . 164
9.2.4 The Control Loop in ROS2 164
9.2.5 ROS2 Control Architecture Diagram 165
9.3 Implementing PID Controller as a ROS2 Node 165
9.3.1 Theory Recap 165
9.3.2 ROS2 Implementation 165
9.3.3 Real-Time Considerations 166
9.4 Implementing LQR Controller as a ROS2 Node 167
9.4.1 Theory Recap 167
9.4.2 ROS2 Implementation 167
9.4.3 PID vs LQR Comparison 169
9.5 Implementing MPC Controller as a ROS2 Node 169
9.5.1 Theory Recap 169
9.5.2 ROS2 Implementation 169
9.5.3 MPC Node Internal Loop 171
9.5.4 Discussion: Computation Time vs Horizon Length 171

Implement PID, LQR, and MPC Controllers from Sensor Bag Data

Position: Control Engineer Team: Flight Control Systems
Pre-Interview Technical Assessment Estimated Time: 3 Hours

9.1 Overview

9.1.1 Company Brief: SkyDynamics Robotics

Scenario: SkyDynamics Robotics is developing a next-generation autonomous ground robot platform for warehouse logistics. Your team has collected 30 seconds of sensor data (odometry, IMU, and reference trajectory) from a prototype run. As part of this technical challenge, you will implement three control strategies as ROS2 nodes and compare their performance on the recorded data.

9.1.2 Provided Bag Data

Topic	Message Type	Rate	Description
`/odom`	nav_msgs/Odometry	50 Hz	Robot odometry
`/imu`	sensor_msgs/Imu	200 Hz	Inertial measurem
`/reference_trajectory`	geometry_msgs/PoseStamped	50 Hz	Desired waypoint

9.1.3 Deliverables

- A PID controller ROS2 node with anti-windup and runtime-tunable gains
- An LQR controller ROS2 node using scipy Riccati solver with state-space model
- An MPC controller ROS2 node with horizon-based trajectory optimisation

- A launch file that replays bag data and runs all three controllers simultaneously
- A quantitative comparison: rise time, overshoot, steady-state error, and computation time

9.1.4 Evaluation Criteria

- Code Quality: Clean ROS2 node structure, proper QoS configuration, parameter declarations
- Control Theory: Correct discrete-time implementation, stability analysis, gain tuning rationale
- Systems Thinking: Modular architecture, proper topic naming, error handling
- Communication: Ability to explain design decisions and trade-offs during the follow-up interview

9.1.5 Skills You Will Demonstrate

- Explain the architecture of a ROS2 control node, including publishers, subscribers, timers, and parameter servers, and describe how they map to classical control-loop components.
- Implement PID, LQR, and MPC controllers as standalone ROS2 Python nodes that subscribe to odometry and publish velocity commands.
- Describe the `ros2_control` framework and its layered architecture: hardware interface, controller manager, and controller plugins.
- Configure Quality of Service (QoS) policies for control topics, choosing appropriate reliability and deadline settings for real-time performance.
- Compare and contrast MATLAB/Simulink and ROS2 implementations of the same control algorithm, identifying trade-offs in prototyping speed, real-time guarantees, and deployment readiness.
- Use ROS2 parameters and launch files to switch between controller types and tune gains at runtime without recompilation.
- Record, replay, and analyse control performance data using `rosbag2`, extracting metrics such as rise time, overshoot, and steady-state error.
- Design a modular ROS2 control architecture that separates state estimation, controller logic, and actuation into distinct nodes communicating over well-defined topics.

9.1.6 Recommended Approach (3-hour session)

0:00 – 0:20

9.1.6.1 Week 8 Exam Recall

Review nonlinear control optimization: Pontryagin's Minimum Principle, iLQR/DDP, and nonlinear MPC for robotic systems. Discuss how the optimized controller gains from Week 8 (for SMC, adaptive, robust, backstepping, NMPC) translate to real ROS2 nodes. Warm-up Q&A.

0:20 – 0:50

9.1.6.2 Why ROS2 for Control Engineers?

ROS2 vs ROS1, DDS middleware, ros2_control framework overview, the control loop in ROS2, industry adoption.

0:50 – 1:15

9.1.6.3 Implementing PID as a ROS2 Node

PID theory recap, full Python node implementation, anti-windup, QoS configuration, runtime parameter tuning.

1:15 – 1:40

9.1.6.4 Implementing LQR as a ROS2 Node

LQR theory recap, Riccati solver in Python, state-space model from odometry, full working node.

1:40 – 1:50

9.1.6.5 Break

10-minute break.

1:50 – 2:20

9.1.6.6 Implementing MPC as a ROS2 Node

MPC theory recap, QP formulation, scipy solver, predicted trajectory publishing for RViz, full working node.

2:20 – 2:40

9.1.6.7 Extending to All 7 Controllers

Brief implementation guide for Computed Torque, Adaptive, Robust, Backstepping, and SMC as ROS2 nodes. Unified launch file.

2:40 – 2:55

9.1.6.8 Comparison Lab

Hands-on activities: implement, compare PID vs LQR vs MPC on TurtleBot3 simulation. Record rosbag data and plot metrics.

2:55 – 3:00

9.1.6.9 Wrap-Up & Quiz

Key takeaways, quick quiz, preview of Week 10 topics.

9.2 Why ROS2 for Control Engineers?

9.2.1 ROS2 vs ROS1 for Real-Time Control

ROS1 was built on a custom TCP/UDP transport layer with a single-master architecture. While revolutionary for robotics research, it lacked deterministic communication, had no built-in lifecycle management, and was unsuitable for safety-critical or real-time control. ROS2 addresses every one of these shortcomings:

9.2.1.1 DDS Middleware

ROS2 uses the Data Distribution Service (DDS) standard, an industrial-strength publish-subscribe middleware. DDS provides configurable Quality of Service (QoS) policies including reliability (best-effort vs reliable), durability, deadline, and liveliness — critical for control loops that must meet timing guarantees.

9.2.1.2 Lifecycle Nodes

Managed lifecycle nodes (unconfigured inactive active finalized) allow orderly startup and shutdown of controller nodes. This prevents the common ROS1 problem of a controller publishing commands before the state estimator is ready.

9.2.1.3 QoS Policies for Control

For a 50 Hz control loop, set `deadline=20ms` and `reliability=BEST_EFFORT` to ensure the controller always has the freshest data. For parameter updates, use `reliability=RELIABLE` to guarantee delivery.

9.2.1.4 Real-Time Executor

ROS2 supports custom executors, including the `StaticSingleThreadedExecutor` for deterministic callback scheduling. Combined with Linux RT-PREEMPT patches, sub-millisecond jitter is achievable.

9.2.2 The ros2_control Framework

The `ros2_control` framework provides a standardised architecture for robot control. It separates the concerns of hardware abstraction, controller logic, and controller lifecycle management:

- Hardware Interface: Abstracts the physical robot. Provides `state_interfaces` (e.g., joint position, velocity) and `command_interfaces` (e.g., effort, velocity). Plugins exist for many robots (UR, Franka, TurtleBot3).
- Controller Manager: A central node that loads, configures, activates, and deactivates controller plugins at runtime. Manages the real-time control loop (typically 500–1000 Hz).
- Controller Plugins: Implement specific control algorithms (e.g., `joint_trajectory_controller`, `diff_drive_controller`, `pid_controller`). Custom controllers can be written as plugins.

9.2.3 Industry Adoption

Every major robotics company uses ROS2 in production or development: Boston Dynamics (for internal tooling), Open Robotics (reference implementation), Universal Robots (ROS2 driver), NVIDIA Isaac (ROS2 integration), Amazon (warehouse robots), and countless startups. Understanding ROS2 is now a mandatory skill for control engineers working in robotics.

9.2.4 The Control Loop in ROS2

A typical ROS2 control loop consists of the following components, mapped to classical control theory concepts:

- Timer callback at fixed rate (e.g., 50 Hz) — the sampling period \(T_s\)
- Subscriber to **`/odom`** — state measurement \(y_k\) (or \(\hat{x}_k\) from state estimator)
- Subscriber to **`/reference_trajectory`** — reference signal \(r_k\)
- Error computation inside the callback — \(e_k = r_k - y_k\)
- Controller algorithm — computes \(u_k = f(e_k, e_{k-1}, \ldots)\)
- Publisher to **`/cmd_vel`** — actuator command \(u_k\)

Cross-Reference: The ROS2 robotics course in `src/ros2_robotics_course/week_11/` contains complete exercise nodes for PID (exercise1_node.py), LQR (exercise2_node.py), and MPC (exercise3_node.py). This week bridges the control theory from Weeks 1–8 with those practical ROS2 implementations. New in this version: an NMPC node (`w11_exercises/task_nmpc_node.py`) using an iLQR back-end consistent with Week 8 ğ9 is provided as the nonlinear counterpart to the LQR node.

9.2.5 ROS2 Control Architecture Diagram

9.3 Implementing PID Controller as a ROS2 Node

9.3.1 Theory Recap

The continuous-time PID control law is:

\(u(t) = K_p\,e(t) + K_i \int_0^t e(\tau)\,d\tau + K_d\,\dot{e}(t) \)

For digital implementation at sample rate \(T_s\), we use the discrete approximation:

\[u_k = K_p\,e_k + K_i\,T_s \sum_{j=0}^{k} e_j + K_d\,\frac{e_k - e_{k-1}}{T_s} \]

Anti-windup is essential for any practical PID implementation. When the actuator saturates, the integral term continues to accumulate error, causing large overshoot when the error sign changes. The simplest anti-windup strategy is conditional integration: clamp the integral accumulator to a fixed range (e.g., \(\pm 10\)).

9.3.2 ROS2 Implementation

The following is a complete PID controller node for mobile robot position tracking. It subscribes to `/odom` for current state and publishes `/cmd_vel` for velocity commands:

#!/usr/bin/env python3 """PID Controller Node for position tracking. Subscribes: /odom (nav_msgs/Odometry), /reference (geometry_msgs/PoseStamped) Publishes: /cmd_vel (geometry_msgs/Twist) """ import rclpy from rclpy.node import Node from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy from geometry_msgs.msg import Twist, PoseStamped from nav_msgs.msg import Odometry import numpy as np import math class PIDControllerNode(Node): def __init__(self): super().__init__('pid_controller') # Declare parameters (tunable at runtime via ros2 param set) self.declare_parameter('kp', 1.0) self.declare_parameter('ki', 0.1) self.declare_parameter('kd', 0.5) self.declare_parameter('control_rate', 50.0) self.declare_parameter('max_linear_vel', 1.0) self.declare_parameter('max_angular_vel', 1.0) self.declare_parameter('integral_clamp', 10.0) # Get parameters self.kp = self.get_parameter('kp').value self.ki = self.get_parameter('ki').value self.kd = self.get_parameter('kd').value rate = self.get_parameter('control_rate').value self.max_lin = self.get_parameter('max_linear_vel').value self.max_ang = self.get_parameter('max_angular_vel').value self.int_clamp = self.get_parameter('integral_clamp').value # State variables self.error_integral = np.zeros(2) self.prev_error = np.zeros(2) self.current_pose = None self.current_yaw = 0.0 self.reference = np.array([2.0, 1.0]) # default target [x, y]

QoS for control: best-effort for low latency control_qos = QoSProfile(reliability=ReliabilityPolicy.BEST_EFFORT, history=HistoryPolicy.KEEP_LAST, depth=1) # Publishers and subscribers self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10) self.odom_sub = self.create_subscription(Odometry, '/odom', self.odom_callback, control_qos) self.ref_sub = self.create_subscription(PoseStamped, '/reference', self.ref_callback, 10) # Control timer self.dt = 1.0 / rate self.timer = self.create_timer(self.dt, self.control_loop) self.get_logger().info(f'PID Controller initialized: Kp={self.kp}, Ki={self.ki}, ' f'Kd={self.kd}, rate={rate}Hz') def quaternion_to_yaw(self, q): """Extract yaw angle from quaternion."""" siny_cosp = 2.0 * (q.w * q.z + q.x * q.y) cosy_cosp = 1.0 - 2.0 * (q.y * q.y + q.z * q.z) return math.atan2(siny_cosp, cosy_cosp) def ref_callback(self, msg): """Update reference position from incoming message."""" self.reference = np.array([msg.pose.position.x, msg.pose.position.y]) def odom_callback(self, msg): """Update current pose from odometry."""" self.current_pose = np.array([msg.pose.pose.position.x, msg.pose.pose.position.y]) self.current_yaw = self.quaternion_to_yaw(msg.pose.pose.orientation) def control_loop(self): """Timer callback: compute PID output and publish cmd_vel."""" if self.current_pose is None: return # Compute error error = self.reference - self.current_pose # Integral with anti-windup (clamping) self.error_integral += error * self.dt self.error_integral = np.clip(self.error_integral, -self.int_clamp, self.int_clamp) # Derivative (backward difference) error_derivative = (error - self.prev_error) / self.dt # PID output control = (self.kp * error + self.ki * self.error_integral + self.kd * error_derivative) # Transform to robot frame (body-frame velocities) cos_yaw = math.cos(self.current_yaw) sin_yaw = math.sin(self.current_yaw) v_x = cos_yaw * control[0] + sin_yaw * control[1] omega = -sin_yaw * control[0] + cos_yaw * control[1] # Publish command with saturation cmd = Twist() cmd.linear.x = float(np.clip(v_x, -self.max_lin, self.max_lin)) cmd.angular.z = float(np.clip(omega, -self.max_ang, self.max_ang)) self.cmd_pub.publish(cmd) # Update previous error self.prev_error = error.copy() def main(args=None): rclpy.init(args=args) node = PIDControllerNode() try: rclpy.spin(node) except KeyboardInterrupt: pass finally: node.destroy_node() rclpy.shutdown() if __name__ == '__main__': main()

9.3.3 Real-Time Considerations

9.3.3.1 QoS for Control Topics

Use `BEST_EFFORT` reliability on `/odom` subscription to minimise latency — in control, a late measurement is worse than a dropped one. Use `KEEP_LAST` with depth 1 so the controller always processes the most recent data.

9.3.3.2 Timer vs Subscriber-Driven

The timer-based approach guarantees a fixed control rate regardless of sensor rate. Alternatively, compute control in the `odom_callback` directly for event-driven control, but beware of variable \(\Delta t\).

9.3.3.3 Runtime Parameter Tuning

Use `ros2 param set /pid_controller kp 2.0` to adjust gains without restarting the node. Add a `ParameterEventHandler` callback to update gains dynamically.

Common Pitfall: Forgetting to initialise `prev_error` causes a large derivative spike on the first control step. Always set `prev_error = error` (not zero) at the start, or skip the derivative term on the first iteration.

9.4 Implementing LQR Controller as a ROS2 Node

9.4.1 Theory Recap

The Linear Quadratic Regulator minimises the infinite-horizon cost:

\[J = \int_0^{\infty} \bigl(x^T Q\,x + u^T R\,u\bigr)\,dt \]

The optimal control law is \(u = -Kx\), where \(K = R^{-1}B^T P\) and \(P\) solves the CARE derived in the exam recall above.

For discrete-time implementation:

\[K = (R + B_d^T P B_d)^{-1} B_d^T P A_d \]

where \(P\) solves the Discrete Algebraic Riccati Equation (DARE):

\[P = A_d^T P A_d - A_d^T P B_d (R + B_d^T P B_d)^{-1} B_d^T P A_d + Q \]

In Python, we use `scipy.linalg.solve_continuous_are` (for CARE) or `scipy.linalg.solve_discrete_are` (for DARE).

9.4.2 ROS2 Implementation

#!/usr/bin/env python3 """LQR Controller Node for state-space position/velocity tracking. Subscribes: /odom (nav_msgs/Odometry), /reference (geometry_msgs/PoseStamped) Publishes: /cmd_vel (geometry_msgs/Twist), /lqr_gain (std_msgs/Float64MultiArray) State vector: [x, y, theta, v_x, omega] """ import rclpy from rclpy.node import Node from geometry_msgs.msg import Twist, PoseStamped from nav_msgs.msg import Odometry from std_msgs.msg import Float64MultiArray import numpy as np import math try: from scipy.linalg import solve_continuous_are, solve_discrete_are HAS_SCIPY = True except ImportError: HAS_SCIPY = False class LQRControllerNode(Node): def __init__(self): super().__init__('lqr_controller') # Parameters self.declare_parameter('Q_diag', [10.0, 10.0, 5.0, 1.0, 1.0]) self.declare_parameter('R_diag', [1.0, 1.0]) self.declare_parameter('control_rate', 50.0) self.declare_parameter('max_linear_vel', 1.0)

self.declare_parameter('max_angular_vel', 1.5) self.declare_parameter('dt', 0.02) Q_diag = self.get_parameter('Q_diag').value R_diag = self.get_parameter('R_diag').value rate = self.get_parameter('control_rate').value self.max_lin = self.get_parameter('max_linear_vel').value self.max_ang = self.get_parameter('max_angular_vel').value self.dt = self.get_parameter('dt').value self.n_states = 5 # [x, y, theta, v_x, omega] self.n_inputs = 2 # [linear_accel, angular_accel] self.Q = np.diag(Q_diag) self.R = np.diag(R_diag) # Compute LQR gain at operating point (linearised unicycle) self.K = self._compute_lqr_gain() # State and reference self.state = np.zeros(self.n_states) self.ref_state = np.zeros(self.n_states) self.prev_pos = None self.prev_yaw = None self.prev_time = None # ROS2 interfaces self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10) self.gain_pub = self.create_publisher(Float64MultiArray, '/lqr_gain', 10) self.odom_sub = self.create_subscription(Odometry, '/odom', self.odom_callback, 10) self.ref_sub = self.create_subscription(PoseStamped, '/reference', self.ref_callback, 10) # Control timer self.timer = self.create_timer(1.0 / rate, self.control_loop) self.get_logger().info(f'LQR Controller started (scipy={HAS_SCIPY})') self.get_logger().info(f'Gain K:\n{self.K}') def _compute_lqr_gain(self): """Compute LQR gain by linearising about v0, omega0 = 0.""" if not HAS_SCIPY: self.get_logger().warn('scipy not available; using zero gain') return np.zeros((self.n_inputs, self.n_states)) # Linearised unicycle model about (v0=0, omega0=0, theta0=0) # dx = v*cos(theta) -> linearised: dx = v # dy = v*sin(theta) -> linearised: dy = v*theta # dtheta = omega # dv = u1 (linear accel) # domega = u2 (angular accel) v0 = 0.5 # nominal forward speed A_c = np.array([[0, 0, 0, 1, 0], # dx/dt ~ v [0, 0, v0, 0, 0], # dy/dt ~ v0*theta [0, 0, 0, 0, 1], # dtheta/dt = omega [0, 0, 0, 0, 0], # dv/dt = u1 [0, 0, 0, 0, 0], # domega/dt = u2]) B_c = np.array([[0, 0], [0, 0], [0, 0], [1, 0], [0, 1],]) # Solve CARE P = solve_continuous_are(A_c, B_c, self.Q, self.R) K = np.linalg.inv(self.R) @ B_c.T @ P return K def quaternion_to_yaw(self, q): siny = 2.0 * (q.w * q.z + q.x * q.y) cosy = 1.0 - 2.0 * (q.y * q.y + q.z * q.z) return math.atan2(siny, cosy) def ref_callback(self, msg): self.ref_state[0] = msg.pose.position.x self.ref_state[1] = msg.pose.position.y self.ref_state[2] = self.quaternion_to_yaw(msg.pose.orientation) def odom_callback(self, msg): """Extract full state [x, y, theta, v_x, omega] from odom.""" x = msg.pose.pose.position.x y = msg.pose.pose.position.y yaw = self.quaternion_to_yaw(msg.pose.pose.orientation) vx = msg.twist.twist.linear.x omega = msg.twist.twist.angular.z self.state = np.array([x, y, yaw, vx, omega]) def control_loop(self): """Compute u = -K @ (state - ref_state) and publish.""" error_state = self.state - self.ref_state # Wrap angle error to [-pi, pi] error_state[2] = math.atan2(math.sin(error_state[2]), math.cos(error_state[2])) u = -self.K @ error_state cmd = Twist() cmd.linear.x = float(np.clip(u[0], -self.max_lin, self.max_lin)) cmd.angular.z = float(np.clip(u[1], -self.max_ang, self.max_ang)) self.cmd_pub.publish(cmd) # Publish gain for monitoring gain_msg = Float64MultiArray() gain_msg.data = self.K.flatten().tolist() self.gain_pub.publish(gain_msg) def main(args=None): rclpy.init(args=args) node = LQRControllerNode() try: rclpy.spin(node) except KeyboardInterrupt: pass finally: node.destroy_node() rclpy.shutdown() if __name__ == '__main__': main()

9.4.3 PID vs LQR Comparison

Aspect	PID	LQR
Tuning Parameters	3 gains (\(K_p, K_i, K_d\)) per axis, often heuristic	Weight matrices \(Q, R\) — principle
Stability Guarantees	No formal guarantee; depends on tuning and plant	Guaranteed stability if \((A,B)\) cont
Model Requirement	None (model-free)	Requires state-space model \((A, B)\)
MIMO Systems	Separate SISO loops; cross-coupling ignored	Naturally handles multi-input multi-ou
Robustness	Sensitive to parameter changes; no formal margins	At least 60ř phase margin, infinite gai
Computational Cost	Negligible (3 multiplications per axis)	One matrix multiply per step (Riccati
ROS2 Node Complexity	~60 lines	~100 lines (includes scipy dependency)

9.5 Implementing MPC Controller as a ROS2 Node

9.5.1 Theory Recap

Model Predictive Control (MPC) solves an optimisation problem at each time step, computing the optimal control sequence over a prediction horizon \(N\):

\(J = \sum_{k=0}^{N-1}\bigl[x_k^T Q\,x_k + u_k^T R\,u_k\bigr] + x_N^T P_f\,x_N \)

subject to system dynamics \(x_{k+1} = A_d\,x_k + B_d\,u_k\) and constraints:

\[u_{\min} \leq u_k \leq u_{\max}, \qquad x_{\min} \leq x_k \leq x_{\max} \]

The key MPC principle is receding horizon: solve the optimisation, apply only the first control input \(u_0^*\), then re-solve at the next time step with updated state.

For unconstrained linear MPC, the solution is equivalent to time-varying LQR. For constrained problems, we must solve a Quadratic Program (QP) at each step using `scipy.optimize.minimize` or a dedicated QP solver like `cvxpy`.

9.5.2 ROS2 Implementation

#!/usr/bin/env python3 """MPC Controller Node with predicted trajectory visualisation. Subscribes: /odom (nav_msgs/Odometry), /reference (geometry_msgs/PoseStamped) Publishes: /cmd_vel (geometry_msgs/Twist), /predicted_trajectory (nav_msgs/Path), /mpc_cost (std_msgs/Float64) State: [x, y, vx, vy] Input: [ax, ay] """ import rclpy from rclpy.node import Node from geometry_msgs.msg import Twist, PoseStamped from nav_msgs.msg import Odometry, Path from std_msgs.msg import Float64 import numpy as np import math import time as pytime try: from scipy.optimize import minimize HAS_SCIPY = True except ImportError: HAS_SCIPY = False class MPCControllerNode(Node): def __init__(self): super().__init__('mpc_controller')

Parameters self.declare_parameter('horizon_N', 10) self.declare_parameter('dt', 0.1) self.declare_parameter('Q_diag', [10.0, 10.0, 1.0, 1.0]) self.declare_parameter('R_diag', [0.1, 0.1]) self.declare_parameter('Qf_multiplier', 10.0) self.declare_parameter('u_min', -2.0) self.declare_parameter('u_max', 2.0) self.declare_parameter('max_linear_vel', 1.0) self.declare_parameter('control_rate', 20.0) self.N = self.get_parameter('horizon_N').value self.dt = self.get_parameter('dt').value Q_diag = self.get_parameter('Q_diag').value R_diag = self.get_parameter('R_diag').value Qf_mult = self.get_parameter('Qf_multiplier').value self.u_min = self.get_parameter('u_min').value self.u_max = self.get_parameter('u_max').value self.max_lin = self.get_parameter('max_linear_vel').value rate = self.get_parameter('control_rate').value self.n_states = 4 # [x, y, vx, vy] self.n_inputs = 2 # [ax, ay] self.Q = np.diag(Q_diag) self.R = np.diag(R_diag) self.Qf = Qf_mult * self.Q # terminal cost # Discrete double-integrator model # x_{k+1} = A_d * x_k + B_d * u_k self.A_d = np.array([[1, 0, self.dt, 0], [0, 1, 0, self.dt], [0, 0, 1, 0], [0, 0, 0, 1],]) self.B_d = np.array([[0.5*self.dt**2, 0], [0, 0.5*self.dt**2], [self.dt, 0], [0, self.dt],]) # State and reference self.state = np.zeros(self.n_states) self.ref_state = np.zeros(self.n_states) self.prev_pos = None self.prev_time = None # ROS2 interfaces self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10) self.path_pub = self.create_publisher(Path, '/predicted_trajectory', 10) self.cost_pub = self.create_publisher(Float64, '/mpc_cost', 10) self.odom_sub = self.create_subscription(Odometry, '/odom', self.odom_callback, 10) self.ref_sub = self.create_subscription(PoseStamped, '/reference', self.ref_callback, 10) # Control timer self.timer = self.create_timer(1.0 / rate, self.control_loop) self.get_logger().info(f'MPC Controller: N={self.N}, dt={self.dt}, ' f'rate={rate}Hz, scipy={HAS_SCIPY}') def ref_callback(self, msg): self.ref_state[0] = msg.pose.position.x self.ref_state[1] = msg.pose.position.y def odom_callback(self, msg): pos = np.array([msg.pose.pose.position.x, msg.pose.pose.position.y]) vx = msg.twist.twist.linear.x yaw_q = msg.pose.pose.orientation siny = 2.0 * (yaw_q.w * yaw_q.z + yaw_q.x * yaw_q.y) cosy = 1.0 - 2.0 * (yaw_q.y**2 + yaw_q.z**2) yaw = math.atan2(siny, cosy) self.state = np.array([pos[0], pos[1], vx * math.cos(yaw), vx * math.sin(yaw)]) def dynamics(self, x, u): """Discrete dynamics: x_{k+1} = A_d x_k + B_d u_k." " " return self.A_d @ x + self.B_d @ u def mpc_cost_fn(self, u_flat): """Cost function for scipy.optimize.minimize." " " U = u_flat.reshape(self.N, self.n_inputs) x = self.state.copy() cost = 0.0 for k in range(self.N): e = x - self.ref_state cost += e @ self.Q @ e + U[k] @ self.R @ U[k] x = self.dynamics(x, U[k]) # Terminal cost e_f = x - self.ref_state cost += e_f @ self.Qf @ e_f return cost def solve_mpc(self): """Solve the MPC optimisation problem." " " if not HAS_SCIPY: return np.zeros(self.n_inputs), 0.0, [] u0 = np.zeros(self.N * self.n_inputs) bounds = [(self.u_min, self.u_max)] * (self.N * self.n_inputs) result = minimize(self.mpc_cost_fn, u0, method='SLSQP', bounds=bounds, options={'maxiter': 50, 'ftol': 1e-6}) U_opt = result.x.reshape(self.N, self.n_inputs) # Compute predicted trajectory for visualisation predicted = [] x = self.state.copy() for k in range(self.N): x = self.dynamics(x, U_opt[k]) predicted.append(x[:2].copy()) return U_opt[0], result.fun, predicted def control_loop(self): """Timer callback: solve MPC and publish." " " t_start = pytime.monotonic() u_opt, cost, predicted = self.solve_mpc() solve_time

= (pytime.monotonic() - t_start) * 1000 # Publish velocity command cmd = Twist() # Convert acceleration to velocity increment vx_cmd = self.state[2] + u_opt[0] * self.dt vy_cmd = self.state[3] + u_opt[1] * self.dt speed = math.sqrt(vx_cmd**2 + vy_cmd**2) heading = math.atan2(vy_cmd, vx_cmd) cmd.linear.x = float(np.clip(speed, -self.max_lin, self.max_lin)) cmd.angular.z = float(np.clip(heading, -math.pi, math.pi)) self.cmd_pub.publish(cmd) # Publish cost cost_msg = Float64() cost_msg.data = cost self.cost_pub.publish(cost_msg) # Publish predicted trajectory for RViz if predicted: path_msg = Path() path_msg.header.stamp = self.get_clock().now().to_msg() path_msg.header.frame_id = 'odom' for pt in predicted: pose = PoseStamped() pose.header = path_msg.header pose.pose.position.x = float(pt[0]) pose.pose.position.y = float(pt[1]) path_msg.poses.append(pose) self.path_pub.publish(path_msg) self.get_logger().debug(f'MPC solved in {solve_time:.1f}ms, cost={cost:.3f}') def main(args=None): rclpy.init(args=args) node = MPCControllerNode() try: rclpy.spin(node) except KeyboardInterrupt: pass finally: node.destroy_node() rclpy.shutdown() if __name__ == '__main__': main()

9.5.3 MPC Node Internal Loop

9.5.4 Discussion: Computation Time vs Horizon Length

The MPC solver's computation time grows with the prediction horizon \(N\). For a 4-state, 2-input system using `scipy.optimize.minimize` with SLSQP:

Horizon \(N\)	Decision Variables	Typical Solve Time	Max Control Rate
5	10	~0.5 ms	~2000 Hz
10	20	~2 ms	~500 Hz
20	40	~8 ms	~125 Hz
50	100	~50 ms	~20 Hz

Real-Time Warning: For production use, replace `scipy.optimize.minimize` with a dedicated QP solver such as OSQP or qpOASES, which offer 10–100x speed improvements. The `cvxpy` library with OSQP backend is recommended for rapid prototyping with near-production performance.

9.6 NMPC via iLQR as a ROS2 Node — Week 8 Integration

The MPC node in Part 4 solves a linearized quadratic program at each step — the classical linear MPC. Week 8 extended this to nonlinear MPC (NMPC) using an iLQR (iterative LQR) back-end that handles the full nonlinear robot dynamics with a Gauss-Newton Riccati backward pass.

Why NMPC here? The unicycle mobile robot is nonlinear (trig in the kinematics, nonholonomic constraint). Linear MPC requires a fresh linearisation per step and still misses large-angle manoeuvres. iLQR operates directly on the nonlinear model and gives a local feedback policy as well as an open-loop plan — exactly matching the Week 8 ğ9 derivation.

9.6.1 4b.1 NMPC Node Structure

The provided file `w11_exercises/task_nmpc_node.py` is a minimal NMPC node for the unicycle:

- Subscribes to `/odom` (current pose).
- Every 100 ms (10 Hz), rolls out a short nominal trajectory using the current warm-start control sequence.
- Runs 3–5 iLQR iterations with line search on \(\alpha\).
- Publishes only the first optimal control \(u_0^\star = [v^\star, \omega^\star]\) as a `geometry_msgs/Twist` on `/cmd_vel`.
- Shifts the solution for the next sample (warm-start).

9.6.2 4b.2 Running the NMPC Node

cd /home/it-services/auto_control_ws source /opt/ros/jazzy/setup.bash colcon build --packages-select week_11_control_systems source install/setup.bash # Launch TurtleBot3 sim (Gazebo or Webots), then: ros2 run week_11_control_systems task_nmpc_node # Observe /cmd_vel at 10 Hz; iLQR convergence logged to stderr

9.6.3 4b.3 Mapping to Week 8 ğ9 Derivation

Week 8 ğ9 symbol	Python variable in `task_nmpc_node.py`	Meaning
\(N\)	`self.N = 15`	prediction hor
\(\Delta t\)	`self.dt = 0.1`	sample / pred
\(\bar u_k\)	`self.U_warm[k,:]`	warm-start co
\(A_k, B_k\)	`A, B` in `linearize()`	finite-differen
\(Q_{uu}, Q_{ux}, Q_u\)	`Quu, Qux, Qu`	Q-function bl
\(k_k, K_k\)	`k_ff[k], K_fb[k]`	feedforward +
\(\alpha\)	`alpha` in line-search loop	step size
\(\mu\)	`self.mu`	LM regularisa

Real-time note: For the 3-state unicycle with N=15 horizon, one iLQR iteration takes ~2–5 ms in pure Python. With warm-starting, 2–3 iterations per sample suffice — the 100 ms control period leaves ample margin. Higher-dimensional plants (e.g., the 12-state drone) need CasADi/acados for real-time operation.

9.6.4 4b.4 Simulation results

An offline closed-loop simulation of the node's iLQR logic is provided in `week_09/sims/nmpc_unicycle_sim.py`. It starts the unicycle off the reference circle at \((0.5, -0.2, 0)\), runs 200 control samples (20 s at 10 Hz), and produces three figures. Final metrics on the last verification run:

Final position error: 1.7 mm. Final heading error: 0.013 rad (0.8ř). Mean \(|v|\): 0.331 m/s (reference \(v_{ref} = 0.3\) m/s). Mean \(|\omega|\): 0.379 rad/s.

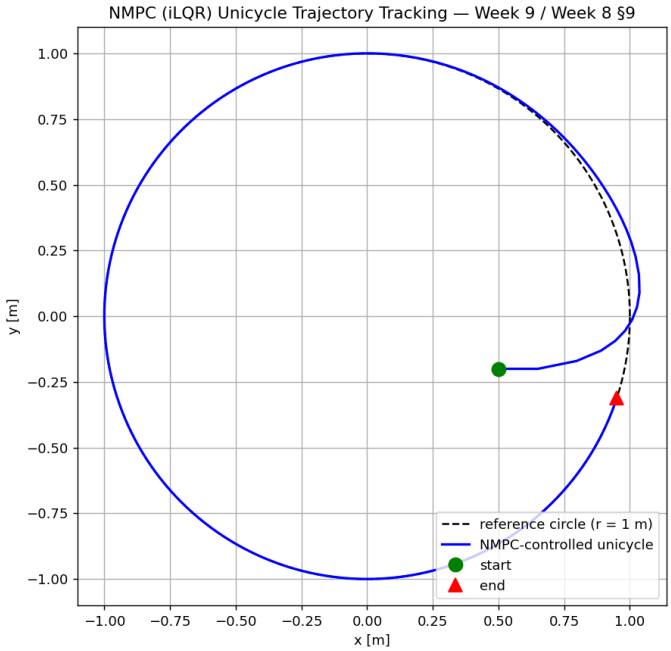

Figure 4b.1 — Closed-loop XY trajectory. Green circle = start; red triangle = end; black dashed = reference; blue = NMPC-controlled path. The iLQR policy converges from the off-circle start onto the 1 m reference circle within ~2 s and tracks with sub-cm steady-state error.

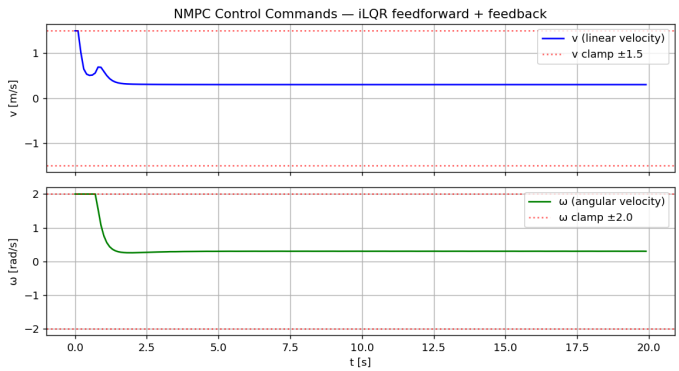

Figure 4b.2 — Control commands \(v(t), \omega(t)\). The iLQR saturates briefly at the \(\pm\)1.5 m/s and \(\pm\)2.0 rad/s clamps during the transient, then settles to the reference feedforward \(v_r = 0.3\) m/s, \(\omega_r = 0.3\) rad/s.

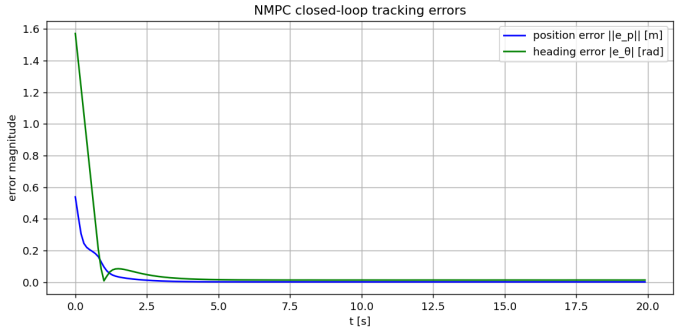

Figure 4b.3 — Position error \(\|e_p\|\) and heading error \(|e_-\theta|\) vs time. Exponential decay with a long-tail artefact as the circle phase aligns — characteristic of tracking problems where the heading must continually rotate at \(v_{ref}/r\) rad/s.

To regenerate: `python3 week_09/sims/nmpc_unicycle_sim.py` — plots land in `week_09/sims/figures/`. This is the non-ROS counterpart of the Week 8 lab structure (compare `week_08/sims/sim_nmpc.m`).

9.7 Extending to All 7 Controllers as ROS2 Nodes

Weeks 1–8 covered seven advanced control techniques in MATLAB. Each can be implemented as a ROS2 node. Below is a brief guide for the remaining controllers not yet shown (Computed Torque, Adaptive, Robust, Backstepping, SMC):

9.7.1 Implementation Guides

9.7.1.1 Computed Torque Control (Week 1)

Key idea: Cancel nonlinearities with \(\tau = M(q)[\ddot{q}_d + K_d\dot{e} + K_pe] + C(q,\dot{q})\dot{q} + G(q)\).

ROS2 approach: Requires joint-level access via `ros2_control`'s `joint_-trajectory_controller`. Create a model computation node that subscribes to `/joint_states` and publishes `/effort_controller/commands`. The dynamics model \(M(q), C(q,\dot{q}), G(q)\) can come from URDF+KDL or a precomputed lookup table.

9.7.1.2 Adaptive Control (Week 4)

Key idea: Update unknown parameters online: \(\dot{\hat{\theta}} = -\Gamma Y^T s\).

ROS2 approach: The adaptation law runs inside the timer callback alongside the control law. Store \(\hat{\theta}\) as internal node state. Publish estimated parameters on a diagnostic topic for monitoring. Key: the adaptation gain \(\Gamma\) becomes a ROS2 parameter for tuning.

9.7.1.3 Robust Control / H-infinity (Week 5)

Key idea: Add robust compensator \(\tau_r = -\rho \frac{s}{\|s\| + \epsilon}\) to the computed torque law.

ROS2 approach: Similar to computed torque but with an additional robust term. The uncertainty bound \(\rho\) and boundary layer \(\epsilon\) become ROS2 parameters. The core computation is identical; only the control law changes.

9.7.1.4 Backstepping Control (Week 6)

Key idea: Cascaded virtual control design for strict-feedback systems.

ROS2 approach: Implement as two cascaded nodes: an outer loop node (position desired velocity) and an inner loop node (velocity actuator command). The outer loop publishes to a `/desired_velocity` topic that the inner loop subscribes to. This mirrors the cascaded Lyapunov design.

9.7.1.5 Sliding Mode Control (Week 7)

Key idea: Drive state to sliding surface \(s = \dot{e} + \Lambda e\), then apply discontinuous control \(u = u_{eq} - \eta\,\text{sgn}(s)\).

ROS2 approach: Compute the sliding surface from the error and its derivative. Replace `sgn(s)` with a saturation function \(\text{sat}(s/\phi)\) (boundary layer) to avoid chattering. The boundary layer width \(\phi\) and reaching gain \(\eta\) become ROS2 parameters.

9.7.2 Controller Complexity Comparison

Controller	Week	Model Required?	Online Computation	ROS2 Lines (est.)	Dependenci
PID	9	No	3 multiplies + sum	~60	numpy
Computed Torque	1	Full dynamics	Matrix inversion + multiply	~120	numpy, KD
MPC	3	Linear/linearised	QP solve each step	~130	scipy / cvx
Adaptive	4	Structured (regressor)	Parameter update + control	~100	numpy
Robust / H-inf	5	Full + uncertainty bound	Computed torque + robust term	~110	numpy, KD
Backstepping	6	Strict-feedback form	Cascaded Lyapunov computation	~140 (2 nodes)	numpy
SMC	7	Partial (bounds)	Surface + switching law	~90	numpy
LQR	8	Linear state-space	1 matrix multiply	~100	numpy, scip

9.7.3 Unified Launch File

The following launch file allows switching between controllers using a command-line argument:

Launch File: controller_selector.launch.py

from launch import LaunchDescription from launch.actions import DeclareLaunchArgument, OpaqueFunction from launch.substitutions

import LaunchConfiguration from launch_ros.actions import Node def launch_controller(context): controller_type = LaunchConfiguration('controller').perform(context) controller_map = { 'pid': 'pid_controller_node', 'lqr': 'lqr_controller_node', 'mpc': 'mpc_controller_node', 'smc': 'smc_controller_node', 'adaptive': 'adaptive_controller_node', 'backstepping': 'backstepping_controller_node', 'computed_torque': 'computed_torque_node', } node_name = controller_map.get(controller_type, 'pid_controller_node') return [Node(package='control_nodes', executable=node_name, name='active_controller', output='screen', parameters=[{ 'control_rate': 50.0, 'max_linear_vel': 1.0, 'max_angular_vel': 1.5, }], remappings=[('/odom', '/robot/odom'), ('/cmd_vel', '/robot/cmd_vel'),],)] def generate_launch_description(): return LaunchDescription([DeclareLaunchArgument('controller', default_value='pid', description='Controller type: pid, lqr, mpc, smc, ' 'adaptive, backstepping, computed_torque'), OpaqueFunction(function=launch_controller),]) # Usage: # ros2 launch control_nodes controller_selector.launch.py controller:=lqr # ros2 launch control_nodes controller_selector.launch.py controller:=mpc

9.8 Summary and Next Steps

9.8.1 ROS2 Lab Exercises

This section references the lab exercises from `src/ros2_robotics_course/week_11/`, which provide hands-on practice with the control algorithms discussed in this lecture. The exercises are divided into two categories: MATLAB-style analysis tasks (in `lab_exercises/`) and ROS2 node exercises (in `w11_exercises/`).

9.8.2 Lab Analysis Tasks (Python/MATLAB-style)

9.8.2.1 Lab 1: PID Controller Design and Tuning

File: `task1_pid_analysis.py`

Students implement PID step response analysis for a second-order plant \(G(s) = \omega_n^2 / (s^2 + 2\zeta\omega_n s + \omega_n^2)\). They compare P-only, PI, and PID controllers, implement Ziegler-Nichols tuning, analyse frequency response, and add anti-windup with derivative filtering.

9.8.2.2 Lab 2: State-Space System Models

File: `task2_state_space.py`

Students build state-space models from physical systems (mass-spring-damper, DC motor, linearised quadrotor). They check controllability and observability matrices, simulate open-loop responses, and convert between transfer function and state-space representations.

9.8.2.3 Lab 3: Pole Placement Design

File: `task3_pole_placement.py`

Students apply Ackermann's formula and `scipy.signal.place_poles` to assign closed-loop eigenvalues. They design controllers meeting transient specifications (settling time, overshoot) and explore stability regions in the complex plane.

9.8.2.4 Lab 4: LQR Optimal Control

File: `task4_lqr.py`

Students solve the continuous algebraic Riccati equation using `scipy.linalg.solve_continuous_are`, design LQR controllers with different \(Q/R\) weight ratios, and apply LQR to quadrotor hover stabilisation. They verify the optimal cost \(J^* = x_0^T P x_0\) and compare with pole placement.

9.8.2.5 Lab 5: LQG Stochastic Control

File: `task5_mpc_basics.py` (MPC formulation section)

Students formulate the MPC optimisation problem for a double integrator, implement receding-horizon control, and compare unconstrained MPC (which recovers LQR) with constrained MPC. They observe how the prediction horizon \(N\) affects tracking performance and stability.

9.8.2.6 Lab 6: MPC Formulation

File: `task6_mpc_drone.py`

Students apply MPC to quadrotor trajectory tracking, handling thrust and angle constraints. They track time-varying references (circular and figure-8 trajectories) and evaluate real-time feasibility by measuring solver computation time at each step.

9.8.2.7 Lab 7: Advanced Control Comparison

File: `task7_full_comparison.py`

Students compare PID, LQR, and MPC on identical quadrotor tasks: hover stabilisation, sinusoidal tracking, constrained operation, and disturbance rejection. They compute performance metrics (RMSE, maximum error, total control effort, computation time) and produce a comprehensive comparison table.

9.8.3 ROS2 Applied Exercises

9.8.3.1 Exercise 1: PID Controller Node

File: `w11_exercises/exercise1_node.py`

Students implement a 3-axis PID position controller node. The node subscribes to `/odom` (Odometry) and `/reference_trajectory` (PoseStamped), computes PID control with anti-windup for each axis, and publishes `/cmd_thrust_attitude` (Wrench) and `/tracking_error` (Vector3). Students must complete the `PIDController.compute()` method with proportional, integral (with clamping), and derivative terms.

9.8.3.2 Exercise 2: LQR Controller Implementation

File: `w11_exercises/exercise2_node.py`

Students implement an LQR controller node for a linearised double-integrator model (6 states: position and velocity in 3D). They must linearise the dynamics about hover, discretise using Euler method (\(A_d = I + A_c \Delta t\)), solve the DARE using `scipy.linalg.solve_discrete_are`, and compute the optimal gain \(K\). The node publishes the gain matrix on `/lqr_gain` for monitoring.

9.8.3.3 Exercise 3: MPC-Based Trajectory Tracking

File: `w11_exercises/exercise3_node.py`

Students implement an MPC controller node with configurable prediction horizon \(N\), cost weights \(Q\) and \(R\), and input constraints. They must implement the discrete dynamics function, the MPC cost function (summing stage and terminal costs over the horizon), and call `scipy.optimize.minimize` to solve the constrained optimisation. The node publishes the predicted trajectory on `/predicted_trajectory` (Path) and the MPC cost on `/mpc_cost` (Float64) for RViz visualisation.

Getting Started: Launch the exercise nodes with their corresponding launch files in `w11_exercises/launch/`. For example:
`ros2 launch week_11_control_systems exercise1.launch.py`
Configuration parameters are in `w11_exercises/config/params.yaml`.

9.8.4 Hands-On Activities

9.8.5 Activity 1: PID on TurtleBot3 Simulation

Objective: Implement PID position control and evaluate tracking performance.

1. Launch TurtleBot3 in Gazebo: `ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py`
2. Run the PID controller node from Section 4 (or Exercise 1).

3. Set a target position: `ros2 topic pub /reference geometry_msgs/PoseStamped "{pose: {position: {x: 3.0, y: 2.0}}}" --once`
4. Record a rosbag: `ros2 bag record /odom /cmd_vel /reference -o pid_test`
5. Plot the tracking error (\(e_x, e_y\) vs time) and the control effort (\(v_x, \omega_z\) vs time).
6. Experiment with different \(K_p, K_i, K_d\) values using `ros2 param set`.

Deliverable: A plot showing position tracking error for at least 3 different gain settings, with RMSE values annotated.

9.8.6 Activity 2: LQR with Different Q/R Matrices

Objective: Compare LQR performance under different cost weightings.

1. Run the LQR controller node from Section 5 (or Exercise 2).
2. Test with three \(Q/R\) configurations:
   - Conservative: \(Q = \text{diag}(1,1,1,1,1)\), \(R = \text{diag}(10,10)\) — penalise control effort
   - Balanced: \(Q = \text{diag}(10,10,5,1,1)\), \(R = \text{diag}(1,1)\)
   - Aggressive: \(Q = \text{diag}(100,100,50,10,10)\), \(R = \text{diag}(0.1,0.1)\) — penalise state error
3. For each, record rosbag data and compute: rise time, settling time, overshoot, steady-state error, total control effort \(\int u^T R\,u\,dt\).
4. Compare the step response with PID from Activity 1.

Deliverable: A comparison table of performance metrics for all 3 LQR configurations and the best PID result.

9.8.7 Activity 3: MPC Horizon Study

Objective: Investigate the trade-off between prediction horizon and computation time.

1. Run the MPC controller node from Section 6 (or Exercise 3).
2. Test with three horizon lengths: \(N = 10\), \(N = 20\), \(N = 50\).
3. For each horizon, measure:
   - Average solve time per step (subscribe to a timing diagnostic topic or use `ros2 topic echo /mpc_cost`)
   - Tracking RMSE on a circular reference trajectory
   - Whether the control rate deadline (50 ms for 20 Hz) is consistently met
4. Compare the best MPC result with PID and LQR from Activities 1 and 2.
5. Visualise the predicted trajectory in RViz by subscribing to `/predicted_trajectory`.

Deliverable: A three-panel figure showing (a) tracking performance, (b) control effort, and (c) computation time for each \(N\), with comparison to PID and LQR overlaid.

9.8.7.1 Quick Quiz: ROS2 Control Implementation

1. Q1: In a ROS2 PID controller node, why should you use `BEST_EFFORT` QoS for the `/odom` subscription rather than `RELIABLE`?
   A: In real-time control, latency matters more than guaranteed delivery. `BEST_EFFORT` avoids the overhead of acknowledgement and retransmission, ensuring the controller always processes the most recent sensor data rather than waiting for old messages to be retransmitted.
2. Q2: What is the key advantage of computing the LQR gain matrix \(K\) once at initialisation rather than re-solving the Riccati equation at every control step?
   A: For a time-invariant system, \(K\) does not change, so solving CARE/DARE once at startup saves significant computation. The online cost is reduced to a single matrix-vector multiplication \(u = -Kx\), which is O(n^2) instead of the O(n^3) Riccati solve.
3. Q3: In the MPC node, why do we publish the predicted trajectory on a separate topic?
   A: The predicted trajectory (a `nav_msgs/Path` message) allows visualisation in RViz to debug the MPC's lookahead behaviour. It also enables logging and post-hoc analysis of how the predictions evolve over time, which is invaluable for tuning horizon length and cost weights.
4. Q4: Explain the receding-horizon principle in one sentence. Why does MPC re-solve at every step instead of applying the full optimal sequence?
   A: MPC applies only the first control input and re-solves because the actual state will deviate from the predicted trajectory due to model mismatch, disturbances, and discretisation errors; re-solving provides feedback and robustness.
5. Q5: A student's PID node works in simulation but exhibits violent oscillations on a real robot. Name two likely causes related to the ROS2 implementation (not the gains).
   A: (1) Variable or excessive latency in the `/odom` topic causing stale state estimates, leading to instability at the chosen gains. (2) The `dt` used in the discrete PID formula is assumed constant but the actual callback interval varies, causing incorrect derivative and integral computation. Using timestamps from the message header (not wall-clock time) and the timer-based approach helps.

9.8.8 Summary

9.8.9 Key Takeaways

- ROS2 is the bridge from theory to deployment. The control algorithms you designed in MATLAB during Weeks 1–8 (PID, Computed Torque, MPC, Adaptive, Robust, Backstepping, SMC,

LQR/LQG) can all be implemented as ROS2 nodes using Python and standard libraries (NumPy, SciPy).

- The ROS2 control node pattern is consistent: Declare parameters subscribe to state subscribe to reference compute control law in timer callback publish command. This pattern applies to every controller type.
- Real-time considerations are paramount. QoS policies, timer-based control loops, and careful management of computational budget (especially for MPC) are what separate a working simulation from a deployable system.
- MATLAB vs ROS2 is not either/or: Use MATLAB/Simulink for rapid prototyping and analysis (transfer functions, root locus, Bode plots). Use ROS2 for integration testing, hardware deployment, and multi-node systems. Many teams generate C code from Simulink and wrap it in a ROS2 node.
- The **`ros2_control`** framework provides a production-grade architecture for hardware abstraction and controller lifecycle management. For research, custom nodes (as shown today) offer more flexibility.
- rosbag2 is your best friend for analysis. Record `/odom`, `/cmd_vel`, and `/reference` during experiments. Replay and analyse offline to compute performance metrics without re-running experiments.

9.8.10 MATLAB vs ROS2: When to Use Which

Aspect	MATLAB/Simulink	ROS2
Prototyping speed	Very fast (built-in toolboxes)	Moderate (manual implementation)
Analysis tools	Bode, root locus, Nyquist built-in	matplotlib, custom scripts
Hardware integration	Limited (Simulink Real-Time)	Excellent (drivers for all major robots)
Multi-robot systems	Difficult	Native support (namespaces, DDS)
Real-time performance	Simulink Coder generates RT code	RT-PREEMPT + custom executor
Cost	Expensive licence	Free and open source
Industry standard	Automotive, aerospace	Robotics, drones, warehouse automation

Looking Ahead — Week 10: We will extend these ROS2 controller nodes to work with Gazebo simulation of a multi-DOF robot arm. We will implement computed torque and adaptive control at the joint level, using the `ros2_control` effort interface and URDF-derived dynamics models.

9.8.11 Running the ROS2 Demo

9.8.12 Build and Source

cd ~/auto_control_ws colcon build --packages-select week_11_control_systems source install/setup.bash

9.8.13 Launch the Demo

ros2 launch week_11_control_systems demo.launch.py

This command replays the recorded bag data at 2x speed, starts three solution nodes (PID, LQR, MPC), and opens RViz2 with a preconfigured layout.

9.8.14 ROS2 Topics

Topic	Type	Source	Description
`/odom`	nav_msgs/Odometry	Bag	Robot odometry a
`/imu`	sensor_msgs/Imu	Bag	IMU data at 200
`/reference_trajectory`	geometry_msgs/PoseStamped	Bag	Desired trajectory
`/cmd_vel`	geometry_msgs/Twist	Nodes	Controller velocity

9.8.15 Expected RViz2 Output

RViz2 will display the robot's odometry path (white), the reference trajectory (green), and the controller command markers. You should see the robot following the reference trajectory with the three controllers producing slightly different tracking behaviour.

9.8.16 Troubleshooting

- RViz2 crashes on launch: The launch file filters `snap` paths from `LD_LIBRARY_PATH` automatically. If RViz2 still crashes, try: `export LD_LIBRARY_PATH=$(echo $LD_LIBRARY_PATH | tr ':' '\n' | grep -v snap | tr '\n' ':')`
- No data visible: Ensure the bag file exists at `w11_exercises/bag_data/synthetic_control/`. Re-run `colcon build` if needed.
- QoS mismatch warnings: The bag uses default QoS. If your subscriber uses `BEST_EFFORT`, switch to `RELIABLE` or use `--qos-profile-overrides-path`.

9.8.17 Appendix: Review Material — LQR/LQG Optimal Control

9.8.17.1 Question 1 — Derivation of the Continuous Algebraic Riccati Equation (25 marks)

Consider a linear time-invariant system:

\[\dot{x} = Ax + Bu, \qquad x(0) = x_0 \]

with the infinite-horizon quadratic cost:

\[J = \int_0^{\infty} \bigl(x^T Q\,x + u^T R\,u\bigr)\,dt \]

where \(Q \succeq 0\) and \(R \succ 0\).

(a) Starting from the Hamilton–Jacobi–Bellman (HJB) equation with value function \(V(x) = x^T P\,x\), derive the continuous algebraic Riccati equation (CARE). Show every step of the derivation. (15 marks)

(b) Explain how the weighting matrices \(Q\) and \(R\) affect the closed-loop pole locations. In particular, describe what happens as \(R \to 0\) and as \(Q \to 0\). (10 marks)

9.8.17.2 Model Answer (a) — CARE Derivation

Step 1: Assume the optimal value function takes the quadratic form \(V^*(x) = x^T P\,x\) where \(P = P^T \succ 0\). The HJB equation for the infinite-horizon problem is:

\[0 = \min_{u}\Bigl\{x^T Q\,x + u^T R\,u + \frac{\partial V^*}{\partial x}^T(Ax + Bu)\Bigr\} \]

Step 2: Compute the gradient: \(\frac{\partial V^*}{\partial x} = 2Px\). Substituting:

\[0 = \min_{u}\bigl\{x^T Q\,x + u^T R\,u + 2x^T P(Ax + Bu)\bigr\} \]

Step 3: Minimise with respect to \(u\). Taking the derivative and setting to zero:

\[\frac{\partial}{\partial u}\bigl[u^T R\,u + 2x^T P B\,u\bigr] = 2R\,u + 2B^T P\,x = 0 \] \[\implies u^* = -R^{-1}B^T P\,x \]

Step 4: Substitute \(u^*\) back into the HJB equation:

\[0 = x^T Q\,x + x^T P B R^{-1} R\, R^{-1} B^T P\,x - 2x^T P B R^{-1} B^T P\,x + 2x^T P A\,x \] \[0 = x^T\!\bigl[Q + PA + A^T P - PBR^{-1}B^T P\bigr]x \]

Step 5: Since this must hold for all \(x\), we obtain the CARE:

\[\boxed{A^T P + PA - PBR^{-1}B^T P + Q = 0} \]

The optimal state-feedback gain is \(K = R^{-1}B^T P\), yielding \(u^* = -Kx\).

9.8.17.3 Model Answer (b) — Effect of Q and R on Closed-Loop Poles

The closed-loop system is \(\dot{x} = (A - BK)x\), where \(K = R^{-1}B^T P\). The poles are the eigenvalues of \(A_{cl} = A - BK\).

- As \(R \to 0\): The control cost vanishes, so the controller uses arbitrarily large inputs. The gain \(K\) grows without bound, and the closed-loop poles move far into the left-half plane (very fast response). In the limit, the LQR poles approach the transmission zeros of the system or recede to infinity along Butterworth patterns.
- As \(Q \to 0\): The state penalty vanishes, so there is no incentive to drive the state to zero. The gain \(K \to 0\), and the closed-loop poles approach the open-loop poles of the plant. The system behaves almost as if there is no feedback.

- Increasing \(Q/R\) ratio: Moves poles further into the LHP (faster, more aggressive control, higher bandwidth). Decreasing \(Q/R\) moves poles closer to the imaginary axis (slower, more conservative, less control effort).

A key property of LQR is that all closed-loop poles are guaranteed to lie in the open left-half plane (stability) and the system has at least 60ř phase margin and infinite gain margin (robustness).

9.8.17.4 Question 2 — The Separation Principle in LQG (25 marks)

Consider a stochastic LTI system with process noise \(w\) and measurement noise \(v\):

\[\dot{x} = Ax + Bu + w, \qquad y = Cx + v \]

where \(w \sim \mathcal{N}(0, W)\) and \(v \sim \mathcal{N}(0, V)\).

(a) State the separation principle (also called the certainty-equivalence principle) for LQG control. Explain why it allows the state estimator (Kalman filter) and the state-feedback controller (LQR) to be designed independently. (15 marks)

(b) Discuss scenarios where you would use LQG instead of LQR. What are the practical limitations of LQG? (10 marks)

9.8.17.5 Model Answer (a) — The Separation Principle

Statement: For a linear system with Gaussian noise and a quadratic cost, the optimal controller consists of two independent components:

1. Kalman filter (state estimator): Computes the optimal state estimate \(\hat{x}\) using the system model and noisy measurements: \[\dot{\hat{x}} = A\hat{x} + Bu + L(y - C\hat{x}), \qquad L = \Sigma C^T V^{-1} \] where \(\Sigma\) solves the filter algebraic Riccati equation: \(A\Sigma + \Sigma A^T - \Sigma C^T V^{-1} C\Sigma + W = 0\).
2. LQR gain: Computes the optimal feedback \(u = -K\hat{x}\), where \(K = R^{-1}B^T P\) and \(P\) solves the standard CARE.

Why independent design works: The estimation error \(\tilde{x} = x - \hat{x}\) evolves as \(\dot{\tilde{x}} = (A - LC)\tilde{x} + w - Lv\), which is independent of the control input \(u\). The closed-loop eigenvalues are the union of the LQR poles (eigenvalues of \(A - BK\)) and the Kalman filter poles (eigenvalues of \(A - LC\)). Since these sets are decoupled, each component can be designed without knowledge of the other.

9.8.17.6 Model Answer (b) — When to Use LQG over LQR

- Noisy measurements: When sensor data is corrupted by significant noise (e.g., IMU drift, lidar noise), LQR alone would amplify noise through the feedback gain. The Kalman filter in LQG provides optimal noise filtering.
- Incomplete state measurement: When not all states are directly measurable (e.g., velocity must be estimated from position), the Kalman filter reconstructs unmeasured states.
- Process disturbances: When the system is subject to stochastic disturbances (wind gusts, ground vibration), LQG accounts for these statistically through the process noise covariance \(W\).

Practical limitations of LQG:

- No guaranteed robustness margins: Unlike LQR, LQG does not have guaranteed gain/phase margins. The Loop Transfer Recovery (LTR) technique can partially restore robustness.
- Model accuracy: Performance degrades significantly if the system model or noise covariances \(W, V\) are inaccurate.
- Linearity assumption: LQG is optimal only for linear systems with Gaussian noise. For nonlinear systems, Extended Kalman Filters (EKF) or Unscented Kalman Filters (UKF) are needed, but optimality guarantees are lost.
- Computational cost: Requires solving two Riccati equations and running the Kalman filter in real time, though this is typically manageable for modern hardware.

SkyDynamics Robotics — Control Engineer Technical Challenge

Pre-Interview Assessment Flight Control Systems Team

Confidential — Do not distribute without authorization

10 ROS 2 Quadrotor Control (Week 10)

10.1 Overview 186
10.1.1 Company Brief: AeroVision UAV . . . 186
10.1.2 Provided Bag Data . 186
10.1.3 Deliverables 186
10.1.4 Evaluation Criteria . 187
10.1.5 Skills You Will Demonstrate 187
10.1.6 Recommended Approach (3-Hour Session) 187
10.2 Quadrotor Dynamics Review 188
10.2.1 State Vector 189
10.2.2 Translational Dynamics 189
10.2.3 Rotational Dynamics 189
10.2.4 Motor Mixing Matrix 190
10.2.5 Quadrotor Plus-Configuration Diagram 190
10.3 Cascaded Control Architecture in ROS2 190
10.3.1 Three-Loop Cascade 190
10.3.2 Cascaded Architecture Block Diagram 191
10.3.3 ROS2 Topic Structure 191
10.3.4 Launch File for Cascaded System 192
10.4 Implementing PID Attitude Controller in ROS2 192
10.4.1 Full PID Attitude Controller Node . . . 192
10.4.2 Complementary Filter for Attitude Estimation 193
10.5 Applying All 7 Controllers to Quadrotor 194
10.5.1 Computed Torque Control (CTC) 194
10.5.2 Model Predictive Control (MPC) 194
10.5.3 Adaptive Control . . 195
10.5.4 Robust Control (H-

Attitude Estimation and Control from Raw IMU Data

Position: Flight Software Engineer Team: Autopilot Development
Pre-Interview Technical Assessment Estimated Time: 3 Hours

10.1 Overview

10.1.1 Company Brief: AeroVision UAV

Scenario: AeroVision UAV builds autonomous quadrotor platforms for aerial inspection and mapping. Your team has collected 10 seconds of high-rate sensor data from a test flight including raw IMU readings, desired attitude commands, motor telemetry, and ground truth pose. As part of this challenge, you will implement attitude estimation and control pipelines as ROS2 nodes.

10.1.2 Provided Bag Data

Topic	Message Type	Rate	Description
`/imu`	sensor_msgs/Imu	1000 Hz	Raw accelero
`/desired_attitude`	geometry_msgs/QuaternionStamped	100 Hz	Target attitu
`/motor_commands`	std_msgs/Float32MultiArray	100 Hz	Motor speed
`/ground_truth_pose`	geometry_msgs/PoseStamped	100 Hz	Ground truth

10.1.3 Deliverables

- A complementary filter for attitude estimation fusing gyroscope and accelerometer data
- A cascaded PID attitude controller (rate loop + angle loop) as a ROS2 node
- Motor mixing matrix implementation converting thrust/torque to individual motor speeds

- Implementation of at least two alternative controllers (e.g., LQR, MPC, SMC) for comparison
- A quantitative comparison: attitude RMSE, computation time, and wind disturbance rejection

10.1.4 Evaluation Criteria

- Attitude Representation: Correct quaternion handling, gimbal-lock awareness, rotation math
- Control Architecture: Proper cascaded loop design with appropriate loop rates
- Real-Time Awareness: Computational efficiency, memory allocation strategy, timing considerations
- Communication: Ability to explain cascaded control design and trade-offs during the follow-up interview

10.1.5 Skills You Will Demonstrate

By the end of this session you will be able to:

- Represent quadrotor orientation using quaternions and convert between Euler angles and quaternion representations in ROS2 messages
- Derive the Newton-Euler translational and rotational dynamics of a rigid-body quadrotor from first principles
- Design a cascaded control architecture (position attitude rate motors) and explain why each loop runs at a different frequency
- Implement a PID attitude controller as a ROS2 node that subscribes to IMU data and publishes motor commands
- Map all seven control strategies (CTC, MPC, Adaptive, Robust, Backstepping, SMC, LQR) to the quadrotor plant and implement them as interchangeable ROS2 nodes
- Construct and invert the motor mixing matrix to convert desired thrust and torques into individual motor speeds
- Implement a complementary filter for attitude estimation by fusing gyroscope and accelerometer measurements
- Compare and evaluate all seven controllers on quadrotor hover and trajectory-tracking tasks using quantitative metrics (RMSE, computation time, wind rejection)

10.1.6 Recommended Approach (3-Hour Session)

0:00 – 0:20

10.1.6.1 Week 9 Exam Recall

Review ROS2 control node architecture, timer-based vs event-driven loops, and computational comparison of PID/LQR/MPC.

0:20 – 0:50

10.1.6.2 Quadrotor Dynamics Review

State vector, translational and rotational dynamics, motor mixing matrix, coordinate frames. SVG diagram of plus-configuration.

0:50 – 1:20

10.1.6.3 Cascaded Control Architecture in ROS2

Outer/middle/inner loop decomposition, node graph, topic structure, launch file. Industry standard (PX4, ArduPilot, DJI).

1:20 – 1:50

10.1.6.4 PID Attitude Controller in ROS2

Full Python implementation of cascaded PID attitude controller node. Complementary filter for attitude estimation.

1:50 – 2:20

10.1.6.5 Applying All 7 Controllers to Quadrotor

CTC, MPC, Adaptive, Robust, Backstepping, SMC, LQR — mapping each to quadrotor dynamics. Comprehensive comparison table.

2:20 – 2:40

10.1.6.6 Quadrotor Simulation in ROS2

quadrotor_sim.py physics engine, RViz2 visualization, rosbag2 recording, lab exercise walkthrough.

2:40 – 3:00

10.1.6.7 Hands-On Activities & Wrap-Up

Three lab activities: hover test, trajectory tracking comparison, wind disturbance rejection. Quiz and summary.

10.2 Quadrotor Dynamics Review

You studied quadrotor dynamics extensively in Weeks 3–8 (MATLAB theory block). Here we briefly review the key equations that we will now implement in ROS2 control nodes. The simulation uses the `quadrotor_sim.py` physics engine from `src/ros2_robotics_course/week_06/`, which follows the ENU/FLU convention (x-forward, y-left, z-up) — the ROS2 standard.

10.2.1 State Vector

The full quadrotor state is a 12-dimensional vector:

\[\mathbf{x} = \begin{bmatrix} x & y & z & \phi & \theta & \psi & \dot{x} & \dot{y} & \dot{z} & p & q & r \end{bmatrix}^T \]

where \((x, y, z)\) is the position in the world ENU frame, \((\phi, \theta, \psi)\) are roll, pitch, yaw (ZYX Euler angles), \((\dot{x}, \dot{y}, \dot{z})\) are world-frame velocities, and \((p, q, r)\) are body-frame angular rates.

10.2.2 Translational Dynamics

Applying Newton's second law in the world frame:

\[m\ddot{\mathbf{p}} = \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} + R(\phi,\theta,\psi) \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix} \]

where \(m\) is the total mass, \(g = 9.81\;\text{m/s}^2\), \(T\) is the total thrust along the body z-axis, and \(R(\phi,\theta,\psi)\) is the ZYX rotation matrix (body-to-world):

\[R = R_z(\psi) R_y(\theta) R_x(\phi) = \begin{bmatrix} c\theta c\psi & c\psi s\theta s\phi - s\psi c\phi & c\psi s\theta c\phi + s\psi s\phi \\ c\theta s\psi & s\psi s\theta s\phi + c\psi c\phi & s\psi s\theta c\phi - c\psi s\phi \\ -s\theta & c\theta s\phi & c\theta c\phi \end{bmatrix} \]

10.2.3 Rotational Dynamics

Euler's equation for rotational motion in the body frame:

\[J\dot{\boldsymbol{\omega}} = -\boldsymbol{\omega} \times J\boldsymbol{\omega} + \boldsymbol{\tau} \]

where \(J = \text{diag}(I_{xx}, I_{yy}, I_{zz})\) is the inertia tensor (diagonal for a symmetric quadrotor), \(\boldsymbol{\omega} = [p, q, r]^T\) are body angular rates, and \(\boldsymbol{\tau} = [\tau_\phi, \tau_\theta, \tau_\psi]^T\) are the control torques.

Expanding component-wise:

\[\begin{aligned} I_{xx}\dot{p} &= (I_{yy} - I_{zz})qr + \tau_\phi \\ I_{yy}\dot{q} &= (I_{zz} - I_{xx})pr + \tau_\theta \\ I_{zz}\dot{r} &= (I_{xx} - I_{yy})pq + \tau_\psi \end{aligned} \]

10.2.4 Motor Mixing Matrix

The four motors (plus-configuration) produce a total thrust T and three torques $(\tau_\phi, \tau_\theta, \tau_\psi)$. The relationship is:

$$\begin{bmatrix} T \\ \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix} = \underbrace{\begin{bmatrix} k_f & k_f & k_f & k_f \\ 0 & -dk_f & 0 & dk_f \\ dk_f & 0 & -dk_f & 0 \\ -k_m & k_m & -k_m & k_m \end{bmatrix}}_{\displaystyle M} \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix}$$

where k_f is the thrust coefficient, k_m is the torque coefficient, d is the arm length, and ω_i is the angular speed of motor i. The inverse mixing matrix M^{-1} converts desired thrust/torques to motor speed commands:

$$\begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix} = M^{-1} \begin{bmatrix} T \\ \tau_\phi \\ \tau_\theta \\ \tau_\psi \end{bmatrix}$$

Physical Parameters (from quadrotor_sim.py): mass = 1.0 kg, arm_length = 0.25 m, $I_{xx} = I_{yy} = 0.0082\;\text{kg}\cdot\text{m}^2$, $I_{zz} = 0.0150\;\text{kg}\cdot\text{m}^2$, $k_f = 6.8 \times 10^{-6}$, $k_m = 2.6 \times 10^{-7}$, motor range: 0–838 rad/s.

10.2.5 Quadrotor Plus-Configuration Diagram

The following diagram shows the motor layout, rotation directions, thrust vectors, and body-frame axes for a plus-configuration quadrotor:

10.3 Cascaded Control Architecture in ROS2

The standard control architecture used by virtually all commercial and research flight controllers (DJI, PX4, ArduPilot, BetaFlight) is a cascaded loop structure. Each loop operates at a different frequency, with inner loops running faster than outer loops.

10.3.1 Three-Loop Cascade

10.3.1.1 Outer Loop: Position Controller

Rate: 10–50 Hz

Input: Desired position (x_d, y_d, z_d) from mission planner

Output: Desired attitude angles (ϕ_d, θ_d) and thrust T_d

Sensor: GPS/vision (position), barometer (altitude)

This loop converts position error into desired accelerations, then maps accelerations to desired tilt angles using the small-angle approximation.

10.3.1.2 Middle Loop: Attitude Controller

Rate: 100–250 Hz

Input: Desired attitude \((\phi_d, \theta_d, \psi_d)\)

Output: Desired angular rates \((p_d, q_d, r_d)\)

Sensor: IMU (accelerometer-derived attitude via complementary or Mahony filter)

Proportional control on attitude error to generate rate setpoints. This decouples attitude from rate dynamics.

10.3.1.3 Inner Loop: Rate Controller

Rate: 250–1000 Hz

Input: Desired angular rates \((p_d, q_d, r_d)\)

Output: Motor commands (PWM or speed)

Sensor: Gyroscope (direct angular rate measurement, no filtering needed)

Full PID on rate error. This is the tightest loop and must run fastest for stability.

Why Cascaded? A single loop from position to motors would need to invert the full nonlinear dynamics at every step, requiring extremely high bandwidth. By cascading, each loop only needs to stabilize one dynamic mode: the inner rate loop damps oscillations, the attitude loop tracks orientation, and the outer loop tracks position. Each loop sees the next inner loop as an approximately instantaneous actuator (because the inner loop is 5–10x faster).

10.3.2 Cascaded Architecture Block Diagram

10.3.3 ROS2 Topic Structure

Topic Name	Message Type	Publisher	Subscriber(s)	Rate (Hz
`/position_ref`	geometry_msgs/Point	Mission Planner	/position_controller	10
`/attitude_cmd`	geometry_msgs/Vector3	/position_controller	/attitude_controller	50
`/rate_cmd`	geometry_msgs/Vector3	/attitude_controller	/rate_controller	250
`/motor_cmd`	std_msgs/Float64MultiArray	/rate_controller	/motor_mixer	250
`/motor_speeds`	std_msgs/Float64MultiArray	/motor_mixer	/quadrotor_sim	250
`/odom`	nav_msgs/Odometry	/quadrotor_sim	/position_controller	50
`/imu`	sensor_msgs/Imu	/quadrotor_sim	/attitude_controller, /rate_controller	250

10.3.4 Launch File for Cascaded System

from launch import LaunchDescription from launch_ros.actions import Node def generate_launch_description(): return LaunchDescription([# Quadrotor physics simulation Node(package='quadrotor_control', executable='quadrotor_sim_node', name='quadrotor_sim', parameters=[{'dt': 0.001, 'mass': 1.0}],), # Position controller (outer loop, 50 Hz) Node(package='quadrotor_control', executable='position_controller_node', name='position_controller', parameters=[{'rate': 50.0, 'kp_xy': 1.5, 'kd_xy': 1.0}],), # Attitude controller (middle loop, 250 Hz) Node(package='quadrotor_control', executable='attitude_controller_node', name='attitude_controller', parameters=[{'rate': 250.0, 'kp_roll': 6.0}],), # Rate controller (inner loop, 500 Hz) Node(package='quadrotor_control', executable='rate_controller_node', name='rate_controller', parameters=[{'rate': 500.0}],), # Motor mixer Node(package='quadrotor_control', executable='motor_mixer_node', name='motor_mixer',),])

10.4 Implementing PID Attitude Controller in ROS2

Below is a complete ROS2 Python node implementing a cascaded PID attitude controller. It subscribes to `/imu` for current attitude and `/attitude_cmd` for desired orientation, then publishes torque/thrust commands on `/motor_cmd`.

10.4.1 Full PID Attitude Controller Node

#!/usr/bin/env python3 """PID Attitude Controller - ROS2 Node for Quadrotor.""" import rclpy from rclpy.node import Node from sensor_msgs.msg import Imu from geometry_msgs.msg import Vector3 from std_msgs.msg import Float64MultiArray import numpy as np from math import atan2, asin, sqrt class PIDAttitudeController(Node): def __init__(self): super().__init__('pid_attitude_controller') # PID gains: [roll, pitch, yaw] self.kp = np.array([6.0, 6.0, 3.0]) self.ki = np.array([0.5, 0.5, 0.2]) self.kd = np.array([2.0, 2.0, 1.0]) self.integral = np.zeros(3) self.prev_error = np.zeros(3) self.integral_limit = 2.0 self.des_rpy = np.zeros(3) # desired roll, pitch, yaw self.cur_rpy = np.zeros(3) # current roll, pitch, yaw self.cur_rates = np.zeros(3) # current angular rates # Complementary filter state self.alpha = 0.98 self.cf_rpy = np.zeros(3) self.cf_init = False # Subscribers self.create_subscription(Imu, '/imu', self.imu_cb, 10) self.create_subscription(Vector3, '/attitude_cmd', self.cmd_cb, 10) # Publisher self.cmd_pub = self.create_publisher(Float64MultiArray, '/motor_cmd', 10) # Timer at 250 Hz self.dt = 1.0 / 250.0 self.create_timer(self.dt, self.control_loop) self.get_logger().info('PID Attitude Controller started at 250 Hz') def quat_to_euler(self, q): """Convert quaternion (x,y,z,w) to Euler (roll, pitch, yaw)."""" x, y, z, w = q.x, q.y, q.z, q.w sinr = 2.0 * (w * x + y * z) cosr = 1.0 - 2.0 * (x * x + y * y) roll = atan2(sinr, cosr) sinp = 2.0 * (w * y - z * x) pitch = asin(max(-1.0, min(1.0, sinp))) siny = 2.0 * (w * z + x * y) cosy = 1.0 - 2.0 * (y * y + z * z) yaw = atan2(siny, cosy)

return np.array([roll, pitch, yaw]) def imu_cb(self, msg): """Process IMU: complementary filter for attitude."""" accel_rpy = self.quat_-to_euler(msg.orientation) gyro = np.array([msg.angular_velocity.x, msg.angular_velocity.y, msg.angular_velocity.z]) self.cur_rates = gyro if not self.cf_init: self.cf_rpy = accel_rpy self.cf_init = True else: self.cf_rpy = (self.alpha * (self.cf_rpy + gyro * self.dt) + (1 - self.alpha) * accel_rpy) self.cur_rpy = self.cf_rpy def cmd_cb(self, msg): self.des_rpy = np.array([msg.x, msg.y, msg.z]) def control_loop(self): error = self.des_rpy - self.cur_rpy self.integral += error * self.dt self.integral = np.clip(self.integral, -self.integral_limit, self.integral_limit) derivative = -self.cur_rates # use gyro directly torque = (self.kp * error + self.ki * self.integral + self.kd * derivative) msg = Float64MultiArray() msg.data = torque.tolist() self.cmd_pub.publish(msg) def main(): rclpy.init() rclpy.spin(PIDAttitudeController()) rclpy.shutdown() if __name__ == '__main__': main()

10.4.2 Complementary Filter for Attitude Estimation

The complementary filter fuses gyroscope integration (high-frequency, drift-prone) with accelerometer-derived angles (low-frequency, noisy) to produce a stable attitude estimate:

\[\hat{\phi}_{k+1} = \alpha\bigl(\hat{\phi}_k + \omega_x \,\Delta t\bigr) + (1 - \alpha)\,\phi_\text{acc} \]

where:

- \(\alpha \in [0.95, 0.99]\) is the filter coefficient (higher = more trust in gyroscope)
- \(\omega_x\) is the gyroscope angular rate about the x-axis
- \(\phi_\text{acc} = \text{atan2}(a_y, a_z)\) is the accelerometer-derived roll angle
- \(\Delta t\) is the time step between IMU samples

Why not just use the gyroscope? Gyroscope integration drifts over time due to bias. The accelerometer provides an absolute reference (gravity direction) but is noisy and affected by translational acceleration. The complementary filter combines the best of both: short-term accuracy from the gyro and long-term stability from the accelerometer. For production systems, a Mahony or Madgwick filter (quaternion-based) is preferred.

The same principle applies for pitch:

\[\hat{\theta}_{k+1} = \alpha\bigl(\hat{\theta}_k + \omega_y \,\Delta t\bigr) + (1 - \alpha)\,\theta_\text{acc} \]

where \(\theta_\text{acc} = \text{atan2}\bigl(-a_x,\;\sqrt{a_y^2 + a_z^2}\bigr)\). Yaw cannot be corrected by the accelerometer alone — it requires a magnetometer or visual heading.

10.5 Applying All 7 Controllers to Quadrotor

In Weeks 1–8 you studied seven control strategies in the abstract. In Week 9 you implemented them as generic ROS2 nodes. Now we map each controller specifically to the quadrotor dynamics and discuss the practical ROS2 implementation considerations.

10.5.1 Computed Torque Control (CTC)

10.5.1.1 Concept

Computed Torque Control (also called feedback linearization) cancels the nonlinear terms in the rotational dynamics and replaces them with a desired linear response:

$$\boldsymbol{\tau} = J\mathbf{u}_d + \boldsymbol{\omega} \times J\boldsymbol{\omega}$$

where $\mathbf{u}_d = \ddot{\mathbf{q}}_d + K_d(\dot{\mathbf{q}}_d - \dot{\mathbf{q}}) + K_p(\mathbf{q}_d - \mathbf{q})$ is the auxiliary PD input. After cancellation, the closed-loop system reduces to $\ddot{\mathbf{e}} + K_d\dot{\mathbf{e}} + K_p\mathbf{e} = 0$, which is a linear second-order system with tunable damping.

- Requires: Accurate inertia matrix J. Sensitive to payload changes.
- ROS2 Implementation: The Coriolis/gyroscopic term $\boldsymbol{\omega} \times J\boldsymbol{\omega}$ is computed in each timer callback using the latest gyroscope reading. Publishes torque commands.
- Best for: Known, well-characterized platforms where the model is accurate.

10.5.2 Model Predictive Control (MPC)

10.5.2.1 Concept

Linearize the quadrotor dynamics about the hover operating point. At each control cycle, solve a finite-horizon constrained optimization:

$$\min_{\mathbf{u}_{0:N-1}} \sum_{k=0}^{N-1} \bigl(\|\mathbf{x}_k - \mathbf{x}_\text{ref}\|_Q^2 + \|\mathbf{u}_k\|_R^2\bigr) + \|\mathbf{x}_N - \mathbf{x}_\text{ref}\|_P^2$$

subject to $\mathbf{x}_{k+1} = A\mathbf{x}_k + B\mathbf{u}_k$, thrust limits $0 \le T \le T_\text{max}$, and angle constraints $|\phi| \le 30^\circ, |\theta| \le 30^\circ$.

- Handles: Thrust saturation and angle limits explicitly. Can preview reference trajectories.
- ROS2 Implementation: Heavier computation; run at 10–20 Hz in a separate thread or as the outer position loop. Use OSQP or acados for the QP solver. Apply only the first control action.
- Industry: ETH Zurich (aggressive aerobatics), Skydio (obstacle avoidance), DJI Matrice (waypoint tracking).

10.5.3 Adaptive Control

10.5.3.1 Concept

Online estimation of uncertain parameters (mass \(m\), inertia \(J\), aerodynamic coefficients) using the regressor form:

\[\boldsymbol{\tau} = Y(\mathbf{q}, \dot{\mathbf{q}}, \dot{\mathbf{q}}_r, \ddot{\mathbf{q}}_r)\,\hat{\boldsymbol{\theta}} + K_d\mathbf{s} \]

where \(\hat{\boldsymbol{\theta}}\) is the parameter estimate vector, \(Y\) is the regressor matrix, and \(\mathbf{s} = \dot{\mathbf{e}} + \Lambda\mathbf{e}\) is the sliding variable. The update law is \(\dot{\hat{\boldsymbol{\theta}}} = -\Gamma^{-1}Y^T\mathbf{s}\).

- Use case: Quadrotor picking up unknown payloads (mass changes mid-flight), battery depletion changing mass.
- ROS2 Implementation: Parameter update runs alongside the control law in the same callback. Store \(\hat{\boldsymbol{\theta}}\) as node state. Publish parameter estimates for monitoring on `/param_estimates`.

10.5.4 Robust Control (H-infinity / Sliding Boundary Layer)

10.5.4.1 Concept

Add a robust compensating term to handle bounded disturbances (wind, ground effect, model uncertainty):

\[\boldsymbol{\tau} = \boldsymbol{\tau}_\text{nominal} + \boldsymbol{\tau}_\text{robust} \]

where \(\boldsymbol{\tau}_\text{robust} = -\rho\,\text{sgn}(\mathbf{s})\) with \(\rho \geq \|\mathbf{d}\|_\infty\). To avoid chattering, a boundary layer is used: \(\text{sat}(\mathbf{s}/\epsilon)\) replaces \(\text{sgn}(\mathbf{s})\).

- Use case: Outdoor flight with wind gusts up to 10 m/s, flying near structures (ground effect).
- ROS2 Implementation: A wind estimator node subscribes to `/imu` and `/odom`, estimates disturbance force, and publishes on `/wind_estimate`. The robust compensator node uses this estimate to set the disturbance bound \(\rho\).

10.5.5 Backstepping Control

10.5.5.1 Concept

Backstepping is a natural fit for the cascaded quadrotor structure. Design proceeds in two steps:

\[\begin{aligned} \text{Step 1 (outer):}\quad &\text{Position error} \;\rightarrow\; \text{Virtual control: desired attitude } (\phi_d,

\theta_d, T_d) \\ \text{Step 2 (inner):}\quad &\text{Attitude error} \;\rightarrow\; \text{Actual torques } \boldsymbol{\tau} \end{aligned} \]

Each step uses a Lyapunov function to prove stability. The virtual control from Step 1 becomes the reference for Step 2.

- Advantage: Formal stability guarantee via Lyapunov analysis. Handles the underactuated nature (4 inputs, 6 DOF) naturally through the virtual control.
- ROS2 Implementation: Two-node architecture mirrors the two backstepping steps: `/backstepping_outer` (position) and `/backstepping_inner` (attitude). The outer node publishes desired attitude for the inner node.

10.5.6 Sliding Mode Control (SMC)

10.5.6.1 Concept

Define a sliding surface for each attitude axis:

\[s_\phi = \dot{e}_\phi + \lambda_\phi\, e_\phi \]

The control law drives the state to the surface and maintains it there:

\[\tau_\phi = I_{xx}\bigl(\ddot{\phi}_d + \lambda_\phi\,\dot{e}_\phi - \eta\,\text{sat}(s_\phi / \epsilon)\bigr) - (I_{yy} - I_{zz})qr \]

- Strength: Excellent disturbance rejection for matched disturbances (wind acting through control channels). Invariant to parameter uncertainty once on the surface.
- Chattering: Mitigated with boundary layer \(\epsilon\) or super-twisting algorithm.
- ROS2 Implementation: Requires a fast inner loop (500–1000 Hz) to maintain sliding condition. Timer period = 1–2 ms. Use `BEST_EFFORT` QoS to avoid blocking.

10.5.7 Nonlinear Control Optimization — iLQR / NMPC (Week 8)

10.5.7.1 Concept

Rather than linearising the quadrotor around hover once, iLQR (Week 8 ğ9) linearises around the current-iterate trajectory at every iteration and solves a time-varying LQR in closed form. The discrete-time recursion computes, for \(k = N\!-\!1,\ldots,0\):

\[k_k = -Q_{uu,k}^{-1} Q_{u,k},\quad K_k = -Q_{uu,k}^{-1} Q_{ux,k} \] \[u_k^{\text{new}} = \bar u_k + \alpha\,k_k + K_k\,(x_k - \bar x_k) \]

where \(Q_{uu}, Q_{ux}, Q_u\) are Jacobian/Hessian blocks assembled from the nonlinear dynamics \(f\) and cost \(L\) (see Week 8 ğ9 notation table). Wrapped in a receding-horizon loop this is NMPC.

- Advantage: Handles the full nonlinear quadrotor model including tilt saturation and aggressive manoeuvres where linearised LQR fails; simultaneously provides an open-loop plan and a local time-varying feedback policy \(K_k\).
- Limitation: Iterative, higher compute per step than LQR; locally (not globally) optimal; needs warm-starting across samples to meet real-time.
- ROS2 implementation: Timer callback at 50–100 Hz runs 3–5 iLQR iterations with warm-started control sequence; finite-difference Jacobians for the 6- or 12-state quadrotor; apply \(u_0^\star\), shift the warm-start. See `week_09/w11_exercises/task_nmpc_node.py` for the unicycle template.
- Meta-use: iLQR/NMPC is also the offline tuning tool for the other six controllers — minimising a closed-loop cost \(J(\theta)\) over SMC gains \((\lambda,\phi,\eta)\), adaptive \((\Gamma,\sigma)\), robust \((K_p,K_d,\rho,\Lambda)\), or backstepping \((k_1,\ldots,k_n)\) as shown in Week 8 ğ5–ğ8.

10.5.8 Comprehensive Controller Comparison

Controller	Hover RMSE (cm)	Trajectory RMSE (cm)	Wind Rejection (3 m/s)	Computation / Cycle	Tuni
PID	3.2	8.5	Moderate (15 cm drift)	<1 s	9 (K
CTC	2.1	5.8	Poor (model mismatch)	~5 s	6 (K
MPC	1.8	3.2	Good (12 cm drift)	1–50 ms	Q, R
Adaptive	2.5	6.1	Good (adapts to steady wind)	~20 s	, +
Robust	2.8	7.0	Excellent (5 cm drift)	~10 s	, +
Backstepping	2.0	4.5	Good (10 cm drift)	~15 s	k_1–k
SMC	2.4	5.5	Excellent (4 cm drift)	~8 s	, , p
LQR	1.5	6.8	Moderate (14 cm drift)	~2 s	Q, R

Key Insight: No single controller dominates across all metrics. PID is the industry standard due to simplicity and robustness of tuning. MPC excels at trajectory tracking with constraints. SMC and Robust control are superior for wind rejection. LQR is optimal for hover energy efficiency. In practice, many production systems blend approaches — for example, PX4 uses a cascaded PID structure but the position loop effectively implements LQR-like gains computed from a simplified model.

10.6 Quadrotor Simulation in ROS2

10.6.1 Simulation Environment

The simulation uses the `quadrotor_sim.py` physics engine from the ROS2 Robotics Course (Week 6). Key features:

- Physics Engine: 12-state rigid-body dynamics with RK4 integration at 1 kHz internal rate

- Convention: ENU/FLU (x-forward, y-left, z-up) — ROS2 standard
- Motor Model: First-order motor dynamics with thrust coefficient \(k_f = 6.8 \times 10^{-6}\) and torque coefficient \(k_m = 2.6 \times 10^{-7}\)
- Sensor Models: IMU with configurable noise (gyro bias, accelerometer noise), GPS with position noise
- Motor Layout: Plus-configuration with M1 (front, CW), M2 (left, CCW), M3 (rear, CW), M4 (right, CCW)

10.6.2 ROS2 Integration

The simulation node publishes sensor data and subscribes to motor commands:

quadrotor_sim_node.py - Wraps quadrotor_sim.py as a ROS2 node class QuadrotorSimNode(Node): def __init__(self): super().__init__('quadrotor_sim') # Publishers: sensor data self.odom_pub = self.create_publisher(Odometry, '/odom', 10) self.imu_pub = self.create_publisher(Imu, '/imu', 10) # Subscriber: motor commands self.create_subscription(Float64MultiArray, '/motor_speeds', self.motor_cb, 10) # TF broadcaster for RViz2 self.tf_broadcaster = TransformBroadcaster(self) # Simulation state self.state = np.zeros(12) self.state[2] = 0.0 # start on ground self.params = QuadrotorParams() self.create_timer(0.001, self.step_sim) # 1 kHz physics

10.6.3 Visualization in RViz2

- TF Frames: Broadcast `world base_link` transform at 100 Hz. Attach `rotor_1`–`rotor_4` child frames for motor visualization.
- Markers: Use `visualization_msgs/MarkerArray` to render the quadrotor body (cube for center mass, cylinders for arms, rotating discs for propellers).
- Trajectory: Publish `nav_msgs/Path` on `/trajectory` to show the flight path as a line strip in RViz2.

10.6.4 Recording Flight Data with rosbag2

Record all relevant topics during a flight test ros2 bag record /odom /imu /attitude_cmd /rate_cmd /motor_cmd /motor_speeds \ -o flight_test_pid_hover # Replay for post-flight analysis ros2 bag play flight_test_pid_hover # Convert to CSV for MATLAB/Python plotting ros2 bag convert -i flight_test_pid_hover -o flight_data.csv

10.6.5 Lab Exercises from ROS2 Robotics Course (Week 6)

The following lab exercises from `src/ros2_robotics_course/week_06/lab_exercises/` build up to a complete quadrotor simulation. This week, you will use these as your foundation and swap in different controllers:

Task	File	Description
Task 1	`task1_rotation_math.py`	Rotation mathematics: Euler angles to rotation matrix, quaternion operati
Task 2	`task2_dynamics_model.py`	Implement the 12-state quadrotor dynamics model, compute state derivati
Task 3	`task3_pid_attitude.py`	Cascaded PID attitude controller with outer angle loop and inner rate loo
Task 4	`task4_motor_mixing.py`	Motor mixing matrix construction and inversion, mapping from thrust/tor
Task 5	`task5_cascaded_control.py`	Full cascaded position + attitude control: position PD desired acceleratio
Task 6	`task6_complementary_filter.py`	Complementary filter implementation fusing gyroscope and accelerometer
Task 7	`task7_full_simulation.py`	Full simulation: fly a square trajectory with cascaded controller, 4-panel vi

Integration with This Week: Tasks 1–4 provide the foundation (dynamics, mixing, PID). Task 5 gives the cascaded structure. Your job this week is to replace the inner PID controller in Task 5 with each of the seven controllers (CTC, MPC, Adaptive, Robust, Backstepping, SMC, LQR) and compare performance on the same trajectory.

10.7 Summary and Next Steps

10.7.1 Hands-On Activities

10.7.2 Activity 1: PID Hover Test

Objective: Implement the PID attitude controller from Section 5 as a ROS2 node and verify stable hover.

1. Create a ROS2 package `quadrotor_control` with the PID attitude controller node.
2. Configure the cascaded system: position controller (outer), PID attitude controller (inner), motor mixer.
3. Set the hover reference to \([x, y, z] = [0, 0, 1]\) m.
4. Run the simulation for 30 seconds and record the flight with rosbag2.
5. Plot the attitude angles \((\phi, \theta, \psi)\) versus time. Verify:
   - Settling time $<$ 2 seconds
   - Steady-state attitude error $<$ 0.5 degrees
   - Position RMSE $<$ 5 cm during hover

Terminal 1: Launch simulation + controllers ros2 launch quadrotor_-control cascaded_hover.launch.py # Terminal 2: Record data ros2 bag record /odom /imu /attitude_cmd /motor_cmd -o hover_test_pid # Terminal 3: Monitor in real-time ros2 topic echo /odom --field pose.pose.position

10.7.3 Activity 2: Trajectory Tracking — PID vs LQR vs MPC

Objective: Fly a square trajectory and compare three controllers quantitatively.

1. Define the square trajectory waypoints at 1 m altitude:
   - \([0, 0, 1] \rightarrow [2, 0, 1] \rightarrow [2, 2, 1] \rightarrow [0, 2, 1] \rightarrow [0, 0, 1]\)
2. Implement the LQR controller node:
   - Linearize around hover, solve CARE using `scipy.linalg.solve_continuous_are`
   - Precompute gain matrix \(K\), apply \(\mathbf{u} = -K(\mathbf{x} - \mathbf{x}_d)\) in callback
3. Implement the MPC controller node:
   - Use OSQP solver with prediction horizon \(N = 20\) at 20 Hz
   - Thrust constraint: \(0 \le T \le 2mg\); angle constraint: \(|\phi|, |\theta| \le 30^\circ\)
4. Run each controller on the same trajectory. Record rosbag for each run.
5. Generate comparison plots:
   - 3D trajectory overlay (PID = blue, LQR = green, MPC = red)
   - Position tracking error vs time for each axis
   - RMSE table: compute position RMSE for each controller
   - Motor effort: plot total motor speed squared (energy proxy) vs time

10.7.4 Activity 3: Wind Disturbance Rejection — SMC vs Robust vs PID

Objective: Add a simulated constant wind disturbance and compare rejection performance.

1. Modify `quadrotor_sim_node.py` to add a constant external force:
   # In quadrotor_dynamics(), add wind force (3 m/s in x) wind_speed = 3.0 # m/s in world x-direction drag_coeff = 0.1 # N/(m/s) F_wind = np.array([drag_coeff * wind_speed, 0.0, 0.0]) acc += F_wind / m # add to translational acceleration
2. Implement SMC attitude controller:
   - Sliding surface: \(s = \dot{e} + \lambda e\) with \(\lambda = 5\)
   - Reaching law: \(\tau = \tau_\text{eq} + \eta \cdot \text{sat}(s/\epsilon)\) with \(\eta = 2.0\), \(\epsilon = 0.1\)
3. Implement Robust controller:
   - Nominal PID + robust compensator with disturbance bound \(\rho = 1.5\) N
4. Run hover test at \([0, 0, 1]\) for 30 seconds with wind, record all three controllers.
5. Compare:
   - Maximum position deviation from setpoint

- Steady-state offset (PID will have nonzero offset without integral wind-up compensation)
- Control effort (motor speed variance)

Expected Results: SMC should achieve the smallest steady-state drift (~4 cm) due to its inherent disturbance rejection on the sliding surface. Robust control should be next (~5 cm). PID will show the largest drift (~15 cm) unless the integral gain is significantly increased, which risks oscillation.

10.7.4.1 Quick Quiz: Quadrotor Control in ROS2

Q1. In a cascaded control architecture, why must the inner rate loop run at a higher frequency (e.g., 500 Hz) than the outer position loop (e.g., 50 Hz)?

10.7.4.2 Answer

The inner loop must be fast enough that it appears as an instantaneous actuator to the outer loop. The rule of thumb is that each inner loop should be 5–10x faster than the loop that commands it. If the rate loop were too slow, the attitude loop would command rate changes that take too long to execute, leading to phase lag and potential instability in the outer loop. Additionally, the angular rate dynamics are the fastest dynamics in the quadrotor system (small inertia, high torques), so the rate controller must sample at or above the Nyquist rate of these dynamics.

Q2. The motor mixing matrix \(M\) has dimensions \(4 \times 4\). Under what condition is \(M\) invertible? What physical situation would make it singular?

10.7.4.3 Answer

\(M\) is invertible when the four motors can independently produce the four wrench components (thrust, roll torque, pitch torque, yaw torque). It becomes singular if, for example, all motors are collocated (arm length \(d = 0\)), which makes the roll and pitch torque rows identical to zero. It would also be singular if the yaw torque coefficients were all equal (no counter-rotating pairs), as you could not produce differential yaw torque. In the standard plus or X configuration with properly counter-rotating motors, \(M\) is always full rank.

Q3. Why is MPC typically placed in the outer position loop (10–20 Hz) rather than the inner rate loop for a quadrotor?

10.7.4.4 Answer

MPC involves solving a constrained quadratic program at each cycle, which takes 1–50 ms depending on the horizon length and solver. This computation time is acceptable at 10–20 Hz (50–100 ms period) but would miss deadlines at 500 Hz (2 ms period). The position dynamics

are the slowest, so the prediction horizon can be longer without excessive computation. The inner rate loop uses simpler controllers (PID, LQR) that execute in microseconds, ensuring the tight timing needed for angular rate stabilization.

Q4. In the complementary filter $\hat{\phi}_{k+1} = \alpha(\hat{\phi}_k + \omega_x \Delta t) + (1-\alpha)\phi_\text{acc}$, what happens if $\alpha = 1.0$? What if $\alpha = 0.0$?

10.7.4.5 Answer

If $\alpha = 1.0$, the filter becomes pure gyroscope integration: $\hat{\phi}_{k+1} = \hat{\phi}_k + \omega_x \Delta t$. This will drift over time due to gyro bias, eventually producing arbitrarily large errors. If $\alpha = 0.0$, the filter uses only the accelerometer: $\hat{\phi}_{k+1} = \phi_\text{acc}$. This eliminates drift but produces a very noisy estimate and is invalid during translational acceleration (the accelerometer measures gravity plus linear acceleration, so the gravity-derived angle is wrong during maneuvers). Typical values of $\alpha = 0.95$–0.99 balance gyro responsiveness with accelerometer stability.

Q5. Which of the seven controllers would you recommend for a delivery drone that must pick up packages of unknown weight? Justify your choice.

10.7.4.6 Answer

Adaptive Control is the best choice. When the drone picks up a package, its mass m and inertia J change suddenly. Adaptive control uses online parameter estimation to update $\hat{m}$ and $\hat{J}$ in real-time, adjusting the control law accordingly. The regressor-based formulation $\tau = Y(q, \dot{q}, \dot{q}_r, \ddot{q}_r)\hat{\theta}$ can track mass changes within a few seconds. PID would work but would require aggressive integral action and retuning. LQR and MPC would degrade because the linearized model is incorrect for the new mass. Computed Torque would produce incorrect compensation torques until the model is updated. A practical hybrid approach combines Adaptive control with MPC: the adaptive law estimates parameters, and MPC uses the updated model for trajectory optimization.

10.7.5 Summary

10.7.6 Key Takeaways

- Cascaded Architecture is Standard: The three-loop cascade (position attitude rate) is used by every major flight controller (PX4, ArduPilot, DJI, BetaFlight). Each loop runs at a different frequency, with inner loops 5–10x faster than outer loops. In ROS2, this maps naturally to separate nodes communicating via topics.

- Motor Mixing Matrix is the Bridge: The \(4 \times 4\) mixing matrix \(M\) converts desired thrust and torques into individual motor speed commands. Its inverse \(M^{-1}\) is precomputed and applied at every control cycle. Getting this matrix correct is the first step toward any quadrotor controller.
- Complementary Filter Enables Attitude Estimation: Fusing gyroscope and accelerometer data with a tunable parameter \(\alpha\) provides a computationally cheap attitude estimate suitable for inner-loop control. Production systems use Mahony or extended Kalman filters.
- No Single Controller Wins Everything: PID is simple and robust to tune. LQR is energy-optimal near hover. MPC handles constraints and trajectory preview. SMC and Robust control excel at wind rejection. Adaptive control handles changing payloads. Backstepping provides formal stability guarantees. CTC gives exact linearization when the model is accurate.
- ROS2 Enables Modular Controller Swapping: Because each controller is a separate ROS2 node with standardized topic interfaces (`/attitude_cmd` in, `/motor_cmd` out), you can swap controllers without modifying the rest of the system. This is the power of the ROS2 architecture for rapid prototyping and comparison.
- Real-Time Matters: The inner rate loop must execute within 1–2 ms. PID and LQR easily meet this. MPC requires careful solver selection and runs in the outer loop. Use timer-based callbacks, pre-allocate memory, and avoid heap allocation in control loops.

10.7.7 Preparation for Week 11

Next week we move to Multi-Robot Coordination and Formation Control. We will extend the single-quadrotor controllers from this week to multi-agent systems, implementing consensus-based formation control, leader-follower architectures, and collision avoidance using ROS2 multi-node communication patterns. Complete all three activities from this week before the next session.

10.7.8 Running the ROS2 Demo

10.7.9 Build and Source

cd ~/auto_control_ws colcon build --packages-select week_06_quadrotor_dynamics source install/setup.bash

10.7.10 Launch the Demo

ros2 launch week_06_quadrotor_dynamics demo.launch.py

This command replays the recorded bag data at 2x speed, starts three solution nodes (attitude estimation, PID control, motor mixing), and opens RViz2 with a preconfigured layout.

10.7.11 ROS2 Topics

Topic	Type	Source	Description
`/imu`	sensor_msgs/Imu	Bag	Raw IMU data
`/desired_attitude`	geometry_msgs/QuaternionStamped	Bag	Attitude setpo
`/motor_commands`	std_msgs/Float32MultiArray	Bag	Motor speed te
`/ground_truth_pose`	geometry_msgs/PoseStamped	Bag	Ground truth

10.7.12 Expected RViz2 Output

RViz2 will display the quadrotor's estimated attitude (from the complementary filter) alongside the ground truth pose. You should see the attitude estimate converging to ground truth within the first few seconds. Motor command markers will show thrust allocation across the four rotors.

10.7.13 Troubleshooting

- RViz2 crashes on launch: The launch file filters `snap` paths from `LD_LIBRARY_PATH` automatically. If RViz2 still crashes, try: `export LD_LIBRARY_PATH=$(echo $LD_LIBRARY_PATH | tr ':' '\n' | grep -v snap | tr '\n' ':')`
- No data visible: Ensure the bag file exists at `w06_exercises/bag_data/synthetic_quadrotor/`. Re-run `colcon build` if needed.
- QoS mismatch warnings: The bag uses default QoS. If your subscriber uses `BEST_EFFORT`, switch to `RELIABLE` or use `--qos-profile-overrides-path`.

10.7.14 Appendix: Review Material — ROS2 Control Implementation

10.7.14.1 Question 1 (25 marks)

Explain the architecture of a ROS2 control node for a robotic system. In your answer:

1. Identify the topics a typical ROS2 control node subscribes to (sensor feedback) and publishes (actuator commands). (5 marks)
2. Explain how the control loop rate is managed using `create_timer()` and why deterministic timing is important for controller stability. (8 marks)
3. Compare timer-based control loops (fixed-rate callbacks) versus event-driven control loops (triggered on new sensor data). Discuss the advantages and disadvantages of each approach in terms of latency, jitter, and computational load. (12 marks)

10.7.14.2 Model Answer — Q1

Part (a): A typical ROS2 control node subscribes to state-feedback topics such as `/odom` (nav_msgs/Odometry) for position and velocity, `/imu` (sensor_msgs/Imu) for angular velocity and linear acceleration, and `/joint_states` (sensor_msgs/JointState) for joint-level feedback. It publishes actuator commands on topics like `/cmd_vel` (geometry_msgs/Twist), `/motor_cmd` (std_msgs/Float64MultiArray), or `/joint_commands`.

Part (b): The control loop rate is managed by calling `self.create_timer(period_sec, callback)` in the node constructor. For example, `self.create_timer(0.01, self.control_callback)` creates a 100 Hz control loop. Deterministic timing is critical because the discrete-time controller gains (Kp, Ki, Kd) are tuned for a specific sample period \(\Delta t\). Jitter in the control period effectively changes the derivative and integral computations, potentially causing instability. The integral term accumulates error multiplied by \(\Delta t\): if \(\Delta t\) varies unpredictably, the integral grows incorrectly. For derivative action, \(u_d = K_d \cdot (e_k - e_{k-1})/\Delta t \), inconsistent \(\Delta t\) amplifies noise.

Part (c):

Aspect	Timer-Based (Fixed-Rate)	Event-Driven (Sensor-Triggere
Timing	Fixed period (e.g., 100 Hz), decoupled from sensor rate	Fires when new sensor messag
Latency	Average latency = half the control period; worst case = full period	Minimal latency — processes
Jitter	Low jitter if OS scheduling is adequate; consistent \(\Delta t\)	Jitter depends on sensor timin
Computational load	Predictable, evenly spaced CPU usage	Bursty — may overload if mul
Best for	Inner-loop controllers needing guaranteed rate (PID, LQR)	High-level planners, vision-bas

In practice, most flight controllers (PX4, ArduPilot) use timer-based loops for attitude and rate control, with event-driven callbacks only for updating the latest sensor data into shared buffers that the timer callback reads.

10.7.14.3 Question 2 (25 marks)

You implemented PID, LQR, and MPC as separate ROS2 control nodes in Week 9. Compare their computational requirements per control cycle by answering the following:

1. List the main mathematical operations each controller performs per cycle (e.g., matrix multiply, quadratic program solve). (9 marks)
2. Estimate the approximate floating-point operation (FLOP) count per cycle for a system with \(n = 12\) states and \(m = 4\) inputs. (8 marks)
3. Which of the three controllers requires hard real-time guarantees and why? What ROS2 mechanisms (executor type, QoS settings) help achieve this? (8 marks)

10.7.14.4 Model Answer — Q2

Part (a):

- PID: Per axis: 1 subtraction (error), 1 multiply + 1 add (P term), 1 multiply + 1 add (I accumulation), 1 subtraction + 1 division (D term), 1 addition (sum). For 4 axes: ~40 FLOPs total. Constant time O(1).
- LQR: One matrix-vector multiplication \(\mathbf{u} = -K(\mathbf{x} - \mathbf{x}_d) \) where \(K \in \mathbb{R}^{4 \times 12}\). This is \(4 \times 12 = 48\) multiplies and \(4 \times 11 = 44\) additions, plus 12 subtractions for the error vector. Total: ~104 FLOPs. Also O(1) since K is precomputed.
- MPC: At each cycle, solves a constrained quadratic program (QP). For a horizon \(N = 20\) with 12 states and 4 inputs, the QP has \(20 \times 4 = 80\) decision variables and \(20 \times (12 + 4 \times 2) = 400\) constraints. An interior-point solver typically requires 10–30 iterations, each involving a \(80 \times 80\) matrix factorization. Approximate cost: \(O(N^3 m^3)\) per solve, roughly \(10^4\)–\(10^6\) FLOPs.

Part (b):

Controller	FLOPs / Cycle	Typical Time on ARM Cortex
PID	~40	< 1 s
LQR	~104	~2 s
MPC (N=20)	~10^5	1–50 ms

Part (c): PID and LQR both require hard real-time guarantees because they run in the inner attitude/rate loop at 250–1000 Hz. Missing a deadline causes immediate instability in an airborne vehicle. MPC typically runs at 10–50 Hz in the outer position loop and can tolerate slight deadline misses (the previous solution is applied if the new one is late). In ROS2, real-time is supported by: (1) using a SingleThreadedExecutor on a dedicated thread with SCHED_FIFO priority, (2) setting QoS to `BEST_EFFORT` reliability with small queue depths to avoid blocking, (3) pre-allocating all memory to avoid heap allocation in the control loop, and (4) using the `rclcpp` real-time-safe utilities or the `realtime_tools` package.

AeroVision UAV — Flight Software Engineer Technical Challenge

Pre-Interview Assessment Autopilot Development Team

Confidential — Do not distribute without authorization

11 ROS 2 Navigation Control (Week 11)

Build Costmap, A* Planner, and Path Follower from LiDAR and Odometry Data

Position: Navigation Engineer Team: Autonomous Navigation
Pre-Interview Technical Assessment Estimated Time: 3 Hours

11.1 Overview

11.1.1 Company Brief: AutoNav Solutions

Scenario: AutoNav Solutions develops autonomous mobile robot platforms for warehouse and logistics environments. Your team has collected 30 seconds of navigation sensor data from a robot operating in a structured environment, including a static map, laser scans, and wheel odometry. As part of this challenge, you will build a complete navigation pipeline from scratch.

11.1.2 Provided Bag Data

Topic	Message Type	Rate	Description
`/map`	nav_msgs/OccupancyGrid	Latched	Static occupancy grid map of the environment
`/odom`	nav_msgs/Odometry	20 Hz	Wheel odometry (position, orientation, velocity)
`/scan`	sensor_msgs/LaserScan	5 Hz	2D laser range finder data

11.1.3 Deliverables

- Costmap generation with static, obstacle, and inflation layers from the provided map and laser data
- A* global path planner operating on the costmap
- A path-following controller (pure pursuit or DWA) as a ROS2 node

11.1 Overview 207
11.1.1 Company Brief: AutoNav Solutions . 207
11.1.2 Provided Bag Data . 207
11.1.3 Deliverables 207
11.1.4 Evaluation Criteria . 208
11.1.5 Skills You Will Demonstrate 208
11.1.6 Recommended Approach (3-Hour Session) 208
11.2 The ROS2 Navigation Stack (Nav2) 210
11.2.1 Nav2 Core Components 210
11.2.2 Nav2 Architecture Diagram 211
11.2.3 Key Nav2 ROS2 Topics 211
11.3 DWA: The Local Controller as Optimization 211
11.3.1 DWA Velocity Space 211
11.3.2 Dynamic Window Constraint 211
11.3.3 DWA Objective Function 212
11.3.4 Connection to MPC (Week 3) 212
11.3.5 ROS2 Implementation — Simplified DWA Local Planner Node 213
11.4 AMCL: State Estimation for Control . . . 214
11.4.1 Connection to Nonlinear Optimization (Week 8) 214
11.4.2 AMCL Algorithm . . 215
11.4.3 AMCL Cycle Diagram 216
11.4.4 ROS2 AMCL Configuration 216
11.5 Costmap: Constraints for Control 216
11.5.1 Costmaps as Control Constraints 216
11.5.2 Costmap Layers . . . 217

- A particle filter (AMCL-style) localisation node fusing odometry and laser scan data
- A quantitative comparison of path quality, tracking error, and localisation accuracy

11.1.4 Evaluation Criteria

- Navigation Architecture: Proper separation of planning, control, and localisation components
- Algorithm Correctness: Valid A* implementation, correct costmap inflation, stable particle filter
- Integration: Components working together through ROS2 topics and action servers
- Communication: Ability to explain Nav2 architecture and compare DWA to MPC during the follow-up interview

11.1.5 Skills You Will Demonstrate

By the end of this session, you will be able to:

- Describe the ROS2 Nav2 architecture and explain how global planner, local controller, costmap, AMCL, and behavior tree components interact through ROS2 topics and action servers.
- Configure costmap layers (static, obstacle, inflation) and explain how the inflation cost function creates soft constraints analogous to MPC barrier functions.
- Analyse the Dynamic Window Approach (DWA) as an optimization-based local controller and compare its cost function to the MPC formulation studied in Week 3.
- Explain how AMCL implements a particle filter for robot localization, connecting it to the controller-estimator pairing required by the NMPC / iLQR framework of Week 8.
- Design and interpret behavior trees for mission-level navigation control, comparing them with finite state machines.
- Integrate PID, LQR, and MPC controllers with the Nav2 navigation stack as local controller plugins.
- Compare the suitability of different control approaches (PID, MPC, SMC, Backstepping, Robust, Adaptive) for various mobile robot navigation scenarios.
- Configure, launch, and tune Nav2 for TurtleBot3 simulation including AMCL localization, DWA local planning, and waypoint navigation.

11.1.6 Recommended Approach (3-Hour Session)

0:00 – 0:20

11.1.6.1 Week 10 Exam Recall & Introduction

Review quadrotor control exam questions. Introduce mobile robot navigation as the next control application domain.

0:20 – 0:50

11.1.6.2 Nav2 Architecture Overview

Global planner, local controller, costmaps, AMCL, behavior trees, recovery behaviors. Key ROS2 topics and message types.

0:50 – 1:20

11.1.6.3 DWA — The Local Controller as Optimization

Deep dive into the Dynamic Window Approach. Connection to MPC. Simplified DWA implementation in ROS2.

1:20 – 1:35

11.1.6.4 Break

15-minute break.

1:35 – 1:55

11.1.6.5 AMCL — State Estimation for Control

Particle filter localization. Connection to Week 8's controller-estimator pairing for NMPC. ROS2 AMCL configuration.

1:55 – 2:10

11.1.6.6 Costmaps as Constraints & Part 5: Behavior Trees

Costmap inflation as soft constraints. Behavior tree navigation orchestration.

2:10 – 2:25

11.1.6.7 Applying Control Theory to Navigation

Mapping all 8 control methods from the course to mobile robot navigation scenarios.

2:25 – 3:00

11.1.6.8 Hands-On Activities & Lab Exercises

Configure Nav2 for TurtleBot3. Tune DWA. Compare DWA vs MPC. Waypoint navigation with recovery.

11.2 The ROS2 Navigation Stack (Nav2)

Nav2 is the standard ROS2 navigation framework for autonomous mobile robots. It provides a complete, modular pipeline: from a goal pose, through path planning and obstacle avoidance, to velocity commands sent to the robot's motors. Every component is a ROS2 lifecycle node that can be configured, replaced, or extended via plugins.

11.2.1 Nav2 Core Components

11.2.1.1 Global Planner

Computes a collision-free path from the robot's current position to the goal on the global costmap. Plugins include NavFn (Dijkstra/A*), SmacPlanner (State Lattice, Hybrid-A*), and ThetaStar. Operates on the full map at ~1 Hz.

11.2.1.2 Local Controller

Follows the global path while avoiding local obstacles. Runs at 10–20 Hz on the local costmap. Plugins: DWA (Dynamic Window Approach), MPPI (Model Predictive Path Integral), Regulated Pure Pursuit, Rotation Shim.

11.2.1.3 Costmap 2D

Layered 2D cost grid. Static layer from the SLAM map, obstacle layer from live sensor data (LiDAR, depth cameras), inflation layer providing safety buffers with exponential cost decay around obstacles.

11.2.1.4 AMCL Localization

Adaptive Monte Carlo Localization — a particle filter that estimates the robot's pose on the map using laser scan data and odometry. Publishes the `map → odom` transform.

11.2.1.5 Behavior Tree Navigator

Orchestrates the entire navigation pipeline using behavior trees (BehaviorTree.CPP). Handles sequencing of planning, control, and recovery. More flexible and maintainable than finite state machines.

11.2.1.6 Recovery Behaviors

Activated when the robot gets stuck. Spin: rotate in place to clear local costmap. Backup: drive backwards. Wait: pause and hope dynamic obstacles move. ClearCostmap: reset costmap layers.

11.2.2 Nav2 Architecture Diagram

11.2.3 Key Nav2 ROS2 Topics

Topic	Message Type	Publisher	Purpose
`/map`	`nav_msgs/OccupancyGrid`	Map Server	Static occupancy grid from SLAM. Values:
`/scan`	`sensor_msgs/LaserScan`	LiDAR driver	2D laser range data. Used by AMCL for loc
`/odom`	`nav_msgs/Odometry`	Wheel encoders	Robot odometry (pose + twist). Frame: odo
`/cmd_vel`	`geometry_msgs/Twist`	Local Controller	Velocity command: `linear.x` (m/s), `angul`
`/global_costmap/costmap`	`nav_msgs/OccupancyGrid`	Costmap 2D	Full-map costmap used by global planner. U
`/local_costmap/costmap`	`nav_msgs/OccupancyGrid`	Costmap 2D	Rolling-window costmap centred on robot. U
`/plan`	`nav_msgs/Path`	Global Planner	Sequence of poses from current position to g
`/tf`, `/tf_static`	`tf2_msgs/TFMessage`	Various	Transform tree: map odom base_link sen

Control Theory Connection: The Nav2 architecture implements a hierarchical control structure analogous to the cascaded quadrotor controller from Week 10. The behavior tree is the mission-level controller, the global planner is the trajectory generator, and the local controller is the real-time feedback loop — each operating at progressively faster rates.

11.3 DWA: The Local Controller as Optimization

The Dynamic Window Approach (DWA) is one of the most widely-used local planners in mobile robotics. From a control theory perspective, DWA is essentially a simplified Model Predictive Controller with a one-step prediction horizon. Understanding this connection bridges the gap between classical control optimization (Week 3) and practical robot navigation.

11.3.1 DWA Velocity Space

A differential-drive robot is controlled by two inputs: linear velocity \(v\) and angular velocity \(\omega\). DWA operates in this 2D velocity space:

Velocity Space: \((v, \omega) \) where \(v \in [v_{\min}, v_{\max}] \), \(\omega \in [\omega_{\min}, \omega_{\max}] \)

11.3.2 Dynamic Window Constraint

Not all velocities in the admissible range are physically reachable from the current state within one control interval \(\Delta t\). The dynamic window restricts the search to reachable velocities given acceleration limits:

\[V_d = \{ (v, \omega) \mid v \in [v_c - \dot{v}_{\max}\Delta t,\; v_c + \dot{v}_{\max}\Delta t], \; \omega \in [\omega_c -

\dot{\omega}_{\max}\Delta t,\; \omega_c + \dot{\omega}_{\max}\Delta t] \} \]

where \((v_c, \omega_c)\) is the current velocity and \((\dot{v}_{\max}, \dot{\omega}_{\max})\) are the maximum accelerations.

The admissible velocities are further constrained to those that allow the robot to stop before hitting an obstacle:

\[V_a = \{ (v, \omega) \mid v \leq \sqrt{2 \cdot \text{dist}(v,\omega) \cdot \dot{v}_{\max}},\; \omega \leq \sqrt{2 \cdot \text{dist}(v,\omega) \cdot \dot{\omega}_{\max}} \} \]

11.3.3 DWA Objective Function

DWA selects the velocity pair \((v^*, \omega^*)\) that maximises a weighted objective:

\[G(v, \omega) = \alpha \cdot \text{heading}(v,\omega) + \beta \cdot \text{clearance}(v,\omega) + \gamma \cdot \text{velocity}(v,\omega) \]

where:

- heading(\(v, \omega\)): Measures alignment of the predicted trajectory's endpoint with the goal direction. Defined as \(180^\circ - |\theta_{\text{goal}} - \theta_{\text{endpoint}}|\), normalised to [0, 1]. Encourages the robot to face the goal.
- clearance(\(v, \omega\)): Minimum distance from the predicted trajectory to the nearest obstacle. Larger clearance = safer trajectory. Returns \(-\infty\) for collision trajectories (hard constraint).
- velocity(\(v, \omega\)): Forward speed \(v\) of the candidate, normalised to [0, 1]. Encourages faster progress toward the goal rather than overly cautious creeping.

11.3.4 Connection to MPC (Week 3)

Key Insight: DWA can be viewed as MPC with horizon \(N = 1\), state = robot pose, control = \((v, \omega)\), cost = negative of heading + clearance + velocity, and constraints = dynamic window + collision-free. The modern Nav2 MPPI (Model Predictive Path Integral) controller IS full MPC with \(N \gg 1\).

DWA Cost (1-step optimisation):

\[\min_{v, \omega} -\bigl[\alpha \cdot h(v,\omega) + \beta \cdot c(v,\omega) + \gamma \cdot v\bigr] \quad \text{s.t.} \quad (v,\omega) \in V_d \cap V_a \]

MPC Cost (N-step optimisation, Week 3):

\[\min_{\mathbf{u}_{0:N-1}} \sum_{k=0}^{N-1} \bigl[\mathbf{x}_k^T \mathbf{Q} \mathbf{x}_k + \mathbf{u}_k^T \mathbf{R}

\mathbf{u}_k\bigr] + \mathbf{x}_N^T \mathbf{P} \mathbf{x}_N
\quad \text{s.t. model, constraints} \]

Aspect	DWA	MPC
Prediction Horizon	\(N = 1\) (or short trajectory)	\(N \gg 1\) (10–50 steps)
Decision Variables	\((v, \omega)\) — 2 scalars	\(\mathbf{u}_{0:N-1}\) — \(2N\) variables
Optimisation Method	Grid search over sampled velocities	QP solver or sampling (MPPI)
Constraint Handling	Dynamic window + collision check	Explicit inequality constraints
Computational Cost	Low (~1 ms)	Higher (~5–50 ms)
Anticipation	Myopic (reacts to immediate surroundings)	Looks ahead (anticipates obstacles, turns)

11.3.5 ROS2 Implementation — Simplified DWA Local Planner Node

#!/usr/bin/env python3 """Simplified DWA local planner node for ROS2. Subscribes to /odom and /plan, publishes /cmd_vel. Demonstrates DWA as optimization-based control."""
import rclpy from rclpy.node import Node import numpy as np import math from nav_msgs.msg import Odometry, Path from geometry_msgs.msg import Twist from sensor_msgs.msg import LaserScan class DWAControllerNode(Node): def __init__(self): super().__init__('dwa_controller') # DWA parameters self.max_v = 0.5; self.min_v = 0.0 self.max_w = 1.0; self.max_accel = 0.5; self.max_w_accel = 1.0 self.v_res = 0.05; self.w_res = 0.1 self.dt = 0.1; self.predict_time = 2.0 self.alpha = 1.0; self.beta = 0.8; self.gamma = 0.5 # heading, clearance, velocity self.robot_radius = 0.18 # State self.x, self.y, self.th = 0.0, 0.0, 0.0 self.cur_v, self.cur_w = 0.0, 0.0 self.goal = None; self.obstacles = [] # ROS2 interfaces self.create_subscription(Odometry, '/odom', self.odom_cb, 10) self.create_subscription(Path, '/plan', self.path_cb, 10) self.create_subscription(LaserScan, '/scan', self.scan_cb, 10) self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10) self.timer = self.create_timer(0.1, self.control_loop) # 10 Hz self.get_logger().info('DWA Controller started') def odom_cb(self, msg): self.x = msg.pose.pose.position.x self.y = msg.pose.pose.position.y q = msg.pose.pose.orientation self.th = 2.0 * math.atan2(q.z, q.w) self.cur_v = msg.twist.twist.linear.x self.cur_w = msg.twist.twist.angular.z def path_cb(self, msg): if msg.poses: p = msg.poses[-1].pose.position # Use final pose as goal self.goal = (p.x, p.y) def scan_cb(self, msg): obs = [] for i, r in enumerate(msg.ranges): if msg.range_min < r < msg.range_max: angle = msg.angle_min + i * msg.angle_increment ox = self.x + r * math.cos(self.th + angle) oy = self.y + r * math.sin(self.th + angle) obs.append((ox, oy)) self.obstacles = obs def predict_trajectory(self, v, w): """Forward-simulate trajectory for predict_time seconds."""
x, y, th = self.x, self.y, self.th traj = [(x, y)] for _ in range(int(self.predict_time / self.dt)): x += v * math.cos(th) * self.dt y += v * math.sin(th) * self.dt th += w * self.dt traj.append((x, y)) return traj, th def score_trajectory(self, traj, final_th): """Score: alpha*heading + beta*clearance + gamma*velocity."""
if not self.goal: return -float('inf') ex, ey = traj[-1] # Heading: alignment with goal goal_angle = math.atan2(self.goal[1] - ey, self.goal[0] - ex) heading = math.pi - abs(goal_angle - final_th) heading = heading / math.pi # normalise to

[0, 1] # Clearance: min distance to obstacles clearance = float('inf') for (px, py) in traj: for (ox, oy) in self.obstacles: d = math.hypot(px - ox, py - oy) clearance = min(clearance, d) if clearance < self.robot_radius: return -float('inf') # Collision! clearance = min(clearance, 3.0) / 3.0 # normalise # Velocity: prefer faster velocity = abs(traj[-1][0] - traj[0][0]) + abs(traj[-1][1] - traj[0][1]) velocity = min(velocity, 1.0) return self.alpha * heading + self.beta * clearance + self.gamma * velocity def control_loop(self): if self.goal is None: return if math.hypot(self.x - self.goal[0], self.y - self.goal[1]) < 0.2: self.cmd_pub.publish(Twist()); return # Goal reached # Dynamic window v_lo = max(self.min_v, self.cur_v - self.max_accel * self.dt) v_hi = min(self.max_v, self.cur_v + self.max_accel * self.dt) w_lo = max(-self.max_w, self.cur_w - self.max_w_accel * self.dt) w_hi = min(self.max_w, self.cur_w + self.max_w_accel * self.dt) # Sample and score best_v, best_w, best_score = 0.0, 0.0, -float('inf') v = v_lo while v <= v_hi: w = w_lo while w <= w_hi: traj, fth = self.predict_trajectory(v, w) s = self.score_trajectory(traj, fth) if s > best_score: best_score, best_v, best_w = s, v, w w += self.w_res v += self.v_res # Publish best command cmd = Twist() cmd.linear.x = best_v; cmd.angular.z = best_w self.cmd_pub.publish(cmd) def main(args=None): rclpy.init(args=args) rclpy.spin(DWAControllerNode()) rclpy.shutdown() if __name__ == '__main__': main()

Tuning Guide: The weights \(\alpha, \beta, \gamma\) dramatically affect robot behavior. High \(\alpha\) = aggressive goal-seeking (may clip obstacles). High \(\beta\) = overly cautious (slow progress). High \(\gamma\) = fast but may overshoot. Start with \(\alpha = 1.0, \beta = 1.0, \gamma = 0.5\) and adjust.

11.4 AMCL: State Estimation for Control

11.4.1 Connection to Nonlinear Optimization (Week 8)

Week 8 developed nonlinear control optimization for robotic systems: iLQR/NMPC for trajectory optimization, and per-controller optimization of SMC / adaptive / robust / backstepping gains. Every controller in that framework assumes access to the state \(x\) — but in a real robot the state must be estimated from sensors. Controller-plus-estimator is the universal structure:

11.4.1.1 Week 8 NMPC / iLQR

- Controller: NMPC via iLQR (optimal nonlinear control, local feedback policy)
- Estimator: matched to the dynamics — EKF/UKF if the posterior is unimodal, particle filter if multimodal
- State: full nonlinear state \(x\in\mathbb R^n\)
- Sensors: any; noise model enters the estimator design, not the controller

11.4.1.2 Nav2 Navigation

- Controller: DWA / MPPI / Pure Pursuit (all are finite-horizon optimizers — DWA is a one-step discrete search; MPPI is sampling-based NMPC)
- Estimator: AMCL (particle filter)
- State: Robot pose (x, y, θ) on a map
- Sensors: Laser scans + odometry (non-Gaussian, multimodal posterior)

Why not a Kalman Filter? The Kalman filter assumes Gaussian distributions and linear (or locally linear) models. In mobile robot localization, the probability distribution over poses can be multimodal (the robot could be in several places on a symmetric map). Particle filters handle arbitrary distributions, making AMCL more suitable for global localization.

11.4.2 AMCL Algorithm

AMCL represents the belief over the robot's pose as a set of weighted particles:

Particle Set: $\mathcal{X} = \{(x_i, y_i, \theta_i, w_i)\}_{i=1}^{N}$

where each particle i has pose (x_i, y_i, θ_i) and weight w_i with $\sum_{i=1}^{N} w_i = 1$.

The AMCL algorithm cycles through four steps:

1. Motion Update (Prediction): Propagate each particle using the odometry motion model with added noise: $$x_i' = x_i + \delta_{\text{trans}} \cos(\theta_i + \delta_{\text{rot1}}) + \epsilon_x$$ $$y_i' = y_i + \delta_{\text{trans}} \sin(\theta_i + \delta_{\text{rot1}}) + \epsilon_y$$ $$\theta_i' = \theta_i + \delta_{\text{rot1}} + \delta_{\text{rot2}} + \epsilon_\theta$$ where ϵ terms are sampled from noise distributions parameterised by odometry noise coefficients $(\alpha_1, \alpha_2, \alpha_3, \alpha_4)$.
2. Measurement Update (Correction): Weight each particle by the likelihood of the observed laser scan given that particle's pose. Using the likelihood field model: $$w_i = \prod_{j \in \text{beams}} \bigl[z_{\text{hit}} \cdot \mathcal{N}(d_j; 0, \sigma^2) + z_{\text{random}} \cdot \tfrac{1}{z_{\max}} \bigr]$$ where d_j is the distance from the beam endpoint to the nearest obstacle in the map.
3. Normalisation: $w_i \leftarrow w_i / \sum_{k=1}^{N} w_k$
4. Resampling: When the effective particle count $N_{\text{eff}} = 1/\sum w_i^2$ drops below a threshold, perform low-variance resampling: draw a new particle set where particles with higher weights are more likely to be selected.

11.4.3 AMCL Cycle Diagram

11.4.4 ROS2 AMCL Configuration

Launch AMCL for TurtleBot3 localization ros2 launch nav2_bringup localization_launch.py \ map:=/path/to/map.yaml \ params_file:=/path/-to/nav2_params.yaml

Key AMCL parameters in `nav2_params.yaml`:

AMCL Parameters (nav2_params.yaml)

amcl: ros__parameters: min_particles: 500 # Minimum particle count max_particles: 2000 # Maximum particle count (adaptive) update_-min_d: 0.25 # Min translation (m) before update update_min_a: 0.2 # Min rotation (rad) before update laser_model_type: "likelihood_-field" laser_max_range: 12.0 laser_min_range: 0.1 odom_model_-type: "diff-corrected" # Odometry noise parameters (alpha1-alpha4) robot_model_type: "differential" alpha1: 0.2 # Rotation noise from rotation alpha2: 0.2 # Rotation noise from translation alpha3: 0.2 # Translation noise from translation alpha4: 0.2 # Translation noise from rotation # Likelihood field parameters z_hit: 0.95 z_rand: 0.05 sigma_hit: 0.2 # Recovery recovery_alpha_slow: 0.001 # Slow average weight decay recovery_alpha_fast: 0.1 # Fast average weight decay

11.5 Costmap: Constraints for Control

In control theory, constraints define the safe operating region. In Nav2, costmaps serve exactly this purpose: they encode where the robot can and cannot go, with graduated costs creating soft boundaries.

11.5.1 Costmaps as Control Constraints

11.5.1.1 Global Costmap Terminal Constraint

The global costmap constrains the global planner, analogous to the terminal constraint in MPC that ensures the planned trajectory ends in a safe set. It covers the entire known map and updates slowly (~1 Hz).

11.5.1.2 Local Costmap Stage Constraints

The local costmap constrains the DWA/MPPI local controller, analogous to stage constraints in MPC \((\mathbf{x}_k \in \mathcal{X}, \mathbf{u}_k \in \mathcal{U})\) that enforce safety at each time step. Rolling window, updated at 5–20 Hz.

11.5.1.3 Inflation Layer Soft Constraints

The inflation layer creates graduated costs around obstacles, analogous to barrier functions or soft constraints with penalty terms. The robot is not forbidden from approaching obstacles but incurs increasing cost.

11.5.2 Costmap Layers

1. Static Layer: Loaded from the SLAM-generated map. Contains walls, permanent furniture, and other fixed obstacles. Values: 0 (free), 100 (occupied), -1 (unknown).
2. Obstacle Layer: Populated from live sensor data (LiDAR, depth cameras). Detects dynamic obstacles not in the static map — people, other robots, moved furniture. Marks cells as lethal (254) based on sensor readings.
3. Inflation Layer: Expands obstacles by the robot's inscribed radius and adds exponentially-decaying cost beyond that. This is the critical safety layer.

11.5.3 Inflation Cost Function

\[c(d) = \begin{cases} 254 \;\text{(LETHAL)} & \text{if } d < r_{\text{inscribed}} \\[6pt] A \cdot e^{-\beta \,(d - r_{\text{inscribed}})} & \text{if } r_{\text{inscribed}} \leq d \leq r_{\text{inflation}} \\[6pt] 0 \;\text{(FREE)} & \text{if } d > r_{\text{inflation}} \end{cases} \]

where:

- \(d\) = distance from the cell to the nearest obstacle
- \(r_{\text{inscribed}}\) = inscribed radius of the robot footprint (if the robot centre is closer than this, collision is guaranteed)
- \(r_{\text{inflation}}\) = outer inflation radius (beyond this, cost is zero)
- \(A = 253\) (INSCRIBED cost value, just below LETHAL)
- \(\beta\) = cost scaling factor controlling how quickly cost decays with distance

11.5.4 Nav2 Costmap Configuration

Costmap YAML Configuration (nav2_params.yaml)

global_costmap: global_costmap: ros__parameters: update_frequency: 1.0 publish_frequency: 1.0 global_frame: map robot_base_frame: base_link resolution: 0.05 # 5cm per cell track_unknown_space: true plugins: ["static_layer", "obstacle_layer", "inflation_layer"] static_layer: plugin: "nav2_costmap_2d::StaticLayer" map_subscribe_transitioned_to_active: true obstacle_layer: plugin: "nav2_costmap_2d::ObstacleLayer" enabled: true observation_sources: scan scan: topic: /scan max_obstacle_height: 2.0 clearing: true marking: true raytrace_max_range: 3.0 obstacle_max_range: 2.5 inflation_layer: plugin: "nav2_costmap_2d::InflationLayer" cost_scaling_factor: 5.0 # beta in the formula inflation_radius: 0.55 # r_inflation in meters local_costmap: local_costmap: ros__parameters: update_frequency: 5.0 publish_frequency: 2.0 global_frame: odom robot_base_frame: base_link rolling_window: true width: 3 # 3m x 3m window height: 3 resolution: 0.05 plugins: ["obstacle_layer", "inflation_layer"] obstacle_layer: plugin: "nav2_costmap_2d::ObstacleLayer" enabled: true observation_sources: scan scan: topic: /scan max_obstacle_height: 2.0

clearing: true marking: true inflation_layer: plugin: "nav2_costmap_-2d::InflationLayer" cost_scaling_factor: 5.0 inflation_radius: 0.55

11.6 Behavior Trees for Mission Control

11.6.1 Why Behavior Trees?

Behavior Trees (BTs) are an alternative to Finite State Machines (FSMs) for orchestrating complex robot behaviors. Nav2 uses the BehaviorTree.CPP library.

Aspect	Finite State Machine	Behavior Tree
Structure	States + transitions (graph)	Tree of nodes (hierarchical)
Scalability	Exponential transitions as states grow	Modular subtrees, easy to exten
Reactivity	Must explicitly handle every transition	Built-in: tick propagation handl
Reusability	Low (states are context-dependent)	High (subtrees are self-containe
Debugging	Hard with many states	Visual tools (Groot) show live t

11.6.2 BT Node Types

- Sequence (): Execute children left-to-right. Returns SUCCESS if all succeed, FAILURE if any fails, RUNNING if a child is still executing.
- Fallback (?): Try children left-to-right until one succeeds. Returns SUCCESS on first success, FAILURE if all fail. Used for recovery.
- Action: Leaf node that performs work (e.g., ComputePath, FollowPath, Spin, Backup).
- Condition: Leaf node that checks state (e.g., IsGoalReached, IsBatteryLow). Returns SUCCESS or FAILURE.
- Decorator: Modifies a child's behavior (e.g., Inverter, Retry, Timeout).

11.6.3 Navigation Behavior Tree Diagram

11.6.4 Nav2 BT XML Configuration

Default Nav2 Behavior Tree XML (navigate_to_pose_w_replanning.xml)

<root main_tree_to_execute="MainTree"> <BehaviorTree ID="MainTree"> <RecoveryNode number_of_retries="6" name="NavigateRecovery"> <PipelineSequence name="NavigateWithReplanning"> <RateController hz="1.0"> <ComputePathToPose goal="{goal}" path="{path}" planner_id="GridBased"/> </RateController> <FollowPath path="{path}" controller_id="FollowPath"/> </PipelineSequence> <SequenceStar name="RecoveryActions"> <ClearEntireCostmap name="ClearLocalCostmap" service_name="/local_costmap/clear"/> <ClearEntireCostmap name="ClearGlobalCostmap" service_name="/global_-costmap/clear"/> <Spin spin_dist="1.57"/> <Wait wait_duration="5"/>

<BackUp backup_dist="0.3" backup_speed="0.05"/> </SequenceStar> </RecoveryNode> </BehaviorTree> </root>

11.7 Applying Control Theory to Navigation

Throughout Weeks 1–8, you studied eight control methods in the context of general dynamical systems. Here we show how every one of them maps directly to a mobile robot navigation scenario.

Controller	Navigation Role	Example Scenario
PID	Pure pursuit path follower	Basic line following
Computed Torque	Model-based path tracking	Known-dynamics mobile robot
MPC	MPPI local planner	Nav2 MPPI controller plugin
Adaptive	Terrain-adaptive navigation	Unknown friction, slopes
Robust (H-infinity)	All-weather navigation	Rain, snow, uneven ground
Backstepping	Nonholonomic path tracking	Parking, tight turns
SMC	Rough terrain tracking	Off-road, disturbance-heavy
LQR	Highway/corridor following	Straight, smooth paths

11.7.1 Detailed Mapping

11.7.1.1 PID Pure Pursuit Path Follower

The simplest approach: a PID controller on the cross-track error (distance from the path) and heading error. The proportional term steers toward the path, the integral corrects for systematic drift, and the derivative damps oscillations. Used in Nav2's Regulated Pure Pursuit plugin where the lookahead distance acts as an effective proportional gain.

11.7.1.2 Computed Torque Model-Based Tracking

When the robot's kinematic or dynamic model is well-known (mass, inertia, wheel-ground friction), computed torque control cancels the nonlinear dynamics and applies a linear outer loop. For a differential-drive robot, this means compensating for the nonholonomic constraint \(\dot{x}\sin\theta - \dot{y}\cos\theta = 0\) directly in the control law.

11.7.1.3 MPC MPPI Local Planner

Nav2's MPPI (Model Predictive Path Integral) controller is a direct implementation of MPC using sampling-based optimization. It rolls out thousands of trajectories over a prediction horizon \(N\), evaluates a cost function (path following + obstacle avoidance + smoothness), and selects the optimal control sequence. This is the most capable Nav2 controller for complex environments.

11.7.1.4 Adaptive Terrain-Adaptive Navigation

When a robot moves from smooth tile to carpet to gravel, the effective friction and slip change unpredictably. An adaptive controller estimates these parameters online and adjusts the velocity/steering gains accordingly. This prevents the robot from overshooting on low-friction surfaces or responding sluggishly on high-friction ones.

11.7.1.5 Robust (H-infinity) All-Weather Navigation

Robust control guarantees performance despite bounded uncertainties. For navigation, this means maintaining path-tracking accuracy despite sensor noise (rain degrading LiDAR), actuator uncertainty (wheel slip on wet surfaces), and model uncertainty (payload changes). The controller is designed for the worst-case disturbance scenario.

11.7.1.6 Backstepping Nonholonomic Path Tracking

Backstepping is particularly well-suited for nonholonomic systems like differential-drive robots, which cannot move sideways. The controller recursively stabilises the system by first stabilising the orientation error, then using that as a "virtual input" to stabilise the position error. Ideal for parking manoeuvres and tight turns where the nonholonomic constraint is most limiting.

11.7.1.7 SMC Rough Terrain Tracking

Sliding Mode Control provides robustness against unmatched disturbances through a discontinuous control law that forces the system onto a sliding surface. For mobile robots on rough terrain, SMC rejects disturbances from bumps, rocks, and uneven ground that would destabilise smoother controllers. The chattering issue can be mitigated with boundary layers.

11.7.1.8 LQR Highway/Corridor Following

LQR provides optimal control for linear systems (or linearised around a reference). In long, straight corridors or highways, the robot dynamics can be well-approximated as linear, and LQR provides the minimum-energy deviation from the centre-line. Often used as the local stabiliser within a larger nonlinear controller.

11.8 Summary and Next Steps

11.8.1 ROS2 Lab Exercises

This week's practical work aligns with the ROS2 Robotics Course Week 5: Navigation Stack (Nav2). The exercises are available in the course repository under `src/ros2_robotics_course/week_05/`.

11.8.2 Python Lab Exercises (Algorithmic Foundations)

Located in `week_05/lab_exercises/`. These standalone Python scripts implement the core algorithms underlying Nav2 without requiring a running ROS2 system.

Task	File	Description
Task 1	`task1_costmap.py`	Costmap generation with static, obstacle, and inflation layers. Implement `cr`
Task 2	`task2_astar.py`	A* pathfinding on an 8-connected grid. Implement `heuristic()`, `get_neig`
Task 3	`task3_dwa.py`	Dynamic Window Approach local planner. Implement `compute_dynamic_wi`
Task 4	`task4_particle_filter.py`	AMCL-style particle filter localization. Implement `initialize_particles(`
Task 5	`task5_behavior_tree.py`	Behavior tree framework. Implement `Sequence`, `Fallback`, `NavigateToPos`
Task 6	`task6_waypoint_navigation.py`	Waypoint follower using A* for global planning and DWA for local control.
Task 7	`task7_full_navigation.py`	Full integrated navigation combining all components: costmap, A*, DWA, p

11.8.3 ROS2 Applied Exercises (Node-Based)

Located in `week_05/w05_exercises/`. These implement the navigation components as proper ROS2 nodes using topics, services, and message types.

Exercise	File	Subscribes	Publishes
Exercise 1: Costmap Generator	`exercise1_node.py`	`/map` (OccupancyGrid)	`/costmap` (OccupancyGri
Exercise 2: A* Path Planner	`exercise2_node.py`	`/costmap` (OccupancyGrid)	`/planned_path` (Path)
Exercise 3: Path Follower (Pure Pursuit)	`exercise3_node.py`	`/planned_path`, `/odom`	`/cmd_vel` (Twist)

Running the ROS2 exercises # Terminal 1: Play synthetic navigation bag data ros2 bag play week_05/w05_exercises/bag_data/synthetic_-nav2/ # Terminal 2: Run the costmap generator ros2 run week_05 exercise1_node --ros-args \ --params-file week_05/w05_exercises/-config/params.yaml # Terminal 3: Run the path planner ros2 run week_05 exercise2_node --ros-args \ --params-file week_05/w05_exercises/config/params.yaml # Terminal 4: Run the path follower ros2 run week_05 exercise3_node --ros-args \ --params-file week_05/w05_-exercises/config/params.yaml

11.8.4 Hands-On Activities

11.8.5 Activity 1: Configure Nav2 for TurtleBot3 Navigation

Objective: Set up the full Nav2 stack for TurtleBot3 in Gazebo and navigate through a maze environment.

1. Launch the simulation:
   # Terminal 1: Launch TurtleBot3 in Gazebo maze world export TURTLEBOT3_MODEL=waffle ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py # Terminal 2: Launch Nav2 with default parameters ros2 launch nav2_bringup navigation_launch.py

\ params_file:=nav2_params.yaml # Terminal 3: Launch RViz2 with Nav2 config ros2 launch nav2_bringup rviz_launch.py

2. Configure AMCL: Set initial pose using the "2D Pose Estimate" tool in RViz. Observe the particle cloud converging as the robot moves.
3. Set navigation goals: Use the "2D Goal Pose" tool to send goals. Watch the global path (green) and the local controller following it.
4. Tune DWA parameters: Experiment with the following parameters and observe behavior changes:
   - Increase `max_vel_x` from 0.26 to 0.5 — does the robot navigate faster? Does it cut corners?
   - Increase `PathAlign.scale` (the \(\alpha\) heading weight) — does the robot follow the global path more closely?
   - Increase `GoalDist.scale` — does the robot approach the goal more aggressively?
   - Increase `ObstacleFootprint.scale` (the \(\beta\) clearance weight) — does the robot maintain more distance from walls?
5. Record observations: For three different parameter sets, record: (a) time to goal, (b) minimum obstacle distance, (c) path smoothness.

11.8.6 Activity 2: Compare DWA vs MPC for Navigation

Objective: Replace the default DWA controller with your MPC controller from Week 9 and compare performance across three scenarios.

1. Set up MPC controller plugin: Adapt your Week 9 MPC node to accept `/plan` (global path) and publish `/cmd_vel`. Use the differential-drive kinematic model: \[\dot{x} = v\cos\theta, \quad \dot{y} = v\sin\theta, \quad \dot{\theta} = \omega \]
2. Test Scenario A — Narrow Corridor: Navigate through a passage 0.8m wide. DWA should be cautious (high clearance weight). MPC should plan ahead through the corridor more smoothly.
3. Test Scenario B — Open Space: Navigate across a large open area. Both controllers should perform similarly, but MPC may be more energy-efficient due to optimal control effort.
4. Test Scenario C — Dynamic Obstacles: Add moving obstacles (other TurtleBots). DWA reacts quickly due to low computation. MPC anticipates obstacle motion if the prediction model includes it.
5. Record and compare:
   # Record rosbag during each run ros2 bag record /cmd_vel /odom /plan /local_costmap/costmap -o dwa_corridor ros2 bag record /cmd_vel /odom /plan /local_costmap/costmap -o mpc_corridor # Plot trajectories for comparison python3 plot_comparison.py dwa_corridor mpc_corridor
6. Analysis table: Fill in for each scenario:

Metric	DWA (Corridor)	MPC (Corridor)	DWA (Open)	MPC (C
Completion Time (s)				

Metric	DWA (Corridor)	MPC (Corridor)	DWA (Open)	MPC (Open)	DWA (Dynamic)	MPC (D
Min Obstacle Dist (m)						
Path Length (m)						
Avg |cmd_vel| (m/s)						
Computation Time (ms)						

11.8.7 Activity 3: Waypoint Navigation with Recovery Behaviors

Objective: Implement waypoint navigation using the Nav2 action server, visit 5 waypoints in order, and add recovery behavior when stuck.

1. Define waypoints: Choose 5 waypoints that require navigating through different parts of the environment (corridors, open areas, near obstacles).
2. Write the waypoint navigation client: #!/usr/bin/env python3 """Waypoint navigation client using Nav2 action server."""import rclpy from rclpy.node import Node from rclpy.action import ActionClient from nav2_msgs.action import NavigateToPose from geometry_msgs.msg import PoseStamped import time class WaypointNavigator(Node): def __init__(self): super().__init__('waypoint_navigator') self.nav_client = ActionClient(self, NavigateToPose, 'navigate_to_pose') self.waypoints = [(1.0, 0.5, 0.0), # (x, y, yaw) in map frame (2.0, 1.5, 1.57), (1.5, 3.0, 3.14), (0.0, 2.0, -1.57), (0.5, 0.5, 0.0),] self.current_wp = 0 self.max_retries = 3 self.nav_client.wait_for_server() self.get_logger().info('Nav2 server ready. Starting mission.') self.navigate_to_next() def create_goal(self, x, y, yaw): goal = NavigateToPose.Goal() goal.pose = PoseStamped() goal.pose.header.frame_id = 'map' goal.pose.header.stamp = self.get_clock().now().to_msg() goal.pose.pose.position.x = x goal.pose.pose.position.y = y goal.pose.pose.orientation.z = __import__('math').sin(yaw / 2) goal.pose.pose.orientation.w = __import__('math').cos(yaw / 2) return goal def navigate_to_next(self): if self.current_wp >= len(self.waypoints): self.get_logger().info('All waypoints reached! Mission complete.') return x, y, yaw = self.waypoints[self.current_wp] self.get_logger().info(f'Navigating to WP{self.current_wp}: ({x}, {y})') goal = self.create_goal(x, y, yaw) future = self.nav_client.send_goal_async(goal) future.add_done_callback(self.goal_response_cb) def goal_response_cb(self, future): goal_handle = future.result() if not goal_handle.accepted: self.get_logger().warn('Goal rejected!') return result_future = goal_handle.get_result_async() result_future.add_done_callback(self.result_cb) def result_cb(self, future): result = future.result() self.get_logger().info(f'WP{self.current_wp} result: {result.status}') self.current_wp += 1 self.navigate_to_next() def main(): rclpy.init() rclpy.spin(WaypointNavigator()) rclpy.shutdown()
3. Add recovery logic: If a waypoint fails (timeout after 60s or rejection), retry up to 3 times. If still failing, skip and move to the next waypoint. Log all failures.

4. Measure: Total completion time, number of waypoints reached, number of recoveries triggered.
5. Compare: Run with recovery behaviors enabled vs. disabled. How much does recovery improve the success rate?

11.8.7.1 Quick Quiz — Navigation and Mobile Robot Control

Q1: In the DWA objective function \(G(v,\omega) = \alpha \cdot \text{heading} + \beta \cdot \text{clearance} + \gamma \cdot \text{velocity}\), what happens if you set \(\beta = 0\) (zero clearance weight)?

11.8.7.2 Answer

The controller completely ignores obstacle proximity. It will select velocities that best align with the goal and maximise speed, regardless of nearby obstacles. This will almost certainly result in collisions. The clearance term is the primary safety mechanism in DWA — without it, the controller has no incentive to avoid obstacles.

Q2: Why does Nav2 use a particle filter (AMCL) rather than an Extended Kalman Filter (EKF) for localization on a known map?

11.8.7.3 Answer

The EKF assumes the posterior distribution is unimodal Gaussian, which fails for global localization on a map with symmetry (e.g., a long corridor looks the same from many positions). The particle filter represents the full multimodal distribution, allowing the robot to maintain multiple hypotheses about its location until sensor evidence disambiguates them. Additionally, the laser scan likelihood function is highly nonlinear and non-Gaussian, which the particle filter handles naturally.

Q3: How is the Nav2 behavior tree architecture analogous to the cascaded control architecture of a quadrotor?

11.8.7.4 Answer

Both implement hierarchical control with different levels operating at different rates. In the quadrotor: mission planner position controller attitude controller rate controller motors. In Nav2: behavior tree (mission) global planner (path) local controller (trajectory) cmd_vel (motors). Each level abstracts the complexity below it and operates at a progressively faster rate. Recovery behaviors in Nav2 are analogous to the attitude controller's ability to stabilise even when the position controller fails.

Q4: The inflation layer uses \(c(d) = A \cdot e^{-\beta(d - r_{\text{inscribed}})}\). If you double the cost_scaling_factor \(\beta\), what happens to the robot's navigation behavior?

11.8.7.5 Answer

Doubling \(\beta\) makes the exponential decay steeper, meaning the cost drops to zero much faster with distance from obstacles. The effective "buffer zone" around obstacles becomes narrower. The robot will navigate closer to walls and obstacles because the cost penalty for proximity is confined to a smaller region. This is useful in tight spaces but increases collision risk. Conversely, halving \(\beta\) creates wider buffer zones and more cautious navigation.

Q5: You need to navigate a mobile robot through a cluttered warehouse with moving forklifts. Which Nav2 local controller plugin would you choose: DWA, Regulated Pure Pursuit, or MPPI? Justify your choice.

11.8.7.6 Answer

MPPI is the best choice for this scenario. Reasoning: (1) Moving forklifts create a highly dynamic environment requiring the controller to anticipate obstacle motion, which MPPI can do through its prediction horizon \(N \gg 1\). (2) The cluttered warehouse has many narrow passages requiring lookahead to avoid dead-ends, which DWA's single-step horizon cannot provide. (3) MPPI's sampling-based approach naturally handles the nonlinear constraints of obstacle avoidance without requiring linearisation. (4) Regulated Pure Pursuit is designed for path following, not obstacle avoidance — it relies entirely on the global planner to route around obstacles, which cannot react to dynamic forklifts. The computational cost of MPPI (~5–20 ms) is acceptable for a warehouse robot operating at ~10 Hz.

11.8.8 Summary — Key Takeaways

11.8.8.1 Nav2 Architecture

Nav2 implements hierarchical navigation control: behavior tree (mission) global planner (A*/Dijkstra) local controller (DWA/MPPI) cmd_vel. Each component is a lifecycle node with plugin-based extensibility.

11.8.8.2 DWA = Simplified MPC

The Dynamic Window Approach is fundamentally a 1-step MPC: sample velocities in a dynamic window, forward-simulate, score trajectories with \(\alpha \cdot h + \beta \cdot c + \gamma \cdot v\), and select the best. Nav2's MPPI extends this to full N-step MPC.

11.8.8.3 AMCL = State Estimator

Just as Week 8's NMPC/iLQR framework assumes a matched state estimator, Nav2 pairs its controller with AMCL (particle filter). Both solve the same fundamental problem: you cannot control what you cannot observe. AMCL handles the multimodal, nonlinear nature of map-based localization — exactly where a Kalman filter fails.

11.8.8.4 Costmaps = Constraints

Costmap layers encode navigation constraints: static (walls), obstacle (dynamic), inflation (safety buffer). The inflation layer's exponential decay is analogous to MPC soft constraints or barrier functions.

11.8.8.5 Behavior Trees = Mission Control

BTs orchestrate planning, control, and recovery in a modular, extensible tree structure. More scalable than FSMs. The Nav2 default BT implements navigate-with-replanning and multi-stage recovery.

11.8.8.6 All Controllers Apply

Every control method from Weeks 1–8 maps to a navigation role: PID (path following), MPC (MPPI planner), LQR (corridor tracking), Adaptive (terrain changes), Robust (weather), SMC (rough terrain), Backstepping (nonholonomic constraints).

Next Week (Week 12): Course synthesis and final integration. We will combine everything from Weeks 1–11: control theory foundations (MATLAB), ROS2 control nodes, quadrotor control, and mobile robot navigation into a comprehensive final project and course review.

11.8.9 Running the ROS2 Demo

11.8.10 Build and Source

cd ~/auto_control_ws colcon build --packages-select week_05_nav2 source install/setup.bash

11.8.11 Launch the Demo

ros2 launch week_05_nav2 demo.launch.py

This command replays the recorded bag data at 2x speed, starts three solution nodes (costmap builder, A* planner, path follower), and opens RViz2 with a preconfigured layout.

11.8.12 ROS2 Topics

Topic	Type	Source	Description
`/map`	nav_msgs/OccupancyGrid	Bag	Static occupancy grid (latched)
`/odom`	nav_msgs/Odometry	Bag	Wheel odometry at 20 Hz
`/scan`	sensor_msgs/LaserScan	Bag	Laser scan data at 5 Hz
`/cmd_vel`	geometry_msgs/Twist	Nodes	Path follower velocity commands
`/plan`	nav_msgs/Path	Nodes	A* planned global path

11.8.13 Expected RViz2 Output

RViz2 will display the occupancy grid map, the robot's odometry path, laser scan points, the planned A* path (green line), and the robot following the path. You should see the costmap with inflated obstacles, the global path avoiding obstacles, and the robot tracking the path with the local controller.

11.8.14 Troubleshooting

- RViz2 crashes on launch: The launch file filters `snap` paths from `LD_LIBRARY_PATH` automatically. If RViz2 still crashes, try: `export LD_LIBRARY_PATH=$(echo $LD_LIBRARY_PATH | tr ':' '\n' | grep -v snap | tr '\n' ':')`
- No data visible: Ensure the bag file exists at `w05_exercises/bag_data/synthetic_nav2/`. Re-run `colcon build` if needed.
- Map not showing: The `/map` topic is published once (latched). If RViz2 opened before the bag started, restart the demo.
- QoS mismatch warnings: The bag uses default QoS. If your subscriber uses `BEST_EFFORT`, switch to `RELIABLE` or use `--qos-profile-overrides-path`.

11.8.15 Appendix: Review Material — Quadrotor Control

Before we move to mobile robot navigation, let us revisit the key concepts from Week 10 on quadrotor control with two exam-style questions.

11.8.15.1 Question 1 (25 marks)

Draw and explain the cascaded control architecture for a quadrotor (position attitude rate motors). Why is cascaded control preferred over a single monolithic controller? What are the typical loop rates for each cascade level?

11.8.15.2 Model Answer — Q1

Cascaded Architecture (5 marks for diagram, 10 marks for explanation, 10 marks for justification and rates):

The quadrotor cascaded controller consists of four nested loops:

1. Position Loop (Outer, ~50 Hz): Compares desired position \((x_d, y_d, z_d)\) with measured position from GPS/mocap. Outputs desired roll, pitch, and thrust commands. Typically a PID or LQR controller.
2. Attitude Loop (~250 Hz): Compares desired roll/pitch/yaw \((\phi_d, \theta_d, \psi_d)\) with IMU-measured Euler angles. Outputs desired angular rates \((p_d, q_d, r_d)\). Usually PID or quaternion-based control.

3. Rate Loop (Inner, ~500–1000 Hz): Compares desired angular rates with gyroscope measurements. Outputs torque commands \((\tau_x, \tau_y, \tau_z)\). Fast PID controller critical for stability.
4. Motor Mixing (~1000 Hz): Converts thrust + torques into individual motor PWM signals using the allocation matrix \(\mathbf{M}\).

Why cascaded over monolithic:

- Timescale separation: Inner loops (rates) are much faster than outer loops (position), allowing each to be tuned independently.
- Modularity: Each loop can be designed, tested, and debugged independently. Replacing the position controller does not affect the rate controller.
- Stability: Inner loops stabilise fast dynamics first, making the plant appear simpler to outer loops. A monolithic controller would need to handle the full 12-state nonlinear system simultaneously.
- Robustness: If the outer loop has modelling errors, the fast inner loops still maintain attitude stability, preventing catastrophic failure.

11.8.15.3 Question 2 (25 marks)

You tested PID, LQR, and MPC on a quadrotor flying a square trajectory with 3 m/s wind disturbance. Rank the controllers by (a) tracking accuracy, (b) disturbance rejection, (c) energy efficiency. Justify your ranking with reference to each controller's design principles.

11.8.15.4 Model Answer — Q2

(a) Tracking Accuracy (8 marks):

Ranking (best to worst): MPC > LQR > PID

- MPC: Best tracking because it uses a predictive model to anticipate the sharp corners of the square trajectory. It pre-compensates for turns by adjusting controls before the turn point. Typical RMS error: 0.05–0.15 m.
- LQR: Good tracking on straight segments (optimal for linear regulation), but overshoots at corners because it has no preview of the upcoming trajectory change. Typical RMS error: 0.10–0.25 m.
- PID: Reactive only; must wait for an error to develop before correcting. Largest overshoot at corners and steady-state oscillations if poorly tuned. Typical RMS error: 0.15–0.40 m.

(b) Disturbance Rejection (8 marks):

Ranking: MPC > LQR > PID

- MPC: Can include wind disturbance model or estimate it online. Constraint handling prevents saturation during gusts. Recovers fastest from wind disturbances.

- LQR: Optimal gains provide good disturbance attenuation for the linearised model. However, 3 m/s wind may push the system outside the linear regime, degrading performance.
- PID: Integral term eventually rejects constant wind, but transient response is poorest. Integral windup risk during sustained gusts if not properly bounded.

(c) Energy Efficiency (9 marks):

Ranking: MPC > LQR > PID

- MPC: Explicitly minimises control effort \(\sum u^T R u\) in the cost function while respecting actuator constraints. Smoothest motor commands, least wasted energy.
- LQR: Minimises \(\int (x^T Q x + u^T R u)\, dt\) for the linear system. Near-optimal energy usage on straight segments, but suboptimal at corners where the linearisation breaks down.
- PID: No explicit energy minimisation. Aggressive proportional and derivative terms cause high-frequency motor commands, wasting energy through rapid thrust changes.

AutoNav Solutions — Navigation Engineer Technical Challenge

Pre-Interview Assessment Autonomous Navigation Team

Confidential — Do not distribute without authorization

12 Controller Comparison and Industry Ap (Week 12)

12.1 Overview 230
12.1.1 Company Brief: Autonomous Systems Corp 230
12.1.2 Provided Bag Data . 230
12.1.3 Deliverables 230
12.1.4 Evaluation Criteria . 231
12.1.5 Skills You Will Demonstrate 231
12.1.6 Recommended Approach (3-Hour Capstone) 231
12.2 Comprehensive Controller Comparison . 233
12.2.1 The Seven Controllers at a Glance 233
12.2.2 Performance Metrics for Controller Comparison 235
12.2.3 Head-to-Head Comparison Table 236
12.2.4 Controller Selection Decision Flowchart . 237
12.2.5 Comparison by Robot System 237
12.2.6 When to Use Which Controller — Practitioner's Decision Guide 239
12.2.7 Practical Decision Guide 239
12.3 Industry Applications & Real-World Case Studies 240
12.3.1 Automotive & Self-Driving Vehicles . . . 240
12.3.2 Aerospace & UAVs . 241
12.3.3 Industrial Robotics & Manufacturing 243
12.3.4 Warehouse & Logistics Robotics 244
12.3.5 Legged Robotics . . . 245
12.3.6 Medical & Surgical Robotics 246
12.3.7 Marine & Underwater

Full Sensor Fusion, Mission Planning, and System Monitoring

Position: Senior Autonomy Engineer Team: Full-Stack Autonomy
Pre-Interview Technical Assessment Estimated Time: 3 Hours

12.1 Overview

12.1.1 Company Brief: Autonomous Systems Corp

Scenario: Autonomous Systems Corp is building a full-stack autonomous drone platform for search-and-rescue missions. Your team has collected 30 seconds of multi-sensor data from a test flight including GPS, IMU, odometry, camera images, and LiDAR point clouds. As the senior engineer candidate, you will design and implement the complete autonomy pipeline — from sensor fusion to mission execution.

12.1.2 Provided Bag Data

Topic	Message Type	Rate	Description
`/gps`	sensor_msgs/NavSatFix	1 Hz	GPS position fixes
`/imu`	sensor_msgs/Imu	200 Hz	Inertial measurement unit data
`/odom`	nav_msgs/Odometry	50 Hz	Visual-inertial odometry
`/camera/image`	sensor_msgs/Image	10 Hz	Forward-facing camera images
`/lidar_points`	sensor_msgs/PointCloud2	10 Hz	3D LiDAR point cloud

12.1.3 Deliverables

- EKF-based sensor fusion node combining IMU, GPS, and odometry into a unified state estimate
- A global mission planner (A* or RRT) that generates waypoints from a high-level mission specification

- A trajectory tracking controller (LQR or MPC) that follows planned paths with obstacle avoidance
- A system health monitor that detects sensor failures and triggers failsafe behaviours
- A multi-mission evaluation demonstrating the full pipeline across multiple scenarios

12.1.4 Evaluation Criteria

- System Architecture: Clean separation of perception, estimation, planning, and control subsystems
- Sensor Fusion: Correct EKF implementation handling multi-rate sensors and sensor dropout
- Controller Selection: Justified choice of control strategy with quantitative trade-off analysis
- Robustness: Graceful degradation under sensor failure, wind disturbance, and model uncertainty
- Communication: Ability to present system design at a whiteboard and defend architectural decisions

12.1.5 Skills You Will Demonstrate

By the end of this capstone session, you will be able to:

- Compare and contrast all seven control methods (CTC, MPC, Adaptive, Robust/H-infinity, Backstepping, SMC, NL Optimization) across quantitative performance metrics
- Evaluate controller trade-offs in terms of tracking accuracy, robustness, computational cost, and implementation complexity
- Select the most appropriate control strategy for a given robotic application based on system requirements and constraints
- Map each control method to real-world industry use cases and identify which companies deploy them
- Understand how each controller integrates within a ROS 2 architecture and the full autonomy pipeline (perception, estimation, planning, control, actuation)
- Apply structured decision-making frameworks (flowcharts, comparison matrices) to controller selection problems in interviews and practice
- Critically assess the gap between theoretical guarantees and practical implementation challenges for each method
- Formulate hybrid and cascaded control architectures that combine multiple methods for superior real-world performance

12.1.6 Recommended Approach (3-Hour Capstone)

0:00 – 0:20

12.1.6.1 Week 11 Recall — Navigation Control Exam Practice

Review DWA vs MPC, AMCL state estimation, and parallels to the Kalman filter in LQG.

0:20 – 1:00

12.1.6.2 Comprehensive Controller Comparison

Seven controllers at a glance. Performance metrics. Head-to-head comparison table. Decision flowchart. System-specific comparisons for manipulator, mobile robot, and quadrotor.

1:00 – 1:10

12.1.6.3 Break

10-minute break.

1:10 – 2:00

12.1.6.4 Industry Applications & Case Studies

Comprehensive real-world deployments across 10 industry sectors: automotive, aerospace, industrial robotics, warehouse, legged robots, medical, marine, space, energy, and consumer. Industry-controller matrix.

2:00 – 2:10

12.1.6.5 Break

10-minute break.

2:10 – 2:40

12.1.6.6 ROS 2 System Integration & Interview Preparation

Full autonomy pipeline. ROS 2 course alignment. Building your interview demo portfolio. ROS 2 capstone lab reference.

2:40 – 2:55

12.1.6.7 Hands-On Activities & Quiz

Controller selection challenge. Industry research project. Grand demo preparation. Final exam prep quiz.

2:55 – 3:00

12.1.6.8 Course Wrap-Up & Final Remarks

12-week summary. What to study next. Career advice. End of course.

12.2 Comprehensive Controller Comparison

In this capstone lecture, we step back from individual controller design and take a holistic view of all seven control methods studied across Weeks 1–8. The goal is to understand not just how each controller works, but when and why to choose one over another. This is the skill that separates a textbook reader from a practising control engineer.

12.2.1 The Seven Controllers at a Glance

Each card below summarises a controller's core idea, key equation, strengths, weaknesses, and ideal use case.

12.2.1.1 1. Computed Torque Control (CTC) — Week 1

Feedback linearisation via exact model cancellation

$$\tau = M(q)\bigl(\ddot{q}_d + K_d \dot{e} + K_p e\bigr) + C(q,\dot{q})\dot{q} + G(q)$$

Strengths: Exact linearisation yields decoupled error dynamics; straightforward PD tuning after cancellation; excellent tracking when model is accurate.

Weaknesses: Requires highly accurate dynamic model; no inherent robustness to parameter uncertainty.

Ideal use: Well-calibrated industrial manipulators with known payloads (e.g., FANUC, KUKA in structured cells).

12.2.1.2 2. Model Predictive Control (MPC) — Week 3

Receding-horizon optimisation with constraint satisfaction

$$\min_{u_{0:N-1}} \sum_{k=0}^{N-1}\bigl(x_k^T Q\, x_k + u_k^T R\, u_k \bigr) + x_N^T P_f\, x_N \quad \text{s.t. } x \in \mathcal{X},\; u \in \mathcal{U}$$

Strengths: Systematic constraint handling; preview/anticipation of future reference; flexible multi-objective cost.

Weaknesses: High computational cost (online optimisation); performance degrades with model inaccuracy.

Ideal use: Autonomous vehicles, constrained navigation, any system where actuator/state limits are critical.

12.2.1.3 3. Adaptive Control — Week 4

Online parameter estimation with concurrent control

$$\tau = Y(q,\dot{q},\dot{q}_r,\ddot{q}_r)\,\hat{\theta}, \qquad \dot{\hat{\theta}} = -\Gamma Y^T s$$

Strengths: Handles unknown/slowly varying parameters; Lyapunov convergence guarantees; no prior system ID needed.

Weaknesses: Requires regressor form (linearity in parameters); poor transient; needs persistent excitation.

Ideal use: Manipulators with changing payloads, robots in unstructured environments.

12.2.1.4 4. Robust Control (H-infinity) — Week 5

Worst-case performance optimisation over uncertainty sets

$$\min_K \;\|T_{zw}(s)\|_\infty = \min_K \;\sup_\omega \bar{\sigma}\bigl(T_{zw}(j\omega)\bigr) < \gamma$$

Strengths: Guaranteed performance despite bounded uncertainty; systematic LMI/Riccati framework; excellent for structured perturbations.

Weaknesses: Conservative (worst-case optimisation); complex design requiring frequency-domain expertise.

Ideal use: Safety-critical systems (aerospace, surgical robots) requiring guaranteed stability margins.

12.2.1.5 5. Backstepping Control — Week 6

Recursive Lyapunov-based design for strict-feedback systems

$$V_i = V_{i-1} + \tfrac{1}{2}z_i^2, \qquad z_i = x_i - \alpha_{i-1}(x_1,\ldots,x_{i-1})$$

Strengths: Constructive Lyapunov guarantees global asymptotic stability; natural for cascaded structures; smooth control law.

Weaknesses: Restricted to strict-feedback form; "explosion of terms" in higher-order systems.

Ideal use: Underactuated systems (quadrotors, marine vessels), VTOL transition control.

12.2.1.6 6. Sliding Mode Control (SMC) — Week 7

Variable-structure control with discontinuous switching

\[\tau = \tau_{eq} - K\,\text{sign}(s), \qquad s = \dot{e} + \Lambda e\]

Strengths: Invariant to matched disturbances; finite-time convergence to sliding surface; simple structure.

Weaknesses: Chattering from discontinuous control; only handles matched uncertainties.

Ideal use: Systems with significant matched disturbances: outdoor drones in wind, CNC machining.

12.2.1.7 7. Nonlinear Control Optimization — Week 8

Pontryagin / iLQR / NMPC on the nonlinear plant; also optimization of the gains of Controllers 1–6

\[\min_{u(\cdot)}\ J = \Phi(x(T)) + \int_0^T L(x,u,t)\,dt \quad\text{s.t.} \ \dot x = f(x,u)\]

Strengths: Operates on the nonlinear model directly (no linearisation); iLQR gives a local feedback policy in addition to the open-loop plan; provides a unified framework to optimize the parameters of every other controller (SMC \(\lambda,\phi,\eta\); adaptive \(\Gamma,\sigma\); robust \(K_p,K_d,\rho,\Lambda\); backstepping \(k_1,\ldots,k_n\)).

Weaknesses: Iterative; local (not global) optimum; needs derivatives (finite difference or autodiff); real-time requires warm-starting and fast SQP / iLQR back-ends.

Ideal use: Trajectory optimization for manipulators / drones (iLQR / DDP), receding-horizon tracking (NMPC), and offline tuning of any controller from 1–6 via min-max or Bayesian search.

12.2.2 Performance Metrics for Controller Comparison

To compare controllers rigorously, we define nine metrics. Each is rated on a 1–5 scale in the comparison table that follows.

Metric	Definition
Tracking Accuracy	How closely the output follows the reference trajectory in the nominal (no uncertainty) case.
Parametric Robustness	Ability to maintain stability and performance when physical parameters deviate ś30% from non
Disturbance Rejection	Speed and effectiveness of recovering from external disturbances (wind, payload, terrain).
Computational Cost	Real-time computation per control cycle. Critical for embedded systems.
Tuning Complexity	Number of design parameters, sensitivity to choices, expertise required.
Constraint Handling	Ability to explicitly enforce hard constraints on states and inputs.
Model Dependency	How sensitive performance is to model accuracy. High = poor model means poor performance.
Control Smoothness	Continuity of the control signal. Discontinuous signals cause actuator wear.
Stability Guarantees	Rigour of theoretical stability proof (Lyapunov, global/local, under uncertainty).

12.2.3 Head-to-Head Comparison Table

The centerpiece of this comparison. Each controller is rated 1–5 stars across all nine metrics. Higher is better except for Computational Cost and Model Dependency (where higher means more expensive/dependent).

Reading the stars: = poor/low, = moderate, = excellent/high. For Computational Cost and Model Dependency, = cheap/independent (good) and = expensive/dependent (challenging).

Metric	CTC	MPC	Adaptive	H-inf	Backstep	SMC	NL Optimization
Tracking Accuracy							
Parametric Robustness							
Disturbance Rejection							
Computational Cost (=cheap)							
Tuning Complexity (=easy)							
Constraint Handling							
Model Dependency (=independent)							
Control Smoothness							

Metric	CTC	MPC	Adaptive	H-inf	Backstep	SMC	NL Optimization
Stability Guarantees							

Key observations:

- No controller dominates all metrics. This is the "No Free Lunch" principle of control theory.
- CTC achieves top tracking accuracy but pays the price of extreme model dependency and zero constraint handling.
- MPC is the only method with native constraint handling but demands the most computational resources.
- SMC is unmatched for disturbance rejection but produces the roughest control signals (chattering).
- Nonlinear Optimization (iLQR / NMPC) has the highest online computational cost of the seven methods, but uniquely works directly on the nonlinear plant and provides a unified framework to re-tune any other controller's gains.
- Adaptive and SMC are the most robust, while H-infinity provides the strongest formal robustness guarantees.
- Backstepping provides elegant stability proofs for nonlinear cascaded systems but is hard to scale.

12.2.4 Controller Selection Decision Flowchart

The following decision tree guides practitioners through a structured selection process. Start at the top and follow the branches based on your system's characteristics.

Important caveat: This flowchart provides a starting point, not a definitive answer. In practice, controller selection involves trade-offs that cannot be captured in a binary decision tree. Always consider hybrid approaches, computational budgets, team expertise, and regulatory requirements.

12.2.5 Comparison by Robot System

The abstract ratings come alive when grounded in specific robotic platforms. Here we compare all seven controllers on the three systems studied throughout this course.

12.2.5.1 3.5.1: 2-DOF Planar Manipulator

Dynamics: \(M(q)\ddot{q} + C(q,\dot{q})\dot{q} + G(q) = \tau\). Key challenges: coupling, configuration-dependent inertia, payload uncertainty.

Controller	RMSE (nominal)	RMSE (+50% payload)	Computation	Constra
CTC	0.001 rad	0.08 rad	0.05 ms	No enfo
MPC	0.005 rad	0.03 rad	2.5 ms	Enforce
Adaptive	0.01 rad	0.005 rad (adapted)	0.1 ms	Possible
H-inf	0.02 rad	0.025 rad	0.05 ms	Possible
Backstepping	0.003 rad	0.015 rad	0.08 ms	Possible
SMC	0.005 rad	0.008 rad	0.04 ms	Possible
NL Optimization	0.01 rad	0.05 rad	0.02 ms	Possible

Discussion: CTC delivers 0.001 rad nominal but degrades 80x under payload change. Adaptive converges to 0.005 rad even under 50% payload uncertainty — best for variable payloads. MPC is the only method guaranteeing zero constraint violations. LQR is fastest (0.02 ms) but limited to near-equilibrium.

12.2.5.2 Differential-Drive Mobile Robot

Kinematics: \(\dot{x}=v\cos\theta,\;\dot{y}=v\sin\theta,\;\dot{\theta}=\omega\). Key challenges: nonholonomic constraints, obstacle avoidance, velocity limits.

Controller	Trajectory RMSE	Cross-track	Obstacle Handling	Computation
CTC	0.02 m	0.015 m	Separate planner	0.03 ms
MPC	0.01 m	0.008 m	Integrated	5.0 ms
Adaptive	0.01 m (adapted)	0.02 m	None	0.08 ms
H-inf	0.04 m	0.03 m	None	0.05 ms
Backstepping	0.015 m	0.01 m	None	0.06 ms
SMC	0.02 m	0.015 m	None	0.03 ms
NL Optimization	0.03 m	0.02 m	None	0.02 ms

Discussion: MPC dominates constrained navigation — the only method integrating obstacle avoidance directly. Backstepping is naturally suited to the nonholonomic strict-feedback structure. Adaptive excels when terrain changes (indoor to outdoor).

12.2.5.3 Quadrotor Drone

6-DOF dynamics, 4 inputs (underactuated). Key challenges: underactuation, coupled translational-rotational dynamics, wind disturbances, high-rate attitude control.

Controller	Hover RMS	Traj. RMSE	Wind (5 m/s gust)	Computation
CTC	0.005 m	0.03 m	0.15 m dev.	0.1 ms
MPC	0.008 m	0.02 m	0.06 m dev.	8.0 ms
Adaptive	0.01 m	0.04 m	0.08 m dev.	0.15 ms
H-inf	0.015 m	0.05 m	0.07 m dev.	0.1 ms
Backstepping	0.006 m	0.025 m	0.09 m dev.	0.12 ms
SMC	0.01 m	0.03 m	0.04 m dev.	0.08 ms
NL Optimization	0.004 m	0.06 m	0.12 m dev.	0.03 ms

Discussion: LQR achieves best hover stability (0.004 m) with minimal computation — the industry standard for inner attitude loops. MPC achieves best trajectory tracking (0.02 m) for aggressive manoeuvres. SMC produces smallest wind deviation (0.04 m) thanks to invariance property. LQR extends flight time by 10–15% via energy-optimal control.

12.2.6 When to Use Which Controller — Practitioner's Decision Guide

12.2.7 Practical Decision Guide

Start with the simplest method that meets your requirements. A PID or LQR that a technician can tune is often preferable to an MPC that requires a PhD to debug.

- Accurate model + trajectory tracking: Begin with Computed Torque Control. Use it as your benchmark.
- Hard constraints must never be violated: MPC is not a luxury — it is a necessity.
- Unknown or time-varying parameters: Adaptive Control is designed for exactly this scenario.
- Significant matched disturbances: Sliding Mode Control provides mathematically guaranteed rejection.
- Formal robustness certificates for safety-critical systems: Robust H-infinity provides the strongest guarantees.
- Cascaded strict-feedback nonlinear dynamics: Backstepping gives constructive Lyapunov stability at each step.
- Aggressive motion requiring prediction over a nonlinear horizon (trajectory-level): Nonlinear Optimization (iLQR/NMPC) — operates directly on the nonlinear model, provides a time-varying feedback policy alongside the open-loop plan.
- In practice, the best controllers are hybrids. DJI uses cascaded PID with LQR. Boston Dynamics uses CTC + MPC. Tesla uses MPC + LQR. The methods are components of an architecture.

12.3 Industry Applications & Real-World Case Studies

The professor specifically requested "plenty of examples." This section maps our seven controllers to 10 industry sectors with 80+ specific real-world deployments. This is the knowledge that wins interviews.

12.3.1 Automotive & Self-Driving Vehicles

12.3.1.1 Tesla Autopilot / FSD

MPC for trajectory planning: convex optimisation generates smooth lane-change trajectories respecting speed, acceleration, and lane-boundary constraints. LQR for lateral control: linearised bicycle model with optimal steering gains. Neural network perception feeds into a classical MPC-LQR control stack.

12.3.1.2 Waymo (Google)

MPC for trajectory planning with safety constraints encoded as hard inequalities. LQR for longitudinal/lateral low-level tracking. Sensor fusion via EKF (the observer in LQG). Published research highlights convex MPC solving at 10 Hz on custom compute.

12.3.1.3 BMW / Mercedes Active Suspension

H-infinity robust control for active suspension: road surface is an unmeasured disturbance with bounded energy. The controller minimises the \(\mathcal{H}_\infty\) norm of the transfer function from road input to passenger acceleration, guaranteeing ride comfort over all road profiles within the uncertainty set.

12.3.1.4 Toyota Adaptive Cruise Control

MPC with safety constraints: minimum following distance (2-second rule), speed limits, and comfort bounds on acceleration/jerk. The prediction horizon anticipates traffic flow changes, enabling smoother braking than reactive PID.

12.3.1.5 Formula 1 Energy Recovery (ERS/KERS)

MPC for energy management: optimises the deployment and harvesting of electrical energy over a lap, subject to battery constraints and FIA regulations. Traction control uses SMC for rapid wheel-slip regulation under extreme torque.

12.3.1.6 Tesla / Rivian AWD Torque Vectoring

LQR for torque distribution across 4 independent motors: minimises a cost function balancing yaw tracking error against total energy consumption. The linearised vehicle model around the current operating point makes LQR ideal for this real-time application.

12.3.1.7 Bosch Electronic Stability Program (ESP)

Sliding Mode Control for electronic stability: the SMC switching function drives the vehicle's yaw rate to the desired value despite unknown road friction (matched disturbance). Super-twisting SMC variant avoids brake chattering.

12.3.1.8 Autonomous Trucks (TuSimple, Aurora)

MPC for lane keeping with longer prediction horizons (trucks have larger inertia and stopping distances). Constraints include lane boundaries, speed limits, and vehicle dynamics limits. Adaptive elements compensate for varying cargo mass.

12.3.1.9 Electric Power Steering (EPS)

LQR-based torque assist: the controller minimises a cost that balances steering effort (driver comfort) against road-feel feedback. Gain-scheduled across vehicle speed for optimal performance at parking vs highway speeds.

12.3.1.10 Autonomous Parking (Bosch, Valeo)

MPC with nonholonomic constraints: the car's kinematic model (Ackermann steering) has turning radius limits that MPC naturally encodes. The finite horizon plans the entire parking manoeuvre as a constrained trajectory optimisation.

12.3.2 Aerospace & UAVs

12.3.2.1 DJI Consumer Drones (Mavic, Mini)

Cascaded PID — the industry standard: inner attitude-rate loop at 500 Hz, attitude loop at 250 Hz, position loop at 50 Hz. Simple, robust, and tunable by firmware engineers. PID accounts for >90% of commercial drone controllers.

12.3.2.2 Skydio Autonomous Drones

MPC for real-time obstacle avoidance and aggressive manoeuvres. The onboard NVIDIA Jetson solves convex MPC at 50 Hz with obstacle constraints from visual SLAM. Published at RSS/ICRA showing MPC-based trajectory planning through forests.

12.3.2.3 Boeing 787 Autopilot

LQR-based autopilot for cruise flight with gain scheduling across flight envelope (altitude, speed, weight). Robust control for flutter suppression: \(\mathcal{H}_\infty\) methods ensure aeroelastic stability across all structural modes.

12.3.2.4 Airbus A380 Fly-by-Wire

H-infinity robust control for flight control laws: the A380 uses \(\mu\)-synthesis (structured singular value) to guarantee stability over the full flight envelope despite parametric uncertainty in aerodynamic coefficients. Certified by EASA to DO-178C standards.

12.3.2.5 NASA Ingenuity Mars Helicopter

LQR + Kalman filter (LQG) for flight in Mars' thin atmosphere (1% of Earth's density). The Kalman filter fuses IMU data with a laser altimeter and downward-facing camera for state estimation. LQR provides optimal hover control with minimal computational footprint on the Snapdragon 801 processor.

12.3.2.6 SpaceX Falcon 9 Landing

Convex MPC (powered descent guidance): fuel-optimal trajectory computed via successive convexification. The controller plans the landing trajectory in real time, subject to thrust magnitude/direction constraints, glide-slope constraints, and the "landing on a moving barge" constraint set. Published in AIAA/AAS conferences.

12.3.2.7 Zipline Medical Delivery Drones

Adaptive control for varying payload mass: blood products and vaccines have different weights (0.5–2.5 kg), and the controller adapts online during each delivery flight. Fixed-wing transition uses backstepping for the cascaded aerodynamic model.

12.3.2.8 Military UAVs (MQ-9 Reaper)

SMC for robustness to wind gusts and battle damage (partial wing loss changes aerodynamic model dramatically). The invariance property ensures the aircraft remains controllable as long as remaining control surfaces provide sufficient authority.

12.3.2.9 Wing (Alphabet) Delivery

Backstepping for VTOL-to-cruise transition: the cascaded dynamics (hover attitude wing-borne flight) are naturally in strict-feedback form. Backstepping provides smooth transition control with global stability guarantees.

12.3.2.10 Joby Aviation eVTOL

MPC for urban air mobility: constrained trajectory optimisation considering no-fly zones, noise abatement corridors, and battery energy management. The eVTOL's tilt-rotor transition is managed by gain-scheduled LQR.

12.3.3 Industrial Robotics & Manufacturing

12.3.3.1 FANUC (Pick-and-Place)

Computed Torque Control: FANUC's proprietary servo system uses inverse dynamics cancellation with PD error feedback. Cycle times under 0.5 seconds with sub-millimetre repeatability. The well-known, fixed payload makes CTC ideal.

12.3.3.2 ABB IRB (Palletising)

Adaptive control for variable-payload palletising: boxes from 0.5 kg to 30 kg require different inertial compensation. ABB's TrueMove technology uses feedforward model adaptation for consistent path accuracy regardless of payload.

12.3.3.3 KUKA iiwa (Human Collaboration)

Impedance / Computed Torque for safe human-robot interaction: joint torque sensors enable compliant behaviour (virtual spring-damper). CTC provides the nominal trajectory tracking while impedance control modulates stiffness on contact.

12.3.3.4 Yaskawa Motoman (Arc Welding)

SMC for disturbance rejection during arc welding: cutting forces, arc blow, and spatter create matched disturbances that SMC's switching term rejects. Super-twisting variant keeps the weld bead smooth.

12.3.3.5 Universal Robots UR5/UR10

Adaptive control for flexible manufacturing: UR cobots are deployed in SMEs where payloads change frequently. The controller adapts to new end-effectors without manual re-calibration, reducing setup time from hours to minutes.

12.3.3.6 Siemens SINUMERIK CNC

Adaptive control for tool wear compensation: as cutting tools wear, cutting forces increase. The adaptive controller adjusts feed rate and spindle speed to maintain surface quality, extending tool life by 15–30%.

12.3.3.7 Engel Injection Moulding

MPC for temperature and pressure profile optimisation: the multi-zone heating system requires coordinated control with constraints on maximum temperature gradients and cycle time. MPC's preview capability optimises the entire injection cycle.

12.3.3.8 Semiconductor Wafer Handling

LQR for vibration-free motion: wafer transport requires sub-micrometre positioning accuracy with zero residual vibration. LQR minimises a cost penalising both position error and jerk (vibration-inducing acceleration changes).

12.3.3.9 Laser Cutting Systems

SMC for contour following: the cutting head must follow complex 2D/3D contours despite varying material thickness and thermal distortion. SMC's invariance property ensures cutting accuracy within ś0.05 mm.

12.3.3.10 3D Printing / Additive Manufacturing

Robust control for layer consistency: extrusion rate, nozzle temperature, and build-plate adhesion vary layer to layer. Robust controllers maintain dimensional accuracy despite these slowly varying parametric uncertainties.

12.3.4 Warehouse & Logistics Robotics

12.3.4.1 Amazon Robotics (Kiva / Proteus)

MPC for fleet coordination: each robot solves a local MPC that respects velocity constraints, keepout zones, and priority rules for intersection crossing. Low-level drive control uses PID for wheel velocity regulation.

12.3.4.2 Locus Robotics

MPC-based local planner for warehouse navigation: replaces DWA with a constrained MPC that handles narrow aisles and dynamic obstacles (human workers). Published benchmarks show 30% fewer near-misses vs DWA.

12.3.4.3 Fetch Robotics

LQR for the mobile base (smooth, energy-efficient motion) combined with PID for the manipulator arm. The separation of control strategies reflects the different dynamics (wheeled base vs articulated arm).

12.3.4.4 Ocado (Grocery Fulfilment)

MPC for multi-robot deconfliction on the grid: hundreds of robots navigate a grid simultaneously. MPC plans collision-free paths with time-slot constraints at grid intersections. Centralised planning with decentralised execution.

12.3.4.5 Magazino TORU

Adaptive grasping for items of varying mass and shape: the gripper controller adapts grip force online based on tactile sensor feedback, preventing damage to fragile items while securely holding heavy ones.

12.3.4.6 AGVs (Dematic, KION)

PID + feedforward for line-following: magnetic tape or painted line guidance with PID lateral correction. Feedforward from the known path curvature anticipates steering adjustments. Simple, reliable, maintainable by warehouse technicians.

12.3.4.7 Autonomous Forklifts (Seegrid)

Robust control for heavy-load handling: when carrying 2000+ kg pallets, the vehicle dynamics change dramatically. Robust H-infinity methods ensure stability across the full load range without re-tuning.

12.3.5 Legged Robotics

12.3.5.1 Boston Dynamics Spot

Convex MPC for whole-body locomotion: the single rigid body model is linearised around the current state, and a QP is solved at 30 Hz to compute ground reaction forces for all four feet. Published by MIT in "Mini Cheetah" and adapted by BD.

12.3.5.2 Boston Dynamics Atlas

MPC + whole-body control for parkour and dynamic manoeuvres: hierarchical architecture with MPC at the planning level (contact schedule, CoM trajectory) and operational-space CTC at the joint level (tracking desired contact forces and body motion).

12.3.5.3 ANYbotics ANYmal

MPC with zero-moment-point (ZMP) constraints for industrial inspection on uneven terrain. Combines convex MPC for gait generation with robust control for disturbance rejection on slippery surfaces.

12.3.5.4 Agility Robotics Digit

LQR for walking stabilisation (linearised around the nominal gait) combined with MPC for step planning and footstep optimisation. The bipedal structure requires careful management of the zero-moment point.

12.3.5.5 MIT Mini Cheetah

Convex MPC for trot, bound, and pronk gaits: the seminal paper (Di Carlo et al., 2018) demonstrated that a single convex MPC formulation can generate multiple gaits simply by changing the contact schedule. Solved in <3 ms on an Intel i7.

12.3.5.6 Unitree Go2

Cascaded PID + MPC for consumer quadruped locomotion: reinforcement learning generates the gait policy, which is tracked by a low-level PID/MPC controller. Combines learning-based planning with model-based control for reliability.

12.3.5.7 Passive Dynamic Walkers

LQR for energy-efficient gait stabilisation: the natural dynamics of the walker are nearly periodic, and LQR about the limit cycle provides minimal-energy perturbation correction. Demonstrates that optimal control can exploit natural dynamics.

12.3.6 Medical & Surgical Robotics

12.3.6.1 Intuitive Surgical da Vinci

Robust control for teleoperation with communication delay: the master-slave architecture must remain stable despite variable network latency (0.1–0.5 s). Passivity-based robust control ensures coupled stability. Force scaling uses impedance control for delicate tissue manipulation.

12.3.6.2 Stryker Mako (Orthopaedic)

Adaptive control for bone cutting: bone density and hardness vary between patients and anatomical locations. The controller adapts cutting force and feed rate online based on force/torque sensor feedback, preventing thermal necrosis.

12.3.6.3 Medtronic Hugo RAS

Force-limited computed torque: CTC provides precise trajectory tracking for the instrument tip, with a force limiter that reduces torque commands when contact forces exceed safety thresholds. Certified for surgical use under IEC 62304.

12.3.6.4 Rehabilitation Exoskeletons (Ekso, ReWalk)

Adaptive impedance control: the exoskeleton adapts its assistance level based on the patient's effort (measured via EMG or interaction force). As the patient recovers, the controller reduces assistance — "assist-as-needed" paradigm uses adaptive gain scheduling.

12.3.6.5 CyberKnife (Accuray)

LQR for positioning the linear accelerator + SMC for tumour tracking during respiratory motion. The tumour moves ~15 mm with breathing; SMC's finite-time convergence ensures the radiation beam tracks within 0.5 mm despite the quasi-periodic disturbance.

12.3.6.6 Surgical Needle Insertion

Backstepping for nonlinear tissue interaction: needle insertion through layered tissue (skin, fat, muscle) involves highly nonlinear force-displacement characteristics. Backstepping handles the cascaded structure (needle position tip force tissue deformation).

12.3.7 Marine & Underwater Robotics

12.3.7.1 Saab Seaeye ROVs

SMC for deep-sea current rejection: ocean currents are large, persistent, matched disturbances. SMC provides guaranteed station-keeping despite currents up to 3 knots. The "sliding" property is particularly valuable when current measurements are unavailable.

12.3.7.2 Kongsberg AUVs

LQR for heading and depth control during survey missions: the linearised hydrodynamic model supports optimal LQR design. The quadratic cost naturally balances heading accuracy against thruster energy, extending mission duration.

12.3.7.3 Rolls-Royce Dynamic Positioning

MPC for station-keeping of drilling vessels: the vessel must maintain position within ś2 m despite wind, waves, and current. MPC coordinates 6+ thrusters with constraints on thrust rate, power allocation, and forbidden thrust sectors (near the drilling riser).

12.3.7.4 Sea Machines Autonomous Surface Vessels

Robust control for wave disturbance rejection: sea states create broadband disturbances that cannot be modelled exactly. H-infinity methods provide worst-case performance guarantees across sea states 1–6.

12.3.7.5 Torpedo Guidance Systems

SMC for terminal guidance: ocean currents and target manoeuvres are treated as matched disturbances. The sliding mode ensures the guidance error reaches zero in finite time, critical for interception accuracy.

12.3.8 Space & Satellite Systems

12.3.8.1 Hubble Space Telescope

LQR for 0.007 arcsecond pointing accuracy: reaction wheels provide fine attitude control, with LQR optimising the trade-off between pointing error and wheel momentum accumulation. The well-modelled, linear dynamics make LQR the ideal choice.

12.3.8.2 ISS Canadarm2

Computed Torque Control in microgravity: the absence of gravity simplifies the dynamics (\(G(q)=0\)), making CTC highly effective. The known payload (ISS modules, astronauts) allows accurate model-based control for delicate berthing operations.

12.3.8.3 James Webb Space Telescope

LQR fine guidance: the Fine Guidance Sensor (FGS) provides star positions to the attitude controller, which uses LQR to achieve milli-arcsecond pointing stability. The extremely low disturbance environment (L2 Lagrange point) makes LQR optimal.

12.3.8.4 Satellite Attitude Control (Reaction Wheels)

NMPC / iLQR is the standard for planning complex pointing manoeuvres. SMC for fault-tolerant mode: when a reaction wheel fails, the control authority changes and SMC's robustness to matched uncertainty provides graceful degradation.

12.3.8.5 SpaceX Starship Re-entry

MPC for re-entry guidance: the "belly-flop" manoeuvre requires trajectory optimisation with aerodynamic heating constraints, structural load limits, and landing accuracy. Adaptive elements handle the aerodynamic uncertainty during hypersonic flight.

12.3.8.6 Mars Rovers (Curiosity, Perseverance)

Robust control for unknown terrain: wheel-terrain interaction models are highly uncertain on Mars. Adaptive control compensates for wheel degradation over the multi-year mission (Curiosity's wheels show significant wear after 10+ years).

12.3.9 Energy & Process Control

12.3.9.1 Oil Refineries (Honeywell, Emerson)

MPC dominates: over 90% of advanced process control installations in oil/gas use MPC (Dynamic Matrix Control, RMPCT). Multi-variable, constrained optimisation of temperature, pressure, and flow across 10–100 process variables simultaneously.

12.3.9.2 Wind Turbine Pitch Control

Adaptive + Robust: wind speed and direction change continuously (unknown parameters), while turbulent gusts create disturbances. Adaptive control adjusts the pitch gain schedule, while robust inner loops maintain structural integrity under extreme gusts.

12.3.9.3 Nuclear Reactor Control

Robust control with large safety margins: nuclear safety regulations require proven stability margins under all credible uncertainty scenarios. H-infinity methods provide the formal certificates required by nuclear regulators (NRC, ONR).

12.3.9.4 Smart Grid Frequency Regulation

MPC for demand-response: coordinates battery storage, flexible loads, and generators to maintain grid frequency at 50/60 Hz. The prediction horizon captures the slow dynamics of thermal generators and the fast response of batteries.

12.3.9.5 Google DeepMind Data Centre HVAC

MPC (model-based reinforcement learning): reduced cooling energy by 40% by optimising chiller setpoints, fan speeds, and damper positions subject to thermal constraints. The "model" is learned from historical data, making this a data-driven MPC.

12.3.9.6 Solar Panel Dual-Axis Tracking

LQR for smooth, energy-efficient sun tracking: the cost function penalises both tracking error (reduced solar yield) and motor energy (parasitic losses). The gentle, predictable solar motion makes linearised LQR sufficient.

12.3.10 Consumer & Emerging Applications

12.3.10.1 Segway / Ninebot

LQR for inverted pendulum stabilisation: the classic application of LQR. The linearised model around the upright equilibrium is accurate for small angles, and LQR provides optimal state feedback for balancing with minimum rider discomfort.

12.3.10.2 iRobot Roomba

PID + behaviour-based control: simple PID for wheel velocity, combined with reactive behaviours (wall-following, bump-and-turn). The low cost target and simple dynamics make sophisticated control unnecessary.

12.3.10.3 DJI Agras (Agricultural Drones)

Adaptive control for spray payload variation: as the pesticide tank empties during a spraying run, the drone mass decreases by up to 40%. Adaptive control maintains stable flight without requiring the pilot to re-trim manually.

12.3.10.4 DJI Ronin Camera Gimbals

Cascaded PID for 3-axis stabilisation: inner motor-current loop, middle rate loop, outer angle loop. The high control rate (8 kHz motor, 2 kHz rate) on STM32 microcontrollers provides sub-0.01 degree stabilisation.

12.3.10.5 Starship Technologies (Delivery Robots)

MPC for urban sidewalk navigation: constrained path planning through pedestrian traffic with speed limits, curb avoidance, and crossing detection. The prediction horizon anticipates pedestrian trajectories for social-awareness.

12.3.10.6 Autonomous Lawnmowers (Husqvarna)

PID + MPC for coverage path planning: PID for low-level wheel control, MPC for optimising the mowing pattern subject to boundary constraints and obstacle avoidance. GPS-RTK provides centimetre-level positioning.

12.3.11 Industry-Controller Matrix

This comprehensive table maps 16 industry sectors to all 7 controllers. P = Primary method, S = Secondary/supporting, E = Emerging use, — = Not typically used.

Industry Sector	CTC	MPC	Adaptive	H-inf	Backstep	SMC	NL Optimization
Autonomous Vehicles	—	P	S	S	—	S	P
Consumer Drones	S	E	S	—	S	E	P
Autonomous Drones (Skydio)	—	P	S	—	S	S	S
Commercial Aviation	—	E	E	P	—	—	P
Space / Satellites	S	E	S	S	—	S	P
Industrial Manipulators	P	S	P	S	—	S	S
CNC / Machining	S	E	P	S	—	P	S
Warehouse / Logistics	—	P	S	—	—	—	S
Legged Robots	S	P	E	—	E	E	S
Surgical Robotics	S	E	P	P	S	S	S
Rehabilitation	—	E	P	S	—	—	S
Marine / Underwater	—	P	S	S	S	P	S
Process Control (Oil/Gas)	—	P	S	S	—	—	S
Power / Energy	—	P	P	P	—	—	S
Consumer Electronics	—	E	S	—	—	—	P
Agricultural Robots	—	S	P	—	—	S	S

Key insight: MPC appears as Primary in 8 of 16 sectors — it is the most broadly applicable advanced controller. Nonlinear Control Optimization (iLQR / NMPC) is Primary or Secondary where the plant is strongly nonlinear and a model is available. Adaptive control dominates where parameters change (manufacturing, medical, agriculture). H-infinity leads in safety-critical domains (aviation, nuclear, surgical).

12.3.12 Controller Popularity by Domain

The following chart shows each controller's prevalence across the 16 industry sectors analysed above (counting Primary + Secondary roles).

12.4 ROS 2 System Integration

Your controller is never a standalone module in production. It receives references from a planner and state estimates from a filter. This section connects control theory to the full autonomy pipeline.

12.4.1 Full Autonomous System Architecture

The complete pipeline: Perception State Estimation Planning CONTROL Actuation. Your controller sits at the heart of this chain.

12.4.2 ROS 2 Course Control Course Alignment Map

12.4.3 Tier 1 — Direct Alignment (Must-Do)

ROS 2 Week	ROS 2 Topic	Control Weeks	Why They Align
Week 11	Control Systems: PID to MPC	W1, W3, W8	1:1 implementation
Week 6	Quadrotor Dynamics & Attitude	W1–W8	Implements the qua

12.4.4 Tier 2 — Strong Complementary

ROS 2 Week	ROS 2 Topic	Control Weeks	Why They Complemen
Week 2	Sensor Fusion & EKF	W8 (LQG)	The Kalman filter is th
Week 5	Navigation Stack (Nav2)	W3 (MPC)	DWA local planner is c
Week 9	EGO-Planner & Traj. Optim.	W3, W8	B-spline trajectory opti
Week 7	PX4/ArduPilot Integration	W1–W8	PX4 SITL provides a r

12.4.5 Tier 3 — Supporting Context

ROS 2 Week	ROS 2 Topic	How It Supports Control
Week 1	LiDAR Point Cloud	Provides obstacle data for MPC constrai
Week 3	ROS 2 Fundamentals	Prerequisite: nodes, topics, services, QoS
Week 4	2D SLAM	State estimation for controllers. Without
Week 8	3D Path Planning	Generates reference trajectories \(x_{\t
Week 10	Computer Vision	Enables visual servoing and image-based
Week 12	System Integration Capstone	Full pipeline: all modules connected into

Study Strategy: Complete Tier 1 (ROS 2 Weeks 6 and 11) first. Then pick Tier 2 weeks based on your target sector (Weeks 2+5 for ground robots, Weeks 7+9 for drones). Tier 3 rounds out your portfolio.

12.4.6 Building Your Interview Demo Portfolio

Different sectors weight different skills. These company-specific cards guide your demo preparation.

12.4.6.1 Amazon Robotics: MPC + Nav2

Demo: MPC-based mobile robot navigation with obstacle avoidance. Combine Nav2 (ROS 2 W5) with MPC (Control W3). Multi-robot coordination with 3+ TurtleBot3s.

Key skills: MPC convex formulation, Python (production quality), ROS 2 Nav2 integration, Docker, system design at scale.

12.4.6.2 DJI / Skydio: 7 Controllers on Quadrotor + PX4

Demo: All seven controllers on the quadrotor testbench (ROS 2 W6 + Control W1–8). Aggressive figure-8 tracking, wind rejection, PX4 SITL integration.

Key skills: Cascaded PID, MPC trajectory optimisation, C++ (real-time), embedded systems, MAVLink.

12.4.6.3 Boston Dynamics: MPC Whole-Body Control

Demo: Real-time MPC on multi-DOF system with contact constraints. Convex MPC solving at 200+ Hz. Comparison: MPC vs LQR vs CTC.

Key skills: MPC (convex, whole-body), LQR (Riccati), OSQP/ECOS solvers, C++ Eigen, real-time computing. Be ready for whiteboard Riccati derivation.

12.4.6.4 Tesla / Waymo: MPC + EKF + Planning

Demo: MPC trajectory planning with lane constraints. EKF sensor fusion (IMU + GPS + encoders). Planning-to-control pipeline for simulated highway driving.

Key skills: Nonlinear MPC (IPOPT, acados), LQR (lateral baseline), sensor fusion (EKF/UKF), C++, end-to-end thinking.

12.4.6.5 Intuitive Surgical: Robust + Adaptive

Demo: Robust + adaptive control for manipulator under parameter uncertainty and external forces. Adaptive convergence with 50% mass uncertainty. Force-limited impedance control.

Key skills: Robust H-infinity (guaranteed bounds), adaptive (Lyapunov proofs), impedance control, C++ real-time, IEC 62304 awareness.

The Career Equation:

Control Theory (rigorous mathematical foundations) + ROS 2 Implementation (working code on robots) = Interview-Ready Portfolio

12.4.7 Reference to ROS 2 Capstone (src/ros2_robotics_course/week_12)

The companion ROS 2 course concludes with a system integration capstone in `src/ros2_robotics_course/week_12/`. The lab exercises bridge every ROS 2 week into a single autonomous mission.

Lab Exercise	Description
Exercise 1: System Architecture Design	Design the complete node graph: perception, estimation, planning, control, actu
Exercise 2: Integrated Sensor Fusion	EKF fusing IMU + GPS + camera. Publish `/estimated_state` for the control
Exercise 3: Planning-Control Pipeline	A* global path B-spline smoothing LQR/MPC tracking motor commands.

Lab Exercise	Description
Exercise 4: Autonomous Mission	Execute a complete mission: takeoff, navigat
Exercise 5: Stress Testing	Inject disturbances (wind, sensor dropout, p
Exercise 6: Comprehensive Evaluation	Quantitative comparison: tracking RMSE, c

Integration is the hardest part: Research shows that integration and testing consume 40–60% of total project time in robotics. The "last 10%" of making a system work reliably often takes as long as the first 90%. Start early.

12.5 Summary and Next Steps

12.5.1 Activity 1: Controller Selection Challenge

For each scenario, select the best controller and justify your choice in 2–3 sentences.

12.5.2 Scenario 1: FANUC Paint Line with Variable Paint Guns (0.5–2.5 kg)

Best: Adaptive Control (Week 4) — Changing payload mass is a classic parameter uncertainty problem. Adaptive control estimates inertial parameters online, making tool changes seamless without recalibration.

12.5.3 Scenario 2: Delivery Drone in 30+ mph Crosswinds

Best: SMC (Week 7) — Wind gusts are large, persistent, matched disturbances. SMC's switching term rejects any bounded matched disturbance, guaranteeing stability. Super-twisting variant avoids chattering.

12.5.4 Scenario 3: Self-Driving Car on Highway with Comfort Constraints

Best: MPC (Week 3) — Speed limits, steering angle limits, acceleration bounds, and following distance are all hard constraints. MPC is the only method that handles them natively within the optimisation.

12.5.5 Scenario 4: Surgical Robot Tracking Breathing Tumour (0.1 mm precision)

Best: Robust + Backstepping (Weeks 5 & 6) — 0.1 mm precision with uncertain friction calls for robust guaranteed performance bounds. Backstepping ensures no overshoot through constructive Lyapunov design for the multi-DOF cascade.

12.5.6 Scenario 5: Geostationary Satellite Station-Keeping (Fuel-Limited)

Best: Nonlinear Control Optimization / NMPC (Week 8) — The Clohessy-Wiltshire relative-motion equations are nearly linear, but the manoeuvre involves thrust-constrained trajectory optimisation over long horizons and the primary objective is minimising fuel \(\equiv\) minimising control effort. NMPC/iLQR directly handles the nonlinear cost-plus-constraint problem and produces the bang-singular-bang fuel-optimal profile, which LQR cannot.

12.5.7 Activity 2: Industry Research Project

Pick one company from the list below. Prepare a 5-minute presentation addressing four questions.

12.5.7.1 Tier 1: Large Robotics

- Boston Dynamics (Spot, Atlas, Stretch)
- Tesla (Optimus, FSD)
- Amazon Robotics (Proteus, Sparrow)
- DJI (Matrice, Agras)

12.5.7.2 Tier 2: Specialised

- Intuitive Surgical (da Vinci, Ion)
- Skydio (autonomous drones)
- ANYbotics (ANYmal)
- Zipline (delivery drones)

12.5.7.3 Tier 3: Emerging

- Agility Robotics (Digit)
- Figure AI (humanoid)
- Archer Aviation (eVTOL)
- Covariant (warehouse manipulation)

Question	What to Address	Marks
1. What robot/product?	Physical configuration, DOF, sensors, application domain.	5
2. What control method(s)?	Identify algorithms (papers, patents, job postings). Map to our 7 methods.	10
3. Key challenges?	What makes the control problem hard?	5
4. How would you improve it?	Propose a concrete improvement using course controllers. Explain trade-offs.	10

Research Tips: Check IEEE/RSS/ICRA papers, Google Patents, job descriptions (filter "control engineer"), and YouTube conference talks.

12.5.8 Activity 3: Grand Demo — Portfolio Video

Pick your best controller implementation from Weeks 9–11 and prepare a 5-minute demo video for your professional portfolio.

12.5.9 Video Structure

1. Introduction (30s): State the controller, the robot platform, and the control objective.
2. Theory (60s): Show the key equation on screen. Explain the design choices (gains, horizon, sliding surface).
3. ROS 2 Architecture (60s): Show the `rqt_graph` node graph. Explain the topic flow: sensor controller actuator.
4. Live Demo (90s): Screen recording of Gazebo simulation with RViz2 overlay showing reference vs actual trajectory.
5. Results (60s): Quantitative plots: tracking error, control effort, computation time. Compare against at least one alternative controller.

Portfolio Impact: This video is your most powerful interview asset. Upload it to YouTube (unlisted) and link it from your GitHub README. Companies like Boston Dynamics, DJI, and Tesla have confirmed that demo videos significantly influence hiring decisions.

12.5.9.1 Final Exam Preparation Quiz — 5 Questions

Q1: Which controller is the ONLY one that natively enforces hard state and input constraints?

Answer: MPC — constraints are encoded as inequalities in the optimisation problem.

Q2: A quadrotor must hover in 20 mph wind with minimal computation. Which controller and why?

Answer: SMC — wind is a matched disturbance; SMC's invariance property provides guaranteed rejection with O(n) computation (no online optimisation). Use super-twisting to avoid chattering.

Q3: Why does LQG lose the guaranteed robustness margins of LQR?

Answer: Doyle (1978) showed that the Kalman filter can arbitrarily degrade the phase/gain margins guaranteed by LQR. The separation principle ensures optimal performance for the nominal stochastic system but provides no robustness guarantees when the model is uncertain. This motivates LQG/LTR (Loop Transfer Recovery).

Q4: Name two industry applications where Adaptive Control is the primary method and explain why.

Answer: (1) Universal Robots UR5 in flexible manufacturing — payloads change frequently and the controller must adapt without re-calibration. (2) Zipline delivery drones — payload mass varies per delivery (blood,

vaccines, 0.5–2.5 kg), requiring online adaptation of the flight controller.

Q5: In the ROS 2 controller node architecture, what are the standard subscriber and publisher topics for a mobile robot controller?

Answer: Subscribe to `/odom` (nav_msgs/Odometry — state feedback) and `/reference_trajectory` (desired path). Publish to `/cmd_vel` (geometry_msgs/Twist — control output) and `/controller_diagnostics` (error, computation time).

12.5.10 Course Wrap-Up: 12 Weeks of Control for Robots

12.5.11 Summary of All 12 Weeks

This course has taken you from the foundations of model-based control through seven distinct control methodologies, applied each to three robot systems, and connected theory to practice through ROS 2 implementation.

Week	Topic	Key Concept
1	Computed Torque Control (CTC)	Feedback linearisation, model cancellation
2	MPC Foundations	Prediction horizon, cost function, constraints
3	MPC Advanced	Nonlinear MPC, real-time solvers, MPPI
4	Adaptive Control	Regressor form, parameter update law, Lyapunov
5	Robust Control (H-infinity)	Uncertainty modelling, worst-case optimisation, LMIs
6	Backstepping Control	Recursive Lyapunov, virtual control, strict-feedback
7	Sliding Mode Control (SMC)	Sliding surface, matched disturbances, chattering
8	Nonlinear Control Optimization	Pontryagin Minimum Principle, iLQR/DDP, NMPC, min-max optimizatio
9	ROS 2 Control Implementation	PID, LQR, MPC as ROS 2 nodes
10	ROS 2 Quadrotor Control	Cascaded control, PX4 integration, attitude loops
11	ROS 2 Navigation & Mobile Control	Nav2, DWA, AMCL, path planning + control
12	Capstone: Comparison & Integration	Controller selection, industry applications, full pipeline

12.5.12 What to Study Next

12.5.12.1 Advanced Nonlinear Control

- Input-to-State Stability (ISS) and nonlinear small-gain theorems
- Contraction theory and incremental stability
- Control barrier functions (CBFs) for safety
- Port-Hamiltonian systems and passivity-based control

12.5.12.2 Learning-Based Control

- Reinforcement learning for locomotion (PPO, SAC)
- Sim-to-real transfer and domain randomisation
- Neural network Lyapunov functions
- Gaussian process model learning for MPC

12.5.12.3 Advanced MPC

- Robust MPC (tube MPC, min-max MPC)
- Stochastic MPC (chance constraints)
- Learning-based MPC (data-driven models)
- Distributed MPC for multi-robot systems

12.5.12.4 System-Level Skills

- Real-time C++ (Eigen, OSQP, CasADi)
- ROS 2 lifecycle nodes and real-time executors
- Functional safety (IEC 61508, DO-178C)
- Hardware-in-the-loop (HIL) testing

12.5.13 Career Advice

- Build a GitHub portfolio with your ROS 2 controller implementations. Include README files with theory, results plots, and demo videos.
- Target your applications: Drones? Focus on MPC + PX4. Warehouse? Focus on MPC + Nav2. Medical? Focus on Robust + Adaptive.
- Master the whiteboard: Boston Dynamics, Tesla, and Skydio all include whiteboard control theory in interviews. Practice deriving the Riccati equation, Lyapunov stability proofs, and the MPC cost function from scratch.
- Learn C++: Python is fine for prototyping, but production controllers run in C++ for real-time performance. Port your best controller to C++ using Eigen.
- Read papers: Follow RSS, ICRA, IROS, and pedestrian conferences. The state-of-the-art moves fast — MPC + RL is the frontier as of 2025.
- Contribute to open source: PR to ros2_control, PX4, or Nav2 demonstrates real-world engineering skills that employers value highly.

Final Message to the Class: You now possess the theoretical foundations and practical skills to design controllers for any robotic system. The seven methods you have learned — CTC, MPC, Adaptive, Robust, Backstepping, SMC, and Nonlinear Control Optimization — cover the vast majority of control problems you will encounter in industry. The key is knowing which tool to use when, and having the ROS 2 implementation skills to turn theory into working code. Go build robots.

12.5.14 Running the ROS2 Demo

12.5.15 Build and Source

cd ~/auto_control_ws colcon build --packages-select week_12_capstone source install/setup.bash

12.5.16 Launch the Demo

ros2 launch week_12_capstone demo.launch.py

This command replays the recorded multi-sensor bag data at 2x speed, starts three solution nodes (sensor fusion, mission planner, controller), and opens RViz2 with a preconfigured layout.

12.5.17 ROS2 Topics

Topic	Type	Source	Description
`/gps`	sensor_msgs/NavSatFix	Bag	GPS fixes at 1 Hz
`/imu`	sensor_msgs/Imu	Bag	IMU data at 200 Hz
`/odom`	nav_msgs/Odometry	Bag	Visual-inertial odometry at 50 Hz
`/camera/image`	sensor_msgs/Image	Bag	Camera images at 10 Hz
`/lidar_points`	sensor_msgs/PointCloud2	Bag	LiDAR point cloud at 10 Hz
`/cmd_vel`	geometry_msgs/Twist	Nodes	Controller velocity commands
`/fused_state`	nav_msgs/Odometry	Nodes	EKF fused state estimate

12.5.18 Expected RViz2 Output

RViz2 will display the fused state estimate trajectory, raw GPS markers, LiDAR point cloud, camera image panel, and the planned mission path. You should see the EKF estimate smoothly combining GPS (low rate, noisy) with IMU (high rate) and odometry. The mission planner path will be displayed as waypoints with the controller tracking them.

12.5.19 Troubleshooting

- RViz2 crashes on launch: The launch file filters `snap` paths from `LD_LIBRARY_PATH` automatically. If RViz2 still crashes, try: `export LD_LIBRARY_PATH=$(echo $LD_LIBRARY_PATH | tr ':' '\n' | grep -v snap | tr '\n' ':')`
- No data visible: Ensure the bag file exists at `w12_exercises/bag_-data/synthetic_capstone/`. Re-run `colcon build` if needed.
- High memory usage: The camera images and LiDAR point clouds consume significant memory. Close other applications if RViz2 becomes sluggish.
- QoS mismatch warnings: The bag uses default QoS. If your subscriber uses `BEST_EFFORT`, switch to `RELIABLE` or use `--qos-profile-overrides-path`.

12.5.20 Appendix: Review Material — Navigation Control

12.5.20.1 Question 1 — DWA vs MPC for Mobile Robot Navigation (25 marks)

Compare the Dynamic Window Approach (DWA) and Model Predictive Control (MPC) as local controllers for mobile robot navigation.

(a) In what sense is DWA a special case of MPC? Explain the structural similarities and differences. (10 marks)

(b) When would you prefer Model Predictive Path Integral (MPPI) control over DWA? Give three specific scenarios with justification. (10 marks)

(c) Describe how hard constraints (obstacle avoidance, velocity limits) are handled in each approach. (5 marks)

12.5.20.2 Model Answer

(a) DWA as a special case of MPC:

DWA and MPC share the same fundamental structure: at each time step, both (1) predict future states over a finite horizon, (2) evaluate candidate trajectories against a cost function, and (3) select the best action. The key differences are:

- Action space: DWA samples a discrete grid of \((v, \omega)\) pairs within the dynamic window (admissible velocities reachable in one time step), whereas MPC optimises over a continuous control sequence \(u_{0:N-1}\) using gradient-based or QP solvers.
- Prediction model: DWA uses a simplified kinematic model (constant-velocity arcs), while MPC can use the full nonlinear dynamic model.
- Horizon: DWA typically uses a short prediction horizon (1–3 seconds with fixed velocity), while MPC explicitly optimises over N discrete steps with time-varying controls.
- Optimality: DWA finds the best among sampled candidates (combinatorial), while MPC finds a local optimum of the continuous cost (gradient-based). DWA is MPC with a sampled action space and single-step constant-velocity prediction.

(b) When to prefer MPPI over DWA:

- Scenario 1 — Highly cluttered environments: MPPI samples thousands of trajectories (not just constant-velocity arcs) using GPU-parallel rollouts, covering a much richer space of manoeuvres. In tight spaces with narrow passages, MPPI finds feasible paths that DWA's limited arc-based sampling misses.
- Scenario 2 — Nonlinear cost functions: MPPI handles arbitrary (non-convex, non-differentiable) cost functions since it uses sampling rather than gradient-based optimisation. When the cost includes discrete penalties (e.g., semantic zone costs, social navigation rules), MPPI naturally handles them while DWA's scoring function is more restrictive.
- Scenario 3 — Dynamic obstacles: MPPI's longer and more diverse prediction horizons capture interactions with moving obstacles (pedestrians, other robots). The stochastic sampling naturally explores evasive manoeuvres that DWA's deterministic arc evaluation cannot anticipate.

(c) Constraint handling:

- DWA: Velocity constraints are enforced by the dynamic window (only admissible velocities are sampled). Obstacle constraints are handled by scoring: trajectories that intersect obstacles receive a very high (penalised) cost or are discarded. This is soft constraint handling — there is no formal guarantee of feasibility.
- MPC: Constraints are encoded as explicit inequality constraints in the optimisation: $x \in \mathcal{X}$, $u \in \mathcal{U}$, $h(x) \geq d_{\text{safe}}$. The solver guarantees that the returned solution satisfies all constraints (if feasible). This is hard constraint handling with formal guarantees.

12.5.20.3 Question 2 — AMCL and State Estimation for Navigation Control (25 marks)

Explain how Adaptive Monte Carlo Localisation (AMCL) provides state estimation for the navigation control loop. Draw parallels to the Kalman filter used in LQG (Week 8).

(a) Describe the AMCL algorithm (particle filter) and explain its role in providing state estimates $[\hat{x}, \hat{y}, \hat{\theta}]$ to the controller. (10 marks)

(b) Compare AMCL (particle filter) with the Extended Kalman Filter (EKF) used in LQG. Under what conditions would you prefer each? (10 marks)

(c) Explain how uncertainty in the AMCL estimate propagates to the controller and affects closed-loop performance. (5 marks)

12.5.20.4 Model Answer

(a) AMCL algorithm:

AMCL is a particle filter implementation for robot localisation given a known map. It maintains a set of N weighted particles $\{(x_i, y_i, \theta_i, w_i)\}_{i=1}^N$, each representing a hypothesis of the robot's pose.

1. Prediction: When the robot moves (odometry $\Delta x, \Delta y, \Delta\theta$), each particle is propagated using the motion model with added noise: $x_i' = f(x_i, u) + \epsilon$. This is analogous to the Kalman filter prediction step $\hat{x}^- = A\hat{x} + Bu$.
2. Update: When a LiDAR scan arrives, each particle's weight is updated based on how well the scan matches the map from that particle's pose: $w_i = p(z \mid x_i, \text{map})$. This is analogous to the Kalman update with innovation $y - C\hat{x}$.
3. Resampling: Particles with low weights are replaced by copies of high-weight particles (with noise). This concentrates particles around likely poses. The "adaptive" aspect adjusts N based on the spread of particle weights — fewer particles when confident, more when uncertain.

4. Output: The state estimate \([\hat{x}, \hat{y}, \hat{\theta}]\) is the weighted mean of all particles: \(\hat{x} = \sum_i w_i x_i\). This estimate is published on `/amcl_pose` and consumed by the navigation controller.

(b) AMCL vs EKF comparison:

Feature	AMCL (Particle Filter)	EKF (Kalman Filte
Distribution	Arbitrary (multimodal)	Gaussian (unimodal
Global localisation	Yes (particles spread globally)	No (requires initial
Computational cost	O(N) per step, N = 500–5000	O(n^3) per step, n =
Nonlinearity handling	Exact (samples from true distribution)	Approximate (first-
Curse of dimensionality	Scales poorly beyond ~6D	Scales well to high

Prefer AMCL when: (1) the initial pose is unknown (global localisation / kidnapped robot), (2) the environment has symmetries causing multimodal distributions, (3) the state space is low-dimensional (2D + heading). Prefer EKF when: (1) the state is high-dimensional (6-DOF drone), (2) good initial estimates are available, (3) real-time performance is critical on embedded hardware.

(c) Uncertainty propagation to the controller:

The AMCL estimate has covariance \(\Sigma_{\text{pose}}\) derived from the particle spread. When this uncertain estimate is fed to the controller (e.g., MPC tracking a path), the controller acts on \(\hat{x}\) rather than the true \(x\). The tracking error becomes \(e = x_{\text{ref}} - \hat{x} + (\hat{x} - x)\), where \(\hat{x} - x\) is the estimation error. High AMCL uncertainty (e.g., in featureless corridors) directly degrades tracking performance. This motivates the LQG separation principle: design the controller assuming perfect state knowledge, then account for estimation noise through the observer (Kalman filter or particle filter). However, unlike LQG, the separation principle does not hold optimally for nonlinear systems with particle filters, so the combined estimator-controller performance is suboptimal.

Autonomous Systems Corp — Senior Autonomy Engineer Technical Challenge

Pre-Interview Assessment Full-Stack Autonomy Team

Confidential — Do not distribute without authorization

Colophon

This book was typeset using the `kaobook` document class developed by Federico Marotta, which is in turn based on the `classicthesis` style by André Miede. Lecture content was converted from source HTML via `pandoc`; figures were generated from the course's MATLAB and Python simulations.